University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Piotr Styczyński
Student no. 386038

An integrated and practical database solution for storage and efficient processing of large genomic datasets

Master's thesis
in COMPUTER SCIENCE

Warsaw, 2026-07-28

Abstract

Genomic workflows commonly connect specialized programs through files. This arrangement separates sequences from relational metadata, repeats parsing and indexing work, and prevents the database planner from representing the cost of alignment. This thesis presents BioSeqDB, a PostgreSQL extension that stores compact biological sequences and runs a minimap2-derived seeding, chaining, and extension pipeline during database scans.

BioSeqDB provides alphabet-generic packed sequence views, scalar and single instruction, multiple data (SIMD) minimizer extraction, and Vulkan implementations for selected sketching and chaining operations. Bounded CPU dynamic-programming kernels and GPU alignment routes perform final extension. Optional SneakySnake and GateKeeper filters reject unsuitable candidate regions before that stage. The database layer adds persistent minimizer indexes, bounded ingestion, external-provider relation definitions, coordinated caching, and generated PostgreSQL boundary declarations.

Alignment is not evaluated as an independent scalar function for every row. Planner support introduces custom paths whose execution nodes batch sequences, execute the explicitly selected implementation when the input and hardware pass admission, and place alignment results into planned expressions. The same operation can therefore participate in joins, filtering, metadata projection, and persistent storage. The thesis describes how the mapper's data and control flow become a database execution primitive while retaining explicit resource bounds and support for heterogeneous processors.

Keywords

sequence alignment, bioinformatics, minimap2, PostgreSQL, databases, genomic data management, SIMD, GPU computing

Thesis domain (Socrates-Erasmus subject area codes)

11.3 Informatics and Computer Science

Subject classification

Applied computing, life and medical sciences, bioinformatics

Tytuł pracy w języku polskim

Zintegrowane i praktyczne rozwiązanie bazodanowe do przechowywania i wydajnego przetwarzania dużych zbiorów danych genomicznych

Contents

Introduction . . . 7
Timeline . . . 10

1. Biological foundations of sequence analysis . . . 11
1.1. Biological context . . . 11
1.1.1. DNA, ribonucleic acid (RNA), and protein alphabets . . . 12
1.2. Alignment operation . . . 12
1.3. Technical considerations . . . 13
1.3.1. FASTA . . . 13
1.3.2. FASTQ . . . 14
1.3.3. CIGAR . . . 15

2. Sequence alignment and search strategies . . . 17
2.1. Alignment foundation and specification of a problem . . . 17
2.2. Dynamic programming definition . . . 18
2.3. Seed and extend . . . 20
2.4. Limitations of existing tools . . . 22

3. Minimizer extraction . . . 25
3.1. From every base to a sparse, comparable sketch . . . 25
3.2. Methods of minimizer generation . . . 27
3.2.1. Scalar rescan . . . 27
3.3. A1: Naive SIMD rescan . . . 29
3.4. A2: Two-stack algorithm . . . 30
3.4.1. One window minimum from two saved values . . . 30
3.5. A3: Chronological SIMD . . . 32
3.6. Bioseq prototype . . . 34
3.7. Seeding prototype V1–V6: Naive SIMD rescan . . . 35
3.8. Seeding prototype V8: Implementing A2 algorithm . . . 36
3.8.1. Seeding prototype V8.5: Deferring hash calculation . . . 37
3.9. Seeding prototype V9–V11: Stream interleaving and forced inline . . . 37
3.9.1. Seeding prototype V12: Forced inline . . . 38
3.9.2. Seeding prototype V13: Existmator fix . . . 39
3.9.3. Seeding prototype V14: directly use nthash . . . 40
3.9.4. Seeding prototype V15: Smaller minimizer space . . . 41
3.9.5. Seeding prototypes summary . . . 41
3.10. GPU acceleration for seeding . . . 42
3.10.1. Sketch pseudocode and work . . . 43

3.11. BioseqDB GPU seeding . . . 44
3.12. Packed minimizer representation . . . 47
3.13. Configured CPU algorithms . . . 49
3.13.1. Vector width and indexed queries . . . 50

4. Chaining seed anchors . . . 53
4.1. From local seed hits to candidate mappings . . . 53
4.2. Anchor geometry and ordering . . . 53
4.3. Intuitive interpretation . . . 55
4.4. Chaining components . . . 55
4.5. Chaining algorithms . . . 57
4.5.1. Shared pair score . . . 57
4.6. B1: minimap2 dynamic programming . . . 59
4.6.1. Skipping with counters and marked anchors . . . 61
4.7. BioseqDB SIMD DP: exact and bounded batch . . . 65
4.7.1. Candidates in SIMD vectors . . . 65
4.7.2. Summary of forward chaining algorithms for CPU . . . 69
4.8. Reverse chaining (backtracking) . . . 69
4.8.1. Formal definition for backtracking . . . 70
4.8.2. Minimap2 backtracking . . . 70
4.8.3. BioseqDB SIMD backtracking . . . 74
4.9. Statistical modelling of CPU forward and backward passes . . . 78
4.9.1. Problematic chains . . . 78
4.9.2. Score distribution for random cases . . . 86
4.9.3. Does the bounded SIMD U limit help? . . . 87
4.9.4. BioseqDB `max_drop` analysis . . . 88
4.9.5. Conlusions . . . 90
4.10. BioseqDB SIMD performance comparison . . . 91
4.11. Range minimum query chaining . . . 91
4.11.1. Minimap2 RMQ . . . 92
4.11.2. RMQ re-chaining in the pipeline . . . 95
4.11.3. RMQ in BioseqDB . . . 96
4.12. GPU chaining . . . 97
4.12.1. G3SA algorithm . . . 97
4.12.2. BioseqDB Vulkan tiled DP . . . 98
4.13. Selecting primary and secondary chains . . . 100

5. Final alignment and postprocessing . . . 103
5.1. Base-level alignment . . . 103
5.2. How Block Aligner follows the alignment . . . 105
5.3. Postprocessing . . . 106
5.4. SneakySnake prefiltering . . . 108

6. Sequence representations and lazy evaluation . . . 111
6.1. Designing sequence abstraction . . . 111
6.2. Packed symbols and a common C++ interface . . . 115
6.3. Slices keep logical coordinates . . . 117
6.4. Composing sequences without joining their payloads . . . 119
6.5. Reading only what the algorithm requires . . . 120

6.5.1. Copying ranks from a PostgreSQL sequence . . . 122

7. PostgreSQL API and custom scans . . . 125
7.1. Designing SQL API for an aligner . . . 125
7.2. Integrating Postgres . . . 128
7.2.1. Designing SQL API . . . 128
7.2.2. Plan lifecycle . . . 129
7.2.3. Different types of scans . . . 131
7.3. Dispatch buffer as glue between Postgres scan and alignment pipeline . . . 132
7.3.1. Returning results back to Postgres . . . 132
7.3.2. Supporting joins . . . 134
7.3.3. Dispatch buffer internals . . . 134
7.3.4. Memory admission before dispatch . . . 138
7.3.5. MVCC support . . . 138
7.3.6. Filter placement . . . 138
7.4. Joins support . . . 139
7.5. Caching results . . . 142
7.5.1. Reusing fragments of queries . . . 143

8. Physical sequence storage in PostgreSQL . . . 145
8.1. Store sequences together with their metadata . . . 145
8.2. Columnar storage . . . 146
8.2.1. Columnar layout . . . 147
8.2.2. Smart reads . . . 150
8.2.3. Compression . . . 151
8.3. Raw byte storage . . . 152
8.4. Locator implementation . . . 154
8.4.1. Two levels of sharding . . . 156
8.5. MVCC and deletion for sequence fragments . . . 158
8.5.1. BioseqDB all cache layers . . . 160

9. Persistent sequence indexes . . . 163
9.1. Keep the lookup work between queries . . . 163
9.2. What existing sequence indexes store . . . 163
9.2.1. Indexing: minimap2 . . . 163
9.2.2. Indexing: mmseqs2 . . . 164
9.3. BioseqDB indexing abstractions . . . 164
9.4. BioseqDB supported indexing . . . 169
9.4.1. Index versioning . . . 171
9.4.2. Symbol count index . . . 174
9.5. Building MPS . . . 174
9.6. Selective reads . . . 176
9.7. Other index considerations . . . 176
9.7.1. Reading saved minimizers . . . 177

10. Benchmarking and performance evaluation . . . 179
10.1. Benchmarking and performance evaluation . . . 179
10.1.1. What the results show . . . 180
10.1.2. Measurement procedure and reading the full tables . . . 190

10.1.3. Complete raw results . . . 191
10.1.4. Comparison with other aligners . . . 232
10.1.5. Vulkan on a Vast.ai GPU instance . . . 237

11. Conclusions . . . 239

Authorship, development and AI assistance . . . 241

References . . . 243

1. A runnable SQL walkthrough . . . 249

2. Configuration reference . . . 261
2.1. Physical storage . . . 261
2.2. Planning and execution . . . 263
2.3. Task scheduling . . . 264
2.4. Index construction and reading . . . 265
2.5. Minimizers and seed filtering . . . 267
2.6. Chaining . . . 268
2.7. Primary and secondary alignments . . . 269
2.8. Base alignment . . . 270
2.9. Vulkan execution . . . 271
2.10. Diagnostics . . . 272
2.11. Optional query-result cache . . . 273
2.12. Controls present only in test builds . . . 273
2.13. Per-search options . . . 274

Introduction

Sequencing instruments turn biological material into collections of strings. The useful interpretation of those strings rarely exists in isolation. A read has an instrument and sample provenance. A reference interval belongs to an assembly and annotation release. An alignment has a score, a strand, a position, an edit path, and a relationship to experimental metadata. Conventional workflows keep the strings in FASTA and FASTQ sequence files, Sequence Alignment/Map (SAM) and Binary Alignment/Map (BAM) alignment files, or mapper-specific index files while keeping much of the surrounding information in tables or ad-hoc scripts. Each tool sees only the portion of the workflow represented by its command-line arguments.

Relational database systems solve a different part of the problem. PostgreSQL provides durable storage, transactions, typed values, indexing, joins, extensible operators, and a cost-based planner. It does not, by itself, provide the representations or execution kernels required for billions of packed symbols and computationally expensive sequence searches. A naive user-defined function is not sufficient. Evaluating one alignment function independently for every row prevents batching, hides accelerator opportunities, and gives the planner little information about a predicate that can dominate the query.

BioSeqDB investigates the boundary between these two systems. It is implemented as a PostgreSQL extension whose non-database core is written in C++26. Biological symbols are represented by generic compact ranges rather than by a deoxyribonucleic acid (DNA)-only byte string. The alignment path is derived from minimap2: representative k-mers are extracted as minimizers, matching minimizers become anchors, compatible anchors are chained, and CPU or GPU alignment routes extend the selected regions. The port is not a line-for-line translation. The implementation introduces template-selected alphabets and layouts, Highway SIMD kernels, task-based parallel execution, Vulkan compute paths, persistent PostgreSQL-backed indexes, and prefilters for candidate regions. These changes must preserve the semantics that make minimap2 useful while operating on data that may be lazy, fragmented, cached, or obtained from an external provider.

The database boundary is equally important. An SQL expression such as `mmsearch(a, b)` has a conventional function definition so that PostgreSQL can type and plan it. The ordinary function body is not an alignment fallback: it fails closed if no BioSeqDB custom executor owns the expression. Its planner-support function assigns the unowned expression a high startup cost, while structural planner hooks can construct an owning custom path with explicit nonnegative costs. The resulting custom scan batches input, invokes the alignment pipeline, and makes its results available to planned expressions through injected PostgreSQL nodes. This is what permits alignment to participate in filters, projections, and joins while retaining the execution structure required by central processing unit (CPU) vector units and graphics processing units (GPUs).

The central research question is therefore not whether an existing aligner can be called from SQL. It is whether the aligner's data and control flow can become a database execution

primitive without losing algorithmic fidelity, resource bounds, or the ability to exploit modern processors. The investigation is organized around four supporting questions:

RQ1. Which parts of seeding, chaining, and extension can be expressed over a common compact sequence abstraction without eager conversion?

RQ2. Which stages admit SIMD or GPU implementations, and which input and resource constraints govern their selection?

RQ3. How can PostgreSQL plan and execute a set-valued, batch-oriented search while presenting an ordinary typed SQL interface?

RQ4. How should sequences, indexes, ingestion, external data, caches, and generated boundary code fit around the kernels so that the resulting system remains inspectable and reliable?

This thesis answers these questions through implementation and analysis of algorithmic and execution contracts. Historical designs appear only when they explain a constraint or a rejected optimization. They are not presented as simultaneous production architectures. Comparisons with minimap2, SIMD-Minimizers, the G3SA GPU seed-chain-extend system, Block Aligner, SneakySnake, GateKeeper, and Citus Columnar distinguish inherited mechanisms from BioSeqDB-specific changes.

The main contributions developed in the following chapters are:

C1. An alphabet-generic sequence representation that supports packed, fragmented, lazily composed, and PostgreSQL-backed data through a common range contract.

C2. A staged minimap2-derived mapper with scalar, SIMD, and Vulkan components and explicit runtime dispatch.

C3. Optional GPU-capable SneakySnake and GateKeeper candidate filters integrated without changing the meaning of an accepted alignment.

C4. Persistent sequence and minimizer-index storage derived from a columnar PostgreSQL access method, with bounded loading, caching, and selective cold-index reads.

C5. A planner and custom-scan interface that connects SQL expressions and joins to batch-oriented alignment.

C6. Structural generation and validation of SQL installation definitions and PostgreSQL function-manager boundary declarations from annotated C++ sources.

The document first establishes the biological and algorithmic context, then states the shortcomings that motivate an integrated system. The algorithm chapters follow data through minimizer extraction and chaining. They then describe the expensive downstream alignment stage before stepping back to the prefilter that guards that stage. This expository inversion makes the filter's safety and cost contract concrete without changing the runtime order, which remains chaining, prefiltering, and alignment. It then moves outward through sequence abstractions, PostgreSQL planning, physical storage, persistent indexes, ingestion, external providers, and generated SQL boundary code.

- Chapter 1 introduces biological sequences and sequencing data. It explains variation and the computational questions that motivate database-resident sequence analysis.

- Chapter 2 develops exact dynamic programming and seed-and-extend mapping. It also presents graph-based approaches and the limits of existing tools.
- Chapter 3 explains minimizer sketching and rolling hashing. It compares the SIMD and Vulkan implementations with minimap2 and SIMD-Minimizers.
- Chapter 4 derives the chaining objective and compares its scalar RMQ SIMD and GPU forms. It then states the selection policy.
- Chapter 5 compares KSW2 and Block Aligner; explains SneakySnake filtering and distinguishes the reported alignment statistics.
- Chapter 6 develops compact sequence ranges and bit-level views. It then presents fragmented and lazy sequences with PostgreSQL-backed adapters and alphabet dispatch.
- Chapter 7 follows an mmsearch expression through planning and batched execution. It explains joins and per-row result substitution.
- Chapter 8 explains the columnar-derived storage layout and fragment metadata. It also covers caching and request coordination.
- Chapter 9 describes index construction and storage with compatibility checks and selective cold loading. It also explains updates and differences from minimap2 indexes.
- Chapter 10 measuring overall performance of BioseqDB
- Chapter 11 answers the research questions and summarizes the final system. It identifies the principal limitations and possible extensions.

Timeline

This thesis required several years of exploration and experiments. I began without a formal background in computational biology, so learning the field was part of the research. I proposed the database idea and prepared the initial design for our internal team. I directed the 2022 prototype, whose implementation was written by colleagues.[1] The MMseqs2 discussions did not lead to a collaboration. The Python prototypes that led to the implementation studied here began in December 2023.



Selected milestones from my project history. Spacing is not proportional to elapsed time. Linked entries point to the related discussion and prototype repositories.

[1] The current BioSeqDB is a separate project, designed and written by me. It contains no code or intellectual property from the early BWA-MEM integration experiments.

[2] I was a co-founder (shareholder) until 15 July 2022 (KRS update on 3 Jun 2022) in Covid Genomics Sp. z o.o. (registered in Poland in Jun 2020, KRS 0000846289) which then changed its legal name to CGen Sp. z o.o. (on 3 Jun 2022) and then changed its branding name to Deepflare (on Jul 2022). I was also a Chief Engineering Officer from 1 Apr 2020 till 31 Aug 2022.

Chapter 1

Biological foundations of sequence analysis

1.1. Biological context

"Significant scrutiny and attention have been given to genome-sequencing costs and how they are calculated since the beginning of the field of genomics in the late 1980s". Institute, The Cost of Sequencing a Human Genome NHGRI (National Human Genome Research Institute) has carefully tracked costs per genome and expose this data publically.

In the last 15 years, there has been seen a massive development of sequencing technologies. Companies like Applied Biosystems, Illumina, Roche, and Pacific Biosciences have developed a plethora of sequencing platforms, e.g. SOLiD, GS FLX, and PacBio. These tools are able to generate an enormous amount of data in a matter of days. As an example, Illumina NextSeq generates up to 360 Gbp, split into reads of length up to 150 bp in merely 2 days (data taken from Illumina website).

Although those platforms vary in detail, the general idea behind them is as follows:

1. DNA is fragmented through physical and chemical methods. Adapters are appended to the ends of the fragments to form what is called a sequencing library.
2. Fragments of unwanted size or self-ligated adapters are removed from the library.
3. The library is immobilized on a chemical bead so that it can be steadily observed.
4. Amplification is performed, e.g. through PCR emulsion
5. A combination of optical devices and fluorophores is used to make reads.

With new advancements in sequencing technologies, the cost of sequencing a genome has been decreasing significantly to the point where most popular open databases store more than 10s of petabytes of data. To make this data usesable researchers often need to perform search operations:

- For bioinvestigative resarch: does this genomic sequence already contains known genes? Does it contain a known mutation? Is it a known pathogen?
- For evolutionary biology: is this genomic sequence a known evolutionary lineage? What is the genetic distance between this sequence and a known sequence?
- For legal research: is this DNA sample from known individual? Does it represent known kinship to someone else?

This search operation is using an operation called ”sequence alignment” which is the core concept of this thesis.

1.1.1. DNA, ribonucleic acid (RNA), and protein alphabets

DNA is a polymer containing one of four bases: adenine, cytosine, guanine and thymine. Their conventional one-letter symbols form the alphabet

$$\Sigma_{\text{DNA4}} = \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}.$$

Four symbols are sufficient for an unambiguous DNA string, but experimental data introduces ambiguity. The symbol `N` conventionally means that the nucleotide at a position is unknown or was not resolved. Other International Union of Pure and Applied Chemistry (IUPAC) codes describe sets of possible bases, such as a purine at a position.

RNA uses adenine, cytosine, guanine and uracil, so a basic RNA alphabet replaces `T` with `U`. Many RNA molecules are transcribed from DNA. Crick, “Central Dogma of Molecular Biology”

A protein sequence is written over the twenty common amino-acid symbols. It’s alphabet is notably larger, but in most cases we can translate it to a DNA sequence and use the DNA alphabet directly.

Sequence class	Core symbols
DNA4	`A,C,G,T`
RNA4	`A,C,G,U`
DNA5	`A,C,G,T,N`
RNA5	`A,C,G,U,N`
PROT20	Twenty common amino-acid symbols

Table 1.1: Biological alphabets and the distinctions that a sequence-processing system must not erase. The table describes biological data in general. It does not assert that every alphabet or ambiguity convention is exposed by every BioSeqDB SQL path.

1.2. Alignment operation

Due to nature of biological sequences, search must be able to match fragments that are non-exact, thus alignment operation is similar to popular fuzzy-search operations on normal text. The alignment can be viewed as a processes that optimises gaps (insertions and deletions) in a way to optimise similarity metric. High similarity score between parts of sequences can indicate that they share same evolutionary origin or have a common purpose. By comparing newly found sequences to already researched ones, we can deduce lots of relevant biological data. Juliusz K and W, “Master thesis: An integrated and practical database solution for storage and efficient processing of large genomic datasets”

We will consider finite nonemptystrings over the DNA4 alphabet. A gap is a symbol indicated by `-`. An alignment is a process of adding gaps to the sequences so that they are of the same length and at the same time, there is no index where a gap occurs in both sequences.

As an example, let us present an alignment of $GATCAGATA$ and $CATTCAATCA$. One of the possible alignments is:

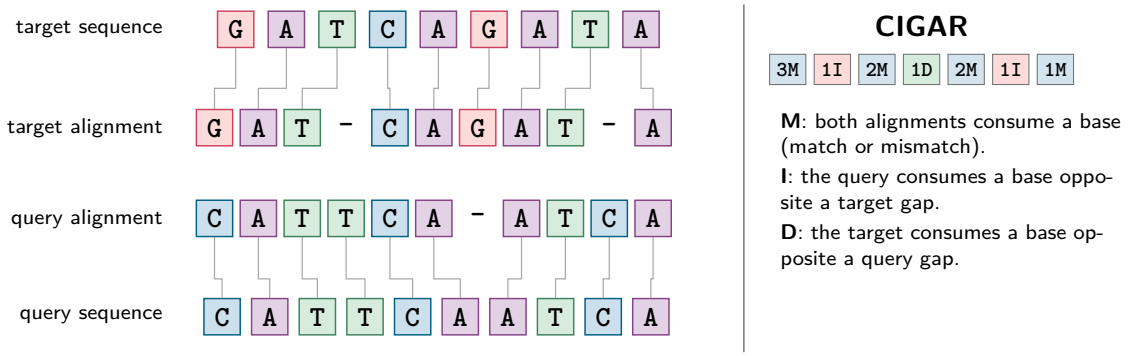


Figure 1.1: Alignment of the target sequence `GATCAGATA` and query sequence `CATTCAATCA`. Orthogonal connectors map each ungapped sequence position to its aligned column; gaps are intentionally unboxed.

There are 4 cases that can occur during alignments:

1. Insertion - a gap has been added to the first sequence
2. Deletion - a gap has been added to the second sequence
3. Mismatch - letters at a given index are different
4. Match - letters at a given index are the same

1.3. Technical considerations

A typical workflow for sequence analysis includes 3 data formats - FASTA, FASTQ, and SAM/BAM. Reference genome, stored in FASTA, is aligned with millions of reads in the FASTQ file and the result of alignment is saved as SAM/BAM file. The reason for the reference genome being in FASTA and reads in FASTQ is that we assume that the reference genome is correct, but reads produced by sequencers can contain various errors. Moreover, we're more likely to score alignments with high-quality bases higher than the ones with low-quality.

1.3.1. FASTA

FASTA is a text-based file format for storing nucleotide sequences. It is one of the most widely used formats in bioinformatics. The current standard specification is given by NCBI. Below is a description of the usually encountered version.

It consists of a series of entries separated by a new line. Each entry starts with a single-line header. The header starts with `>` and contains various metadata about the following sequence, like the description of the sequence, library accession number, and submission date. After the header comes the sequence itself represented with standard IUB/IUPAC acid codes, split into lines of length usually not exceeding 80 characters. Aside from A, C, T, G commonly encountered is code N, which means any of A/C/T/G. Lowercase letters are treated the same as uppercase. The usual extension is .fasta and .fa. An example FASTA file:

```
>PartOfHumanGenome|HG_9357/2022|2022-03-01
ACTTTGAGCTAGCATCGATTGGATCAGATGCAT
ACTCATCGGCAGTCGCATCAGCTTGGACTACGT
>PartOfMouseGenome|MG_5417/2019|2019-08-14
ACTATAGGAGCAGANNNNNNNNCTAGAGACATC
TCTGCGTNTNTTCCGGACGTACGTAGCATAGTC
```

The advantages of this format are simplicity and human readability. The downside is that it is not optimal in terms of size. To adhere to this issue, compression software is used, like gzip.

1.3.2. FASTQ

FASTQ is an extension of the FASTA format. It enhances FASTA by adding quality scores of sequences. It is a standard output format of sequencers.

Each entry consists of 4 lines. The first line starts with `@` and contains metadata about the sequence. The second line contains the sequence itself. The third line starts with `+` and may contain some additional metadata. The fourth line contains quality scores, represented by ASCII characters with codes from 33 to 126, one character per nucleotide. The file extension used is .fq and .fastq. Again, gzip can be used to reduce file size. An example FASTQ file:

```
@HumanGenomeSequencerRead
ACTACTAGACGG
+Genome20200301
*!]^##[$*@*!(
```

The quality score is usually a phread score, meaning Q satisfies $Q = -10 \log_{10} p$, where p is the probability of a corresponding nucleotide being incorrect. ASCII codes from 33 to 126 are mapped from 0 to 93.

1.3.3. CIGAR

CIGAR is an acronym for Concise Idiosyncratic Gapped Alignment Report and has been originally defined by the Exonerate alignment program. The CIGAR format is designed to contain the minimal information necessary for the reconstruction of an alignment. This format does not include nucleotides. Sweden, The Cost of Sequencing a Human Genome

- M - match or mismatch
- D - deletion
- I - insertion
- S - soft clipping. Can occur only at the beginning or at the end of the alignment. Indicates that nucleotides are clipped from the alignment.
- H - hard clipping. The same as soft clipping, except that the clipped part is not present in the SEQ field of the SAM file.
- N - skipped region. Used for representing introns (part of the gene that is non-coded) in the alignment.
- = - match
- X - mismatch

Chapter 2

Sequence alignment and search strategies

2.1. Alignment foundation and specification of a problem

Alignment can be done in different ways, where the core algorithms stays the same, but some of the conbditions change. It's important to understand what are the capabilities of standard dynamic programming algorithms and how they can be used to solve different problems.

Formally: Let $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$ be the sequences we align. An alignment is a sequence of columns

$$A = ((a_1, b_1), \dots, (a_L, b_L)), \qquad (a_r, b_r) \in (\Sigma \cup \{-\})^2 \setminus \{(-,-)\},$$

where first and second elements reciver contents of X and Y after gaps are removed:

1. An element of shape $a_r, -$ signifies an insertion in X
2. An element of shape $-, b_r$ signifies a deletion in Y
3. An element of shape m_r, m_r signifies a match
4. An element of shape a_r, b_r where $a_r \neq b_r$ signifies a substitution

This alignment is an optimized sequence correspondence under a scoring model. For substitution score $s(a, b)$ and gap cost $g(\ell)$, the score is

$$S(A) = \sum_{\text{symbol pairs } r} s(a_r, b_r) - \sum_{\text{gap runs } u} g(|u|).$$

The scoring model may be formulated in different ways, but one of the simplest and most-common conventions is to use an affine gap cost model:

$$g(\ell) = q + \ell e, \qquad \ell \geq 1, \tag{2.1}$$

where q is the opening cost and e is the extension cost.

A gap of size one costs then $q + e$. Some scientific papers mentioning $q + (\ell - 1)e$ use a different parameter convention. Affine scoring charges one opening cost per gap run and can be evaluated with a fixed number of states per matrix cell Gotoh, "An improved algorithm for matching biological sequences."

This scoring model isn't sufficient to define entire problem:

1. Global alignment consumes both complete sequences Needleman and Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins"

2. Local alignment may restart at zero and selects the best pair of substrings Smith and Waterman, "Identification of common molecular subsequences"
3. Semi-global and overlap goals consume selected ends while allowing specified prefix/postfix to remain unaligned
4. Extension begins from a fixed anchor and chooses an endpoint under a stopping rule
5. Clipping will just finish on the rightmost gap and keep postfix unaligned
6. Local-extension fixes the start but permits an interior endpoint

Figure 2.1 shows schematic examples of these modes.

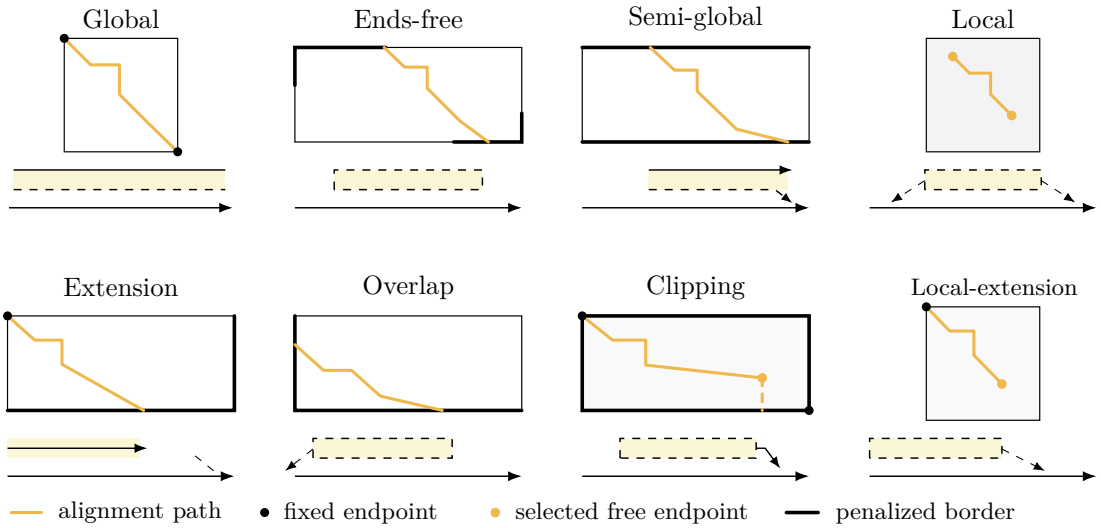


Figure 2.1: Eight alignment boundary modes. Yellow lines depcict possible tracebacks. Black dots mark fixed endpoints (for example for global alignment we always align entire sequence so endpoints are 0 and len(Y)-1), yellow dots mark selected free endpoints, and bold border segments mark penalized or anchored boundaries (usually the border is the length of sequence, but under certain conditions we might have limits on the alignment length and coordinates). In each lower schematic, the solid arrow is the reference and the yellow dashed track is the query. These panels showcase how different libraries/algorithms handle different goals across number of scientific resarch

Various practical implementations also use special stopping criteria, which usualy can be upper bound on the $S(A)$ score.

2.2. Dynamic programming definition

Let $H_{i,j}$ be the best score after consuming i symbols of X and j symbols of Y. For a simpler linear gap cost model d, we can define the algorithm's state matrix as:

$$H_{i,j} = \max\{H_{i-1,j-1} + s(x_i, y_j), H_{i-1,j} - d, H_{i,j-1} - d\}. \tag{2.2}$$

This is the intuitive definition based on 3 possible options (replacement or two variants of gap). Free-end alignment (like local) change the score values for the cells from which traceback may begin.

For affine scoring model we need to hold main state and one state for each gap orientation. Let $I_{i,j}$ and $D_{i,j}$ end in gaps in X and Y, respectively, and let $M_{i,j}$ end in a symbol pair. Under Equation 2.1, one formulation is

$$\begin{aligned} M_{i,j} &= H_{i-1,j-1} + s(x_i, y_j), \\ I_{i,j} &= \max\{H_{i,j-1} - q - e,\ I_{i,j-1} - e\}, \\ D_{i,j} &= \max\{H_{i-1,j} - q - e,\ D_{i-1,j} - e\}, \\ H_{i,j} &= \max\{M_{i,j}, I_{i,j}, D_{i,j}\}. \end{aligned} \tag{2.3}$$

Using H to open a gap permits a transition from any previous state. Local alignment also includes zero in the final maximum.

Algorithm 1 Affine-gap alignment with an explicit boundary contract

Require: sequences $X[1 \dots n]$, $Y[1 \dots m]$, scores s, q, e, and a boundary contract B
Ensure: score, endpoints, and an optional CIGAR string

```
initialize H, I, and D according to B
for i ← 1 to n do
    for j ← 1 to m do
        update M_{i,j}, I_{i,j}, D_{i,j}, and H_{i,j} with Equation 2.3
        store the predecessor of each winning state when traceback is requested
select the endpoint permitted by B
follow predecessors to an allowed start and run-length encode the operations
```

The algorithm state matrix can be used to retrieve the edit path as CIGAR string. This is the standard compact representation of gapped alignments described in chapter 1SAM/BAM Format Specification Working Group, Sequence Alignment/Map Format Specification

Affine model is some scenarios might not be enough as it doesn't distinguish between short and long gaps. The two-piece affine model can support such feature without complicating the scoring function too much:

$$g_2(\ell) = \min(q_1 + \ell e_1,\ q_2 + \ell e_2) \tag{2.4}$$

This model permits different cost slopes for short and long gaps. In our software we use single affine model for the fastest alignment modes (adaptive block), while KSW2 variants support dual affine model.

For large datasets evaluating entire large $O(nm)$ matrix can make significant practical difference:

- We can evaluate alignments only close to diagonal. Being close to diagonal means that there aren't many large gaps between matching fragments. Designing algorithms around diagonals can make us skip evaluation of many far cells that do not contribute to the result.
- In case of features that do not require traceback (CIGAR construction requires traceback, but score or coordinates of best match do not) we can just store relevant slice in memory and avoid allocating large memory region.
- We can introduce X-drop stop condition: stop when score falls below best score seen so far by more than X

- We can introduce Z-drop stop condition: the same as X-drop, but if the distance from diagonal is too high then stop as well (this means that we stop when there are too many gaps cummulatively)

Smart evaluation startegy can combine all these approaches. KSW2 and Block aligner for example use all of them. Block aligner Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices" evaluates only the most promising rectangular region in the matrix while KSW2 Suzuki and Kasahara, "Introducing difference recurrence relations for faster semi-global alignment of long sequences" focuses on diagonal-centered evaluation.

2.3. Seed and extend

Applying local dynamic programming between a query of length m and a reference of length N costs $O(Nm)$. Most reference positions don't contribute to the result. Long-read mappers therefore organize search into three stages Li, "Minimap2: pairwise alignment for nucleotide sequences."

First step is to use divide and conquer approach and split sequence into representatives known as seeds:

$$\begin{aligned}\mathsf{Seed}(x_1,\ldots,x_n) = \{(s_1,i_1,j_1),\ldots,(s_k,i_k,j_k)\},\\ \text{where } 1 \le i_a, j_a \le n \quad \text{for all } a \in \{1,\ldots,k\}.\end{aligned} \tag{2.5}$$

Seed (s_a, i_a, j_a) is some represenative of the sequence substring $x_a = x_{i_a} x_{i_a+1} \ldots x_{j_a}$. This can be implemented in different ways, this definition is the most abtract one. If a seed occurs in both query and reference, then we can consulde that two represented substrings are very similar. Using some concrete seeding function we can obtain intersection for query and reference:

$$S_{X\cap Y} = \left\{ (s, i_x, j_x, i_y, j_y) \;\middle|\; \begin{matrix}(s,i_x,j_x) \in \mathsf{Seed}(X),\\ (s,i_y,j_y) \in \mathsf{Seed}(Y)\end{matrix} \right\}. \tag{2.6}$$

1. Seeding Generate $Seed(X)$ and $Seed(Y)$ and obtain $S_{X\cap Y}$
2. Chaining As matching seeds will cover only parts of both sequences that match without errors, we still need to fill-in the gaps between them. We connect seeds from $S_{X\cap Y}$ into chains using similar dynamic programming approach as in usual aligner. This gives us some approximate alignments.
3. Extension Perform usual alignment algorithm on the approximate fragments to obtain final results.

Minimizers are one of many possible seeding model definitions. Given parameters w and k, a window W of length $l = w + k - 1$ contains w consecutive k-mers. The minimizer of the window is the smallest k-mer (subsequence of length k) in the window. For random minimizers, k-mers are ordered by a pseudo-random order, usually given by comparing hashes of the k-mers. In case of ties, the leftmost smallest k-mer is chosen. Roberts et al., "Reducing storage requirements for biological sequence comparison" Overlapping windows often retain the same representative, which is emitted once.

Canonical minimizers: DNA is double-stranded and majority of sequencing technologies do not distinguish these two strands. Genomic sequences thus need to satisy additional constraint: sequence and its reverse complement ($A \leftrightarrow T, C \leftrightarrow G$) should be handled in the same

way by the software (they should be indistinguisable) In our case this means that for minimizer of a window W at position p we need to have minimizer $\overline{W^c}$ at position $w-1-p=|W|-k-p$. Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast" In most of the software due to simplicity truly cannonical minimizers aren't used. Instead we take minimum of hashes of k-mer and its reverse complement: $m^{\text{cannonical}} = \min(m, \overline{m^c})$

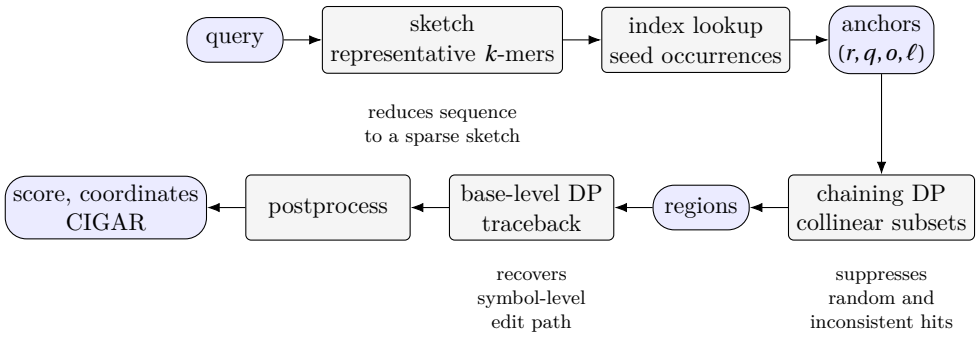


Figure 2.2: The seed–chain–extend decomposition

In case of Minimap2 and BioseqDB $S_{X\cap Y}$ is represented as tuple (r,q,o,ℓ) which holds reference coordinate r, query coordinate q, orientation o and span ℓ (length).

After sorting anchors by ending reference coordinate, with implementation-specific tie keys, a chaining score can be written

$$f(i) = \max\Big(w_i,\ \max_{j<i}\{f(j)+\alpha(j,i)-\beta(j,i)\}\Big), \tag{2.7}$$

where α rewards newly covered matching bases and β penalizes large gaps and diagonal changes. A representative minimap2 reward is

$$\alpha(j,i) = \min\big(\min(y_i - y_j,\ x_i - x_j),\ w_i\big).$$

Testing every $j<i$ is $O(A^2)$ for A anchors. Minimap2 scans a bounded recent window and stops after a configured number of unproductive indices, making the practical cost much smaller Li, "Minimap2: pairwise alignment for nucleotide sequences."

Algorithm 2 Conceptual bounded predecessor chaining

1. Input. Anchors $a_1, \ldots, a_A$ in canonical coordinate order.
2. Endpoint loop. For each $i = 1, \ldots, A$, perform the following steps:
   (a) Initialize $f[i] \leftarrow \mathsf{span}(a_i)$ and $p[i] \leftarrow \perp$.
   (b) For each candidate $j < i$ in the active distance and iteration window, perform the following steps:
      i. If a_j and a_i have valid order, strand, gap, and bandwidth, compute
      $$v \leftarrow f[j] + \mathsf{reward}(a_j, a_i) - \mathsf{gapPenalty}(a_j, a_i).$$
      ii. If the candidate is valid and $v > f[i]$, set $f[i] \leftarrow v$ and $p[i] \leftarrow j$.
      iii. Optionally stop the predecessor loop after the configured skip heuristic fires.
3. Backtracking. Backtrack from high-scoring endpoints through p.
4. Result. Return the chains that remain after filtering by score, seed count, and overlap.

2.4. Limitations of existing tools

Most of the current most popular tools implement variations of the seed and extend approach described in the previous section. However the algorithm outlined here is heavily underspecified to be directly implemented as a computer program:

- $S_{X \cap Y}$ can be implemented as interection of two hash maps where $Seed(X)$ is realized as map from seed to its start/end: $S \to \natural^2$ where seed itself is represented by it's hash $S = \mathbb{Z}$. This opens few important questions: what hashing strategy to choose? Which map implementation to choose?
- As $Seed(X)$ can be computed before alignment starts we can store it in memory and persist it to disk to speed up processing in the future. Which binary format we want to choose?
- Anchors as tuples will be implemented as some concrete binary structure and authors need to specify their representation (do we store them as AOS or SOA? What are widths of elements and their types?). Each decision will imapct performance as there can be millions of anchors per long sequence and additional bytes sum up to MBs of additional memory to be processed (and slowdowns).
- Finally, what is the most effective way to parallelize this computation (the most straightforward approach is to split input sequences X and Y into chunks)

Most of the current state of the art tools choose to:

1. File-based operation
2. Suboptimal or non-extensible internal data structures
3. Optimise only selected parts of the algorithm like hash function properties or one specific stage.

This leads to different consequences for the user.

File-based operation: Operating directly on the disk with simplistic wire format means that seed index is usually one giant binary blob not prepared for highly-concurrent access and interface doesn't offer modern tools like buffer pooling or lazy loading. The format of indexes are largely undocumented by authors. Missing of lazy loading feature is the most hurting to the user. If index is large it will be loaded entirely into memory and can overwhelm SWAP space. This degrades performance by several orders of magnitude and can be avoided by splitting sequences into smaller chunks. User doesn't know what is the exact relationship between sequence length and required memory space, so this process takes time and involves guessing. Additionally index implementation sometimes offer complex parameters to adjust its partitioning (like in case of MMseqs2), but entire responsibility is still shifted to the user and that leads to user frustration andrecurring OOM issues or severly degraded performance. Youngblut, MMseqs2 Github issue 338 Zhan, MMseqs2 Github issue 517 Software not only treats indexes as files, but it operates on FASTA files directly. This means the user cannot easily sort/split/extract or filter records without using external tools.

Suboptimal or non-extensible internal data structures: Minimap2 uses 31 bits to encode anchor's position meaning that practically entire human genome (>3Gbps) cannot be aligned at once and require splitting via other tools before any operation is performed. LexiMap (not using seed and extend approach) limits maximum genome size to 268Mbp Shen, LexicMap FAQs. Additionally software is implemented without proper abstractions - minimap2 code is non-easily extensible C code, while G3SA doesn't offer extension points for new types of indexes or sequences interfaces.

Selective optimisations: Optimisations are usually fixed for one single platform (AVX512 or SSE) and either get severe performance penalties (scalar fallbacks) or user cannot use them at all. SIMD support is quite popular even though it supports one or two selected platforms, but GPU support exists usually only for NVIDIA graphics cards. This leads to a difficult situation for the user as they need to choose between several versions of the same tool (Saurabh Kalikar, mm2-fast: Accelerating minimap2 for long-read sequencing applications on modern CPUs, Sadasivan H, mm2-gpu: Accelerating Minimap2 for Accurate Long Read Alignment on GPUs), while better, more modern software like G3SA exists purely on NVIDIA hardware Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment" Modern hardware like MacBook Pro M3 with arm64 architecture won't be able to utilise any integrated GPU acceleration. While minimap2 offers compatibility layer to port SSE code to NEON, it does not vectorize all operations and code ported from SSE doesn't account for lane widths and instruction level parallelism available in M2-M4 chips. Feintzeig, SIMD + Parallelism: Ray Tracer Optimization Part 3

I wanted to address these limitations by implementing BioseqDB, Postgres extension that will provide the following features:

- Tight integration with Postgres meaning that user can operate on sequences (loaded FASTA files) directly using SQL rather than resorting to external toolkits.
- Support for indexes and storage structures designed with parallelism and lazy-loading from the start, meaning that software can be run on very RAM-constrained device and still perform well. At the same time design will incorporate assumptiuon about arbitrary large sequences so the user will be able to operate on >3Gbps records without any additional steps.

- Provide layer of abstractions written using latest C++26 standard, so the algorithms can be reused in separate software or extended to different types of new sequences and indexes.
- Provide optimised kernels for SIMD and GPU architectures written using architecture-agnostic, but performant code.

Tight integration with Postgres also means that as a side effect, we can utilise some benefits of persistent state: deferring expesive operations, keeping large indexing structures in memory in case use reruns query and more effective memory-disk caching schemes.

Chapter 3

Minimizer extraction

3.1. From every base to a sparse, comparable sketch

The first computational stage of the BioSeqDB aligner needs to compute $\mathsf{Seed}(X)$. A target seed becomes a key in an index and then query seed is looked up in that index, and each occurrence (so shared seed between query and target) becomes an anchor for the chaining stage described in Section 4.1.

We expect the seed set to be sparse enough to reduce index and lookup costs, but dense enough that related sequences have common seeds. As described before minimap2 and BioseqDB operate on minimizers. I didn't specifically added support for "true" cannonical minimizers in a sense that I adhered to the minimap2 approach i.e taking minimum of hashes of k-mer and its reverse complement.

Formally, let $S = s_0 s_1 \dots s_{n-1}$ be a nucleotide sequence. The k-mer beginning at zero-based position i is

$$K_i = S[i, i+k).$$

For DNA, the forward and reverse-complement encodings are

$$F_i = \mathrm{enc}(K_i), \qquad R_i = \mathrm{enc}(\overline{K_i}).$$

A canonical k-mer uses the same key on both strands. A common choice is

$$C_i = \min(F_i, R_i), \qquad z_i = \mathbf{1}[R_i < F_i],$$

where z_i records the selected strand. Applying a hash function H gives a candidate

$$x_i = (h_i, p_i, z_i), \qquad h_i = H(C_i).$$

The position p_i may denote the first or final base of K_i. BioSeqDB's minimap-compatible schemes use the final base because that is the convention used by minimap2 Li, "Minimap2: pairwise alignment for nucleotide sequences."

For a window of w consecutive candidates,

$$W_j = (x_j, x_{j+1}, \dots, x_{j+w-1}), \qquad m_j = \min_{x \in W_j} h(x).$$

The expected density is theoretically $2/(w+1)$ minimizers per candidate position. Shur, On Minimizers of Minimum Density, however practically it might vary. Minimap2 for example uses HPC (homopolymer-compressed) encoding which means that repeated letters are compressed

so we don't extract repeated minimizers for sections that are larege single-letter segments. In BioseqDB I do not support HPC compression.

There are also other practical and theoretical consideratiosn affecting minimizer extraction like different formulations of hashing functions or heuristics to avoid having very low minimizer density in specific repeated regions, but I won't go into very specific details.

Figure 3.1 shows what happens to one k-mer through hashing, selection of window that its belongs to and dedupliction of hashes.

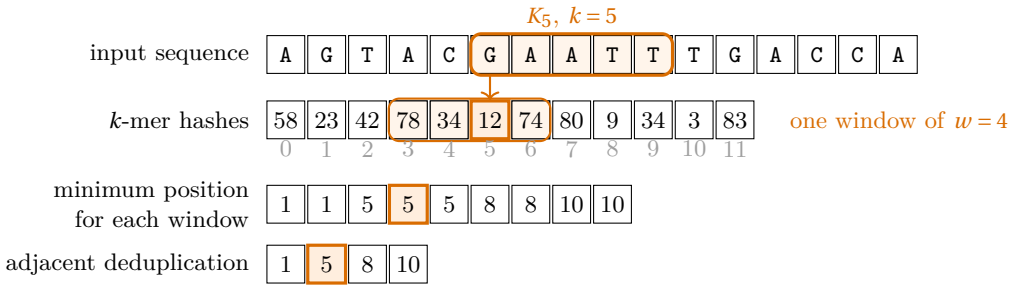


Figure 3.1: Minimizer extraction for $k=5$ and $w=4$ with illustrative hashes.

The hashing function can affect heavily minimizer desnity:

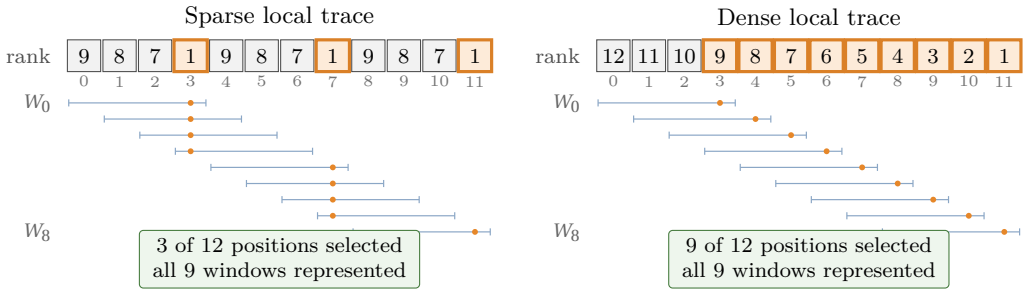


Figure 3.2: Two minimizer traces for $w=4$. Thin brackets mark candidate windows, and orange dots mark their selected minima. The left trace reuses positions 3 and 7. The right trace selects a new position after each slide. Everything depends on numeric properties of used hashing scheme

I wanted to emphasize that the topic of seed-and-extend is largely based on measuring practical stochastic properties of minimizers distribution and properties and in BioseqDB I only focus on the simplest approaches where in most cases I use the same hashing scheme as minimap2 to guarantee almost exact sensitivity and in one case discussed later incorporate use of rolling hash (nthash).

Figure 3.3 uses fixed four-base seeds on both sides. The symbols h_1, h_2, and h_3 are some computed hashes. The picture shows that in general we want to build one hash table, compare query to the hashes in the hash table and always filter out the most occuring hashes (as this means that this hash does not ultimately distinguish the part that we maching and will cause lots of false positives not contributing to the final result)

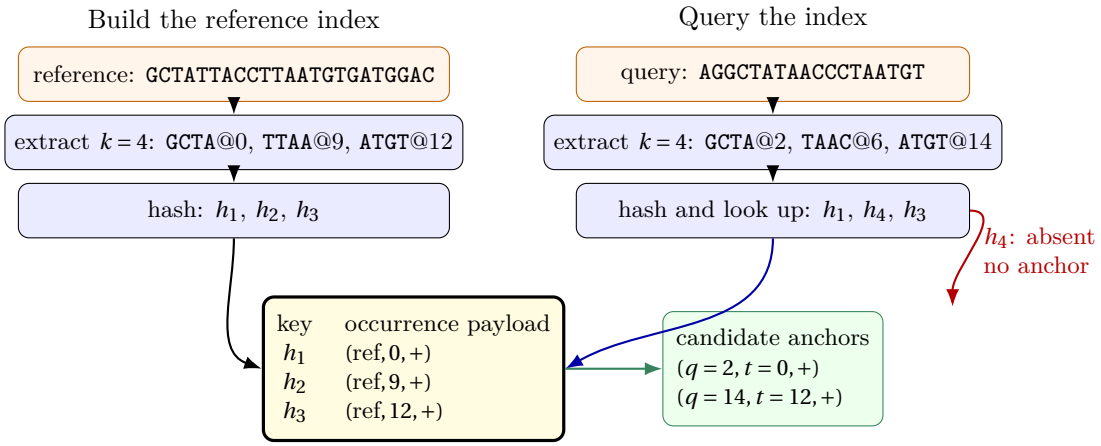


Figure 3.3: Reference-index construction and query lookup. Positions are seed starts in this example. A usable seed index stores some seed metadata like position and strand

Frequent seeds can produce very large occurrence lists. BioSeqDB can exclude seeds whose target frequency exceeds a configured threshold before anchors are constructed. This is common practice in other software as well: minimap2 can remove most common minimizers for specific sequence by using percentage threshold (`--q-occ-frac` $\in [0, 1]$)

3.2. Methods of minimizer generation

3.2.1. Scalar rescan

The most straightforward algorithm to select smallest hash in a given sliding window is to implement scalar rescan. We keep track of current minimum until it leaves the window. Then we need to do ”rescan” part which calculates new minimum from scratch. Additionally we perform deduplication of adjacent hashes, so if the minimum is unchanged, we don’t emit new minimizer. However if there’s new equal hash on different position, then we treat that as a new minimizer.

This approach is implemented by minimap2.

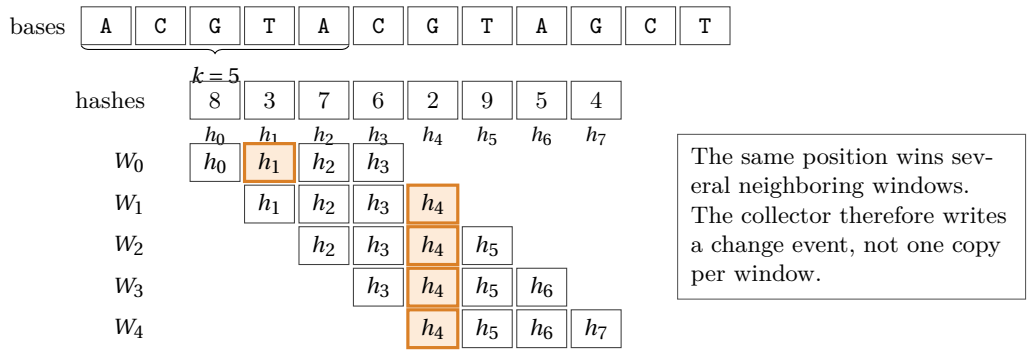


Figure 3.4: Minimizer extraction with illustrative $k = 5$, $w = 4$ hashes. Each row is one window of four adjacent k-mers. Orange marks the selected position. The hash value 2 remains selected while the window moves, so those windows do not require repeated output.

Equal hashes at different positions are not duplicate records. We wanmt to keep both to guarantee higher minimizer density.

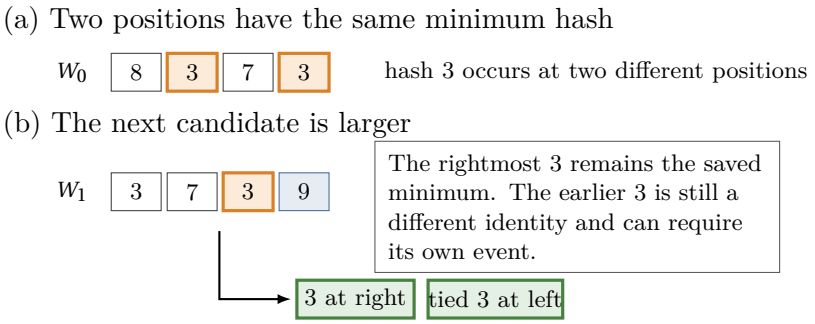


Figure 3.5: Why a hash value alone does not define the output. Both orange cells have hash 3, but they refer to different k-mer positions. Minimap2 keeps the rightmost equal position as its saved minimum and has explicit rules for writing other tied identities.

Minimap2 stores the most recent w candidates in a circular buffer It also stores the current minimum and the array slot containing it.

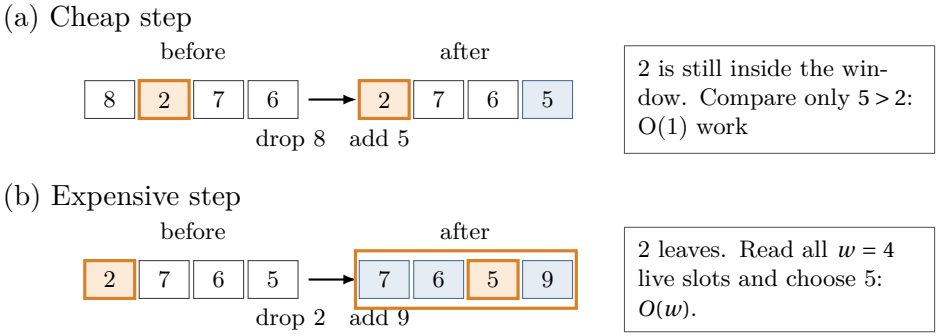


Figure 3.6: The two costs of one minimap2 window advance. Gray cells are already in the ring, blue is the entering candidate, and orange is the saved or newly found minimum. A normal step performs one comparison. When the saved minimum leaves, the algorithm reads the complete window again (rescan)

Minimap2 stores kmers and apply hash function each time from scratch.

Algorithm 3 Scalar minimap2-style rescan

1. Store k-mer binary representation and its reverse complement and update them each step. Calculate hash for both of them and take the minimum.
2. Write this hash into the next slot of the w-slot circular buffer.
3. If new hash is less than or equal to the saved minimum, emit new minimizer.
4. Else, if the overwritten slot held the current minimum, emit new minimizer and scan all w slots from oldest to newest, and save the rightmost record having the smallest hash as new minimum.
5. At the end of the sequence, write the remaining saved minimum from buffer as final minimizers.

3.3. A1: Naive SIMD rescan

Let's assume we have scalar linear iteration over data. The first though is to unroll the loop so each element of a SIMD vector corresponds to one iteration. This approach however works only if each iteration step is independent. If we have any condition that requires knowledege of results of previous step the SIMD vector's elements will be dependent on each other and won't give us any advantage. The other approach would be to threat each element not as consecutive input items, but as separate entities handling different parts of the input. Each lane will handle separate parts of iteration, so we will avoid data dependency of vector's elements on each other and replace it with the dependency between current state vector and previous state vector. In this document, I will define a SIMD lane as one numbered element of a vector register.

Each SIMD lane will load fragments of input data:

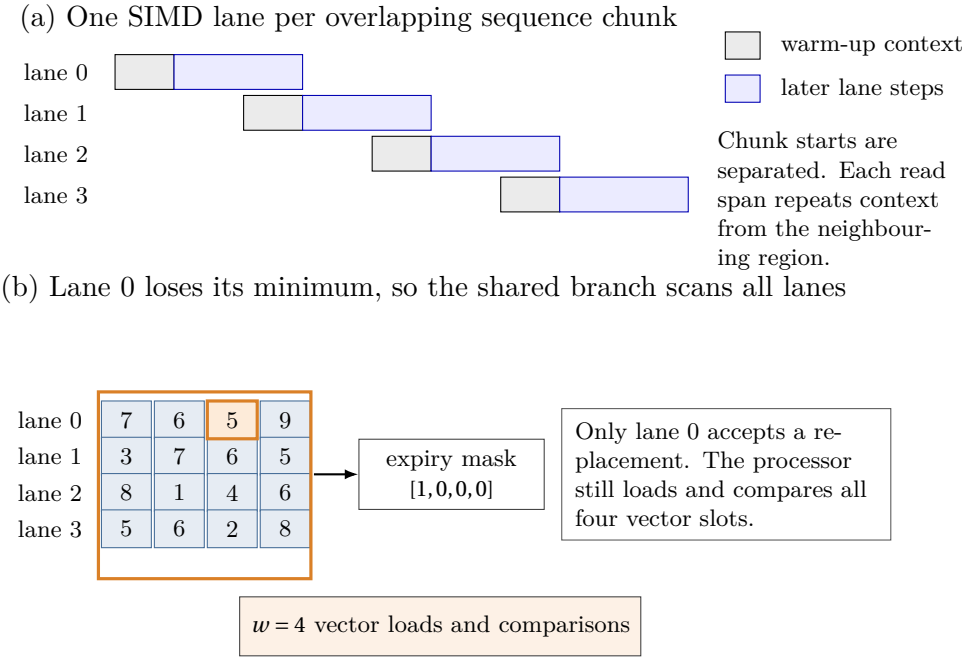


Figure 3.7: Why direct SIMD rescan is costly. Each lane follows a different sequence piece. A nonzero expiry mask enters one shared loop over all w vector slots. Masking protects lanes 1–3 from changing their answers, but it does not remove their loads and comparisons.

Note that algorithm fundamentally stays the same, we only now divided input sequence into chunks and translated each operation (hashing, comparisons) into SIMD instructions:

Algorithm 4 SIMD minimap2-style rescan, with the output cases stated plainly

1. Split input sequence into chunks each corresponding to the lane of SIMD vector. Each lane handles different chunk (chunks overlap slightly to make sure we don't omit any window)
2. Store k-mer binary representations and its reverse complements and update them each step for each lane. Calculate hashes for both of them and take the minimum for each lane.
3. Write all hashes for each laneinto the next slot of the w-slot circular buffer (this buffer now stores SIMD vectors instead of single hashes).
4. If the new hash vector is less than equal to the minimum vector element-wise, extract corresponding elements and emit new minimizers.
5. Else (new hash vector is greater then minimum vector on each position), if the overwritten slot held the current minimum for any lane, emit new minimizers and scan all w slots from oldest to newest, and save the new minimums vector.
6. At the end of the sequence, emit all remaining minimums as final minimizers.

As you can see this algorithm works well only if new hash values are greater than current minimum on each position. If not, we still have branching. This is problematic as vectorized algorithms works well if the branching is minimal and CPU pipeline isn't penalized for wrong branch prediction. Even if we assume the predictor is perfect, we still do expensive rescan for each lane if at least one needs to be updated. Assume one lane needs a rescan with probability $1/w$, then a group of L independent lanes enters the shared slow path with probability

$$1 - \left(1 - \frac{1}{w}\right)^L.$$

For $w = 10$ and $L = 4$, this is about 34.4%. This means that we will need to duplicate work much more often than in scalar case.

3.4. A2: Two-stack algorithm

In different approach we can preserve prefix and suffix minima to calculate the window minima wrom both of them. SIMD-minimizers paper introduces this approach as well as usage of rolling hashes (nthash) instead of calculating hash each time from scratch. Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast"

3.4.1. One window minimum from two saved values

Split the hash stream into blocks of w values. Within each block, let P_i be the smallest value from the block start through position i, and let Q_i be the smallest value from position i through the block end. A window starting at j and ending at $e = j + w - 1$ crosses one block boundary and has minimum

$$\min(Q_j, P_e).$$

The suffix values are recalculated in a fixed w-step loop each w steps. This means that we still have "rescan" part, but it becames unconditional.

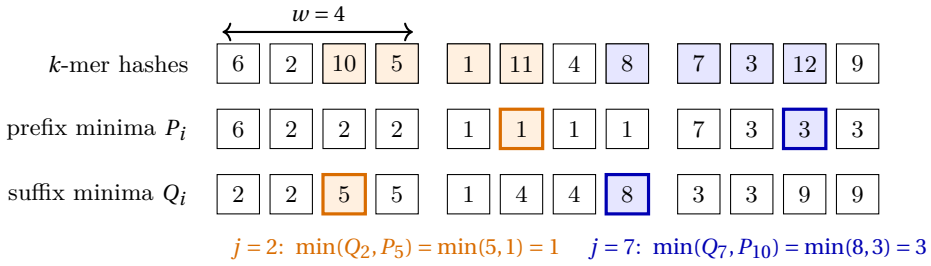


Figure 3.8: Sliding minima from blockwise prefixes and suffixes. The two shaded crossing windows each use one suffix value from the starting block and one prefix value from the ending block.

Fixed-width SIMD vectors

The SIMD-Minimizers use fixed-vector size architecture, where the SIMD vector is set to be uint32 times 8. The split per lane is the same as discussed previously. On Neon architecture only 128-bit-wide register is available, so the 256 fixed vector generates separate store intructions.

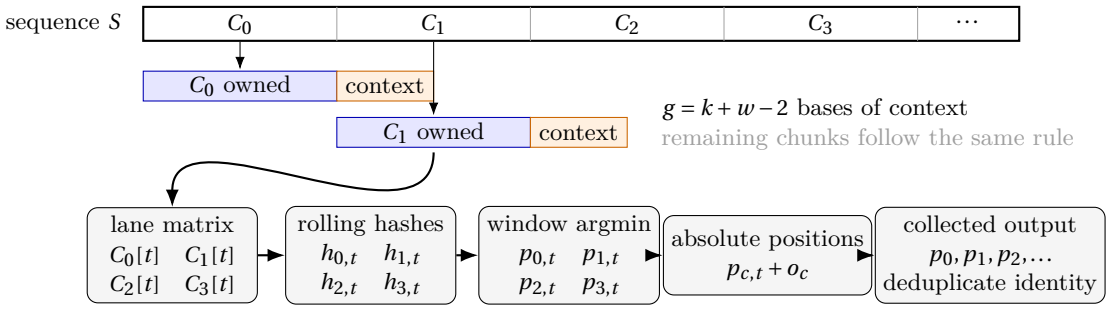


Figure 3.9: Vectorized minimizer extraction: Each chunk owns a disjoint range and reads enough context to complete its windows.

Note that in this setup we can store full minimum vectors and later use matrix transposition to deduplicate them (in case that minimum stays the same over window): At first, one SIMD vector contains the selected positions for several far-apart streams at the same step. After several steps, the SIMD-Minimizers paper algorithm rearranges that output matrix so one row contains consecutive windows from one stream. Consecutive windows often select the same position. Once those positions are adjacent in a row, the paper compares neighbors and keeps only positions that changed. Finally, when the ordered stream results are joined, it removes the repeated boundary position caused by the overlapping input pieces.

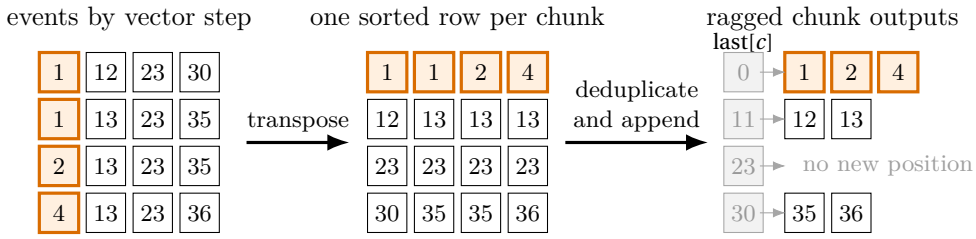


Figure 3.10: Transposition and boundary deduplication for four logical chunks. The input rows are vector steps and the columns are chunks. Transposition makes each chunk a sorted row. Each row has its own previous-position carry, shown in gray. Only new positions are appended, so padding cells are not part of the output.

3.5. A3: Chronological SIMD

Looking at A1 we observed that the main problem was that far-apart sequence chunks can have different characterisistics which cause us to do lots of additional work. Instead of splitting sequence that way, we can work on just one part of neighbouring positions.

We will try to calculate minimums for all local windows i.e calculate result for $X[p, p+1, ..., p+k+w-1]$ (this is window of size w for kmers of size k). Physically we only have L SIMD lanes so we will call that a group and advance only by L positions at the time (i.e from $X[p, p+1, ..., p+k+w-1]$ to $X[p+L, p+L+1, ..., p+k+w-1+L]$).

Potentially we can have more than one group per call. It offers some advantages discussed later. If the group number is higher, then we just execute their operations serially and perform result emission only after all groups execute.

The idea for one group is as follows:

1. We load L new sequence positions into L SIMD lanes each one is shifted by 1 using load instructions

2. For each lane we compute minimum along with strand direction and information which position it is. We call that new candidates: $x_0, \ldots, x_{L-1}$

3. We iterate through all candidates i.e $D \leftarrow T \parallel (x_0, \ldots, x_{L-1})$ where T are previous $w-1$ candidates and calculate new minimums. This gives us M which are L minimums for entire parts of sequence:

$$\begin{aligned}&\{\mathrm{Seed}(X[p, p+1, \ldots, p+k+w-1]),\\ &\mathrm{Seed}(X[p+1, p+2, \ldots, p+k+w]), \ldots,\\ &\mathrm{Seed}(X[p+L, p+L+1, \ldots, p+k+w+L])\}.\end{aligned}$$

   Now this along last previous result c^* forms P: $\mathbf{P} \leftarrow (c^\star, M_0, \ldots, M_{L-2})$

4. For each lane ℓ we compare P_ℓ with x_ℓ. If $h(x_\ell) \le P_\ell$ then P_ℓ is a new minimizer and we emit it. Otherwise if $\mathrm{pos}(P_\ell) = \mathrm{pos}(x_\ell)$ then we see P_ℓ expired so we emit it.

5. If the expiry happens then we go through all w windows and check if their minimum has the same value but different position. This only requires checking $m_0, m_1, ..., m_{w-1}$. If yes then we do slow path: extract the ties in order (hence the name of the algorithm: chronological)

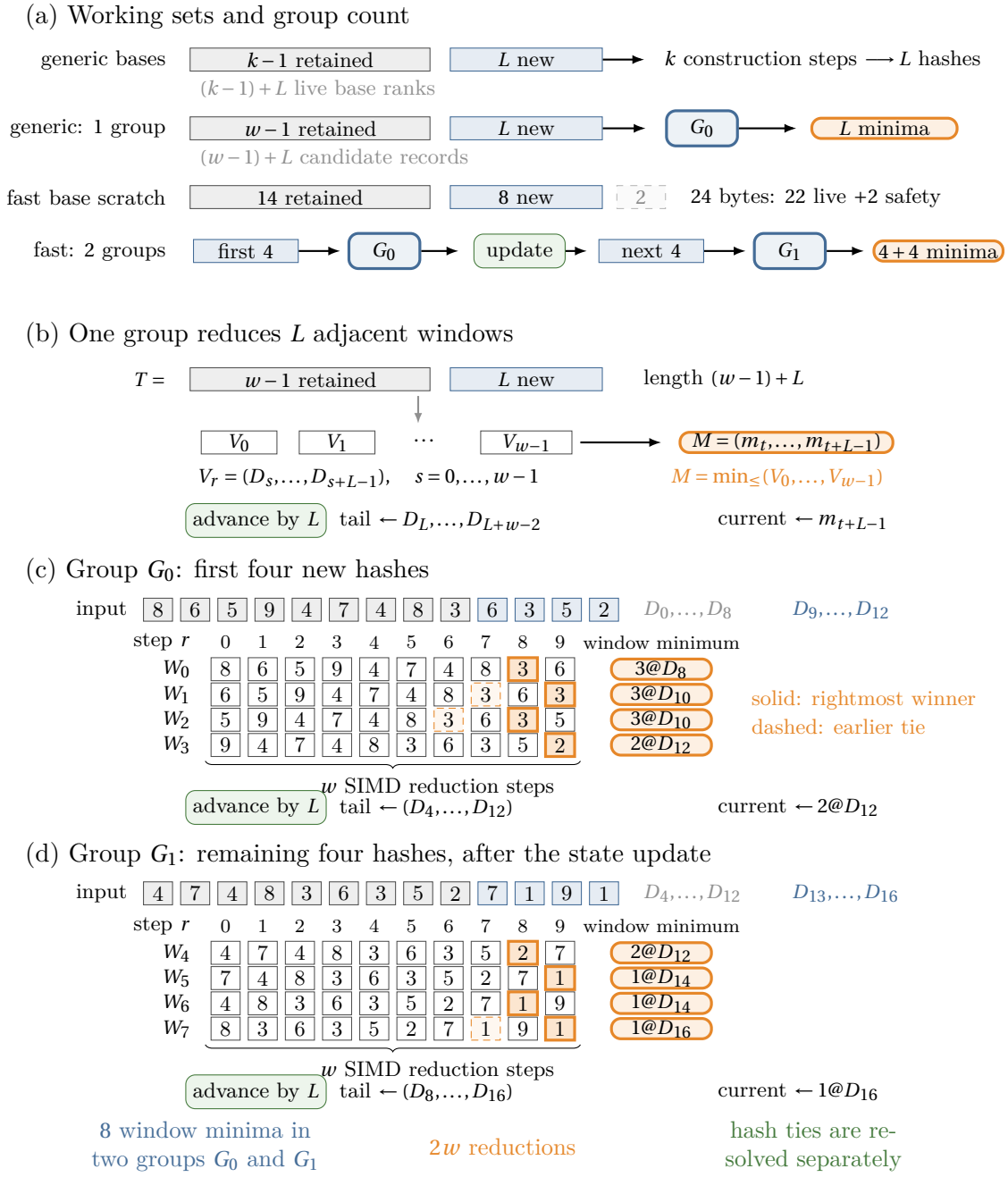


Figure 3.11: Working sets and execution of the chronological adjacent-window kernel. One group combines previous $w-1$ candidates with L new candidates, then loads w shifted vectors, and reduces them componentwise to the rightmost minima of L adjacent windows. Orange cells represent selected minimums (candidates for potential minimizer emit). A solid orange cell is the rightmost equal minimum selected by the source's $\leq$ comparison and a dashed cell is an earlier equal minimum.

The figure 3.11 illustrates calculation of window minima for $L = 4$ lanes: $x_0, x_1, ...x_{L-1}$. The entire algorithm looks as follows:

Algorithm 5 One iteration: build L k-mers and rescan L adjacent windows

Carry: C = last $k-1$ bases, T = last $w-1$ candidates, $c^\star$ = preceding window minimum
while the next L candidate bases are available do
  $U \leftarrow$ next L bases
  $Y \leftarrow C \,\|\, U$
  $\mathbf{F}, \mathbf{R} \leftarrow \mathbf{0}$
    Build forward/reverse kmers symbol by symbol.
  for $t = 0, \ldots, k-1$ do
    $\mathbf{z} \leftarrow \mathrm{load}_L(Y[t \,..\, t+L))$
    $\mathbf{F} \leftarrow F \,\|\, z$
    $\mathbf{R} \leftarrow \overline{z} \,\|\, R$
  Calculate hashes from kmers.
  for all lanes ℓ do
    $a_\ell \leftarrow H(\min(F_\ell, R_\ell))$
    if $F_\ell = R_\ell$ or $a_\ell = h_{\max}$ then
      $x_\ell \leftarrow \infty$
    else
      $x_\ell \leftarrow \mathrm{pack}(a_\ell, \mathrm{position}_\ell, [R_\ell < F_\ell])$
  $D \leftarrow T \,\|\, (x_0, \ldots, x_{L-1})$
  $\mathbf{M} \leftarrow \infty$
    Calculate M, which holds the rightmost minima of all w windows.
  for $s = 0, \ldots, w-1$ do
    $\mathbf{V} \leftarrow D[s \,..\, s+L)$
    $\mathbf{M} \leftarrow \mathrm{select}(h(\mathbf{V}) \le h(\mathbf{M}), \mathbf{V}, \mathbf{M})$
  One step = three field loads + one compare + three selects.
  Hence M_ℓ is the rightmost minimum of the complete window ending at x_ℓ
  $\mathbf{P} \leftarrow (c^\star, M_0, \ldots, M_{L-2})$
  for all lanes ℓ do
    $e \leftarrow \emptyset$
    if $x_\ell = P_\ell$ then
      $e \leftarrow \mathrm{event}^{\mathrm{NEW}}(P_\ell)$
    if $\mathrm{pos}(P_\ell) = \mathrm{pos}(x_\ell)$ then
      $e \leftarrow \mathrm{event}^{\mathrm{EXPIRED}}(P_\ell)$ expiration
    if e is Expired then
      Check all w windows for an equal hash on a different position
      $e \leftarrow \mathrm{event}^{\mathrm{TIE}}(P_\ell)$
    if e is Tie then
      Reread the w records only to construct ordered output slow path
  $C \leftarrow Y[L \,..\, L+k-1)$
  $T \leftarrow D[L \,..\, L+w-1)$
  $c^\star \leftarrow M_{L-1}$
if $0 < q < L$ bases remain then
  Run the same body once with q active lanes and sentinels elsewhere
  Both loops still execute k and w steps
  $C \leftarrow Y[q \,..\, q+k-1)$, $T \leftarrow D[q \,..\, q+w-1)$, $c^\star \leftarrow M_{q-1}$

3.6. Bioseq prototype

We established different approaches to calculate minimizers in previous sections.Now we want to discuss practical considerations when it comes to implementing those algorithms. The work started by exploring minimap2 source code and understanding its behaviour. The first step was to port minimap2 code into modern C++. This was required to have a stable code where

we could test and benchmark different algorithms. I encountered several key problems that lagged implementation by considerable number of months:

1. Minimap2 code uses direct C arrays access for lots of different data structures. Minimizers are encoded as uint128 type represented by a struct with two uint64 fields with abstract filed names. Circular buffers are represented by single raw pointer and require macros for any operations. The encoding and semantics of fields can sometimes change even though underlying uint128 representation stays the same (it's the same field name). This complicates any manual static analysis of the code I conducted.

2. Minimap2 uses raw instrinsics for SIMD operations which makes translation to generic SIMD very tedious.

3. Some parts of minimap2 hash logic are undocumented either in official paper or in-code.

The porting involved several iterations over the minimap2 source to understand how index is split, how parallelization works and how exactly minimizers are represented. After achieving runnable pipeline (with seeding, chaining, alignment and estimator) I started to work specifically on the minimizer extraction part as it's first in the pipeline.

Label	Main change What it means for the minimizer algorithm
V1–V6	naive SIMD rescan approach equivalent to A1
V7–V8	hybrid algorithm A1 where we use two-stack for minimum calculation
V8.5	defer minimap2 hashing to later stages
V9-V10	hand-written input and matrix loop
V11	two independent Highway lane groups
V12	force the per-base body inline
V13	fixes of estimator

Table 3.1: History of the first prototypes

3.7. Seeding prototype V1–V6: Naive SIMD rescan

The first approach (apart from direct minimap2 scalar rescan already existing in the ported code) was implementation of A1 SIMD rescan.

As correctly predicted our benchmark (Table 3.2) showed that the scalar `mm_sketch` took 12.5 ms on a 903 kbp pair, while the "vectorised" `mm_sketch_simd` took 24 ms – a 2× slowdown. This was caused by the fact that any minimum going outside the window causes rescan of all lanes.

Phase	scalar (ms)	V6 SIMD (ms)	V6 over scalar
`mm_sketch`	12.50	14.94	+19%
`mm_sketch_simd`	0	23.88	—
`target_index_build`	16.0	14.3	−11%
`collect_seed_hits`	6.5	11.0	+69%
`mg_lchain_dp`	16.0	0.0	— (replaced)
`mg_lchain_simd`	0.0	8.9	—
`mm_est_err`	0.7	7.2	+929%
`mm_align_skel.`	87.5	86.4	−1%
Pipeline total per pair	158.4	170.7	+8%

Table 3.2: Phase breakdown at 903 kbp, V3 baseline – pre-SIMD-Minimizers work. The naive SIMD path (V6) is +8% slower than scalar despite four lanes of parallelism. The slowdown is split between three regressions: (a) the SIMD minimizer's `mm_sketch_simd` is itself slower than scalar `mm_sketch`; (b) the SIMD chainer feeds anchors into `collect_seed_hits` that hit a worse code path; and (c) the SIMD chain anchors are not cleanly indexable into the sorted query-minimizer array, so `mm_est_err` explodes. The V13 work mostly fixes (a) and (c).

The downstream regressions on `collect_seed_hits` and `mm_est_err` were caused by SIMD chainer producing a slightly different anchor distribution than scalar. This problem is orthogonal to the minimizer extraction and we mention it later.

3.8. Seeding prototype V8: Implementing A2 algorithm

The single most important external input to this work was the SIMD-Minimizers paper of Groot Koerkamp & Martayan Groot Koerkamp and Martayan, "SimdMinimizers: Computing Random Minimizers, fast." Its central insight is that the rescan method is fundamentally data-dependent, and that any data-dependent branch costs more in SIMD than in scalar because lanes' branch targets are tied to each other.

Algorithm 6 sketches the V8 inner loop.

Algorithm 6 V8 `collect_minimizers_two_stack_simd` (per-step body)

state: prefix_min[L], ring[w][L], step, ring_idx
hash ← `rolling_kmer_hash`.Next($d, c_{\text{in}}, c_{\text{out}}$)
strand, palindrome ← StrandMask(hash)
valid ← ¬palindrome
hs ← IfThenElse(strand, hash | 1, hash & ~1) strand bit packed in LSB
elem ← IfThenElse(valid, ((hash & 0xffff_0000) | pos_local), 0xffff_ffff)
ring[ring_idx] ← elem single ring buffer slot per lane
better ← elem < prefix_min
prefix_min ← IfThenElse(better, elem, prefix_min)
ring_idx += 1, step += 1, pos_local += 1
if ring_idx = w then rebuild suffix
  ring[i] ← min(ring[$i..w-1$]) for all i $O(w)$ uniform branch
  prefix_min ← elem
  ring_idx ← 0
window_min ← min(prefix_min, ring[ring_idx])
if step ≥ $k+w-1$ and window_min.pos ≠ last_emit_pos then
  emit window minimum to per-lane scalar collector
  last_emit_pos ← window_min.pos

The V8 numbers were already encouraging: `mm_sketch_simd` dropped from V6's 24 ms to 17 ms at 903 kbp – a 30 % reduction on the minimizer-only phase, which translated into the SIMD pipeline being only 1.4 % behind scalar overall (down from V6's 8 % behind).

Note that in this prototype we weren't using nthash rolling hash, but the original minimap2 hash function.

3.8.1. Seeding prototype V8.5: Deferring hash calculation

Till this time we were using minimap2 hash function for minimizers2, but the observation was that we can use cheaper hash function for ordering values inside the inner loop and then hash the emitted values. If the hash function used will be significantly cheaper we can have proper speedup. The cheap inner hash needs only one property: be a bijection on the $2k$-bit kmer space, so that $\text{lex}(\text{kmer}_1) = \text{lex}(\text{kmer}_2) \iff \text{kmer}_1 = \text{kmer}_2$. We use the "lexicographic-canonical hash" – the bit-packed min(forward, rev_compl) but multiplied by Knuth's golden-ratio constant $0x9E3779B9$ Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching.

The arithmetic of the saving:

$$\underbrace{2.25 \cdot 10^5}_{\text{iters / Mbp}} \times \underbrace{(7\,\text{stages} - 2\,\text{stages})}_{\text{ops saved per iter}} \approx 1.1 \cdot 10^6\,\text{SIMD ops saved}$$

The measured saving was ~ 5 ms per 1 Mbp pair on `mm_sketch_simd`, taking the cost from V8's 17 ms down to 12 ms.

3.9. Seeding prototype V9–V11: Stream interleaving and forced inline

A close reading of the assembly created by Apple Clang LLVM revealed inner loop has unfriendly data dependency chain:

`shift_fwd → or → and → Min(fwd, rc) → Mul(lex, golden).`

The Apple M1 has four 128-bit NEON pipes that issue independently, so a single 4-lane stream wastes three out of four pipes for any given cycle of the dependency chain. This feature is called ILP (Instruction Level Parallelism).

This is limitation that wasn't observed in the original SIMD minimizers paper as Rust code used fixed-size SIMD vectors, where we use adaptive vector size equal to possible physical register size. The possible optimisation here is to keep two independent 4-lane streams in flight simultaneously and interleave their operations in source order:

```
shift_fwd_lo, shift_fwd_hi = ShiftLeft(state_fwd_lo / hi)
state_fwd_lo, state_fwd_hi = And(Or(shift_fwd_, bp_), kmer_mask)
state_rc_lo,  state_rc_hi  = Or(ShiftRight(state_rc_), new_high_)
lex_lo,       lex_hi       = Min(state_fwd_, state_rc_)
hash_lo,      hash_hi      = And(Mul(lex_, golden), hash_mask)
elem_lo,      elem_hi      = IfThenElse(valid_, hash_|pos_, max)
```

We chose 2 streams rather than 4 execution stream because the per-stream state plus the shared ring buffer for $w = 10$ already consumes ~ 26 NEON registers per stream. Doubling it could make the ring buffer spill to stack losing the ILP benefit.

The wall-clock effect at 903 kbp was a ~ 4 ms reduction on `mm_sketch_simd` (from 12 ms to 8 ms); Figure 3.12 plots the full evolution.

3.9.1. Seeding prototype V12: Forced inline

V11 was good but disappointing: the wall-clock saving was less than the theoretical 2× ILP win we expected. `xcrun llvm-objdump -d build/bioseq/bioseq_bench` on the V11 wrapper revealed the underlying problem.

```
ldp q0, q1, [sp, #0x150]       // load bp_row from stack
ushr.4s v2, v1, #0x2           // extract per-BP value
movi.4s v1, #0x3
stp q0, q2, [sp, #0x150]       // store back
and.16b v0, v0, v1
and.16b v1, v2, v1
add x0, sp, #0x5, lsl #12      // prepare lambda's captured-this
add x0, x0, #0x770
movi.2d v2, #0xffffffffffffffff
add x1, x22, #0x8
bl 0x1002acca4 <step_one_bp_pair lambda>   // the offending call!
```

Despite the per-step body being decorated with `[[gnu::always_inline]]`, Clang lowered the lambda as a function call at every BP step. Each call paid:

- Function-call prologue/epilogue (`stp x28-x30 [sp,...]`).
- Captured-state spill/reload via stack – every per-stream state register was loaded from the captured-this pointer at the start of each lambda invocation, and stored back at the end.
- No ILP across iterations – each lambda call is its own function frame, so dependent state spills to memory between consecutive iterations.

Figure 3.12 shows full evolution of the prototypes. The scalar part involves unavoidable SIMD overhead of collecting results into map, scalar scramble hashing (minimap2 hash) and encoding output into the bit structure.

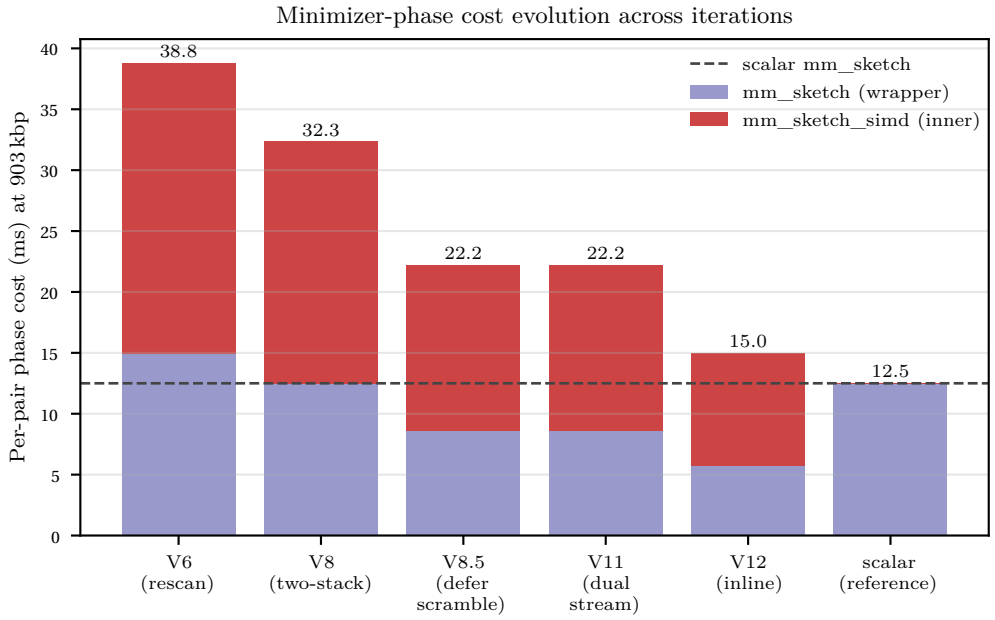


Figure 3.12: Evolution of the minimizer-phase cost (`mm_sketch` wrapper plus `mm_sketch_simd` inner) at 903 kbp across the iteration rounds discussed inuujj §3.7–§3.9. Numbers are means of three runs at `-O3 -DNDEBUG`. V8 is the two-stack port, V8.5 defers the minimap2 hash to emit time, V11 adds dual-stream interleaving, and V12 forces inlining of the per-step body via a free-function template. The dashed horizontal line is the scalar `mm_sketch` baseline. Even with some optimisations still SIMD routine is slower than scalar execution.

3.9.2. Seeding prototype V13: Existmator fix

For each chain, minimap2 estimates the per-base sequence divergence from a Poisson model on the minimizer-set intersection Li, "Minimap2: pairwise alignment for nucleotide sequences":

$$\hat{\epsilon} = \frac{1}{k} \log \frac{n}{m}$$

where n is the number of minimizers that are part of chains and m is the number of all query minimizers between first match and last match. Computing n requires knowing which positions are query minimizers, which is a set membership test against the query's minimizer-position set.

The divergence estimator is used to measure the quality of the seeding. If the divergence is too high, the seeding is not good and the chain is not valid.

When porting minimap2 code we had hidden one important detail visible on table 3.2. The `mm_est_err` section had significant 9-times slowdown. This is because we are using separate chunks for each lane, but the output minimizers were stored in the buffer. Minimap2 normally receives its `mini_pos` array already sorted (because of how scalar loop works). This means that in our case we did the most naive approach:

1. Perform `std::sort` on the array to sort minimizers by positions
2. Perform `std::lower_bound` on the array to test membership

This caused severe slowdown, which we mistakenly attributed to seeding algorithm itself. More on divergence is described in the postprocessing sections.

Figure 3.13 stacks the per-phase cost at 903 kbp for V12-pre versus V13. The two large gains are `mm_est_err` (from 7.88 ms to 0.85 ms) and the V12 `mm_sketch` stack (from 14.99 ms to 9.23 ms, already covered in previous section.

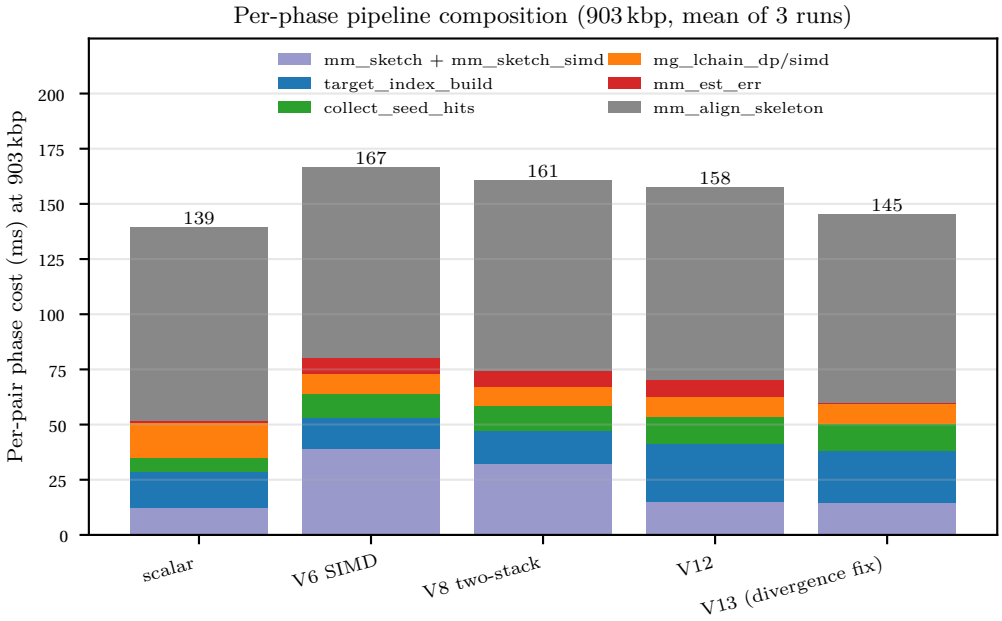


Figure 3.13: Per-phase composition of the per-pair pipeline at 903 kbp we can see that V13 reduced exploded divergence phase

3.9.3. Seeding prototype V14: directly use nthash

So far we utilised minimap2 hash function directly. One other approach was replacing this hash function directly. This should have given us measurable speedup. For nthash we can construct LUT (lookup table) accessed via SIMD instruction.

- `mm_sketch_simd` cost increased by 2 ms (the table lookup used NEON `tbl.16b` instruction and hashing required two rotates, and some numbers XORs per stream per sequence symbol).
- `collect_seed_hits` dropped by 0.4 ms (this isn't significant enough).

My investigation pointed me to the fact that I was trying to optimise incorrect region of computations. The benchmark during all experiments evolved and improved with each iteration. This time the measurement didn't report that most of the time was spent on inserting minimizers into the index.

The gap in performance didn't come from the minimizer hash distribution (or hashing speed itself) but from the usage of `std::unordered_map`. The prototype's target index used STL's default hash table, while the C minimap2 path uses an open-addressing `khash_t<uint64_t>`. This shows another problem with directly porting minimap2 into C++. `std::unordered_map` is known problem with standard library that we overlooked during implementation similarily to how `std::sort` was used for divergence estimation. The cleanest solution here was implementing indexing structures as flat hash maps directly.

3.9.4. Seeding prototype V15: Smaller minimizer space

In all of the experiments we used adjustable minimizer represetations with hash width of 16 bits (minimizer representation is explained in separate section). However smaller represetations mean more efficient algorithms. Existing iterations of seeding algorithm all used adjustable SIMD vector widths provided by C++ Highway library. This means that narrower input type will automtically increase number of available lanes L thus increasing potential performance in all algorithms. However having smaller type means that we need to either get rid of strand information (1 bit) or use narrower position representation (this is not always possible) or use narrower hash representation. We conducted early experiment testing V12 prototype with narrower hash of 14 bits.

	V12 (pre)	V12 (post-Goal-2)	delta
mm_sketch_simd	9.0 ms	9.5 ms	+0.5 ms (insignificant change)
target_index_build	24.7 ms	4.9 ms	−19.8 ms (~85% fewer keys)
collect_seed_hits	12.3 ms	842.0 ms	+830 ms (70x regression)
mg_lchain_simd	8.9 ms	299.9 ms	+291 ms (34x regression)
mm_align_skeleton	87.2 ms	4811.0 ms	+4724 ms (56x regression)

Hash-space collision amplification

The kmer space stored in `target_index` hash buckets collapsed to ~16000 distinct keys for 168000 query minimizers. Each query had ~10 collisions per bucket, meaning the chain-anchor count exploded, and the alignment had to walk through all of them. This means that we sped up index lookups significantly (less memory accesses required), but at the same time the more costly parts of the pipeline get more false positives that need to be verified.

3.9.5. Seeding prototypes summary

TODO

Cite this stack overflow answer: https://stackoverflow.com/a/25027750

The combination of the V12 inlinening fixes, the V13 estimator fix and replacing `boost::unordered_map` (marked as $V13^*$) yielded observable improvements. We didn't want to implement new hashing scheme at this time, only see the impact of using different hash storage.

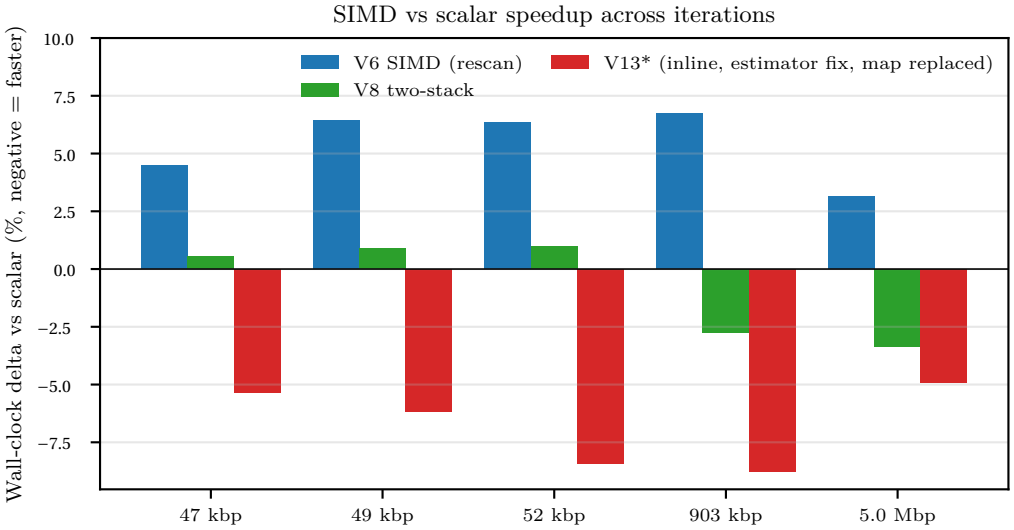


Figure 3.14: Wall-clock delta versus scalar for the three SIMD configurations, expressed as percent (negative = faster). $V13^*$ now performs better than scalar, however this is nowhere near results reported in SIMD Minimizers paper.

The summary table:

Length	scalar (ms)	V13 (ms)	V13 vs scalar	V13 vs V6 SIMD
47 kbp	14.56	13.78	−5.4 %	−9.4 %
49 kbp	14.94	14.02	−6.2 %	−11.8 %
52 kbp	15.09	13.82	−8.4 %	−13.9 %
903 kbp	188.93	172.38	−8.8 %	−14.5 %
5.0 Mbp	1171.0	1113.3	−4.9 %	−7.8 %

Table 3.3: Total measured time for alignment (mean of 3 runs from `-O3 -DNDEBUG`

3.10. GPU acceleration for seeding

G3SA Han et al., "G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment" provides implementation of multiple useful kernels including one specific to minimizer generation phase.[1]

G3SA seeding kernel
mm_sketch_device 83:1

G3SA implements the A3 adjacent-window computation graph, in CUDA SIMT. Let $L = 32$ be the number of CUDA threads. For group $r \in \{0,1,2,3\}$ and thread $t \in [0, L)$, define

$$j^\star_{r,t} = \max \underset{0 \le j < w}{\{\ \operatorname{argmin}\}} h(C_{rL+t+j}), \qquad W_{rL+t} = C_{rL+t+j^\star_{r,t}}.$$

The outer maximum keeps the rightmost record among equal minimum hashes. The vector $(W_{rL}, \ldots, W_{rL+L-1})$ is the componentwise reduction of w shifted candidate vectors. This is the same exact computation as Algorithm 5, but CUDA threads replace SIMD lanes. One difference is that there's no some global state preserved so it's need to be recomputed at initialisation.

[1] Inspected code from official G3SA repository that supplies CUDA kernels along with CPU host-side code, https://github.com/sunwookim028/G3SA

The computation is divided according to the following geometry:

$$L = 32, \qquad G = 4, \qquad n_{\text{owned}} = LG = 128, \qquad n_{\text{shared}} = L(G+1) = 160.$$

We have 128 blocks of 32 threads. Block b handles reads $b, b+128, b+256, \ldots$, one read at a time. For each read it advances by $n_{\text{owned}} = 128$ window starts. In one advance, thread t attempts candidate slots $t, 32+t, \ldots, 128+t$ and then reduces the four windows beginning at $t, 32+t, 64+t$, and $96+t$. Figure 3.15 shows this transpose between candidate construction and window reduction.

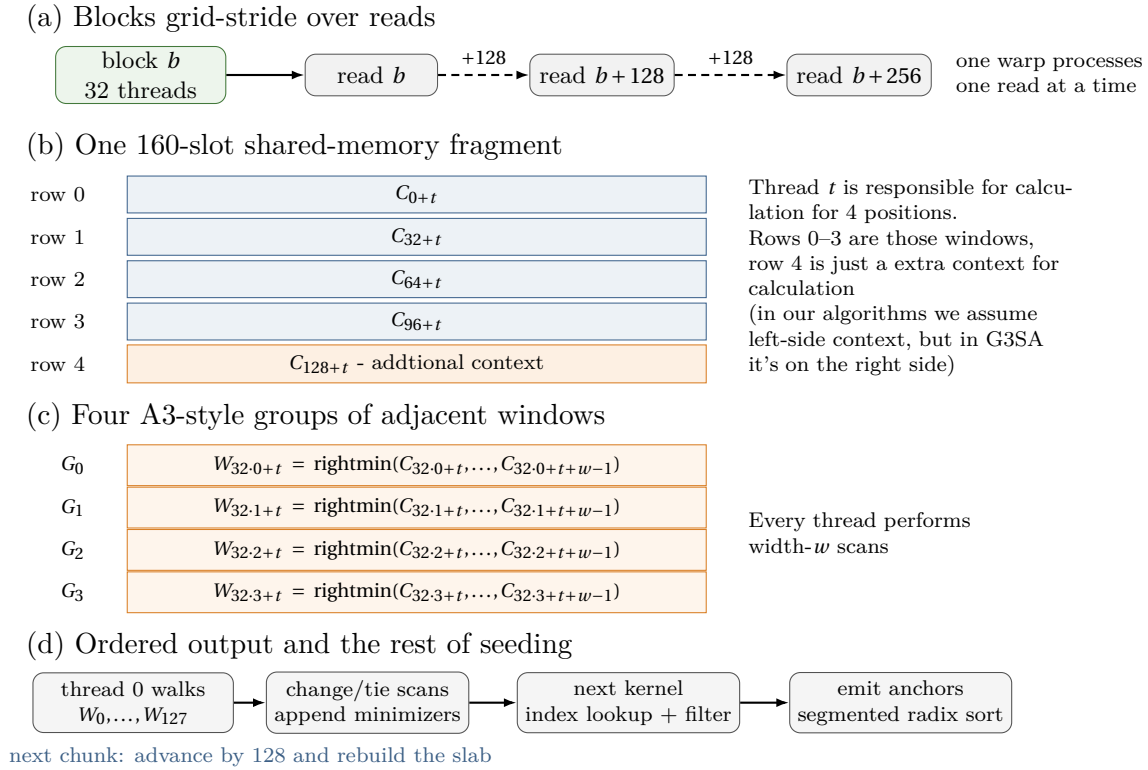


Figure 3.15: G3SA algorithm reconstructed from the CUDA code. This is similar to A3, but has specific work split per thread/warp (standrd CUDA SIMT model)

3.10.1. Sketch pseudocode and work

Here, PublishBlockWrites returns a block-scoped phase token. The token becomes ready only after every thread in the block has reached that phase and the named shared-memory writes are visible to every thread; awaiting it is therefore a collective rendezvous.

Algorithm 7 G3SA sketching CUDA algorithm

Require: concatenated reads S, offsets O, lengths N, index parameters k, w

shared $Base[160]$, $Cand[160]$, $Winner[128]$
procedure SketchBlock(b, t) b is a block; $t \in [0,32)$ is its thread
  for $q \leftarrow b$; $q < |N|$; $q \leftarrow q + \text{gridDim}_x$ do
    $p \leftarrow O[q]$, $e \leftarrow O[q] + N[q]$, $m \leftarrow 0$
    while $p < e$ do
      for $r \leftarrow 0$ to 4 in every thread do
        $u \leftarrow 32r + t$
        $Base[u] \leftarrow (p + u < e)$? $S[p + u]$: NUL
        $Cand[u] \leftarrow \infty$
      $baseReady \leftarrow$ PublishBlockWrites($Base, Cand$)
      await($baseReady$)
      for $r \leftarrow 0$ to 4 in every thread do
        $u \leftarrow 32r + t$
        $Cand[u] \leftarrow$ GetHashedKmer($Base, u, k, p - O[q]$)
      $hashReady \leftarrow$ PublishBlockWrites($Cand$)
      await($hashReady$)
      for $r \leftarrow 0$ to 3 in every thread do
        $s \leftarrow 32r + t$, $best \leftarrow \infty$
        for $j \leftarrow 0$ to $w - 1$ do
          if $Cand[s + j].hash \le best.hash$ and $p + s + j + k - 1 \le e$ then
            $best \leftarrow Cand[s + j]$
        if $p + s + w + k - 1 \le e$ then
          $Winner[s] \leftarrow best$
        else if $N[q] < w + k - 1$ and $s = 0$ and $best < \infty$ then
          $Winner[0] \leftarrow best$ source short-read case
        else
          $Winner[s] \leftarrow 0$
      $winnerReady \leftarrow$ PublishBlockWrites($Winner$)
      await($winnerReady$)
      if $t = 0$ then
        $m \leftarrow$ AppendChangesAndHashTies($Cand, Winner, w, O[q], m$)
      $p \leftarrow p + 128$
    if $t = 0$ then
      $minimizerCount[q] \leftarrow m$

3.11. BioseqDB GPU seeding

The BioseqDB minimizer generation uses minimap2's scalar rescan approach ported to GPU. Specifically input is divided into tiles, where each invocation handles one tile using rescan algorithm.

When talking about GPU shaders, Vulkan's workgroup correspond to CUDa thread block, while subgroup is a direct equivalent of CUDA warp.

Let Q be the uploaded query batch and let $\mathcal{T} = (t_0, \ldots, t_{T-1})$ be its flat tile list. The ordered dispatch uses $G = \min(\lceil T/64 \rceil, G_{\max})$ workgroups, where $G_{\max}$ is the device's workgroup limit.

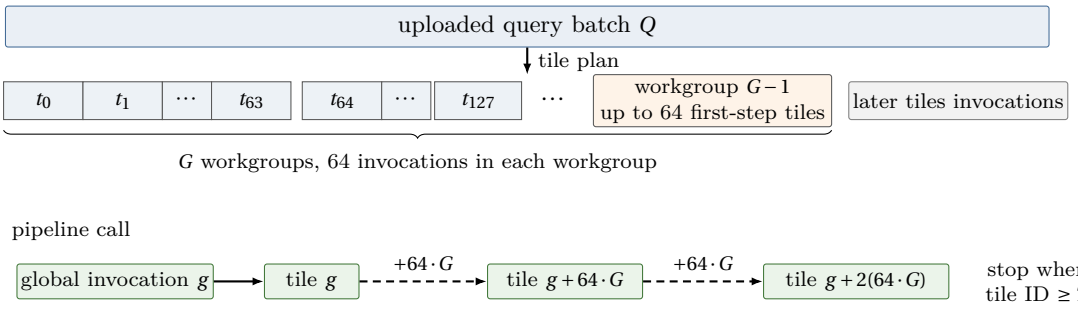


Figure 3.16: Tile assignment in an ordered-sketch dispatch. Each of the G workgroups has 64 invocations. Invocation g processes one tile at a time and advances by $64 \cdot G$ while another tile remains.

When we collect results we need to either: have fixed memory offsets per tile and perform another host pass to extract results or we can do lazy approach where we execute pipeline twice. First we use count mode to caculate where to place results and then do a second pass to actually write them. This makes pipeline do twice as much work, but avoids any additional host pass and we can directly use output results. The limtiations of our work is that we didn't measure the difference between these approaches or between G3SA approach.

The core part is OrderedSketchTile which runs the rescan algorithm in count mode and writes Count. The Count$[j]$ is the number of output records for tile j. PrefixScanBlock reads Count. Each of its B single-invocation workgroups scans at most $S = 256$ entries. It writes local offsets into Offset and one total per block into Block. Two stages (calls) of PrefixScanAdd are used to calculate global offsets.

The positive value A is the pessimistic output capacity planned by the caller using heuristic. HostAndDevice creates an A-record device output before count. If the actual pipeline overflows, then we return error.

Algorithm 8 Vulkan scheduling in BioseqDB

Require: packed query batch Q, k, w, device group limit $G_{\max}$

Require: positive planned output capacity A

Require: policy $P \in \{\text{HostAndDevice}, \text{DeviceOnly}\}$

$\mathcal{T} \leftarrow \text{PlanTiles}(Q, k, w, G_{\max})$
$T \leftarrow |\mathcal{T}|$
return without a GPU dispatch if $T = 0$
$G \leftarrow \min(\lceil T/64 \rceil, G_{\max}), \quad S \leftarrow 256, \quad B \leftarrow \lceil T/S \rceil$
PrepareBoundDeviceInputs$(Q, \mathcal{T})$
allocate device buffers $\underline{\text{Count}}[T]$, $\underline{\text{Offset}}[T]$, and $\underline{\text{Block}}[B]$
if $P = \text{HostAndDevice}$ then $\mathit{Output} \leftarrow \text{HostCreateDeviceOutput}(A)$
BeginCommandBuffer
RecordHostToGPUBarrier
$\mathit{countTask} \leftarrow \text{RecordOrderedSketchTile}(\text{Count}, Q, \mathcal{T}, k, w, G, \mathit{Count})$
AwaitGPU($\mathit{countTask}$)
$\mathit{localTask} \leftarrow \text{RecordPrefixScanBlock}(\mathit{Count}, \mathit{Offset}, \mathit{Block}, T, S, B)$
AwaitGPU($\mathit{localTask}$)
$\mathit{blockTask} \leftarrow \text{RecordPrefixScanAdd}(0, \mathit{Block}, B)$
AwaitGPU($\mathit{blockTask}$)
$\mathit{offsetTask} \leftarrow \text{RecordPrefixScanAdd}(1, \mathit{Offset}, \mathit{Block}, T, S)$
if $P = \text{DeviceOnly}$ then
  SubmitAndHostAwait($\mathit{offsetTask}$)
  $N_{\text{out}} \leftarrow \mathit{Offset}[T-1] + \mathit{Count}[T-1]$
  return overflow if $N_{\text{out}} > A$
  $\mathit{Output} \leftarrow \text{HostCreateDeviceOutput}(N_{\text{out}})$
  BeginCommandBuffer
  RecordHostToGPUBarrier
AwaitGPU($\mathit{offsetTask}$)
$\mathit{emitTask} \leftarrow \text{RecordOrderedSketchTile}(\text{Emit}, Q, \mathcal{T}, k, w, G, \mathit{Offset}, \mathit{Output})$
SubmitAndHostAwait($\mathit{emitTask}$)
if $P = \text{HostAndDevice}$ then
  $N_{\text{out}} \leftarrow \mathit{Offset}[T-1] + \mathit{Count}[T-1]$
  return overflow if $N_{\text{out}} > A$
  $\mathit{HostOutput} \leftarrow \text{CopyMapped}(\mathit{Output}[0 \ldots N_{\text{out}}))$
else
  $\mathit{HostOutput} \leftarrow \varnothing$
return $(\mathit{Output}, \mathit{HostOutput}, N_{\text{out}})$

Algorithm 9 `RecordOrderedSketchTile` call

```
procedure RecordOrderedSketchTile(g, mode)
Require: global invocation ID 0 ≤ g < 64 · G
Require: dispatch-wide mode ∈ {Count, Emit}
Require: bound Q, 𝒯, T, k, w, G, Count
         from Algorithm 8
Require: bound Offset and Output in Emit mode
    j ← g
    while j < T do
        localCount ← 0
        for all record e streamed by RunScalarRescanTile(Q, 𝒯[j], k, w) do
            if mode = Emit then
                Output[Offset[j] + localCount] ← e
            localCount ← localCount + 1
        if mode = Count then
            Count[j] ← localCount
        j ← j + 64 · G
```

3.12. Packed minimizer representation

When running BioseqDB minimizer generation we need to have a way to store the results in a format that can be resused in other stages of the pipeline.

Minimap2's `mm_sketch` emits each minimizer in an `mm128_t`, two `uint64_t` words named `x` and `y`. The C++26 declaration below shows only interpretation of the fields, while minimap2's itself uses binary shifts and masks to set and extract specific values (which made its manual code analysis very challenging).

Listing 3.1: Logical fields packed into a minimap2 minimizer.

```
struct MinimapSketchFields {
  std::uint64_t kmer_hash : 56;   logical k-mer hash
  std::uint64_t span      :  8;   input-base span
  std::uint64_t strand    :  1;   canonical orientation bit
  std::uint64_t last_pos  : 31;   zero-based final-base position
  std::uint64_t rid       : 32;   reference/query-segment ID
};
```

Minimap2's C code construction is done via:

$$x = (hash \ll 8) \mid span, \tag{3.1}$$

$$y = (rid \ll 32) \mid (last_pos \ll 1) \mid strand. \tag{3.2}$$

Homopolymer compression

Normally span equals k (or is $< k$ for first $k-1$ bases). Minimap2 suppors homopolymer compression (HPC) togglable by command line option `-H` which collapses strings of one repeating character into single character occurence. If HPC is enabled, span can have values $0 \leq span \leq 255$. If span is increased beyond 255 (there are 255 same characters in the input sequence next to each other), then minimizer is completely abandoned until new symbol is found.

TODO

Find out if BioseqDB uses HPC.

The `rid` value identifies a reference during indexing and a query segment during query sketching. Note that the ID itself isn't useful during seeding, chaining or alignment, it's only useful at the end when reporting results.

When porting the core of minimap2 algorithms we observed the following facts:

1. The span contains no useful information as HPC is disabled by default and we don't have to support it at this time. For HPC disabled it can only carry information if the k-mer was at the start of the sequence (this can be checked via last_pos field)
2. rid is completely useless during most of the pipeline
3. last_pos which is the end position of the k-mer is only 31 bits which introduces requirement to split long sequences before using minimap2.
4. kmer_hash is wide, but using more bits for hash than binary representation of k-mer is not giving any advantage (for example for DNA4 alpohabet if we encode each symbol in 2 bits then k=15 requires 30 bits for k-mer representation. Hashing 30-bits into 56-bit wide hash doesn't give any advantage as huge part of hashing space won't be used)

BioSeqDB's minimizer producers and collectors use `minimizer_representation<L,K>`. Every form stores the hash, zero-based final position, and strand.

The longest form still uses span set to k, but it's not used for anything other than just padding of fields. Here L is the number of position bits and K is the k-mer length. This L is unrelated to the SIMD lane count used earlier in the chapter.

The type first reserves

$$H = \min(2K, 56)$$

hash bits. Let $R = H + L + 1$, where the last term is the strand bit. The type uses the first width $W \in \{16, 32, 64\}$ for which $R \leq W$. Every bit above the position and strand fields then belongs to the hash, so the stored hash field has $W - L - 1$ bits. If $R > 64$, the record uses two 64-bit words. One word contains a 56-bit hash and an 8-bit span. The other word contains the position, the strand bit, and any unused high bits.

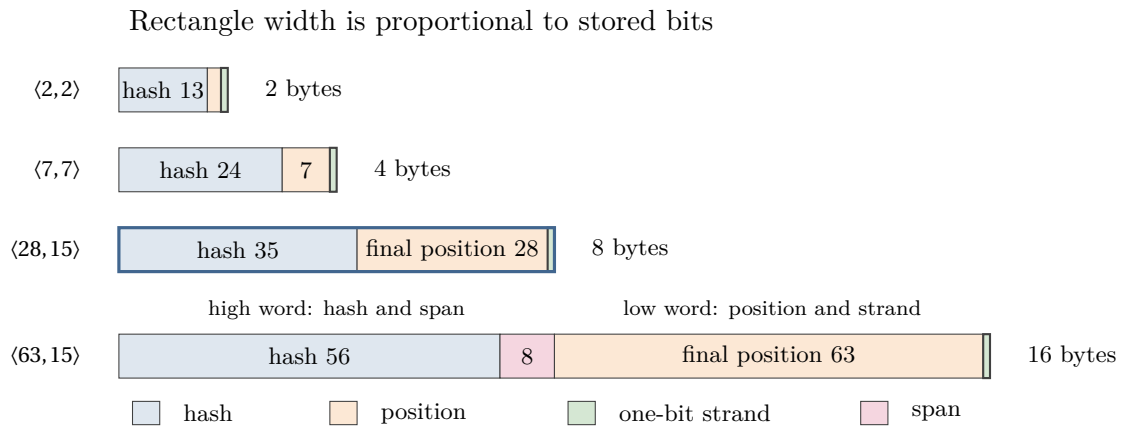


Figure 3.17: Bit fields in four `minimizer_representation<L,K>` instantiations, shown from high bits on the left to low bits on the right. Blue is the hash, orange is the zero-based final position, green is the one-bit strand, and purple is the span in the two-word form. The highlighted $\langle 28,15\rangle$ record is the default 64-bit sketch-data form.

The L-bit field represents final positions from zero through 2^L-1, so it can describe a local sequence span of at most 2^L bases. For $K=15$, values through $L=33$ still fit in 64 bits, while $L=34$ requires the two-word form. The $\langle 28,15\rangle$ form has a 35-bit hash field, but its minimap2 integer hash occupies only 30 bits.

Currently the main parameter that drives used representation width is the number of position bits L (this is compared against kmer_size_buckets array). Theoretically the template can produce a two-byte record, but this bucket isn't suppored at this moment (such small width make's it problematic to adjust as we need to have very short hash, which isn't ideal as shorter k-mers are more prone to collisions or limit the size of sequence to less than 256, which would be very specific optimisation) The $\langle 2,2\rangle$ on the figure is just the demonstrative example to show that template is supported even in very degenerated edge cases (we don't actually use special form for sequences with less than 4 bases as it would make very little sense).

3.13. Configured CPU algorithms

The PostgreSQL setting `bioseq.aligner.minimizer.algorithm` chooses the minimizer implementation. Its current default is `bounded_nthash_simd`. Table 3.4 maps the four CPU values to the algorithm families described earlier. Additionally we allow dispatching Vulkan kernel.

CPU instruction-set selection. Highway selects a CPU target that was compiled into the extension and is supported by the running processor. BioseqDB selects its kernel table once per alignment batch. This chooses an implementation of the requested algorithm, rather than changing the minimizer algorithm itself.

Setting value	Current CPU route
`scalar`	Scalar rescan with the minimap2 integer hash.
`simd_minimap2_reference`	A1 with the minimap2 integer hash for DNA4 and a registered rank-based SIMD route for DNA5.
`bounded_nthash_simd`	A2 with 32-bit ntHash for DNA4 and DNA5. A specialized $k = 15, w = 10$ kernel accompanies the generic runtime kernel.
`two_stack_nthash`	Normally A3 chronological SIMD with the minimap2 integer hash.

Table 3.4: CPU minimizer implementations exposed by the PostgreSQL setting.

Kernel selection

- Bounded ntHash accepts $1 \le k \le 31$ and $1 \le w \le 64$ within a compatible registered representation. Changing k, w does not automatically select scalar code.
- Older measurements can contain an explicitly recorded SIMD-reference-to-scalar fallback. Such a row describes that historical effective path, not the current DNA5 SIMD implementation.
- The Vulkan implementation uses the rescan approach described in Section 3.11; it is not G3SA's kernel.

In the bounded DNA5 kernel, `N` has its own registered ntHash seed and participates in rolling hashes like the other alphabet symbols. This differs from minimap2, which skips k-mers spanning an ambiguous base; DNA5 therefore changes seed semantics as well as packed storage.

3.13.1. Vector width and indexed queries

A wider vector does not by itself predict the speed of a complete query. The bounded kernel maintains eight logical streams: it uses two four-lane vectors on NEON and one eight-lane vector where that shape is available. This applies the instruction-level parallelism idea from Section 3.9, while the cost of each vector operation still depends on the processor. The two-stack resident route also has a dual-stream implementation, but indexed queries require its chronological continuation route. These are separate implementations, not a universal doubling of every SIMD stage.

Reusing a target index removes target sketch construction from a query. It does not remove sketching of the incoming query sequence. The selected minimizer scheme must agree on both sides, as explained in Section 9.7.1. Consequently, nearly equal indexed times can mean that query sketching contributes little to the total; they do not establish that the selected kernels are identical.

Position locality

In case of generating minimizers we can operate on local coordinates meaning that our minimizer representation can just track position relative to the sequence fragment. Sequence fragmentation in storage is described in later chapters. However for next steps of processing (chaining) we use global coordinates. It would be possible to take advantage of that locality for example by building index that stores only minimizers for some shorter sequence fragments (this is also described in later chapters), but for simplicity even in that case we just map local coordinates afterwards to global ones at all times. This have the cost of anchors being wider in memory (as we need to use int64 for global coordinates).

Chapter 4

Chaining seed anchors

4.1. From local seed hits to candidate mappings

When minimizers are generated and we take the intersection (by hash value) of the minimizer sets between the query and the reference, we get a set of seed hits which is encoded as anchors.

4.2. Anchor geometry and ordering

A seed hit is represented by an anchor

$$a_i = (r_i, q_i, \sigma_i, \ell_i, g_i, \phi_i),$$

where r_i and q_i are inclusive seed-end coordinates on the reference and query, σ_i is the strand, ℓ_i is the query span, g_i identifies a query segment, and ϕ_i stores additional ordering flags.

Below you can see the physical representation of the anchor in memory in minimap2 and in our implementation. We could potentially use adaptive representation similar to minimizers, but anchors are fewer than minimizers, so the benefit would be limited for additional complexity of C++ templates.

```
typedef struct {
  uint32_t n;                    Size of cr array (number of matches in the reference sequence)
  uint32_t q_pos;                Query position
  uint32_t q_span       : 31;    Query minimizer span in symbols
  uint32_t flt          : 1;     If the minimizer is filtered out (very frequently occuring in the refe
  uint32_t seg_id       : 31;    Query segment identifier
  uint32_t is_tandem    : 1;     Tandem repeat flag
  const uint64_t *cr;            Multiple reference coordinates per anchor
} mm_seed_t;
```

```
struct anchor {
  std::uint64_t ref_position;    Absolute reference coordinate
  std::uint64_t query_position;  Absolute query coordinate
  std::uint32_t is_reverse;      Reverse-strand flag encoded as zero or one
  std::uint8_t query_span;       Query minimizer span in symbols
  std::uint8_t flags;            Seed flags propagated by collection
  std::uint16_t segment_id;      Query segment identifier
};
```

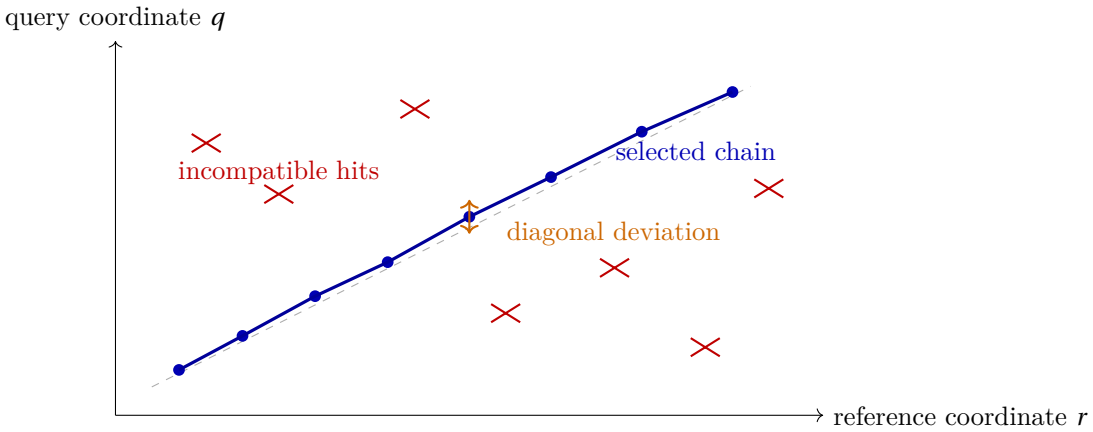


Figure 4.1: Chaining selects growing, approximately collinear path so we can get closer to the final result. Base-level alignment (later stage of the pipeline) will later resolve gaps between anchors.

TODO

Improve anchor API

Figure 4.1 shows viasual interpretation. Valid anchors that can be used for chaining lie below and to the left of the current anchor. A chain can move between nearby diagonals to represent an insertion or deletion, but large diagonal changes receive a penalty or are not considered at all.

Why we look for anchors that are colinear?

In the perfect scenario without gaps $r_i - q_i$ is constant. An increasing diagonal coordinate indicates that in reference we matched more symbols than in the query (meaning insertion/deletion has occured). Chaining needs to specify costs for abandoning the main diagonal too much (intuitevely we don't want the case in which we introduce very large gap in the reference sequence just to match one or two symbols in the query, because it's improbable that some very large fragment was replaced during evolutionary processes by just a few bases)

As we would like to have some well defined order of anchors, we sort them by strand and reference coordinate first, then by the additional metadata:

$$\begin{aligned} K_r(a_i) &= (\sigma_i \ll 63) \mid \big(r_i \,\&\, (2^{63}-1)\big), \\ K_q(a_i) &= (g_i \ll 48) \mid (\phi_i \ll 40) \mid (\ell_i \ll 32) \mid (q_i \bmod 2^{32}). \end{aligned} \tag{4.1}$$

Comparison is lexicographic in K_r, K_q, and the full q_i. The hashing doesn't need full 64 bits for query position i.e if query coordinates q_i exceed 2^{32} we still have correct order as the leftmostpart is still the fragment number g_i (fragments won't be larger than 2^{32}). Fragmentation of sequences is discussed later in the storage chapter.

The ordering is required, because the chaining wants to process anchors in sequential order (increasing query positions), while we build chains of anchors using reference positions.

4.3. Intuitive interpretation

We need to first understand how intuitively human would align sequences. First we choose a random fragment of query and use text search in our editor to find that exact fragment. The longer fragment is better, because it's less likely that we found it just because of random subsitutions ocurring. At first we won't find it, but we repeatadly delete chacters from the fragment start or end until match is found. Each match corresponds to possible location in the target sequence. If we repat the process enough for other fragments of query we will establih the most probable locations of query in the target sequence (by seeing groups of matches). Throughout this chapter we will never gave any reason to treat query and target asymetrically as scoring functions don't distinguish between both inputs. The distinction was rather coming from their role in the alignment flow (the larger set of sequences can be called the set of references or targets, while the smaller one could be called queries).

Longest increasing sequence

If the reference coordinates are increasing, then naive approach would be finding LIS (longest increasing subsequence) of query coordinates. You can think of the chaining as "improving" this simplistic approach by changing query coordinates to more complex scoring function which can handle gaps and other complex cases.

The sorting mentioned in section 4.2 is required, because what we would like to intuitively do is check where the first reference fragment can be placed in query sequence and when we move to the next anchor (increasing reference positon), the position of query should increase too (logically if the middle of query matches somewhere, then we should see the end of it, not the beggining). This introduces some emergent assymetry: it will be very likely to see one anchor in query matched with multiple anchors in referece (query can occur multiple times in reference), however what happens if one reference fragment is matched to multiple query fragments? This situation means that is hard to distinguish which part of the query is best to match:

- Reference $\leftarrow XYZVW$, Query $\leftarrow YZV$ - it's immediately obvious that for V in the reference we found the end of query
- Reference $\leftarrow XYZVW$, Query $\leftarrow VXVXVXV$ - we don't know what part (is this start or the end of query, or the middle part?) we should consider without looking at neighbours of V.

Emergent assymetry

In the following chapters I will present new optimisations that are not discussed in the field of computational biology. The emergent assymetry will play important role when talking about the experimental results in separate chapter 4.9.

4.4. Chaining components

We defined the anchors that are input to chaining pipeline stage, however chaining itself isn't a monolith. It can be viewed as composition of several steps.

Chaining pipeline

Although chaining can be decomposed into several steps that are dispatched to differrent functions, in BioseqDB this is an internal implementation detail and we don't allow users to alter for example which backtracking backend is used (although we could expose parts of chaining as composable components in the future)

Forward chaining (which is the core component) produces the score array f and predecessor array p. Second most important step is backtracking (as typically in case of DP algorithms which fill some state matrices) reads the sorted anchors, f, and p. The backtracking writes theflat output anchor array supplied by the caller, denoted by A (A can be viewed as type `anchor A`

).

The chaining can be fully decomposed into the following steps:

1. Forward chaining reads the sorted anchors and writes the temporary f and p arrays.
2. Backtracking reads these inputs and writes the accepted chain records plus full anchor copies in A.
3. Region generation converts each chain record into one alignment region while retaining A.
4. Region selection filters the regions while retaining A.
5. Base-level alignment reads the selected regions, their slices of A, the query and target sequences, and the alignment parameters. It fills the base-level fields in the regions supplying exact start/end coordinates, match qualities etc.

BioseqDB represents every chain record with the following structure:

```
struct chain {
  int32_t score        = 0;   Accepted-chain score
  int32_t anchor_count = 0;   Anchors in the chain
  int32_t anchor_start = 0;   Start of the chain slice in A
};
```

Where `anchor_start` is the index of the first anchor in the chain in the array A.

For a non-empty record c, its anchors occupy consecutive positions in forward order:

$$A[c.\texttt{anchor_start}], \ldots, A[c.\texttt{anchor_start} + c.\texttt{anchor_count} - 1].$$

The array A is constructed using f and p arrays. For every anchor i, the two stored values mean:

f_i the score of the best chain that the selected forward algorithm found with anchor i at its end.

p_i the index of the anchor immediately before i in that chain. The value is -1 when the best chain found for i starts at i

For example,

$$p_7 = 5, \qquad p_5 = 2, \qquad p_2 = -1 \quad \Longrightarrow \quad 7 \to 5 \to 2 \to -1.$$

4.5. Chaining algorithms

Similar to the minimizer chapter here we will discuss different approaches to chaining. The algorithm itself is based on dynamic programming scoring, but the details differ.

4.5.1. Shared pair score

Let's say we have two anchors with coordinates q_j, q_i and r_j, r_i on the query and reference. Let's assume the minimizer span is always k. We now can say that after incorporating (r_j, r_i) anchor we will cover $q_i - q_j$ symbols on the query and $r_i - r_j$ symbols on the reference, however those two jumps can be interpreted as taking some symbols from query and assuming they form replacement and the rest forms a gap. For example let's say $q_i - q_j > r_i - r_j$. This means that we match more symbols in query than in reference, so indicates some deletion in the reference. So we can say $r_i - r_j$ symbols are replaced (or stay the same) while the leftover $(q_i - q_j) - (r_i - r_j)$ symbols form this deletion. This thinking is symmetrical so we just only need to use minimum and absolute value to make it more generic.

Now we introduced $g(j,i)$ (the advance shared by the reference and query) and $\delta(j,i)$ (the remaining advance in whichever sequence moves farther). As k-mers represent hashed fragment of length k we can assume that in our match we have exactly $\min\{q_i - q_j, r_i - r_j, k\}$ new symbols that matched. Hence we now define our first primitive score as $\min\{q_i - q_j, r_i - r_j, k\} = \min\{g(j,i), k\}$.

This scoring has some problems with the most important one being that it doesn't penalize the case when we have a large gap in the reference sequence just to match one or two symbols in the query. This is improbable that some very large fragment was replaced during evolutionary processes by just a few bases. For that reason we will use function $\lfloor \text{Penalty}(\delta, g) \rfloor$. The penalty function should satisfy the following properties:

- There's no penalty for doing nothing at all: $\text{Penalty}(0,0) = 0$
- For $g \leq k \wedge \delta = 0$ there's no gap so there's shouldn't be any penalty: $\text{Penalty}(0, g) = 0$
- Penalty should be monotonically increasing with respect to δ: $\text{Penalty}(\delta, g) \leq \text{Penalty}(\delta + 1, g)$
- Penalty should be increasing slower than linearly with respect to g: $\text{Penalty}(\delta, g+1) < \text{Penalty}(\delta, g) + 1$. This guarantees that increasing g without touching δ benefits overall score (we prefer replacement region over gap region)
- For small g and δ our penalty function should prefer increase in g rather than δ. This is equivalent of saying that we want to stay close to the diagonal and reduce the gap size introduced: $\text{Penalty}(\delta + 1, g) \geq \text{Penalty}(\delta, g + 1)$
- For large g and δ our penalty function should be asymptotically the same for both δ and g. This is equivalent of saying that if we encounter very large gaps, then it doesn't matter if we increase the matching region between two fragments or increase the gap as both fragments are very dissimilar: $O_\delta(\text{Penalty}(\delta, g = \text{const})) = O_g(\text{Penalty}(\delta = \text{const}, g))$

Symmetry

Using g and δ as only parameters for penalty function guarantees that scoring is symmetric. Symmetry makes sure that gap costs are the same for both query and reference.

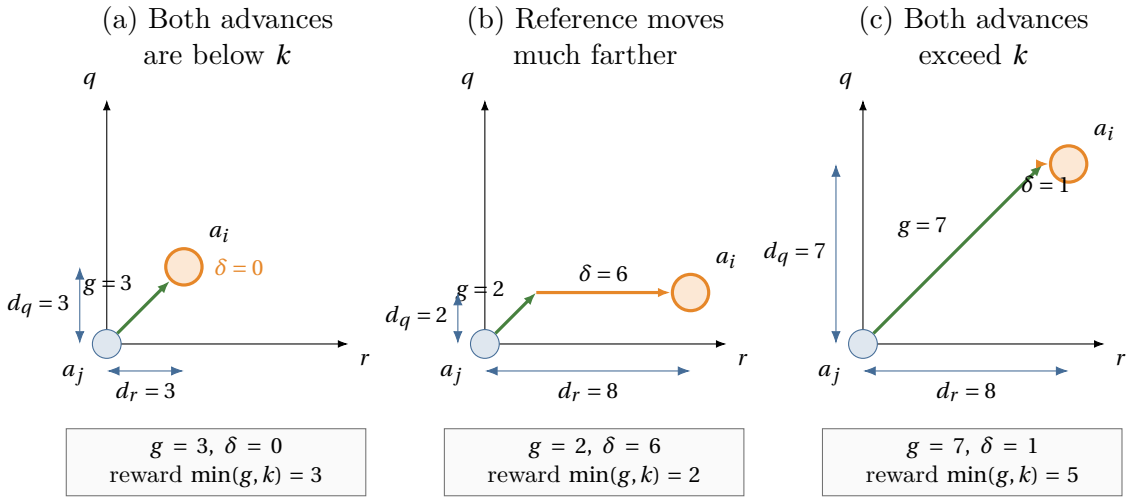


Figure 4.2: Three anchor displacements for $k = 5$. The blue brackets are the actual reference and query advances. The green diagonal step advances both coordinates by $g = \min(d_r, d_q)$. The orange horizontal step is the remaining $\delta = |d_r - d_q|$. It would be vertical when $d_q > d_r$. Panel (c) also shows that the reward stops at k, even though $g > k$.

Our final score is then something of the following structure:

$$\begin{aligned} \text{Score}(j,i) &= \min\{g(j,i), k\} - \lfloor \text{Penalty}(\delta(j,i), g(j,i)) \rfloor \\ &= \min\{g(j,i), k\} - \lfloor \alpha * g(j,i) + \beta * \delta(j,i) + \gamma * f(\delta(j,i)) \rfloor \\ &\qquad 0 \le \alpha, \beta, \gamma \le 1 \end{aligned} \tag{4.2}$$

The minimap2 scoring definition E looks exactly as what we have just specified:

$$\begin{aligned} d_q(j,i) &= q_i - q_j, & d_r(j,i) &= r_i - r_j, \\ \delta(j,i) &= |d_r(j,i) - d_q(j,i)|, & g(j,i) &= \min\{d_r(j,i), d_q(j,i)\}. \end{aligned} \tag{4.3}$$

$$E(j,i) = \min\{\ell_j, g(j,i)\} - P(j,i), \tag{4.4}$$

where

$$P(j,i) = \begin{cases} 0, & \delta(j,i) = 0 \ \wedge \ g(j,i) \le \ell_j, \\ \left\lfloor \lambda_g \delta(j,i) + \lambda_s g(j,i) + \frac{1}{2} \text{mglog}_2(\delta(j,i)+1) \right\rfloor, & \text{otherwise.} \end{cases} \tag{4.5}$$

The function mglog_2 is minimap2's polynomial approximation of $\log_2$. So far in the abstract definition of scoring we didn't mention diagonal, but nonnegative coefficient λ_g can be viewed as diagonal-gap penalty (further away from diagonal we penalize more), where the nonnegative coefficient λ_s is the penalty for distance. The value ℓ_j is the query span of predecessor j. (In our example above we used k for all k-mers sizes, but as mentioned before minimap2 with specific settings can compress regions of same repeating symbols so we just replace k with kmer span ℓ_j. All the logic stays the same)

Minimap2 chaining link score
comput_sc 113

Score units

In this definition we view scoring function as expressed in number of matched symbols (not some arbitrary large numbers), so the coefficients must be set accordingly so the values are scaled correctly. In the dynamic programming formulation we then can take maximum of E value and ℓ_i to make sure that in the worst case scenario we assume that we at least matched the length of the minimizer. There are no formal limitations on this normalization, but it makes reasoning about the scores easier.

It's worth noting that defined scoring is also symmetrical in terms of coordinates i.e the score doesn't take r_i, q_i positions as parameters so reversing coordinate systems doesn't change the outcome. However, in dynamic programming we often want to use the locally close predecessor as the best one.

We see that scoring is not easy to work with as it depends on both anchors. If we used a linear gap scoring similar to what we discussed in alignment foundations chapter 2.1 we would get a score that is not dependent on both anchors. Note that the following is true due to trivial relationship between arithmetic average, distance and minimum:

$$\begin{aligned} \phi(j,i) &= \frac{d_q(j,i) + d_r(j,i)}{2} \\ \delta(j,i) + 2 * g(j,i) &= \phi(j,i) \end{aligned} \tag{4.6}$$

So we can simplify our scoring function to be only dependent on $\phi(j,i)$ (I replaced $min(\ell_j, g(j,i))$ with just ℓ_j and replaced penalty function with a simple linear function dependent on $\phi(j,i)$):

$$E(j,i) = \ell_j - \lambda_\phi * \phi(j,i) \tag{4.7}$$

Now if we assume that every anchor has constant span (as most of them will have length k), then we can define:

$$\begin{aligned} f_j &= \max_{i<j}\{f_i + E(j,i)\}, \\ \text{GlobalScore}(i) &= f_i + \lambda_\phi * (q_i + r_i) \end{aligned} \tag{4.8}$$

We can observe that:

$$\begin{aligned} f_i + E(j,i) > f_k + E(k,i) &\iff \\ f_i + \ell_j - \lambda_\phi * \phi(j,i) > f_k + \ell_k - \lambda_\phi * \phi(k,i) &\iff \\ \text{GlobalScore}(i) + \ell_j > \text{GlobalScore}(k) + \ell_k &\iff \\ \text{GlobalScore}(i) > \text{GlobalScore}(k) \end{aligned} \tag{4.9}$$

This means that we can compute GlobalScore(i) independently for each anchor and use it to calculate global score. This is what was done in minimap2 as described in minigraph paper Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph." The problem is that the new linear gap scoring is very simplistic and does not exhibit some of the properties I discussed earlier.

4.6. B1: minimap2 dynamic programming

As our scoring model $E(i,j)$ depends only on two anchors we can define recursive equations for dynamic programming:

$$f_i = \max\left(\ell_i, \max_{j \in \mathscr{C}_i}\{f_j + E(j,i)\}\right), \qquad p_i = \arg\max_{j \in \mathscr{C}_i}\{f_j + E(j,i)\}. \tag{4.10}$$

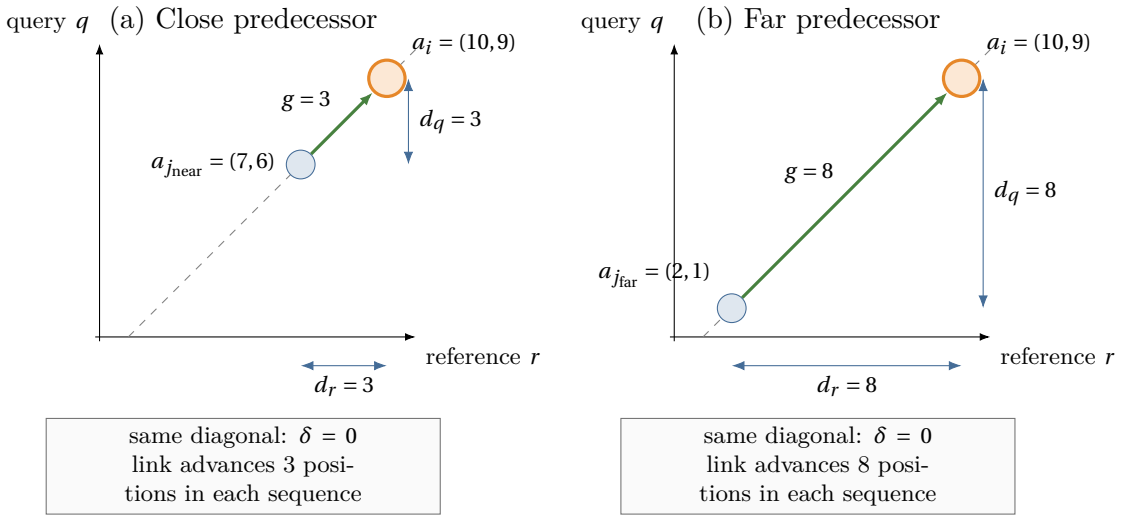


Figure 4.3: A close and a far predecessor for the same current anchor $a_i = (10,9)$. Both candidates lie on the diagonal $r - q = 1$, so both have $\delta = 0$. Their link lengths are nevertheless different: the close link has $g = 3$, whereas the far link has $g = 8$. Thus zero diagonal deviation does not mean zero or equal distance.

Where $\mathscr{C}_i$ is the set of valid predecessors (selected by being close to diagonal - this is form of specifying cutoff condition so we don't need to check all possible predecessors all the time). Maximum function is used here to make sure that in worst case if our definition of $E(i,j)$ yield very low score we can say that we matched at least length of the minimizer span.

If no predecessor improves the base value of score ℓ_i, then $p_i = -1$.

We will call iteration over $\mathscr{C}_i$ a scan. Minimap2 does this scan by visiting anchors in order from newest to oldest.

After sorting, minimap2 numbers the anchors $0, 1, \ldots, n-1$. When the DP algorithm reaches anchor i, the scores $f_0, \ldots, f_{i-1}$ and predecessor indices $p_0, \ldots, p_{i-1}$ have already been calculated. It starts a new chain at i with

$$f_i = \ell_i \qquad \text{and} \qquad p_i = -1,$$

Minimap2 tests earlier anchors in the order $j = i-1, i-2, \ldots$ (larger index is tested first). For every pair that passes the distance checks, it calculates

$$c_j = f_j + E(j,i).$$

If $c_j > f_i$, anchor j becomes the new predecessor: $f_i \leftarrow c_j$ and $p_i \leftarrow j$ (strict equality is important here so we avoid using older anchors with the same score)

Trying every earlier anchor would make this dynamic program expensive. Minimap2 applies the following strategies to limit the work performed:

- `max_iter` limits how many previous anchors we scan: this is the number of highest-numbered array positions before i.
- `max_skip` is the limit on skip counter s. It starts at zero for every new i and is increased by one on marked anchor that fails to improve the score and decreased by one on marked anchor that improves the score. If skip counter exceeds limit we finish scan loop early and do not scan other previous anchors.

- `m` is an extra candidate that is tested after the scan loop. It is the anchor with the highest score among all previously tested anchors (this is a measure to prevent skipping good chains due to too aggressive skip counter limit)
- `CanScore(j,i)` checks the anchor distances to make sure the anchor lies within the band (close to diagonal). It uses D_x and D_y for limits on reference and query distances respectively and B for distance from diagonal.

Minimap2 contains separate rules for multi-segment and cDNA input. The key algorithm stays the same and differences are very subtle:

- There are differences in handling scoring case of $g_i = g_j$
- Multi-segment non-cDNA input adds special requirement $d_r \le D_y$
- cDNA input does not force $D_y \ge B$
- There are differences in penatly factors for scoring function $E(j,i)$

In description below I assume we talk about default case of one-segment, non-cDNA input.

Different kinds of input

Multi-segment input is in case of Minimap2 described as a situation in which single query maps to different fragments of the reference section. I do not support this use case in BioseqDB out of the box at this time. This situation is also known as split-read or supplementary alignment and normally occurs due to due to structural variations, translocations, or chimeric artifacts in the genome. cDNA in this case describes the same situation, but has slightly different scoring parameters. Even though that can be interesting, we don't want to describe these modes here since those are very specialized heuristics

4.6.1. Skipping with counters and marked anchors

As discussed above minimap2 scores only valid pairs of anchors:

$$0 < d_r(j,i) \le D_x, \qquad 0 < d_q(j,i) \le \min(D_x, D_y), \qquad \delta(j,i) \le B, \tag{4.11}$$

where $B = \texttt{bw}$ where all parameters like `bw` are minimap2's CLI arguments.

$$D_x = \max(\texttt{max_dist_x}, B), \qquad D_y = \max(\texttt{max_dist_y}, B).$$

After a candidate j passes this checks, we calculate $E(i,j)$ score and then update p_j if we see better score. We want to that p_j was updated in scan for i. This can be understood as building chains in parallel where we want to save information about predecessor for each anchor:

$$t_{p_j} \leftarrow i \qquad \text{when } p_j \ge 0.$$

If some time later in other scan we see that $t_{p_j} = i$ then we know that p_j was updated in scan for i (we already tested candidate with a larger anchor). If marked anchor x fails to improve f_i, then s increases by one. A failed unmarked anchor leaves s unchanged. An improvement subtracts one from a positive s. It does not necessarily reset the counter to zero. The loop stops only when $s > \texttt{max_skip}$

Interpretation

This marking with skip counter can be intuitevely understood as a situation in which we try to improve the best chain by adding new anchor. If repeatedly this chain fails to improve the score, we don't want to waste more time on it.

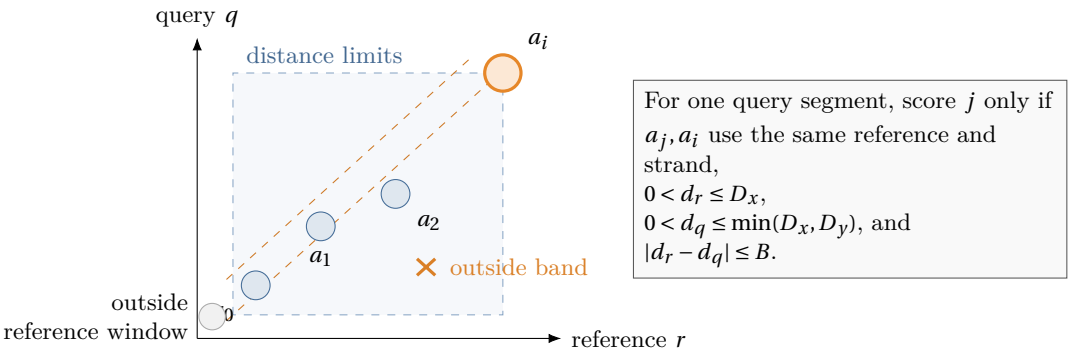
We don't perform this marking when this is candidate thatraises s above `max_skip` (in that case we stop the scan loop early).

Minimap2 keeps the loop index in e. If the skip test stopped the loop while it was processing j, then $e = j$. e is used to check if we can score anchor m (if $m < e$ then we can).

m is additional index saved by minimap2 on the side, which we call the saved high-score anchor. The saved value is an anchor index whose previously calculated f_m is large. If m is absent or has left the reference-distance window (is too far), minimap2 first rebuilds it by finding the largest f_j in the current `max_iter` set.

This saved anchor can then remain saved after $m < i - \texttt{max_iter}$. Each time we check $e < m$ (see if we already scored this anchor) and if not we calculate $E(m, i)$ and do update checks similar to other anchors. m is introduced to make sure to take into consideration anchor that could be potentially a little bit further away. Minimap2 doesn't use priority queue or list here, because chaining accuracy is already good enough and this approach is much simpler.

(a) Which earlier anchors can be scored



(b) One complete backward scan with concrete counter values

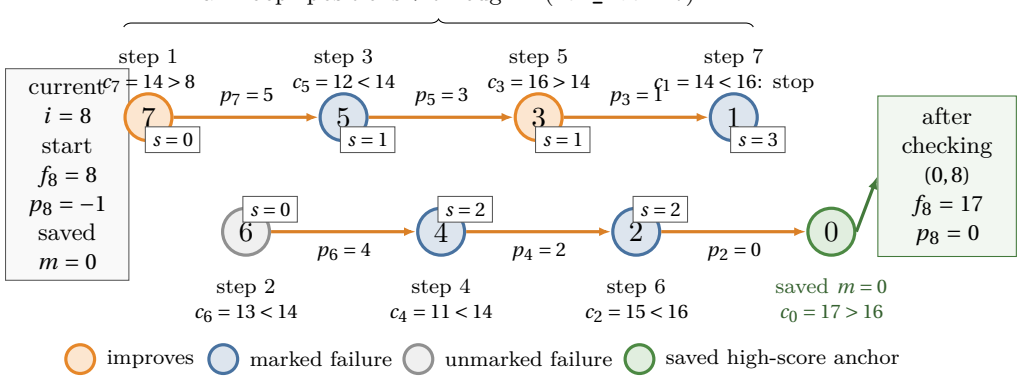


Figure 4.4: (a) The reference and query limits and diagonal band decide whether a pair can be scored or not. (b) If distance checks pass, then we perform scans in a loop. The picture shows one pass of the scan for the current anchor $i = 8$. The numbers inside the circles are anchor indices. The orange arrow from j to p_j shows the predecessor previously saved for anchor j. After j is scored without stopping, the arrow's destination receives the current mark $i = 8$. The numbered steps visit anchors $7, 6, \ldots, 1$. The small s boxes show the single skip counter for current anchor 8. Step 2 doesn't affect s as this anchor doesn't have a predecessor in this scan pass (it's unmarked). Step 5 improves the score and changes $s = 2$ to $s = 1$. Anchor 0 is already outside the main loop because `max_iter` $= 7$, so we don't scan it, but we check it later with $m = 0$. Step 7 stops with $e = 1$, so saved $m = 0$ satisfies $m < e$ and is checked once. The score of $m = 0$ equal to 17 wins, but in some other scan other anchor could have a better score.

Algorithm 10 Minimap2's ordered predecessor scan for one query segment

Require: sorted anchors $a_0, \ldots, a_{n-1}$; ordinary non-cDNA mode; configured distance limits, $B = \texttt{bw}$, `max_iter`, and `max_skip`
Ensure: chain scores f_i and predecessor indices p_i

$D_x \leftarrow \max(\texttt{max_dist_x}, B)$
$D_y \leftarrow \max(\texttt{max_dist_y}, B)$
function CanScore(j, i)
    $d_q \leftarrow q_i - q_j, \quad d_r \leftarrow r_i - r_j$
    if a_j, a_i use different references or strands then
        return false
    if $d_r \le 0$ or $d_r > D_x$ then
        return false
    if $d_q \le 0$ or $d_q > D_x$ or $d_q > D_y$ then
        return false
    if $|d_r - d_q| > B$ then
        return false
    return true
$t_j \leftarrow 0$ for every j; $m \leftarrow -1$; $st \leftarrow 0$ m is one saved anchor index
for $i \leftarrow 0$ to $n-1$ do
    while $st < i$ and (a_{st} has a different reference or strand from a_i, or $r_i - r_{st} > D_x$) do
        $st \leftarrow st + 1$
    $st \leftarrow \max(st, i - \texttt{max_iter})$ hard lower bound for the main scan
    $f_i \leftarrow \ell_i, \quad p_i \leftarrow -1, \quad s \leftarrow 0, \quad e \leftarrow st - 1$
    for $j \leftarrow i-1$ down to st do
        if CanScore(j, i) then
            $c \leftarrow f_j + E(j, i)$
            if $c > f_i$ then
                $f_i \leftarrow c, \quad p_i \leftarrow j$
                if $s > 0$ then
                    $s \leftarrow s - 1$
            else if $t_j = i$ then
                $s \leftarrow s + 1$
                if $s > \texttt{max_skip}$ then
                    $e \leftarrow j$
                    break
            if $p_j \ge 0$ then
                $t_{p_j} \leftarrow i$ mark one older anchor for this scan
    if $m < 0$, or a_m has a different reference or strand from a_i, or $r_i - r_m > D_x$ then
        $m \leftarrow -1$
        for $j \leftarrow i-1$ down to st do
            if $m < 0$ or $f_j > f_m$ then
                $m \leftarrow j$
    if $m \ge 0$ and $m < e$ then
        if CanScore(m, i) then
            $c \leftarrow f_m + E(m, i)$
            if $c > f_i$ then
                $f_i \leftarrow c, \quad p_i \leftarrow m$
    if $m < 0$, or (a_m, a_i use the same reference and strand, $r_i - r_m \le D_x$, and $f_i > f_m$) then
        $m \leftarrow i$

Algorithm details

We check if a_{st} has a different reference or strand from a_i, because anchors in minimap2 can originate from different refences. In BioseqDB this won't happen, but we still need to check for strand direction. Also please note that formally we use different values for marking $t_{p_j} \leftarrow i$, but it's main goal is just to detect visited anchors for the same scan pass (like in binary map, but assigning i just means that we don't have to clear it.)

4.7. BioseqDB SIMD DP: exact and bounded batch

For one target anchor a_i, scalar B1 tests possible predecessors one at a time. During the sacan it starts at a_{i-1} and walks towards the lower end of the search window. SIMD implementations both implement the same outer loop, but the scan itself is rewritten: We place each predecessor candidate in batches of size L (each batch is one SIMD vector) - one vector call can calculate as many as L values of $E(j,i)$. We have two kinds of SIMD algorithm:

1. Exact SIMD B1 called **`exact_visited`** It it the most naive implementation: it calculates up to L pair scores together in a SIMD vector, but F, P, and s are still single scalars as in original B1.

2. Bounded batch SIMD B1 called **`bounded_batch_approximate`**. Algorithm removes the mark array completely, while scalar skip counter in now counting batches. Lane λ keeps its own temporary best score H_λ and predecessor P_λ. A shared counter u records how many batches in a row failed to improve any lane (the algorithm stops when u reaches its batch limit U). We take the largest score across the lane winners and test the saved anchor m once.

We will discuss bounded batch SIMD B1 first, as it is more interesting one.

4.7.1. Candidates in SIMD vectors

The candidates are placed in batches. So now in loops we use $b = 0$ instead of j where 0 is the first batch, $b = 1$ is the next older batch, and so on. Within one batch, $\lambda \in \{0, \ldots, L-1\}$ is the SIMD lane number. The candidate placed in that lane is $j_{b,\lambda}$.

First we introduce the most boring thing - coordinates mapping:

$$\begin{aligned} \mathrm{end}_b &= i - bL, & \mathrm{begin}_b &= \max(st_i, \mathrm{end}_b - L), \\ a_b &= \mathrm{end}_b - \mathrm{begin}_b, & j_{b,\lambda} &= \mathrm{begin}_b + \lambda. \end{aligned} \tag{4.12}$$

Here end_b is the position immediately after the batch, begin_b is its first candidate, and a_b is its number of active lanes. As candidates are split into batches, the last one can be partial (so a_b will be used to mask out the inactive lanes).

For lane λ, the pair scorer first calculates $\epsilon_{b,\lambda} = E(j_{b,\lambda}, i)$. The code then loads the earlier chain score $f_{j_{b,\lambda}}$ and adds it to $\epsilon_{b,\lambda}$. The complete proposed chain score is $C_{b,\lambda}$, and $\mathbf{C}_b$ is the vector containing one such total per lane. A rejected candidate or an inactive lane receives $-\infty$.

Figure 4.5 depicts that now instead of single candidate being checked, we check batches:

- $C_{b,\lambda}$ holds total score for the candidates which corresponds to c in original B1 algorithm

- $\mathbf{j}_b$ holds position of the candidate which corresponds to j in original B1 algorithm
- $\mathbf{H}^{(b)}$ holds the best score for the batch which corresponds to f_i in original B1 algorithm (note that now we hold vector instead of scalar, so at the end we need to select minimum value)
- $\mathbf{P}^{(b)}$ is shown later on the next figure and holds the index for $\mathbf{H}^{(b)}$

Note that only $\mathbf{H}^{(b)}$ and $\mathbf{P}^{(b)}$ are presisted between scans. The superscript (-1) over H means that we got this value from previous scan.

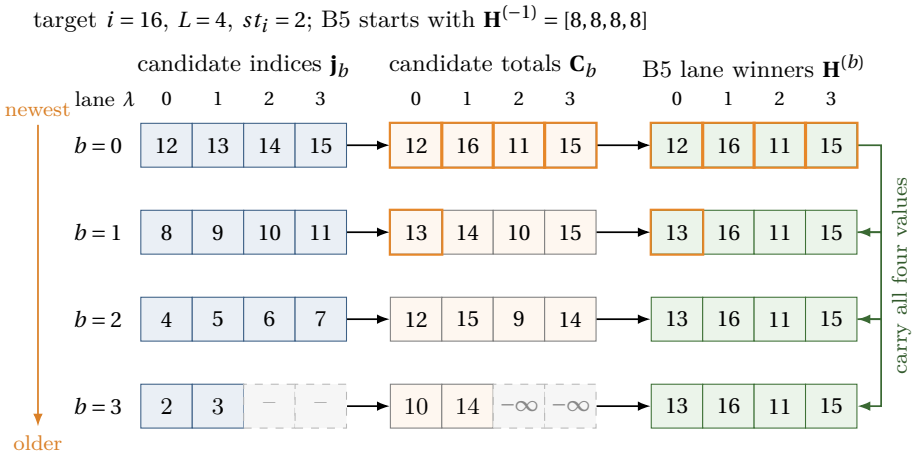


Figure 4.5: How predecessor candidates are placed in four SIMD lanes. The lane number λ selects the corresponding column in all three vectors. Blue boxes are candidate indices, orange boxes are calculated scores, and green boxes are best scores (the scores are improved vertically). When a lane improves its new maximum has an orange outline. Batch 3 is partial (we exhausted all candidates), so its two dashed lanes are inactive and their totals are $-\infty$.

We operate on batches so we need to adjust our definition of skip counter. Let $S =$ `max_chain_skip`:

$$U = \max\left\{1, \left\lfloor \frac{S}{L} \right\rfloor + 1\right\} \tag{4.13}$$

As a helper we introduce $I_{b,\lambda}$ which is just binary mask checking lane-wise which C value is better than H, so we update skip counter according to the following rules:

$$u_b = \begin{cases} 0, & \text{if any } I_{b,\lambda} \text{ is true,} \\ u_{b-1} + 1, & \text{otherwise.} \end{cases} \tag{4.14}$$

When $u_b \geq U$ the algorithm stops. In original B1 we had guarantee that we will stop after S skipped candidates, but here we stop after U such that: $S < LU \leq S + L$. This doesn't however bother us at all, as this is only very minor implementation detail user doesn't need to see.

The figure below depcits two kinds of SIMD B1 algorithms. The first one (a) wasn't discussed in detail as it only puts the candidates in a single SIMD vector to speed up only few operations like finding the maximum, where the rest of the code stays unchanged. The second parts of figure (b) is more interesting as it shows the approach we discussed with vector batching.

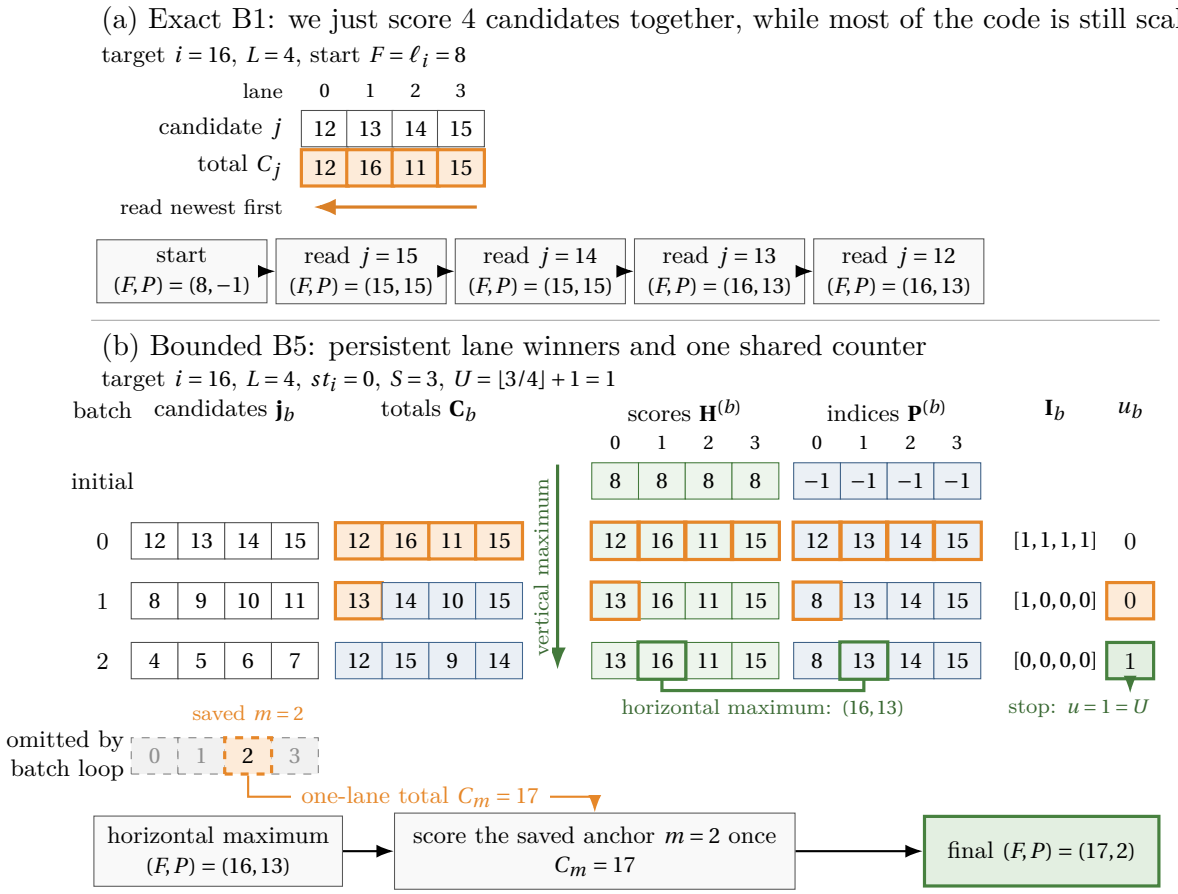


Figure 4.6: Two strategies for calculating SIMD B1 (a) Exact B1 calculates four totals together, then applies state changes in the scalar order whenever some lane can improve. (b) Bounded SIMD is more clever - it starts with $\mathbf{H}^{(-1)} = [8,8,8,8]$ and $\mathbf{P}^{(-1)} = [-1,-1,-1,-1]$. Each row shows both vectors after its batch. Green boxes hold scores and blue boxes hold their predecessor indices. An orange border around C marks a new improvement (maximum). Orange borders around H and P show the stored pair that changed. All four lanes change in batch 0. In batch 1, only lane zero changes, to $(H_0, P_0) = (13, 8)$. Batch 2 changes neither vector, so $u = 1 = U$ and candidates 0 through 3 (bottom gray area) are omitted. The downward arrow shows that each lane keeps its best (H, P) pair while the scan moves through older batches. The horizontal maximum then compares the four lane scores. Lane 1 supplies the pair selected by that horizontal maximum, $(H_1^{(2)}, P_1^{(2)}) = (16, 13)$, marked by the green bracket and outlines. The orange box marks saved candidate $m = 2$, whose separate one-lane total $C_m = 17$ replaces the result with $(17, 2)$.

After the last scored batch, the algorithm takes horizontal maximum of a vector once more:

$$f_i = \max_{\lambda} H_\lambda, \qquad p_i = \max\{P_\lambda : H_\lambda = f_i\}. \tag{4.15}$$

The algorithm is described in detail in Algorithm 11.

Algorithm 11 Bounded SIMD B1

Require: target i, window $[st_i, i)$, SIMD width L, $S = \texttt{max_chain_skip} > 0$, $\texttt{max_chain_iter} > 0$, saved index m

Ensure: f_i, p_i

$U \leftarrow \max\{1, \lfloor S/L \rfloor + 1\}$ whole-batch stopping limit

$\mathbf{H} \leftarrow [\ell_i]^L, \quad \mathbf{P} \leftarrow [-1]^L, \quad u \leftarrow 0$ one temporary winner per lane

$o \leftarrow st_i$ no omitted interval yet: $[st_i, o)$ is empty

for $b \leftarrow 0, 1, \ldots$ while $\mathbf{end}_b > st_i$ do

  calculate $\mathbf{begin}_b, \mathbf{end}_b, a_b, \mathbf{j}_b$ using Equation 4.12

  $\mathbf{A} \leftarrow [\lambda < a_b]_{\lambda=0}^{L-1}$ mask unused lanes in the oldest partial batch

  $\boldsymbol{\epsilon} \leftarrow \text{PairScoreBatch}(\mathbf{j}_b, i, \mathbf{A})$ $E(j, i)$, or $-\infty$ for a rejected pair

  $\mathbf{C} \leftarrow \text{Select}(\boldsymbol{\epsilon} \neq -\infty, \mathbf{f}[\mathbf{j}_b] + \boldsymbol{\epsilon}, -\infty)$ total candidate scores

  $\mathbf{I} \leftarrow (\mathbf{C} > \mathbf{H})$ strict lane-wise improvements

  $\mathbf{H} \leftarrow \text{Select}(\mathbf{I}, \mathbf{C}, \mathbf{H})$

  $\mathbf{P} \leftarrow \text{Select}(\mathbf{I}, \mathbf{j}_b, \mathbf{P})$ equal older candidates do not replace

  if $\text{Any}(\mathbf{I})$ then

    $u \leftarrow 0$

  else

    $u \leftarrow u + 1$

  if $u \geq U$ and $\mathbf{begin}_b > st_i$ then

    $o \leftarrow \mathbf{begin}_b$ batch loop omits $[st_i, o)$

    break

$f_i \leftarrow \text{HorizontalMax}(\mathbf{H})$

$\mathbf{T} \leftarrow \text{Select}(\mathbf{H} = f_i, \mathbf{P}, -1)$

$p_i \leftarrow \text{HorizontalMax}(\mathbf{T})$ largest predecessor index wins a score tie

if $f_i = \ell_i$ then

  $p_i \leftarrow -1$

if $m < st_i$ or m is outside the current reference and strand window then

  $m \leftarrow \arg\max_{st_i \leq j < i}(f_j, j)$, or -1 if this interval is empty compare score first, then index

if $st_i \leq m < o$ then

  $\epsilon_m \leftarrow \text{PairScoreOneSIMDLane}(m, i)$ $-\infty$ when the pair is rejected

  if $\epsilon_m \neq -\infty$ then

    $c \leftarrow f_m + \epsilon_m$

    if $c > f_i$ then

      $f_i \leftarrow c, \quad p_i \leftarrow m$

if $m < 0$ or $f_i \geq f_m$ then

  $m \leftarrow i$ B5 moves the cache to the newer anchor on a tie

4.7.2. Summary of forward chaining algorithms for CPU

mm2-fast is another version of minimap2 where specific parts were accelerated. The chaining algorithm in BioseqDB was inspired by mm2-fast (but note that as in case of other scientific work, mm2-fast provided only AVX2/AVX512 kernel, not universal code like in case of BioseqDB). There are some slight differences in high-level algorithm design:

- mm2-fast doesn't seem to have counter of anchors that does not improve the best score in the current batch (In BioseqDB bounded SIMD B1 we introduced such counter and in the pseudocode it's called U)
- mm2-fast uses scalar code to check for the lane winner, while BioseqDB bounded SIMD B1 uses SIMD instructions
- mm2-fast fallbacks to scalar code for fewer anchors than lane count

Table 4.1: Summary of all forward chaining implementations for CPU

Method	How candidates are processed	When the scan stops	How the result is selected
minimap2 B1	Score one predecessor j at a time. After each candidate, update the shared best pair and scalar skip/mark state	At the end of the chain, or when the scalar `max_skip` rule fires while j decreases	The shared (f_i, p_i) is already final
BioseqDB exact SIMD B1	Score one predecessor per active lane. Inspect a batch in decreasing j order when it may improve the best score or locate the exact `max_skip` stop	At the same candidate as scalar B1	The shared (f_i, p_i) is already final
mm2-fast	Score one predecessor per active lane. Each lane keeps its best pair across blocks	At the end of chain as there is no `max_skip`	Scalar code scans the lane winners
BioseqDB bounded batch SIMD B1	Score one predecessor per active lane. Each lane keeps its best pair, and one counter tracks batches with no lane-local improvement	At the end of the chain, or after U such consecutive batches	Choose the best lane result

Experimental justification

Note that although I was inspired by mm2-fast I still decided to apply the U limit with a counter (for counting anchors that do not improve score). This is not justified for now. Experimental data supporting this decision is presented in a separate section

4.8. Reverse chaining (backtracking)

The forward pass has already produced one score f_i and one predecessor p_i for every anchor. Backtracking follows the predecessor links and turns them into the chain records and flat anchor array A defined at the start of this chapter.

4.8.1. Formal definition for backtracking

Anchors have their score assigned: f_i. We would like to start by considering the highest scores as they are the best chains according to our score function. We sort the anchors in descending order of their scores: $f_{a_0} \geq f_{a_1} \geq \ldots \geq f_{a_{n-1}}$.

Now each anchor is the chain end so we need to find its start. We will traverse anchors according to p_i i.e $i \to p_i \to p_{p_i} \to \ldots \to -1$. This traversal doesn't have to go till the end of the chain. Usually there will be some part of the chain that is too low quality (not contributing to the score significantly).

We define function $\Delta_e(b)$ as the score assigned to the anchors followed from e up to, but not including, b:

$$\Delta_e(b) = \begin{cases} f_e, & b = -1, \\ f_e - f_b, & b \geq 0. \end{cases} \tag{4.16}$$

The value of $\Delta_e(b)$ will be used to determine the best cut point for each anchor:

$$\text{Cut}([\Delta_e(e), \Delta_e(p_e), \Delta_e(p_{p_e}), \ldots, \Delta_e(-1)]) \in [e, p_e, p_{p_e}, \ldots, -1] \tag{4.17}$$

The backtracking is then defined as:

$$\text{Backtrack}(e) = \{c = (a_i, p_{a_i}, \ldots \text{Cut}(a_i)) | a_i \in [a_0, a_1, \ldots, a_{n-1}] \ \wedge f_{a_i} \geq F_{\min} \wedge |c| \geq C_{\min}\} \tag{4.18}$$

The backtracking function finds best cutoff point and filters anchors for minimal score $F_{\min}$ and minimal chain length $C_{\min}$.

4.8.2. Minimap2 backtracking

Minimap2 uses two types of markings. The array t (similar to forward chaining), records which anchors have already been marked:

1. 0 means unmarked: we didn't visited this anchor yet
2. 1 means permanently marked: we won't consider this anchor as end of the chain, because it's already part of some found best chain. However anchors marked as 1 still can be part of the chain. This marking prevents a situation in which we would find chain, then it's exact subchain etc.
3. 2 means temporarily marked while one path is tested: this marking is used for Cut function and cleared after the path is tested. It purpose was to detect cycles (see cycle note below)

> **$t = 2$**
>
> Minimap2's implementation could remove usage of $t = 2$ markings, because under usual conditions there will be never a cycle of chains (anchor will always point to other anchor will lower position due to how forward chaining works). In listings below I provide the pseudocode corresponding to the actual current implementation, not some optimized one.

Minimap2 cut function marks visited anchors with $t = 2$ and stops on any anchor with non-zero t mark. We remember the best cut with the largest value of Δ_e. If a later cut is worse than the best cut by more than `max_drop`, the search stops and returns the best cut seen so far. After everything we clean the $t = 2$ temporary markings.

Figure 4.7 shows three interations of the main loop: we detect cuts for three anchors a_{11}, a_{10}, a_9. The example uses `min_sc` = 40 (this is minimap's minimum score $F_{\min}$), `min_cnt` = 2 (this is minimap's minimum chain length $C_{\min}$), and `max_drop` = 5 (maimum allowed score drop between best cut and current anchor in the chain). A Δ value inside anchor a_b is the score that would result if a_b was the selected cut. Orange boxes are anchors where either cut happens or we stop the search for the cut. Additionally I added "marked" label to a_5 to depict the fact that a_5 was marked by first $e = 11$ chain, so it cannot be end of the chain (it's still selected for $e = 10$, because it's start of the chain) Green boxes are anchors that are copied into the resulting array (everything excluding the cut anchor itself).

The minimap2 backtracking uses the same technique as BioseqDB Vulkan minimizer seeding: in the first pass we only count the number of copied anchors, while in the second pass we store them in the resulting array. This phase $\in \langle$count, write$\rangle$ iteration is depicted in (c) panel. The last step of minimap2's backtracking depicted in panel (d) is reversing the chains (so the start is on the left rather than on the right side) and sorting by reference coordinates of the chain's starts, so the chains are in natural left-to-right order.

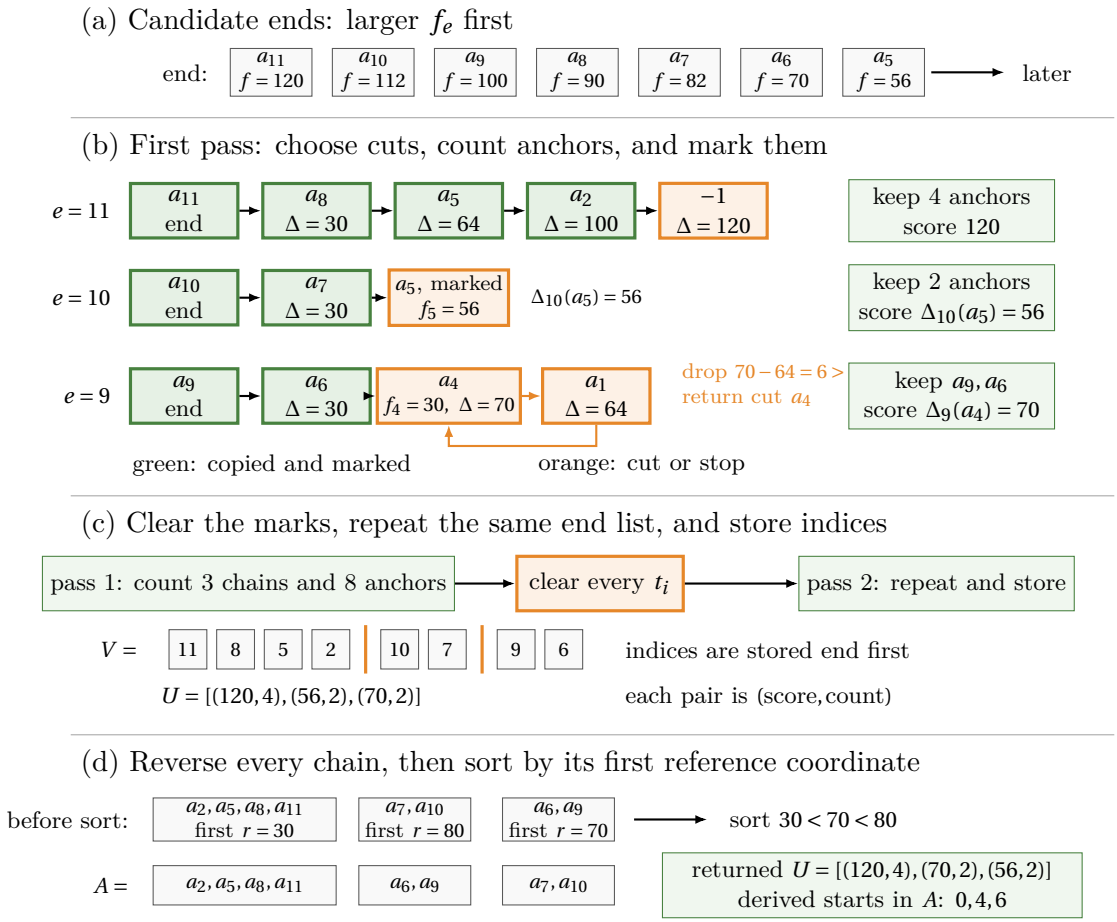


Figure 4.7: Minimap2 scalar backtracking on three anchors. Panel (b) shows that the anchor that triggers `max_drop` doesn't have to be the cut that is returned. Panel (c) shows the separate count and write passes. Panel (d) shows the final forward anchor order.

Algorithm 12 shows the code for the cut search loop.

Algorithm 12 Minimap2 cut selection function for anchor e

Require: unmarked end e, scores f, predecessors p, marks t

Require: drop limit $D = \texttt{max_drop}$
Ensure: cut b, which is excluded from the new chain

```
function ChooseCut(e, f, p, t, D)
    i ← e,   b ← e,   δ_best ← 0
    repeat
        t_i ← 2   temporary mark
        i ← p_i
        if i < 0 then
            δ ← f_e
        else
            δ ← f_e − f_i
        if δ > δ_best then
            δ_best ← δ,   b ← i
        else if δ_best − δ > D then
            break
    until i < 0 ∨ (i ≥ 0 ∧ t_i ≠ 0)
    h ← i,   j ← e   h is where this search stopped
    while j ≥ 0 and j ≠ h do
        t_j ← 0,   j ← p_j   clear only temporary marks
    return b
```

Algorithm 13 shows also how two passes for count/write are integrated. Minimap2 uses first pass to count all anchors i.e N_C (this value is used to allocate array U). Array U holds anchor count for a chain in lower 32 bits and score in higher 32 bits (this packing is defined by Pack function). U is then chain-reversed into B_γ array, which undergoes sorting by reference coordinates ReferencePos($B_\gamma[0]$) The final array A contains all anchors, while output array U contains packed `uint_64t` metadata for chain (count and score).

Algorithm 13 Minimap2 scalar backtracking and final chain order

Require: input anchors $a_0, \ldots, a_{n-1}$, scores f, predecessors p

Require: limits $S = \texttt{min_sc}$, $K = \texttt{min_cnt}$, and $D = \texttt{max_drop}$
Ensure: packed minimap2 records U and forward-order anchor array A

define $\mathrm{Pack}(s, c) = (\mathrm{uint64}(s) \ll 32) \mid c$
$Z \leftarrow [(f_i, i) \mid 0 \le i < n \text{ and } f_i \ge S]$
sort Z by increasing score only — read it backward below
use temporary array $V[0 \ldots n)$ — reuse an existing n-entry array
for phase $\in \langle \mathrm{count}, \mathrm{write} \rangle$ do
    $t[0 \ldots n) \leftarrow 0, \quad N_C \leftarrow 0, \quad N_V \leftarrow 0$ — accepted records and retained indices
    for $k \leftarrow |Z| - 1$ down to 0 do
        $e \leftarrow Z_k.\mathrm{index}$
        if $t_e = 0$ then
            $b \leftarrow \mathrm{ChooseCut}(e, f, p, t, D)$
            $N_V^{\mathrm{start}} \leftarrow N_V, \quad i \leftarrow e$ — cursor before tracing this end
            while $i \ne b$ do
                if phase = write then
                    $V[N_V] \leftarrow i$
                $N_V \leftarrow N_V + 1, \quad t_i \leftarrow 1, \quad i \leftarrow p_i$
            $c \leftarrow N_V - N_V^{\mathrm{start}}$
            if $b < 0$ then
                $s \leftarrow f_e$
            else
                $s \leftarrow f_e - f_b$
            if $s \ge S$ and $c > 0$ and $c \ge K$ then
                if phase = write then
                    $U[N_C] \leftarrow \mathrm{Pack}(s, c)$
                $N_C \leftarrow N_C + 1$
            else
                $N_V \leftarrow N_V^{\mathrm{start}}$ — discard indices but keep the marks
    if phase = count then
        allocate $U[0 \ldots N_C)$
$o \leftarrow 0$
for $\gamma \leftarrow 0$ to $N_C - 1$ do
    $c_\gamma \leftarrow \mathrm{uint32}(U[\gamma])$ — low 32 bits store the anchor count
    $B_\gamma \leftarrow [a_{V[o+c_\gamma-1]}, \ldots, a_{V[o]}]$ — reverse one chain
    $o \leftarrow o + c_\gamma$
$\pi \leftarrow \mathrm{argsort}_{0 \le \gamma < N_C} \mathrm{ReferencePos}(B_\gamma[0])$ — increasing ReferencePos
$U_{\mathrm{old}} \leftarrow U, \quad U \leftarrow [\,], \quad A \leftarrow [\,]$
for $m \leftarrow 0$ to $N_C - 1$ do
    $\gamma \leftarrow \pi_m$
    append $U_{\mathrm{old}}[\gamma]$ to U
    append every anchor in B_γ to A
return (U, A)

Start positions

Minimap2 stores only the score and anchor count, but it does not store an anchor start. The start of a chain is the sum of the counts of all earlier entries in U and later code uses that sum. I were initially very suprised and confused by summing coordinates, but this is the explanation.

4.8.3. BioseqDB SIMD backtracking

Exact SIMD B1 and bounded SIMD B1 call the same SIMD backtracking function. Its main differences from minimap2 are the following.

1. Minimap2 follows and marks one end at a time. BioseqDB handles the end with largest (f_e, e) alone, then collects up to L later paths together.
2. Minimap2 can cut a path with `max_drop`. BioseqDB follows a path until -1 or an anchor that is already marked.
3. Minimap2 repeats the complete backtracking pass so it can count chains before allocating U. BioseqDB runs one pass and appends accepted chains to output arrays that grow as needed.
4. Minimap2 does not specify an anchor order for equal scores. BioseqDB tries the higher end position first on a score tie.
5. Minimap2 finally sorts complete chains by ReferencePos of their first anchor. BioseqDB leaves accepted chains in the order in which it processes their ends.

In Bioseq chaining we consider 3 different operations:

1. Single-path: This is a special case for consequitive anchor indices. If anchor has a chain built from consecutive anchor indices, then we can follow only one path at the time (SIMD vector contains only fragment of one chain) and mark anchors in bulk.
2. Multi-path: This is expected case of SIMD-ified algorithm. In SIMD vector we store multiple anchors and follow each of the chains in parallel.
3. Multi-path marking: This is the step that we do after multi-path - we need to mark all anchors and we do that by transposing the matrix created in multi-path step, where now each of L lanes contains consecutive indices for one chain. (In reality we don't transpose any matrix, it's here only as illustration as in the code we put chains directly into final path rows)

In BioseqDB we only have binary marks:

$$t_i = 0 \quad \text{means unclaimed,} \qquad t_i = 1 \quad \text{means claimed.}$$

There is no temporary value 2, which was obsolete as mentioned earlier. A mark is never cleared, including when the marked prefix later fails the score or count test.

If L consecutive links satisfy $p_j = j - 1$, then we run single-path algorithm version, where one vector comparison checks the L links. Formally this condition is specified as:

$$\mathrm{Run}_L(j, c) \iff j \geq L-1 \ \wedge \ c \leq n - L \ \wedge \ \bigwedge_{u=0}^{L-1} p_{j-u} = j - u - 1.$$

Chain order heuristic

At first it might not be intuitive why we are interested in chains of consecutive anchors. We can think of inverse of that condition. If the anchors are not consecutive, then it means that for some anchor there exists alternative very good match i.e for one target position there are multiple competing chainings for different query positions that fight over few sibling anchors. This means that fragment in query is not easily recognisable as query is repetitive and each repetition is equally good (very low divergence rates). For now we will just assume that this condition $\mathsf{Run}_L(j,c)$ is non-trivial algorithm specialisation that is discussed in detail in separate chapter.

If the $\mathsf{Run}_L(j,c)$ condition is met then we use only two store operations to mark L anchors of that single chain (we pay the cost of checking the condition, but the we benefit by using less instructions). If not "multi-path" version follows one chain per lane. This concept is illustrated in Figure 4.8 section (b). At first each lane contains consecutive indices which are highest-scoring anchors used as ends. Then we iterate trhough all of them until we reach the stop condition. Each lane λ follows one chain and can stop earlier than others. We don't want to explicitly hold this structure as matrix and transpose it. Instead for iteration (depth) d we directly store (scatter)lanes:

$$V_{\lambda,d} \quad \overrightarrow{\text{memory address}} \quad \text{Address}(V) + (\lambda n + d) * \text{sizeof}(V_{0,0}).$$

where n constant is the arbitrary number of SIMD vectors we want to store ($d < n$)

Figure 4.8 illustrates the algorithm with number of lanes: $L = 4$, minimum score $F_{\min}$: $S = 30$, and minimum chain length $C_{\min}$: $K = 2$. The relevant scores for the example are:

$$f_{19} = 100, \quad f_{18} = 90, \quad f_{17} = 85, \quad f_{16} = 80, \quad f_{15} = 75, \quad f_9 = 40, \quad f_{11} = 55, \quad f_{10} = 50.$$

- First we start with f_{19} as this is the highest-scoring anchor. We look for $p[19]$ and load $[p[19], p[18], p[17], p[16]] = [9,8,7,6]$ into SIMD vector. This is compared to vector $[p[19], p[19], p[19], p[19]] - [0,1,2,3]$, which in this case are equal. This means (toegether with bound checks) that $\mathsf{Run}_L(19,0)$ is true. Hence we run the single-path algorithm. I specifically split the second vector in the comparison to illustrate how SIMD vectors are constructed and compared in the real code. Then the single-path (a) will use store operation to mark L anchors as visited and reverse the indices and store them in the result vector.

- For first path (a) when we encounter 1 (the end of the array of anchors) we fallback to multi-path behaviour (this is the simplest way to avoid out-of-bounds access. Otherwise we would need to implement complicated masking). This is observed in (a) as 1 and 0 being in separate fragments rather than single one.

- The multi-path is illustrated in (b). The next highest scoring anchor after f_{19} is f_{18} with score 90. We look for $p[18]$ and load $[p[18], p[17], p[16], p[15]] = [14,13,12,10]$ into SIMD vector. The comparison in this case will yield false as $p[15] \neq 11$. This means that we follow each index one by one. Each time we load markings for the index and check what lanes were already marked (this forms a SIMD binary mask). If all lanes are masked-out then we can stop iteration completely. At each iteration we store lanes values in correct indices in our V array.

- After multi-path (b) runs we still needs to filter the chains by score and length.

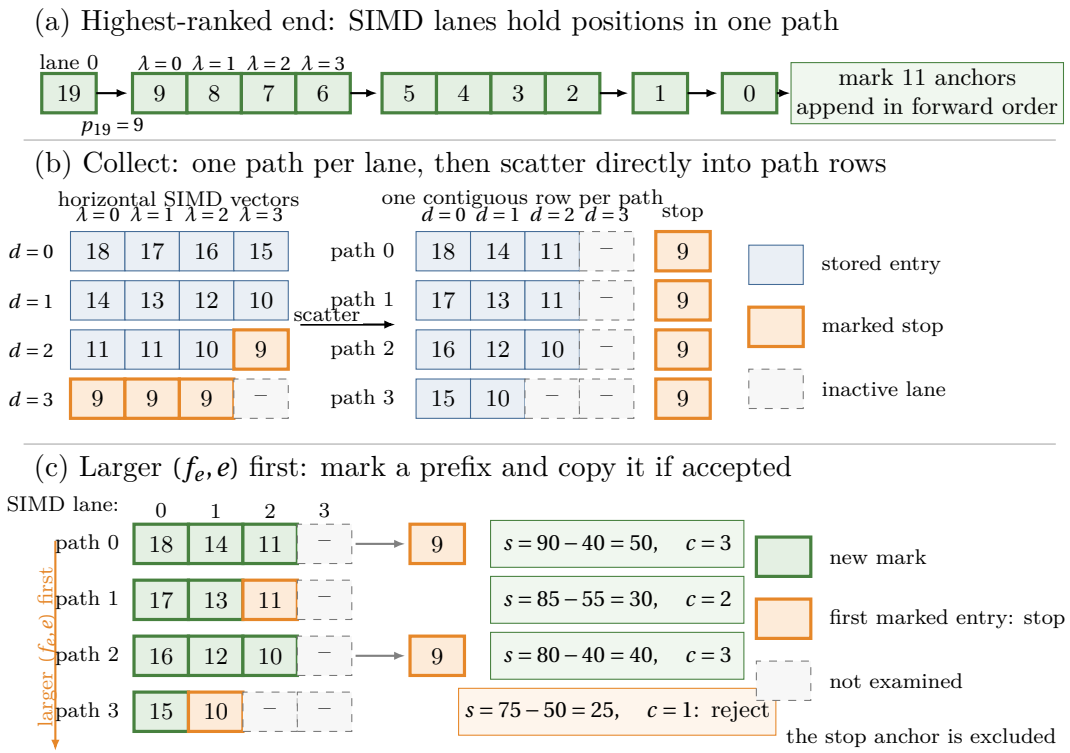


Figure 4.8: BioseqDB SIMD backtracking with four L lanes. Panel (a) shows the special handling of the first path. If we detect Run_L is true, then we use entire SIMD vector just for this one chain (SIMD vector contains anchors for $L = 4$ different chains). Panel (b) shows the other case when Run_L if false. The SIMD vector changes with each iteration $d = 0, 1, 2, 3$ and it's used to store (scatter) anchors indices into the buffer (on the right size). Panel (c) reads one row at a time from the buffer starting at highest-scoring anchors first (now the SIMD vector as in case of Panel (a) contains one chain), we check for marked nodes via FirstMarked function. If there are no marked anchors detected then we iterate this processing multiple times going through the chain. After reaching either end or marked node, we save the chain if it's score is greater than required threshold and it's length is sufficient.

The accepted output in this example is

$$C = [(100, 11, 0), (50, 3, 11), (30, 2, 14), (40, 3, 16)],$$
$$A = [a_0, \ldots, a_9, a_{19} \mid a_{11}, a_{14}, a_{18} \mid a_{13}, a_{17} \mid a_{10}, a_{12}, a_{16}].$$

Each record in C is (score, count, start). For a_{15} (in the Panel (c)) we mark it before testing its thresholds, but then the next anchor has too low score and is rejected. Because the chain length is just 1 which is lower than minimum required length, we don't copy that chain into A.

In the listing below, I present the code for this backtracking algorithm.

In the pseudocode we use Run_L to set g which is either L or 1 depending if we choose multi-path or single-path iteration. Additionally, we didn't define $FirstMarked$ function in the pseudocode as it's just helper to determine left-most marked anchor in the row (limited to only g entries):

$$\text{FirstMarked}(t, V_\lambda, o, g) = \min\big(\{u \mid 0 \le u < g \wedge t_{V_{\lambda, o+u}} = 1\} \cup \{g\}\big).$$

It returns g when none of the g entries is marked. On the pseudocode listing for row λ of V denoted as V_λ:

- ℓ_λ is the number of stored entries in that row
- b_λ is the row's stop anchor
- α_λ says whether its collection lane is active (this is SIMD mask to determine if we need to terminate the chain if we either reach -1 or already marked anchor)

Algorithm 14 BioseqDB SIMD path collection and backtracking

Require: anchors $a[0\ldots n)$, scores f, predecessors p, thresholds S, K, width L, and empty outputs C, A

function ChooseCutsSIMD($\mathbf{e}, p, t, V, n$)
  $q \leftarrow |\mathbf{e}|, \quad \mathbf{j} \leftarrow \mathbf{e}, \quad \boldsymbol{\ell} \leftarrow \mathbf{0}, \quad \mathbf{b} \leftarrow -\mathbf{1}, \quad \boldsymbol{\alpha} \leftarrow \mathbf{1}$
  for $d \leftarrow 0, \ldots, n-1$ while Note that all things detnoted by λ are evaluated for $0 \le \lambda < q$ in one SIMD vector $\sum_{\lambda=0}^{q-1} \alpha_\lambda \neq 0$ do
    $h_\lambda \leftarrow [\alpha_\lambda = 1 \wedge t_{j_\lambda} = 1], \quad \eta_\lambda \leftarrow \alpha_\lambda \wedge \neg h_\lambda$ check if we reached marked anchor in the lane
    $b_\lambda \leftarrow j_\lambda$ if $h_\lambda = 1$, and $(V_{\lambda,d}, \ell_\lambda) \leftarrow (j_\lambda, \ell_\lambda + 1)$ if $\eta_\lambda = 1$ scatter
    $j_\lambda \leftarrow \begin{cases} p_{j_\lambda}, & \eta_\lambda = 1 \\ -1, & \text{otherwise} \end{cases}, \quad \alpha_\lambda \leftarrow [j_\lambda \ge 0]$
  assert $\sum_{\lambda=0}^{q-1} \alpha_\lambda = 0$ All of the lanes should terminate chains
  return $(\boldsymbol{\ell}, \mathbf{b})$
function BacktrackSIMD(a, f, p, S, K, L, C, A)
  allocate $V \in \text{int32}^{L \times n}$, and set $t[0\ldots n) \leftarrow 0$ 0: unclaimed, 1: claimed
  $Z \leftarrow \text{sort}_\uparrow[(f_i, i) \mid 0 \le i < n \wedge f_i \ge S], \quad k \leftarrow |Z| - 1$
  if $k \ge 0$ then
    $e \leftarrow Z_k.\text{index}, \quad j \leftarrow e, \quad c \leftarrow 0$
    while $j \ge 0 \wedge c < n$ do
      $g \leftarrow \begin{cases} L, & \text{Run}_L(j, c) \\ 1, & \text{otherwise} \end{cases}$
      $(V_{0,c+u}, t_{j-u}) \leftarrow (j-u, 1)$ for $0 \le u < g$, then $(j, c) \leftarrow (p_{j-g+1}, c+g)$
    assert $j < 0$
    if $f_e \ge S \wedge c \ge K$ then append $(f_e, c, |A|)$ to C and $(a_{V_{0,c-1}}, \ldots, a_{V_{0,0}})$ to A
    $k \leftarrow k - 1$
  while $k \ge 0$ do
    $q \leftarrow \min(L, k+1), \quad \mathbf{e} \leftarrow (e_0, \ldots, e_{q-1}), \quad e_\lambda \leftarrow Z_{k-\lambda}.\text{index}$
    if $(S > 0 \vee K > 0) \wedge \bigwedge_{\lambda=0}^{q-1} t_{e_\lambda} = 1$ then $k \leftarrow k - q$ and continue
    $(\boldsymbol{\ell}, \mathbf{b}) \leftarrow$ ChooseCutsSIMD($\mathbf{e}, p, t, V, n$)
    for $\lambda \leftarrow 0$ to $q-1$ do
      $c \leftarrow 0, \quad o \leftarrow 0$ c holds length of chain and o is the offset in the A where we store it
      while $o < \ell_\lambda$ do
        $g \leftarrow \min(L, \ell_\lambda - o), \quad r \leftarrow \text{FirstMarked}(t, V_\lambda, o, g)$
        $t_{V_{\lambda,o+u}} \leftarrow 1$ for $0 \le u < r$, and $c \leftarrow c + r$
        if $r < g$ then $b_\lambda \leftarrow V_{\lambda,o+r}$ and break, otherwise $o \leftarrow o + g$
      $s \leftarrow \begin{cases} f_{e_\lambda}, & b_\lambda < 0 \\ f_{e_\lambda} - f_{b_\lambda}, & b_\lambda \ge 0 \end{cases}$
      if $s \ge S \wedge c \ge K$ then append $(s, c, |A|)$ to C and $(a_{V_{\lambda,c-1}}, \ldots, a_{V_{\lambda,0}})$ to A
    $k \leftarrow k - q$
  return (C, A)

Minor improvements

Note that we check $\bigwedge_{\lambda=0}^{q-1} t_{e_\lambda} = 1$ for the first iteration to check if we have chains that all go through other chains that were already all marked. This could be implemented via SIMD masks operations, but in my code I count the true bits and compare their count. This means there are still some very minor optimisation opportunities that could be explored

4.9. Statistical modelling of CPU forward and backward passes

The first loop in Algorithm 14 can process L anchors at once only when $\mathsf{Run}_L(j, c)$ is true. We also use update counter U in Algorithm 11 to terminate loop if the consecutive updates don't improve any SIMD lane. At the same time minimap2 uses max_drop in its algorithm 4.7 to terminate backtracking if the score drop is too large.

We never explained any proper justification for using or not using these heuristics. This section talks about random synthetic data and intuitive explanation behind the choices. The subsections talk about:

- Preparing synthetic data (randomly generated sequences with insertions/deletions) and specially constructed examples aimed at absuing the scoring function and thresholds
- Answering if counting number of updates that do not improve score in forward chaining algorithm 11 improve performance
- Answering if the anchors during backtracking 14 are just continous sequences of indices incremented by 1 ($\mathsf{Run}_L(j, c)$ condition)
- Answering if max drop would help us increase performance of SIMD backtracking algorithm or not 4.7

Synthetic data

Primary modelling here uses synthetic data built from randomly generated sequences assuming default substitution rate of 0.001 and insertion/deleteion probability of 0.0005. I prepared special cases of sequence construction using gaps or equal regions to see what happens in cases where extremally large gaps occur as secondary method of analysis. Overall the analysis is intended to motivate the approach and formulate conditions under which an optimisation's cost exceeds its benefit.

4.9.1. Problematic chains

As mentioned in section 4.3 there is a case of emergent assymetry: if one anchor matches with multiple anchors in the query, then we cannot easily tell which part of the query is best to match. The following sections demonstrates what happens in this scenario. The special query/target constructions are prepared in that asymmetry in mind.

Reference and query structures

In our experiments we use δ as input parameter for divergence (controlling substituiton rates). The exact substituion count (number of replaced bases in entire sequence) can be expressed

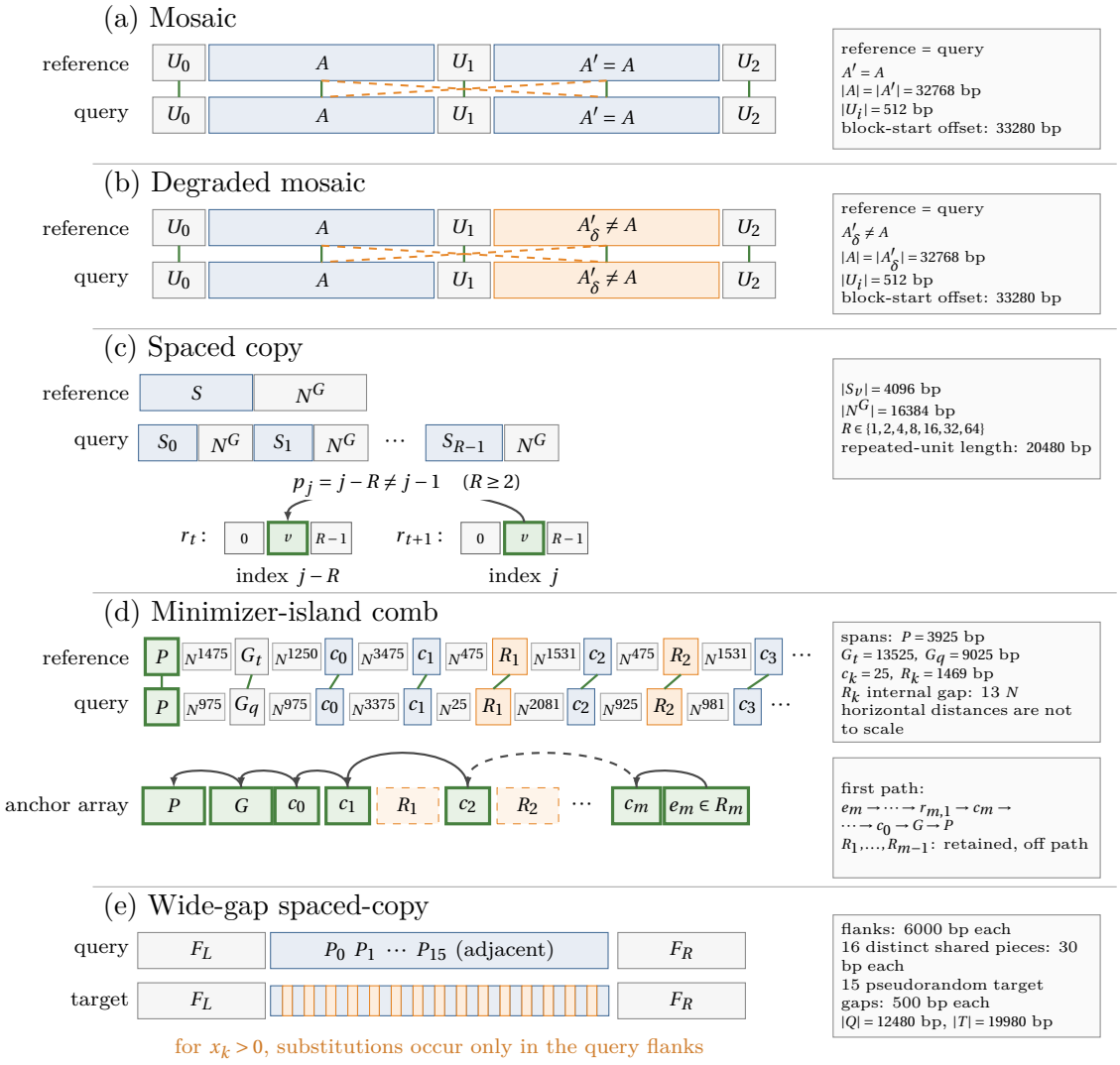


Figure 4.9: Panel (a) presents mosaic, with $A' = A$ so $T = Q = U_0 A U_1 A'_\delta U_2$where$A' = \text{Randomize}_\delta(A)$. Panel (b) is degraded mosaic, so the A and A' are independently generated, so the divergence between them should be very high, used as control example. Panel (c) is shows spaced copy, which is contructed in the following way $T = S N^G$, $Q = (S N^G)^R$, $|S| = 4096$, $G = 16384$ Panel (d) is the Minimizer-island comb. I construct it by assuming that every c_k and $r_{k,\ell}$ is an isolated 25-base minimizer island. Then $r_{k,\ell}$ are grouped together with 13-N intervals to create R_k groups. The construction is calculated in such a way that $|d_r - d_q|$ is lower for c_k anchors than for R_k anchors. Panel (e) is wide-gap spaced-copy, where we insert gaps between 16 30-base pieces of query with absurdly large 500-base gaps.

as:

$$s_\delta = \sum_{\text{fragment} \in \text{query}} \left\lfloor \delta|A| + \frac{1}{2} \right\rfloor \tag{4.19}$$

The $+\frac{1}{2}$ is just used to round the result to the nearest integer. The sum goes over all fragments we can change (this depends on exact construction as some of them need to contain specific exact regions)

Figure 4.9 depicts all of my constructions:

- Random reference and query - parameters to experiments are $|Q|$ and $|T|$, number of occurences of query $1 \leq N$. I first generate random target, then place N query occurences in it and use divergence to create substitution in the occurences.
- (A) Mosaic - reference and query with identical bytes. A is randomly generated and I perform substitutions on it. This is example of exact duplication of long fragment. The specific A length was choosen to be very long to generate long, very similar chains.
- (B) Degraded mosaic - this is version of mosaic, where we use two regions that aren't equal and do not perform any substitutions on them. This is control case, where the generation of both A regions is independent. Meaning the expected divergence should be very high (as sequences are random).
- (C) Spaced copy - we insert very long blocks N^G between parts of reference, where N denotes symbol N (in dna4 alphabet it denotes unknown symbol). Normal processing just ignores these blocks (minimap2 doesn't generate minimizers for them), but they distrupt anchor positions. You can think of this construction as very special case of gapped repeatitions of same sequence, where instead of random gap-filler we use symbol N. That kind of contruction shouldn't occur in real-world example, it's here to illustrate that we can utilize N symbol to sabotage chaining.
- (D) Minimizer-island comb - we also use N symbol as filler between inslands. As we use default minimap2 parameters including $w = 10, k = 15$, we adjusted $|c_k|$ to be $w + k = 25$ bases, so these fragments generate maximum of two non-N minimizers. R_k groups are internally built from the same 25-symbol "islands" separated by 13 N bases (13 was arbitrary choice). This creates very interesting scenario where we can control which minimizers occur where. Then we adjust wider gaps between "islands" with tricky criteria. As DP algorithm selects one "next" best anchor to "connect" chain to, we adjust the gaps so the difference between target and query distances are large for the anchors just at the edge of R_k but smaller for ck_kj. You can look at the example and see that if we try to chain c_1 to c_0 the $\delta(c_0, c_1) = |d_r - d_q| = 3475 - 3375 = 100$ but δ for R_1 will be $\delta(c_0, R_1) = (3475 + 475) - (3375 + 25) = 550$. We exploit the feature of scoring function to convince the minimap2 that c_k is "better" match being closer to diagonal than R_k. This will create long chain starting at R_k and going through all c_k anchors (ommiting all other R)
- (E) This example is more practical one, where we don't overuse N symbols and divide the query to add random gaps between its pieces. However the gaps are very large compared to the "islands" so this example is "exaggerated" as well.

Overuse of N symbol

Usually N is used only to distinbguish processing errors (from the sequencing hardware) and do not occur with high quantities. The construction of the examples use them as gap-fillers to have easier control over occurence of minimizers and their behavior. Cases (A) and (E) are closer to real-world examples, where repetitions of genomes happen frequently.

Constructions (a), (b) and (c) are primary using the same idea. After collecting and sorting anchors the anchor table looks like this for spaced copy (sorted by target coordinate):

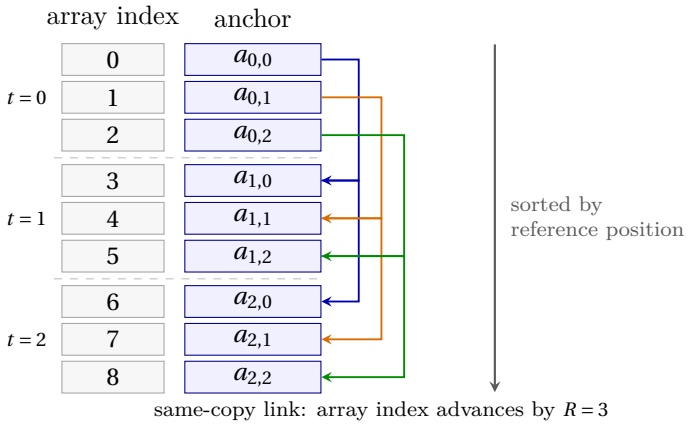


Figure 4.10: Coordinate-sorted spaced-copy anchors for the concrete case $R = 3$. As the query contains exact R copies we will have "layers" ($t = 0, 1, 2...$) of the same anchors with the same reference position. Anchor $a_{t,v}$ is the anchor for v-th copy in the query. Colored arrows follow mark the next anchor in the chain. Naturally each anchor points to the next anchor in the same v-th copy. This means that each anchor will just point R anchors forward.

The construction (d) - minimizer-island comb was created by adjusting gap sizes between query and target islands:

transition	target N-run	query N-run
$P \to G$	1475	975
$G \to c_0$	1250	975
$c_0 \to c_1$	3475	3375
$c_k \to R_k$	475	25 (k odd), 925 (k even)
$R_k \to c_{k+1}$	1531	2081 (k odd), 981 (k even).

Default minimap2 parameters

As we discussed due to larger shifts between target and query coordinates, the R_k anchors will have larger pentalty due to higher distance from diagonal i.e δ. Minimap2 by default uses max bandwith of $B = \text{bw} = 500$. Every anchor in R_k and c_{k+1} differ by 550 between the target and query distances, so those anchors fail the bandwidth check. This means that the best chain will be going through: $R_m, c_m, c_{m-1}, \ldots c_0, G, P$ as depicted in the figure 4.9 The original dimentions of constrution can be calculated from the following formula ($r_{k,\ell}$ is the ℓ-th anchor in R_k), L is the count of "islands" in R_k:

$$B = 500, L = 39, s(k) = \begin{cases} 1 & \text{if } k \text{ is odd} \\ -1 & \text{if } k \text{ is even} \end{cases}$$

$$c_k \to c_{k-1} : (3500,\ 3500 - 100 * s(k)),$$
$$r_{k,\ell} \to c_k : (B + (L-1)(\ell - 1),\ B - 450 * s(k) + (L-1)(\ell - 1)), \quad 1 \le \ell \le L.$$

Experiment design

Minimap2 2.31-r1302 was run with the `map-ont` preset (default one). I added instrumentation to record the score and predecessor arrays.

- Let $L \in \{2, 4, 8, 16, 32, 64\}$ be the number of anchors viewed at the same time similar to our SIMD algorithms.
- Let $\text{Run}_L(j, c) \iff j \ge L - 1 \wedge c \le n - L \wedge \bigwedge_{u=0}^{L-1} p_{j-u} = j - u - 1$ be the condition used in BioseqDB SIMD algorithms.
- Let B_L be the number of evaluations for which Run_L is true
- Let E_L the number of times the first-path loop evaluates Run_L
- Let P_L the number that satisfy both bounds $j \ge L - 1$ and $c \le n - L$.
- Let q be the number of anchors on the first predecessor path.
- Let Fire_L be the observed rate at which my heuristic fires
- Let Contiguous_L be the the ratio Fire_L but with bounds checks excluded (we measure firing rate but only after bounds checks)
- Let Coverage_L be the fraction of path anchors consumed by vector batches (this is a metric that answers the question "How many anchors we can cover by SIMD batches")

The exact definitions are as follows:

$$\text{Fire}_L = \frac{B_L}{E_L}, \qquad \text{Contiguous}_L = \frac{B_L}{P_L}, \qquad \text{Coverage}_L = \frac{LB_L}{q}.$$

Table 4.2: Observed first-path branch-firing rate $100B_L/E_L$, in percent. The Mosaic, Degraded mosaic, and $R = 1$ Spaced copy rows report medians over three inputs generated from fixed random-number-generator seeds. The $R \geq 2$ Spaced copy row covers the tested powers of two from 2 through 64, all of which gave zero. Minimizer-island-comb values are deterministic and agreed with the recorded counters.

Input	$L = 2$	$L = 4$	$L = 8$	$L = 16$	$L = 32$	$L = 64$
(a) Mosaic	1.17	0.59	0.29	0.14	0.07	0.02
(b) Degraded mosaic (independent A')	100.00	99.94	99.74	99.24	97.01	91.12
(c) Spaced copy, $R = 1$	100.00	98.97	94.12	88.89	80.00	66.67
(c) Spaced copy, $R \geq 2$	0.00	0.00	0.00	0.00	0.00	0.00
(d) Minimizer-island comb, $m = 2$	95.74	84.62	45.45	29.41	6.67	0.00
(d) Minimizer-island comb, $m = 1024$	4.21	2.10	0.96	0.48	0.19	0.00

We can make the following observations:

- For minimizer-island the prefix will contain some $C > 0$ anchors, meaning that we will get only continuous parts in R_m at the end and in the sequence prefix: $B_L \simeq \lfloor \frac{39}{L} \rfloor + \lfloor \frac{C}{L} \rfloor, q = m + C$ This means that for $L = const$: $\mathsf{Fire}_L, \mathsf{Coverage}_L < \theta(1/m)$ and for large enough $L > 39 \wedge m \to \infty$ they will approach 0.
- For spaced copy the anchors are always separated by R so Fire_L should be zero for all $R > 0$
- Degraded mosaic is just simulating two random fragments, so Fire_L should be almost 100% with high measured divergence.
- Mosaic in this case was simulated with zero divergence, so the Fire_L should measure anchors only in the fillers between equal parts i.e $0\% < \mathsf{Fire}_L < \frac{|U_i|*3}{|U_i|*3+|A|} < 4\%$

The table 4.2 confirms our assumptions.

Now we would like to see how divergence δ affects the mosaic rates. Table 4.3 shows selected data points from Mosaic runs at δ. s_δ is the number of substitutions made to the target sequence. We can see that the higher number of same-target anchor alternatives (multiple query positions per anchor), the worse results we get (as predicted).

Table 4.3: Few most important data points for mosaic construction for different values of δ and degraded mosaic (should have effective high divergence). Each entry in the last three columns id the median followed by the observed minimum and maximum over three inputs generated from fixed random-number-generator seeds, in percent.

Case	s_δ	$100\hat{\delta}$ (%)	Path anchors with same-target alternatives (%)	$100\,\mathsf{Fire}_4$ (%)	$100\,\mathsf{Coverage}_4$ (%)
$\delta = 0$	0	0.000	97.69 [97.64, 97.69]	0.59 [0.58, 0.59]	2.31 [2.29, 2.33]
$\delta = 0.0025$	82	0.250	93.51 [93.40, 93.52]	1.06 [1.01, 1.18]	4.12 [3.91, 4.57]
$\delta = 0.0050$	164	0.500	89.55 [89.47, 89.68]	1.56 [1.47, 1.79]	5.96 [5.64, 6.79]
$\delta = 0.0100$	328	1.001	82.24 [82.19, 82.29]	2.61 [2.57, 2.80]	9.69 [9.55, 10.34]
$\delta = 0.0200$	655	1.999	69.33 [68.93, 69.42]	5.14 [5.08, 5.30]	17.82 [17.64, 18.28]
$\delta = 0.0500$	1638	4.999	41.41 [41.33, 41.63]	14.97 [14.97, 15.38]	41.32 [41.32, 42.09]
$\delta = 0.1000$	3277	10.001	17.23 [16.81, 17.64]	37.99 [37.64, 38.72]	71.02 [70.71, 71.65]
$\delta = 0.2000$	6554	20.001	3.04 [2.66, 3.36]	79.41 [77.58, 81.93]	93.91 [93.26, 94.77]
Independent A'	n/a	75.043	0.00 [0.00, 0.00]	99.94 [99.90, 100.00]	99.98 [99.98, 100.00]

Let's consider that we are looking at mosaic with divergence rate $\delta = 0$, so $A = A'$. Then we have h pairwise-disjoint "blocking" minimizer windows $W_1, \ldots, W_h$. Each W_i has some

minimizer that now forms an anchor that makes $Run_L(j,c)$ evaluation false. W_i covers $k+w-1$ bases. Each of the windows needs to receive at least one subsitution so they're changed and do not generate one anchor with two different positions. So with probability $\frac{(s_\delta)_h}{(M)_h}$ all h windows will receive at least one substitution (for $(x)_h = x(x-1)\cdots(x-h+1)$). We can choose one base from every window with b^h ways, so this gives us very loose upper bound on the probability:

$$\begin{aligned}\Pr_\pi\big[\mathrm{Run}_L(j,c) \text{ changes from false to true}\big] &\le \Pr_\pi\big[S_\delta \cap W_i \neq \varnothing \text{ for every } i\big] \\ &\le b^h \frac{(s_\delta)_h}{(M)_h} \le (b\widehat{\delta})^h.\end{aligned}$$

Obiously disjoint-window model is a bad assumption as substitution can affect several overlapping windows and conversely with high divergence we can spontaneusly get random matches with other fragments like U_i (although this is unlikely). However we can loosely bound the probability by $\theta((k\widehat{\delta})^h)$. This gives us idea that the worst sequences are the one with perfect repeats (low divergence).

Figure 4.11 shows the results for mosaic for different levels of divergence (the degraded mosaic - as control sample - is show separately as a gray rectangle on the right side of chart)

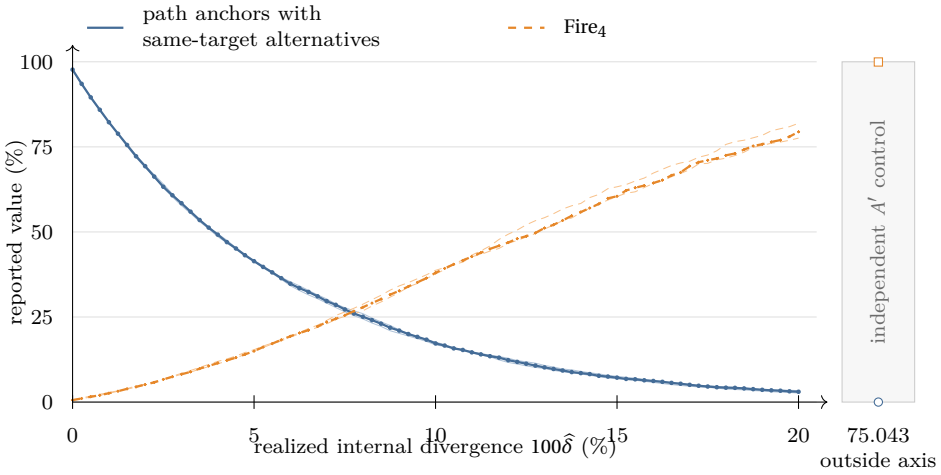


Figure 4.11: The lighter colours represent minimum/maximum while the darker ones represent median. As we can see the chart confirms that the Run_L firing rates are tied to number of anchors with same-target positions which are competing for the same anchor. Additionally we observe that divergence levels of at least 12% mitigate the negative effect of highly-repetive query.

As we can see from Figure 4.11, table 4.3 and table 4.2 the most inconvenient case is the high number of repetitions in the query. These repetitions can be mitigated by increased divergence levels. This was intuitive consulsion that we now sucessfully confirmed with the experiments.

Random-sequence subsitutions

I also wanted to measure effects of random sequences (not specially constructed) on the Run_L firing rates:

- We generate random sequences query/target parameterized by their lengths: $|T|$ and $|Q|$. We choose N positions in target to be the query and replace its fragments ac-

cordingly. By default we test each such occurence base with probability 0.001 to be substituted, 0.0005 to be inserted/deleted.

- Here I also use the default minimap2 parameters: $k = 15$, $w = 10$, occurrence filtering, and chaining scores.

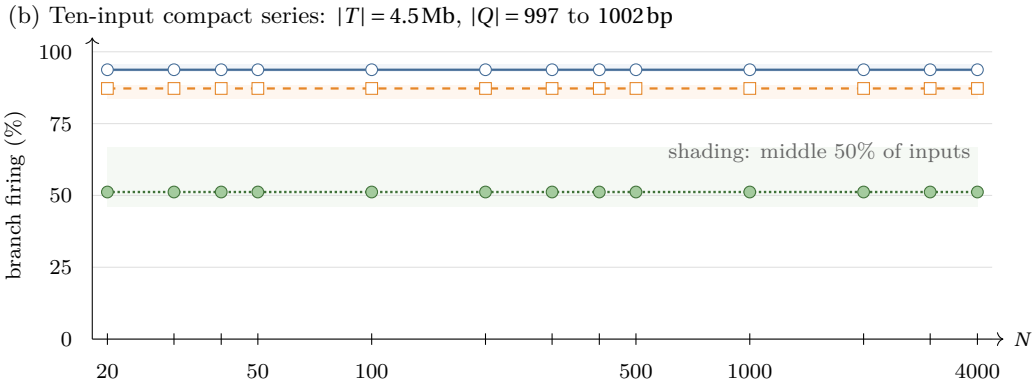


Figure 4.12: Branch firing rate $100\,\mathrm{Fire}_L$ for different values of L and N for random-sequence occurrences. As we can observe the N doesn't affect the firing rate as expected. The light colours represent middle 50% of inputs. As we observe as L gets larger it's more likely to disrupt the condition randomly so the standard deviation should be higher.

Figure 4.12 was used to just confirm that random generation is correct and N doesn't affect the probability (it can affect probability if the target size is too small and query occurences are too crowded and start to overlap). Figure 4.13 shows more interesting data. I tested $Fire_L$ (panel a) and $Reduction_L$ (panel b)for different valujes of δ and L.



Figure 4.13: Each data point represents sum of samples (averaged rates) over N=2,3,4...,4000 randomized experiments. The samples for $N = 1$ were marked with circles as we expect that they will be significantly higher (as there's one ocurrence)

Firing rate

Naturally the firing rate for $L = 4$ is higher, but $Reduction_L$ is initially higher for $L = 16$ meaning that on the hardware supporting more vectors (longer SIMD vectors), we exchange more condition checks (it's harder to guarantee $Run_L(i, j)$), but for more saved instructions (we can skip more L anchors). In practice we should test this on different hardware as the ideal loop reduction on panel (b) doesn't involve real costs of the instructions in cycles or cache latency etc.

4.9.2. Score distribution for random cases

If we plot the anchor count versus the cummulative chain score, we should get a curve that is roughly linear (this follows from formulation of the scoring function). The divergence should break long chains and make the curve score axis more narrow. This can be observed on Figure 4.14. I supply the charts to show how random inputs can behave as supplementary resources to the theoretical chapters. I also provide figure 4.15 to better illustrate "narrowing" of the chart

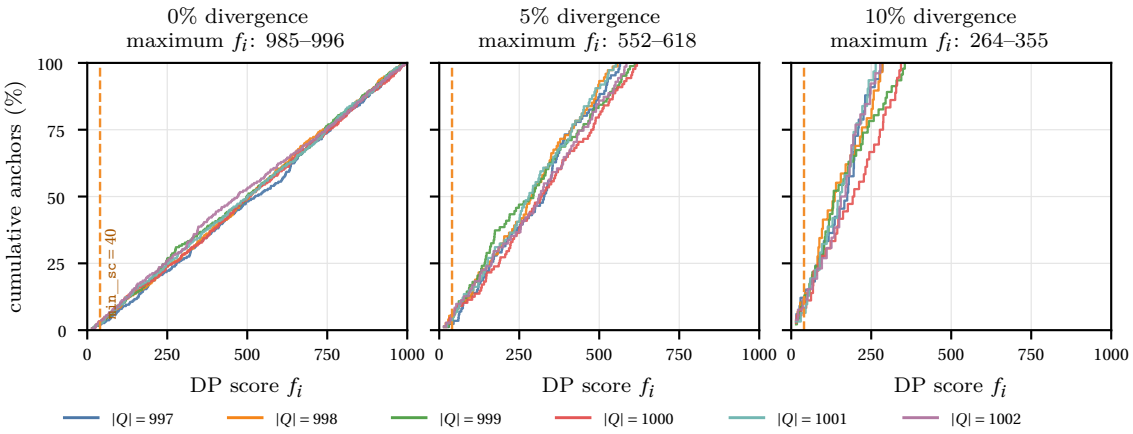


Figure 4.14: Results from randomized sequences for slightly different query lengths (997-1002 bases). The orange line is the minimum score threshold for chain (minimap default of `min_sc = 40`)

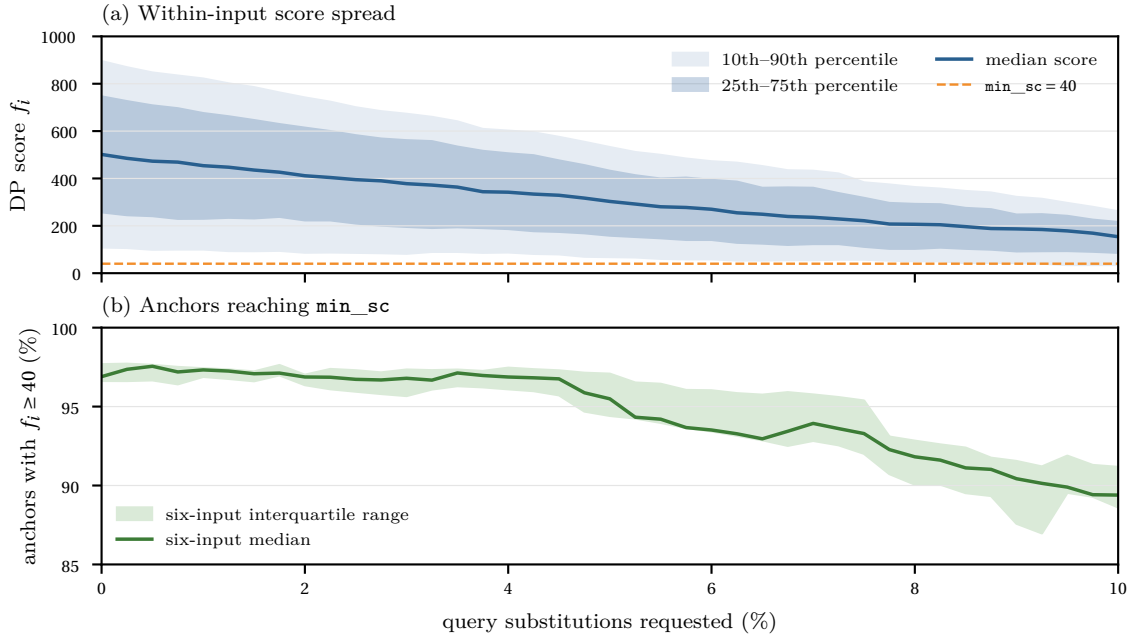


Figure 4.15: Cummulative results for all the experiments previously described (query lenghts 997-1002 bases). In the chart I take 6-input median (median over all 6 experiments with slightly different query lengths)

4.9.3. Does the bounded SIMD U limit help?

We analyzed the Run_L heuristis for chaining backtracking. Now I would like to apply similar measurements to determine how much limit on batch iteration U help us.

As reminder U is the maximum number of batches that consecutively do not improve the score for any chain during forward chaining 11 Minimap2 uses skip heuristic while we treat batch as a whole.

For SIMD width L, let E_L be the number of executed batches (same as in previous sections), Z_L the executed batches for which no lane satisfies $C_\lambda > H_\lambda$ (no score improvement), and Q_L the batches

in the complete window without the U stop. Similarily we define the firing rate and reduction rate:

$$\text{NoUpdate}_L = Z_L / E_L, \qquad \text{Reduction}_L^U = (Q_L - E_L)/Q_L.$$

The measurements on Figure 4.16 are pooled (summed over random-seed experiments), while Table 4.4 contains the separate runs for 0% and 10% divergence rates.

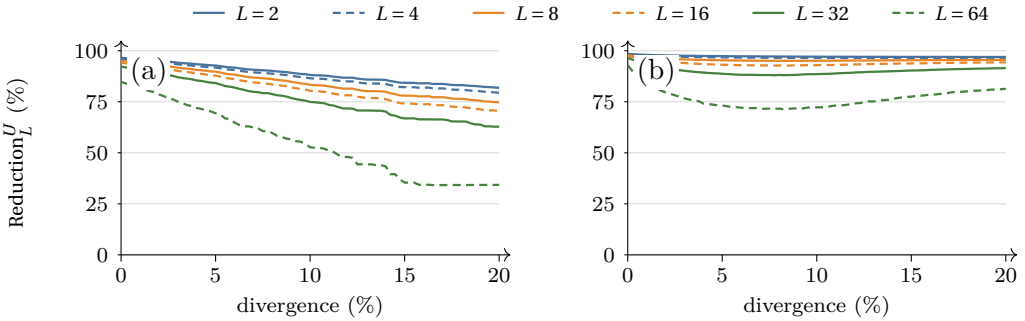


Figure 4.16: Pooled U-only reduction at each divergence for (a) random inputs and (b) sum over mosaic and degraded mosaic

Table 4.4: NoUpdate_L / Reduction_L^U (percent). Random uses $N = 4000$, a 4.5-Mbase target, 997–1002-base queries, and the same divergence levels as presented previously. Spaced copy uses $R \in \{1,2,4,8,16,32,64\}$. The Minimizer-island comb uses $m \in \{1,2,4,\ldots,1024\}$. Defaults are $k = 15$, $w = 10$, $S = 25$, and `max_chain_iter`=5000.

Subset	Input	$L=2$	$L=4$	$L=8$	$L=16$	$L=32$	$L=64$
0% divergence	Random occurrences	92.9/96.5	87.6/96.0	80.1/95.0	66.8/94.1	50.1/92.2	50.1/84.8
	Mosaic	92.9/98.4	87.5/98.2	80.0/97.8	66.6/97.3	49.9/96.4	49.7/92.9
10% divergence	Random occurrences	93.0/88.1	87.7/86.5	80.3/83.3	67.1/80.4	50.5/74.9	50.5/52.7
	Degraded mosaic	91.6/97.1	82.8/96.5	68.9/95.1	47.3/92.9	26.1/88.5	21.2/72.3
Pooled	Random occurrences	93.0/92.9	87.7/92.0	80.2/90.0	67.0/88.2	50.4/84.6	50.4/70.2
	Mosaic and Degraded mosaic	91.7/97.4	83.1/96.9	69.7/95.7	48.8/94.0	27.7/90.5	23.0/77.4
	Spaced copy	97.3/99.3	92.9/99.1	82.2/98.7	62.6/98.5	35.4/97.9	15.4/90.6
	Minimizer-island comb	93.2/53.1	88.0/47.4	80.7/36.8	67.6/28.8	51.0/17.8	27.0/0.0

As we can observe the firing rate is high for most of the cases. This is different to Run_L predicate, because in this case we don't care if we follow different chains as the same time (this is purpose of U limit). We only care about the fact that at some point the chain will end and we won't be able to improve it. The boundary limits like D checking the distances to diagonal will kick in, but we can resonably expect that we will always have some anchor nearby. This means that U heuristic is very useful for forward chaining. My constructions became problematic only for very high L where it's very likely that some chain will improve anyway.

4.9.4. BioseqDB `max_drop` analysis

The last part to analyse is if the max_drop heuristic can be useful for backtracking 4.7. As reminder, the BioseqDB backtracking procedure follows predecessor links until it reaches the sentinel $p_j = -1$ or an anchor that was already assigned to an earlier chain. Minimap2 can stop a predecessor scan earlier when its running score falls sufficiently far below the best score seen on that scan.

Formally let f_i be minimap2's dynamic-programming score for the best chain ending at anchor i. The minimap2 finds an eligible chain end e, let B be the number of steps made

by the BioseqDB-style baseline scan, which stops only at $p_j = -1$ or at a previously marked anchor and let b_m be the boundary reached after predecessor step m, and define

$$s_m = \begin{cases} f_e - f_{b_m}, & b_m \geq 0, \\ f_e, & b_m = -1, \end{cases} \qquad M_m = \max(0, s_1, \ldots, s_m).$$

Define the score drop at step m as

$$d_m = M_m - s_m, \qquad \Delta(e) = \max_{1 \leq m \leq B} d_m.$$

Thus d_m is the distance from the best score already seen in this backtrack to the score at the current step. The value $\Delta(e)$ is the largest such drop in the scan from end e. Minimap2 predicate will want to terminate the execution when $d_m > D$.

I tried measuring how frequently minimap2 fires the predicate. Table 4.5 contains very dissapointing results. By default max drop treshold is $D = 500$, but we didn't even observed that. The max delta is always zero because the score always improves along the chain.

T (Mbp)	query length (bp)	B	$\Delta_{\max}$
1.125	247–252	25 705	0
1.125	997–1002	109 715	0
1.125	3997–4002	440 525	0
1.125	15997–16002	1 757 774	0
4.500	247–252	25 705	0
4.500	997–1002	109 715	0
4.500	3997–4002	440 525	0
4.500	15997–16002	1 757 741	0
18.000	247–252	25 670	0
18.000	997–1002	109 715	0
18.000	3997–4002	440 525	0
18.000	15997–16002	1 757 741	0

Table 4.5: Default-$D = 500$ target- and query-length screen

Thankfully the wide-gap construction can help in this case. The 19,980-base target contains the same parts in the same order, but we insert a random 500-base. Figure 4.9(e) shows this construction. This means that we can get lots of anchors that doesn't match to improve the score at all. The "requested substituiton" rate refers to substitutions in the separated target fragments.

For panel (a) 4.17 I calculate how many steps we perform compared to the place where we could terminate execution (lower percentage means that the heuristic won't save anything). I don't mark the anchors so we always follow chains right to the end. Panel (b) 4.17 demonstrates that when marking chains the panel (a) don't show full truth. Max drop will save us scanned anchors, but because we won't mark the chains the actual work is larger (later we need to scan the anchors).

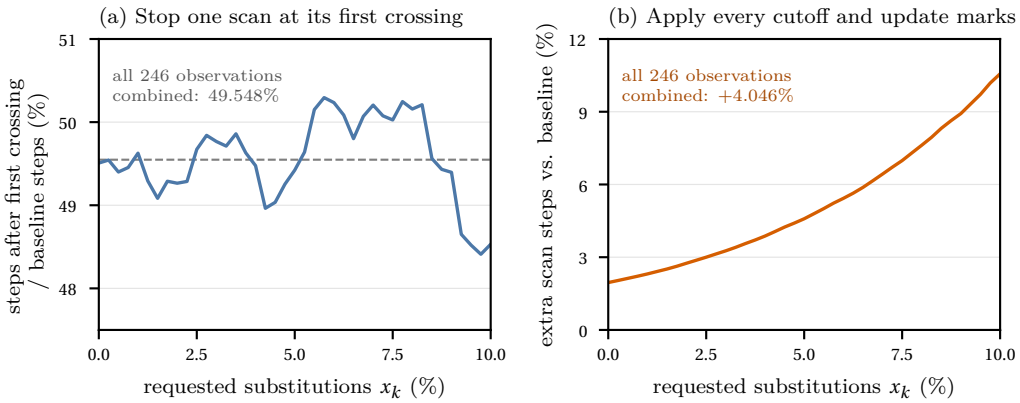


Figure 4.17: Scan-step accounting for Wide-gap spaced-copy at default $D = 500$. Its construction and exact dimensions are shown in Figure 4.9(e). Panel (a) counts the part of each fixed-start baseline scan after we reach max drop for first time. The dashed value is $\sum S / \sum B$ over all 246 observations. Panel (b) applies every cutoff and updates the marks, which increases the complete scan work. These are logical scan steps, not instructions, elapsed time, or a measured slowdown.

After consulting relevant work we see that max drop heuristic isn't used to reduce number of scanned anchors, but rather to prevent chaining unrelated fragments of sequences. The minimap2 will try to chain large gaps as unrelated fragments, which increases amount of work performed. This however is reported to increase matching accuracy Li, "Minimap2: pairwise alignment for nucleotide sequences"

In BioseqDB I didn't implement this heuristic in the SIMD algorithm at this point.

4.9.5. Conlusions

Final conlusions

From the experimental samples we can observe that in the random scenario:

- Run_L is beneficial to performance increase
- U limit is very beneficial to performance increase
- max_drop increases work done scalarily by minimap2, but according to literature increases quality of the alignment

Thus I decided to leave these heuristics in, while the max_drop can be implemented in the future (as observed on the random cases, it only even triggers in case of very large gaps, so it requires careful testing on real-world datasets as well as estimating how SIMD implementation will penalize the execution times)

4.10. BioseqDB SIMD performance comparison

I perfomed intermediate testing to verify if predictions coming from synthetic results can be correct.

The test used $|T| = 500Mbps$ and $|Q| = 5Mbps$ sequences with just one $N = 1$ occurence (this means that this is very optimistic case of random scenario)

TODO

Replace this section with final results or remove it entirely (the b panel is misleading and there's no interpretation of the results.). For anyone reading: this is just old test I found to illustrate performance so it's probably outdated and misleading because for backtracking there is single SIMD algorithm. Benchmarking framwork resuses labels but it's the same algorithm in panel B for B1/B5 and it makes no sense at all

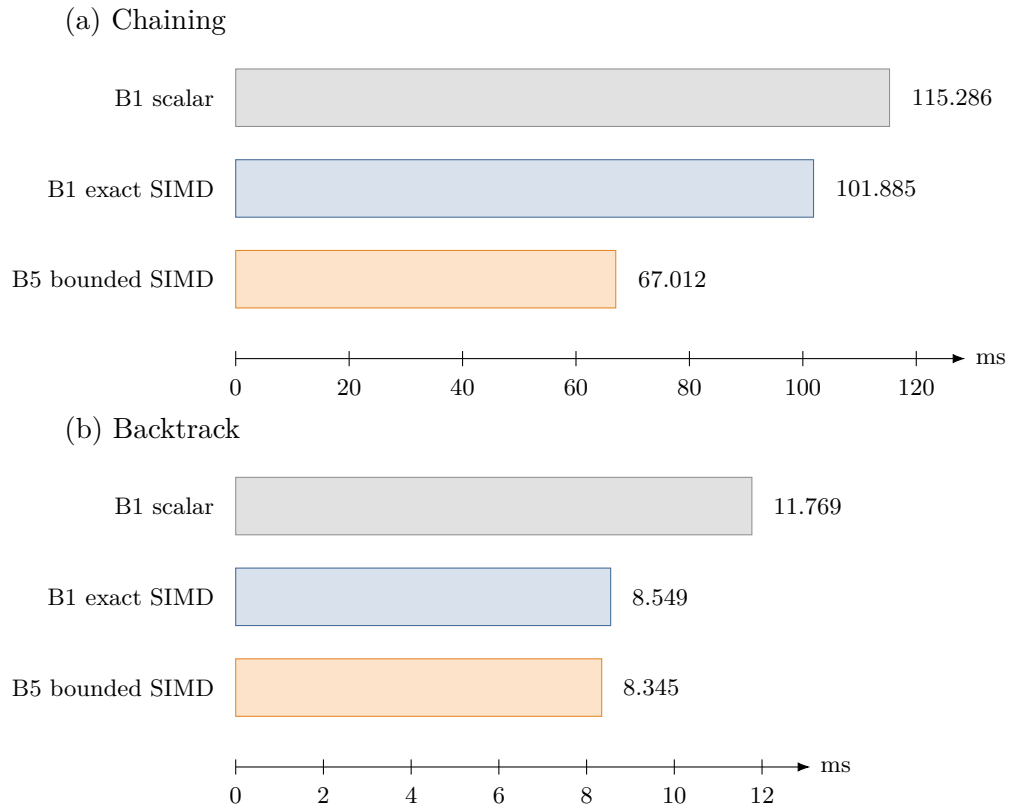


Figure 4.18: Recorded times for the three CPU modes

The figure above depicts only two runs and presents only quick prototyping test measurements to judge if we are going in right direction.

4.11. Range minimum query chaining

The chaining score model is described in Section 4.5.1 gives an ability to better express relationships between anchors we want to optimize. However it's more complex to calculate than simple linear scoring model. As noted in equation 4.8, we can compute GlobalScore(i)

independently for each anchor author of minimap2 did in minigraph paper Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph." The problem then is reduced to finding the maximum among all values in continous segment of anchors.

Range minimum query cp-algorithms contributors, Range Minimum Query (RMQ) algorithm is used to calculate the linear score $h(j)$ for anchor, while minimap2 later calculates actual score $f_j + E(j,i)$ (and if the RMQ result is not sufficiently "good", then it tries again with different parameters)

$$h(j) = f_j + \frac{\lambda_g}{2}(r_j + q_j). \tag{4.20}$$

4.11.1. Minimap2 RMQ

I didn't initially found papers going into details of minimap2 RMQ's implementation, so here I present detailed description of the algorithm. The minigraph paper Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph" describes only the high-level approach, without going into details of the implementation. Minimap2's RMQ is implemented using two AVL trees cp-algorithms contributors, Segment Tree. First of them is used to query anchors in the interval $0 < r_i - r_j \le D$. Each time minimap2 removes nodes from the tree if either they go outside of this interval or node count reaches C (the total limit of nodes per tree). The second tree has the same structure, but is used to query anchors in the interval $0 < r_i - r_j \le D_{\text{in}}$ with the intent to have $D_{\text{in}} \le D$ so if we get an anchor candidate that is not sufficiently "good", we can then perform another query on second tree, where we will get scores for smaller neighbourhood of an anchor. I will call this second lookup the "local scan". Minimap2 D_{in} can be set to 0 to disable second AVL tree entirely.

While the original anchor array is sorted by increasing reference coordinates r_i, the AVL trees order the node primarily by increasing query coordinate, then by increasing anchor index in the array (pairs (q_j, j)). As we want to prevent chaining anchors with the same reference coordinate, we will not add single anchor to the trees (a "reference group" is the continous sequence in the anchor array that has the same reference position, but different query positions). We will add entire reference group to the tree at once. This is implemented via InsertCompletedReferenceGroup(T, a, f, i) pseudocode function which checks if current anchor reference coordinate is different to the saved one.

The trees also need to remove anchors that go outside reference coordinates i.e $0 < r_i - r_j \le D_{\text{in}}, D$. This is implemented via RemoveExpiredEntries(T, a, i, D, C) pseudocode function. Such operation could be implemented by visiting each node in the entire tree, however internally minimap2 saves pointer (cursor) to the first anchor in the window and moves it when going to next anchor. This results in better reemoval theoretical complexity.

The last complex operation is local scan. After best candidate j^* is selected from the main tree T, we calculate it's score $c_{j^\star} = f_{j^\star} + E(j^\star, i)$ and accept it only when the anchor is "exact":

$$\delta(j^\star, i) = 0 \quad \wedge \quad g(j^\star, i) \le \ell_{j^\star}. \tag{4.21}$$

meaning that two anchors advcance equally and no more than k-mer span (as discussed earlier $k = \ell_{j^\star}$ for HPC disabled 3.12). If that's not the case, then minimap2 runs the local scan. The scan goes through all anchors in the query coordinate interval $q_i - D_{\text{in}} \le q_j \le q_i - 1$. This is implemented by first finding the first anchor in the tree with largest q_i value that is strictly less than q_j and then iterating candidates in decreasing order of q_j value (this is ordinary tree traversal). The scan itself is implemented via ScanLocalInterval$(T_{\text{in}}, i, D_{\text{in}}, B, S, f, p, t)$ pseudocode function and is very similar to forward chaining loop in minimap2 scalar algorithm 10. The only difference is that in the forward chaining loop without RMQ minimap2 iteration

was driven by *max_iter* limit for anchors not sorted by query coordinate, while in the RMQ scan we don't have *max_iter* limit (the minimap2 skip counter is used in the same way - it stops iteration on anchors that don't improve score several times in a row).

Theoretical complexity

In the previous forward chaining loop 10 we iterated over all anchors and bounded the number of iterations by *max_iter* limit. In the simplest calcualtion we can say that number of candidates is bounded by $\Theta(n)$ for one anchor, so entire pass does $\Theta(n^2)$ work. In case of RMQ each insertion and deletion (inserting reference groups and removing expired anchors) is amortized to $O(\log n)$ work for each anchor (each anchor can be added and removed at most once), so if the number of the candidates is the same, then overall work is bounded by $\Theta(n \log n + n^2)$. The RMQ is also inherently less friendlier to efficient pipelining and cache locality as we will need to jump across the tree frequently and it uses linear scoring model which can be less accurate. The benefit comes from the fact that RMQ can be more efficient with larger boundary limits, so it can be used for cases of long sequences with large insertions/deletions with lots of noise. Li, Feng, and Chu, "The design and construction of reference pangenome graphs with minigraph"

Figure 4.19 illustrates how RMQ chaining works.

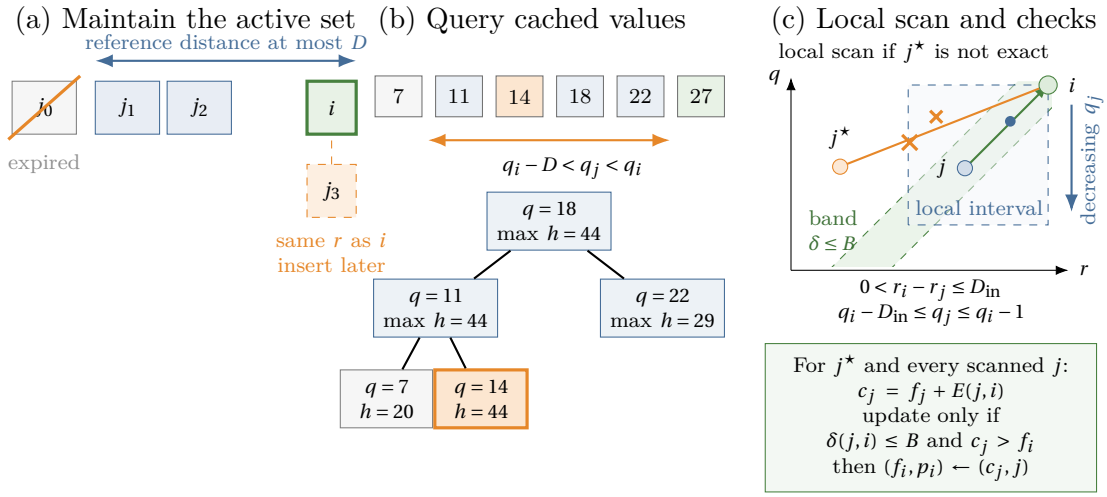


Figure 4.19: Minimap2 RMQ for one target. (a) We maintain the range of reference coordinates in the array to know where the window starts (what is the anchor with lowest reference coordinate that still fits in the $r_i - D$ window). (b) Select the best anchor according to linear scoring model$j^\star$; "max h" is the largest value in a node's subtree. (c) If the selected anchor $j^\star$ is not exact (Equation 4.21), then we run the local scan. The blue rectangle is the local interval the green band satisfies $\delta \le B$, orange crosses mark rejected candidates, including one inside the local interval. A scanned link (blue dot) replaces the current result only if both checks below pass. The local scan can stop early and do not iterate over all anchors in the local interval if for example skip counter S is reached.

Algorithm 15 shows how RMQ forward chaining is implemented inside minimap2. RangeBest function returns the anchor with the largest $h(j)$ in the interval, or $\perp$ if the interval contains

no retained anchor, while CompleteLink function calculates $E(j,i)$ from Equations 4.4–4.5, together with δ and the exact (local scan) condition in Equation 4.21.

Algorithm 15 Minimap2 RMQ forward chaining

Require: sorted anchors $a_0,\ldots,a_{n-1}$, distance D, inner distance D_{in}, bandwidth B, tree cap C, skip limit S
Ensure: scores f and predecessor indices p

```
function Minimap2RMQForward(a, D, D_in, B, C, S)
    D ← max(D, B),   D_in ← min(D, max(0, D_in))
    T ← ∅,   T_in ← ∅   augmented AVL trees
    t ← (0, ..., 0)   B1 marks for the optional local scan
    for i ← 0 to n − 1 do
        InsertCompletedReferenceGroup(T, a, f, i)
        if D_in > 0 then
            InsertCompletedReferenceGroup(T_in, a, f, i)
        RemoveExpiredEntries(T, a, i, D, C)
        if D_in > 0 then
            RemoveExpiredEntries(T_in, a, i, D_in, C)
        f_i ← ℓ_i,   p_i ← −1
        j* ← RangeBest(T, (q_i − D, INT32_MAX), (q_i, 0))
        if j* ≠ ⊥ then
            (e, δ, exact) ← CompleteLink(j*, i)   e = E(j*, i)
            if δ ≤ B ∧ f_j* + e > f_i then
                f_i ← f_j* + e,   p_i ← j*
            if ¬exact ∧ T_in ≠ ∅ ∧ q_i > 0 then
                (f_i, p_i) ← ScanLocalInterval(T_in, i, D_in, B, S, f, p, t)   Algorithm 16
    return f, p
```

Algorithm 16 shows how local scan is implemented. I didn't want to show other tree maintenance operations as there aren't many valuable details to discuss (this is usual AVL tree maintenance).

Similarily to minimap2 forward chaining we use t_j array to mark visited candidates i.e mark $t_j = i$ means that a candidate already tested for target i points to j through its saved predecessor. It must be noted that the for loop in the local scan algorithm 16 utilises the fact that anchors are sorted by query coordinate so we can just go to the leftmost node in the interval and iterate each leaf from left to right (I'm reiterating this to showcase that the for loop hides this complexity in hte pseudocode).

Algorithm 16 Minimap2 local scan after the primary RMQ query

Require: maintained inner tree T_{in}, target i, local limit D_{in}, bandwidth B, skip limit S

Require: completed earlier scores and predecessors f, p, marks t, and current result (f_i, p_i)
Ensure: updated (f_i, p_i) and marks t

```
function ScanLocalInterval(T_in, i, D_in, B, S, f, p, t)
    s ← 0
    for retained j ∈ T_in with q_i − D_in ≤ q_j ≤ q_i − 1, by decreasing (q_j, j) do
        c ← f_j + E(j, i),   δ ← |d_r(j, i) − d_q(j, i)|
        if δ ≤ B then
            if c > f_i then
                f_i ← c,   p_i ← j,   s ← max(0, s − 1)
            else if t_j = i then
                s ← s + 1
            if s > S then
                break
            if p_j ≥ 0 then
                t_{p_j} ← i
    return (f_i, p_i)
```

4.11.2. RMQ re-chaining in the pipeline

Normally Minimaps2 disables RMQ for chaining by default, the manual recommends`--rmq` for alignment through long insertions and deletions Li, Minimap2 manual, but at the same time RMQ is cheaper so minimap2 can use it to perform "re-chaining".

RMQ re-chaining

RMQ re-chaining means running chaining again with a larger bandwidth to try to connect chains across long gaps. This is optional step that runs in minimap2 after the forward chaining and chaining backtracking - after step 3 5. Minimap2 uses array A which is sorted by radix sort by reference coordinates again and provided to RMQ chaining algorithm. Please note that this means that minimap2 does some duplicate work - first it groups anchors by the chain they belong to (placing them in A array), but rechaining sorts anchors again. Minimap2 also discards f and p arrays from first chaining step. RMQ re-chaining will then work on the subset of original input anchors from seeding (not every anchor will form a chain that will be accepted by its length and score limits 13). As RMQ re-chaining is special case not enabled by default in minimap2, it wasn't in scope of this thesis - I didn't measure how many anchors are retained and how RMQ re-chaining affects quality of alignments and performance.

BioseqDB does not support re-chaining

I don't provide any specialized stage in pipeline for re-chaining. Some of the software in literature support increasingly precise queries to improve alignments iterativelyKallenborn et al., "GPU-accelerated homology search with MMseqs2." This can be implemented in the future.

Normally in minimap2 RMQ re-chaining will be only enabled iff all conditions are satisfied:

- RMQ re-chaining is enabled only for specific presets (assembly presets), so user have to select specific preset or use `--rmq` flag explicitly

- There's more than one chain (easy to satisfy)
- There's only 1 query in the mapping call
- Either query length minus span of first chain is greater than 1000 bases or span of first chain is greater than 10% of query length (very long insertions and deletions)

The conditions above were created by me after inspection of source code, so the list might not be complete, but it illustrates why re-chaining wasn't first priority for me.

4.11.3. RMQ in BioseqDB

In BioseqDB I implemented the same algorithm as minimap2 with some differences:

Table 4.6: Differences between minimap2 and BioseqDB RMQ

	Minimap2	BioseqDB
Local scan	Optional scan with complete scores and the B1 skip rule	Local scan is implemented by disabled by default by setting $D_{\text{in}} = 0$
Query interval	Composite bounds in Algorithm 15	Includes both $q_i - D$ and q_i
Bandwidth check	Additionally rejects anchor when $\delta > B$	Does not check that
RMQ re-chaining	At most one extra forward run and backtracking step with a larger bandwidth	No any re-chaining stage is implemented
Backtracking	Uses the minimap2 backtracking algorithm 13	Performs scalar BioseqDB backtracking which is similar algorithm to minimap2, however we could potentially use SIMD backtracking (backtracking isn't separate pipeline stage, so SIMD chaining selects both forward/backtrack in SIMD, while RMQ has it's own scalar backtracking)

⚠ Other algorithmic limitations

There are some other limitations:

- Including $q_j = q_i$ allows a link with no query advance. BioseqDB searches from $(q_i - D, \text{INT64_MIN})$ to $(q_i, \text{INT64_MAX})$, so every anchor at either query boundary is eligible for selection (it should have minor impact on performance)
- Omitting the bandwidth reject i.e $\delta > B$ allows a link farther from the diagonal than the configured B (this was ommited due to simpler code and RMQ having lesser priority than oder algorithms)

I didn't measure effects of these limtations neither on performance nor on quality of alignments.

Future work

The backtracking for RMQ can be removed and I could use SIMD backtracking. This would require separating forward/backward chaining into separate pipeline stages. Moreover in the future I can use specialized GPU acceleartion for RMQ (as noted before G3SA contains kernels for RMQ which weren't used). There exists some liteature Kreis et al., GPU-RMQ: Accelerating Range Minimum Queries on Modern GPUs discussing GPU acceleration for RMQ problems using ray tracing and CUDA cores (one limitation is that in the paper the RMQ base array is static, so it would require additional design work to design GPU-friendly algorithm).

4.12. GPU chaining

4.12.1. G3SA algorithm

The G3SA paper divides GPU kernel into 4 stages:

1. First forward-scoring pass: Either use short kernel (specialized for short sequences) or tiled kernel (tiled kernel is just dividing sequence into smaller chunks and applying modified short kernel). This stage computes total scores as normal chaining $f_j + E(j,i)$.

2. First backtracking and packing: G3SA applies minimap2's cut and marking rule, reverses accepted paths, and packs their anchors into a buffer.

3. Conditional ranking: If the difference between sizes of query and target is big (see Figure 4.20), then kernel compares simplified$h(j)$ and calculates one complete link score for the best canidate (similar to how minimap2 does RMQ rescoring mentioned earlier)

4. Final backtracking: This is done via minimap2 scalar backtracking algorithm

G3SA AVL code

The repository contains CUDA AVL support code and a commented experimental entry point. The active chaining pipeline does not call that entry point. Its second pass uses array-based short and tiled kernels.

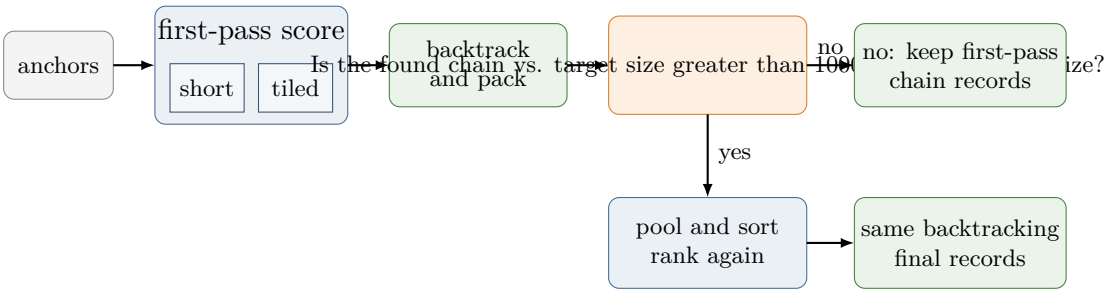


Figure 4.20: Simplified depictions of G3SA pipeline

G3SA chaining pipeline
chaining_kernel_wrapper 775

Commented CUDA AVL entry point
gpu_chain_rmq 406

The main difference between G3SA is that it uses specialised paths depending on data size and it calculates rows instead of columns in the score matrix for anchors: the first kernel instead of looking at previous anchors and updating itself, looks at next anchors and updates them 4.21.

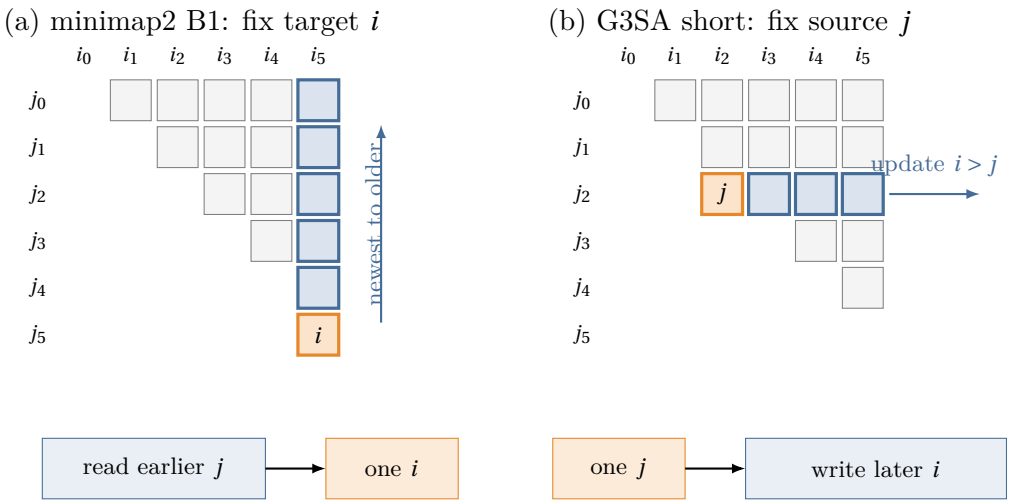


Figure 4.21: Different approaches (transposed) of minimap2 and G3SA

After long analysis I decided to implement the simpler approach. Instead of writing multiple kernels, just naively port existing chaining logic to Vulkan API.

G3SA short ranking kernel
mm_compute_chain_short_rmq 970

G3SA ranking-pass launches
chaining_kernel_wrapper_rmq 595

4.12.2. BioseqDB Vulkan tiled DP

BioseqDB's `vulkan_tiled_dynamic_programming` is the usual chaining algorithm that fixes one anchor i and examines earlier predecessors in batches of 32.

The host first sets

$$D = \max\{\texttt{max_gap}, \texttt{bandwidth}\}.$$

and sorts anchors to filter them by adjacent reference gap greater than D. For a fixed target, lane u first scores predecessor $i-u-1$. The next iteration moves all lanes 32 positions to the back. Every active lane calculates the complete value $f_j + E(j,i)$ to then select one pair (score, j).

Algorithm 17 BioseqDB Vulkan forward chaining for one hard range

Require: sorted range $a_b, \ldots, a_{e-1}$, distance D, scan limit K, skip parameter S
Ensure: scores f and absolute predecessor indices p

$U \leftarrow \infty$ if $S \le 0$, otherwise $U \leftarrow \lfloor S/32 \rfloor + 1$
for $i \leftarrow b$ to $e-1$ do
  $f_i \leftarrow \ell_i, \quad p_i \leftarrow -1$
  $o \leftarrow 1, \quad u \leftarrow 0$
  while $o \le K \wedge i - o \ge b$ do
    $B \leftarrow$ the longest prefix of $(i-o, \ldots, i-o-31)$ with $j \ge b$, $i - j \le K$, and $r_i - r_j \le D$
    if $B = \varnothing$ then
      break
    for all $j \in B$ in 32 lanes do
      $c_j \leftarrow f_j + E(j, i)$ invalid links use $-\infty$
    $j^\star \leftarrow \arg\max_{j \in B}(c_j, j)$ larger j breaks a score tie
    $c \leftarrow c_{j^\star}$
    if $c > f_i$ then
      $f_i \leftarrow c, \quad p_i \leftarrow j^\star, \quad u \leftarrow 0$
    else if $S > 0$ then
      $u \leftarrow u + 1$
      if $u \ge U$ then
        break
    if $|B| < 32$ then
      break
    $o \leftarrow o + 32$
  await(targetUpdates$_i$) make f_i, p_i visible before target $i+1$

4.13. Selecting primary and secondary chains

After chains are collected, we must note that several chains can naturally match the same part of a query in differents parts of the target. Sometimes there can be significant overlap between better chain and slightly worse one. For this reason minimap2 needs to classify chains into two groups: primary chains and secondary chains and it reports all primary chains but limit reported secondary chains to some set runtime limit.[1] Minimap considers chains from the highest score to the lowest:

- Chains are classified either as primary (best one) or secondary
- Specific chain becomes secondary when there's existing primary that covers enough of the same query interval
- Otherise, chain is classified as primary

BioseqDB uses the same classification logic as minimap2. Below, in Algorithm 18, I show how this classification is formally done:

- c is a chain (iterated by decreasing score)
- $I_c = [q_c^{\text{start}}, q_c^{\text{end}})$ is the query interval of chain c (start of first anchor and end of last anchor). We will intersect these intervals to calcualte how many positions chain covers in the query.
- s_c is chain score
- The value u counts symbols (bases) not covered by any accepted primary chain.
- Previously p was specifically parent of an anchor. In this case it represents primary chain.
- $\pi(c)$ is the parent chain of c (I defined previously parents for anchors, but this is extended notation to represent parent of first anchor in chain)
- Subtracting this contribution makes a chain with substantial new coverage less likely to become secondary.
- The default overlap threshold is $\lambda = 0.5$.
- The uncovered-length limit is $L = 2^{31} - 1$, so it normally adds no restriction.

Let $I_c = [q_c^{\text{start}}, q_c^{\text{end}})$ be the query interval of chain c, and s_c its score. Below, in Algorithm 18, I show how classification is formally done:$|I|$ counts query bases. The value u counts bases not covered by any accepted primary. Subtracting this contribution makes a chain with substantial new coverage less likely to become secondary. The default overlap threshold is $\lambda = 0.5$. The uncovered-length limit is $L = 2^{31} - 1$, so it normally adds no restriction.

[1] Stopa, MMseqs2 Github issue 323.

Algorithm 18 Minimap2 primary and secondary chain selection

Require: chains C, query intervals I_c, scores s_c, limits λ, L
Ensure: ordered primary list P and parent $\pi(c)$ for every chain

$P \leftarrow []$
for $c \in C$ in decreasing order of s_c do
    $u \leftarrow \left|I_c \setminus \bigcup_{p \in P} I_p\right|, \quad \pi(c) \leftarrow c$ initially primary
    for $p \in P$ in acceptance order do
        $o \leftarrow |I_c \cap I_p|$
        $a \leftarrow \min(|I_c|, |I_p|), \quad b \leftarrow \max(|I_c|, |I_p|)$
        if $o > 0 \wedge o/a - u/b > \lambda \wedge u \le L$ then
            $\pi(c) \leftarrow p$ secondary alternative to p
            break
    if $\pi(c) = c$ then
        append c to P

A separate filter retains all primaries and limits secondary alternatives. With the defaults, a secondary must score at least $0.8 s_{\pi(c)}$ or be within $2k$ of its parent's score, where k is the seed length. Exact duplicates of the parent's coordinates are removed, and at most five secondaries survive across the list. BioseqDB follows these selection steps and repeats them after base alignment refines the regions (Section 5.3).

Primary chains should not be confused with primary SAM records. Minimap2 marks one surviving nonsecondary alignment as the primary SAM record. The other nonsecondary parts of a split mapping are supplementary. Secondary records instead describe alternative mappings of the same query region.

Chapter 5

Final alignment and postprocessing

5.1. Base-level alignment

In this short chapter I want to discuss last parts of the pipeline's algorithms. The first one is the base-level alignment. Chaining will tell us approximetely where two sequences match, but it isn't sufficient to tell which exact symbols correspond to which. The base-level alignment performs the dynamic programming specified in the introductory sections 2.2 as now instead of matching entire target, we can just process target subsequence of size similar to the query size.

Conditional CIGAR execution

BioseqDB doesn't call base-level alignment if no `bioseq.cigar()` is requested 2.2, so then `bioseq.score()` only reflects the chaining score. If `bioseq.cigar()` call is present in the SQL query, then score represents results of the final alignment (thus score can differ depending on the query). This is explained in the walkthrough appendix 1.

In BioseqDB I used existing implementation of KSW2 (the algorithm by the author of minimap2 that uses AVX SIMD C intrinsics to implement DP calculations) from minimap2 and Block Aligner Li, "Minimap2: pairwise alignment for nucleotide sequences"; Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." The DP formulations follow what was described already easlier 2.2, where KSW2 provides various flavours of functions in its API primarily left/right extension and local mode 2.1. KSW2 uses banded alignment where the maximum distances are fixed (for more details on that see chaining for maximum diagonal distance (bandwith) 4.1) The block aligner is more complex and uses moving square blocks to adaptively set this size of "bandwith", where we compute scores in the score matrix.

Table 5.1: CPU aligners incorporated into BioseqDB.

	KSW2	Block Aligner
DP matrix where scores are calculated	Fixed diagonal band	Moving square that grows and shrinks repeatedly
Gap cost function (single-affine/dual-affine)	$\min(q_1 + e_1\ell, q_2 + e_2\ell)$	$q_1 + e_1\ell$
Special stop conditions (see below)	Z-drop	X-drop after two consecutive low-scoring steps
Parameters that affect sizes of searched regions	"bandwith" is the maximum distance from the diagonal for the diagonal band	Block size, which is the initial size of the square

I previously introduced X-drop and Z-drop concepts 2.2, but formally they can be formulated as follows. Let's say $S^\star$ is the best score reached at $(i^\star, j^\star)$ and we test this best score against the current score S at (i, j) (meaning that we are matching ith symbol of target vs. jth symbol of query):

$$\begin{aligned} &\text{X-drop:} \quad S^\star - S > X, \\ &\text{Z-drop:} \quad S^\star - S > Z + e_g \left|(i - i^\star) - (j - j^\star)\right|. \end{aligned} \tag{5.1}$$

e_g here is gap-extension penalty, which in this context means that we can disard all symbols from the best score to the current position by introducing a gap. This means that effectively we just stop earlier if the distance from the diagonal gets to large, so practically Z-drop is more restrictive for correct setup.

5.2. How Block Aligner follows the alignment

Block Aligner moves a $B \times B$ square through the scoring matrix, and internally compute strips of width $S = 8$ visible on the figure 5.1 below. When the algorithm fails to improve best score for number of steps, then it decides to grow the block as seen on image below 5.1

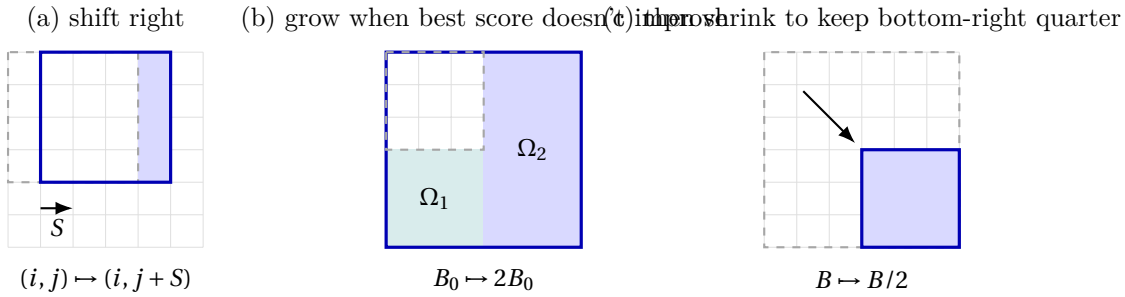


Figure 5.1: Illustration of 3 steps that Block Aligner repeatedly performs. Growth computes the lower rectangle Ω_1 before the right rectangle Ω_2.

The pseudocode below is directly taken from Block Aligner paper Liu and Steinegger, "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." For more details please consult the projects README Liu, Block Aligner: official project README. I use here original paper notation, where q is the query and p the target sequence aligner to. $G^c_{open}, G^c_{close}, G^r_{open}, G_{ext}$ are the gap penalties. Function COMPUTE_RECT$((r, c), (w, h))$ is used to compute all the scores in the given rectangle given as parameters, using SIMD version of regular DP scoring formulation.

The important thing in the paper is: $[r, r + h) \times [c, c + w)$; $Y = \lfloor B/S \rfloor - 1$ which defines threshold, when block assumes the score cannot improve and decides to grow. Note that here S was used by paper's authors as the width of the strip, not the actual score computed.

Algorithm 19 Pseudocode for the Block Aligner algorithm.

```
procedure ALIGN(q, p, X, B_min, B_max, G^c_open, G^c_close, G^r_open, G_ext)
    S ← 8, max_score ← 0, dir ← GROW, B ← B_min, B_prev ← 0, i ← 0, j ← 0
    Initialize D, D', C, R, and the trace array
    while true do  One step of ligner. Note that i, j, B, and dir have already been updated.
        if dir = RIGHT then  Block position (i, j − S) ↦ (i, j)
            curr_score ← COMPUTE_RECT((i, j + B − S), (S, B))
        else if dir = DOWN then  Block position (i − S, j) ↦ (i, j)
            curr_score ← COMPUTE_RECT((i + B − S, j), (B, S))
        else if dir = GROW then
            curr_score ← COMPUTE_RECT((i + B_prev, j), (B_prev, B − B_prev))  Grow down
            curr_score ← COMPUTE_RECT((i, j + B_prev), (B − B_prev, B))  Grow right
        end if

        if curr_score > max_score then
            Update max_score with curr_score and its store its position
            Save i, j, and the block right and bottom border scores as checkpoint
        end if

        if curr_score < max_score − X for 2 consecutive steps, then break  X-drop
        if i + B > |q| and j + B > |p|, then break  Reached end of DP matrix
        if i + B > |q| or j + B > |p|, then shift i, j by S to avoid out of bounds and continue

        if B < B_max and Y steps pass without improving max_score then  Block grow criteria
            B_prev = B, B ← 2B, dir ← GROW
            Restore i, j, and block border scores from previous checkpoint
            continue
        end if

        if B > B_min and max_score is in the bottom-right corner region of block then  Block shrink criteria
            B ← B/2, i ← i + B, j ← j + B
            Save i, j, and the block right and bottom border scores as checkpoint
        end if

        dir ← RIGHT or DOWN based on comparing right and bottom border cells
        Increment i, j by S based on dir
    end while
    return the trace array, and if X-drop, then max_score, else D[|q|][|p|]
end procedure
```

5.3. Postprocessing

After the alignment is done, BioseqDB computes CIGAR string, extracts all values returned by the final aligner (like exact coordinates, score etc.), and also calculates some additional values (if they are requested by the user in query):

- Divergence Matching minimizers give an estimate of the fraction of differing bases
- Mapping quality (MAPQ) This estimates how subjectively good are the mapping locations (are we confident enough that this is a match)

Minimap defines diverges exactly as mentioned before in Section 3.9.2:

$$\hat{\epsilon} = \frac{1}{k} \ln \frac{n}{m}, \qquad \hat{d} = 1 - \exp(-\hat{\epsilon}) = 1 - \left(\frac{m}{n}\right)^{1/k}. \tag{5.2}$$

where n is the number of minimizers that are part of chains and m is the number of all query minimizers between first match and last match BioseqDB similarily to what minimap2 does, returns $\widehat{d}$, which is approximation of $\widehat{\epsilon}$ assuming low sequence divergence. To obtain n in BioseqDB I uise cumulative counts of minimizers by position, so subtracting two counts gives the number in an interval without sorting the emitted minimizers (this avoids cirtical problem I found earlier, which made this step thousands of time slower - Section 3.9.2).

For MAPQ, if two reference locations give nearly equal alignment scores, BioseqDB assigns low MAPQ, because both alignments may match well and we cannot be confident enough that this location is the correct one.

The maximum MAPQ in minimap2 is 60 and it's similar to usual phred score scale used in base calling (for example in SAM formats), where $P = 10^{-Q/10}$[1] equal to 60 means 1 in million chance for error.

BioseqDB uses a direct port of minimap2's heuristic.[2] The equation below assumes positive DP scores for the primary chain and a competing secondary chain.

$$\mathrm{MAPQ} \approx \mathrm{clip}_{C_{\min},C_{\max}}\Big(\min\Big\{C_1 I p(1-\rho^{C_{\mathrm{power}}})\ln\frac{D_{\mathrm{pri}}}{a},\, C_2 I^{C_{\mathrm{power}}}\frac{D_{\mathrm{pri}}-D_{\mathrm{sec}}}{a}\Big\} - C_3\ln(N+1)\Big). \tag{5.3}$$

- D_{pri}: best DP score returned by the aligner for the primary chain.
- D_{sec}: highest DP score returned by the aligner among secondary chains competing with the primary chain for an overlapping region of the query.
- a: reward for one matching base.
- I: matching bases divided by alignment block length.
- p: support penalty between zero and 1, reduced by weak chains, few seeds or repetitive query regions.
- $\rho = (D_{\mathrm{sec}}/D_{\mathrm{pri}})\max(s_{\mathrm{alt}}, s_{\min})/s_{\mathrm{orig}}$: ratio combining alignment and chaining scores. Here s_{alt} is the strongest secondary chaining score, $s_{\min}$ is the minimum chaining-score threshold, and s_{orig} is the primary mapping's original chaining score before later adjustments.
- N: number of competitive secondary mappings.
- $\ln$: natural logarithm.
- $\mathrm{clip}_{C_{\min},C_{\max}}$: leaves values inside $[C_{\min}, C_{\max}]$ unchanged and replaces values outside this interval with the nearest endpoint.
- $C_1 = 40,\ C_2 = 6.02,\ C_3 = 4.343,\ C_{\mathrm{power}} = 2,\ C_{\min} = 0,\ C_{\max} = 60.$

BioseqDB applies rounding offsets and converts intermediate values to integers by discarding their fractional parts. Primary mappings receive at least 1 if $D_{\mathrm{pri}} > D_{\mathrm{sec}}$. Secondary mappings receive zero.

[1] SAM/BAM Format Specification Working Group, Sequence Alignment/Map Format Specification.

[2] Li, minimap2: mapping-quality implementation in `hit.c`.

5.4. SneakySnake prefiltering

We discussed the postprocessing step, but there's also corresponding preprocessing step in the pipeline. I didn't separate it into its own chapter, because the feature is exerimental and uses external dependency.

A prefilter is specialized algorithm to reject sequence quickly before doing any alignment work. In BioseqDB I decided to use SneakySnake algorithm directly. Good prefilter should offer the following properties:

- It should quickly evaluate query and sequence parts and compute estimate of edit distance.
- The estimated edit distance should be lower than actual edit distance to prevent situation in which prefilter rejects valid candidates.

In BioseqDB I used SneakySnake Alser et al., "SneakySnake: a fast and accurate universal genome pre-alignment filter for CPUs, GPUs and FPGAs"; SAFARI Research Group, SneakySnake: official project README, where the core idea is to translate approximate string matching problem into a chip maze problem (formally SNR - Single Net Routing). I won't describe the algorithm in detail here, as I only use SneakySnake as a dependency and it wasn't evalauted in the final benchmarks.

The prefilter can be evaluated if users provides special predicate 7.3.6. In that case BioseqDB for case where the sequence is filtered-out won't fetch the minimizers or recompute them meaning the rest of the pipeline will be entirely skipped (to be precise the order of the filter evaluation is as follows: non-sequence user filters, then prefilter):

Listing:

```
SELECT pk, bioseq.edit(seq0) AS edits
FROM tutorial_mmsearch
WHERE public.mmsearch(seq0, $1::bioseq.seq('dna4', 64))
  AND bioseq.edit(seq0) < 5
ORDER BY pk;
```

The SNR problem is constructed as follows: For input query/target $Q, R \in \Sigma^m$ and given maximum edit distance E, SneakySnake constructs a matrix $Z \in \{0,1\}^{(2E+1)\times m}$ The idea is that the matrix covers one sequence (columns), while the rows cover only $2E+1$ relative positions in the second sequence. We then compare each position in the query and in target and if they match we put 0, otherwise we put 1 in the Z matrix. Now Z matrix represents "obstacles" for 1s and empty spaces for 0s. The SNR problem needs to find a path from left to right (possibly going up and down and to the left, but never to the right), where path is better if it contains less obstacles.

The benefit here is that traverse of the matrix is $O((2E+1) * m)$, which for low E can be very fast. You must note however that I directly plugged SneakySnake into BioseqDB lazy-sequence adapters without proper benchmarking, which means that I don't know how matching quality/performance is affected by calling this prefilter.

Let's define Z matrix formally. First index i is up to $2E+1$ and second index j is position in R: $1 \le j \le m$:

$$\delta_i = \begin{cases} -i, & 1 \le i \le E, \\ 0, & i = E+1, \\ i-E-1, & E+1 < i \le 2E+1. \end{cases} \qquad Z_{i,j} = \begin{cases} 0, & 1 \le j+\delta_i \le m \ \wedge \ Q_{j+\delta_i} = R_j, \\ 1, & \text{otherwise.} \end{cases}$$

Here, δ_i just formally defines relative lookups i.e if we look to the left of position in R up to E positions or to the right up to E positions. In paper authors want to look at the left side first, for example $(-1,-2,-3,0,+1,+2,+3)$, so δ_i looks more complicated than just $i-E$.

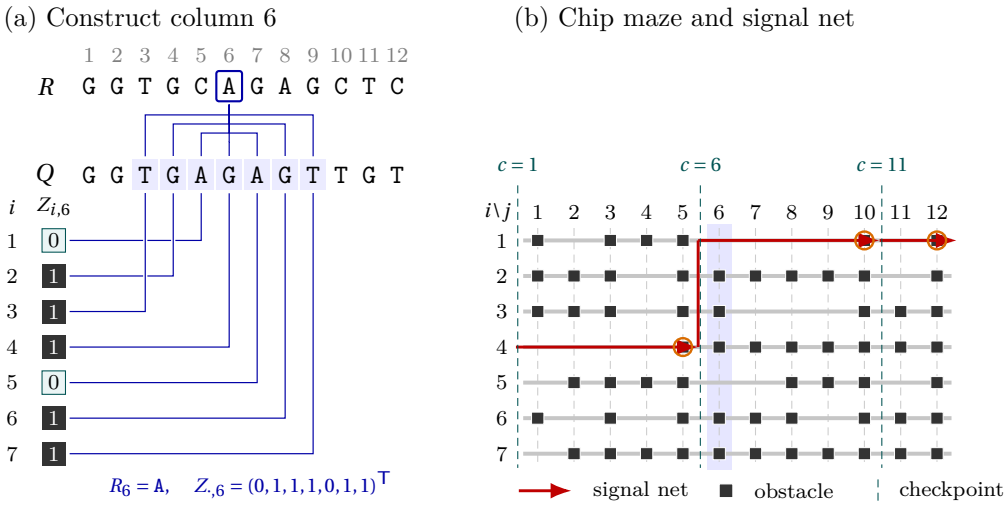


Figure 5.2: Example similar to original figure from paper: Fig. 2(a,b) in Alser et al. (2020) for R = GGTGCAGAGCTC, Q = GGTGAGAGTTGT, and $E = 3$. (a) Comparing R_6 with $Q_3,\ldots,Q_9$ constructs column 6 (blue in (b)). The first three results follow this δ_i reversed order.. (b) The net (best path) crosses three circled obstacles, so the pair passes the filter. The SneakySnake algorithm uses special notion of "checkpoints" so we don't perform path updates until obstacle is met after last checkpoint.

Chapter 6

Sequence representations and lazy evaluation

6.1. Designing sequence abstraction

Suppose user stores large genome sequence in one row, but only wants to align specific small part of it (let's say we know location of one protein-coding human gene and are interested only in finding occurences of that gene). It would be very wasteful if I didn't provide built-in functionality to load only that small region. This region could be stored on disk, so we only have it's locator poiting to a specific chunk in the table.

Physical storage

"Locator" refers to a small structure holding sequence metadata and pointer to its actal location in the storage system. More on is explained in physcial storage chapter 8.1. For now, I assume it's only an opaque pointer to sequence and use term "hydration" describing the process of converting it into a sequence itself or its fragment (as continous byte array in RAM).

On the other hand, we can load sequences from fasta file, where parser can be writing to RAM, so our sequence could refer to location in memory, where all sequences are pooled into one memory region to avoid constant allocation/reallocation. In yet another case, in tests we would like to use standard library containers to prepare and examine sequences (potentially even somebody could directly call some internal BioseqDB functions as standalone alignment library). This created a design challenge: same sequence can arrive as relationship locator, region in memory pool in RAM, can be provided as a whole as standrad container or as only a fragment (if user wants to operate only on specific slice or if it does not fit in RAM or RAM is limited, so we cannot fit entire sequence directly there), while these fragments should also be shareable, so parallel alignment to the same sequence doesn't have to load them multiple times.

There are several existing libraries for interfacing with genomic sequences. SeqAn[1] is a C++ library that provides modern interface for composing efficient sequence-analysis software from generic C++ components. Its library includes sequence representations, indexes and configurable alignment algorithms, with SIMD alignment, but it has it's own limitations.

[1] Reinert et al., "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers"; SeqAn Project, SeqAn Manual.

However SeqAn SIMD algorithms are mostly limited to auto-vectoriztion provided by the compiler (which doesn't have to be portable in most cases) and offers very narrow set of features for custom pipelines. I would be hard to directly reuse it for implementation of mentioned feautres. Using SeqAn would require deep integration, that's why I decided to address the design challenges by designing own C++ system of templates with different specializations driven by C++ concepts (inspired by nice-looking HPC MPI code Uhl, Schimek, and Brommer, Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface).

SeqAn compatibility

Because the architecture of BioseqDB pipeline is very custom, I decided to only use SeqAn's alphabet layer[a] and utilities exposed by its public API. This means that libraries using SeqAn code as base, could be slightly adjusted to be able to work with BioseqDB pipelines.

[a] SeqAn Project, Alphabets in SeqAn.

Table 6.1 lists every specialization of all BioseqDB sequence representations, to showcase how they are used in different scenarios. SeqAn offers abstractions over lazy sequences as well SeqAn Project, SeqAn Tutorial: Sequences In-Depth, but they're not as sophisticated as in my case. The lazy interface is really important for BioseqDB to improve it's performance utilising late-materialization strategies similar to those described in literature[3]

Table 6.1: Iterfaces for sequence representations in BioSeqDB.

Template	Specialization, and when it is selected	Description and usage context
`compact_sequence_range<R>` This is the core type representing sequence that resides completely in memory. Template allows only four available specializations depending on what `R` is and each of them have entirely separate handling logic.	`R` is a container of alphabet values, such as `std::vector<dna4>`	This is only adapter for existing sequence containers to provide compatibility with SeqAn and standard library containers. There is no alphabet packing and the type borrows `const R&` Adapter just provides common sequence interface. Adapting one costs $O(1)$ as we proxy all operations to the underlying container
	`R` is a packing configuration `sequence_custom_alphabet<A, U,b>`	Bitpacked vector that stores sequences according to alphabet rules passed as type paramter. Owns storage. Can convert to and from other bitpacked or not-bitpacked sequences, but if constructed from non-bitpacked sequence, then constructor costs $O(N)$
	`R` is an SeqAn alphabet type, as in `compact_sequence_range<dna4>`	The same as `sequence_custom_alphabet<A,U,b>`, but has sensible defaults for used underlying alphabet's storage unit width etc. Commonly used in tests. Code interfacing directly with BioseqDB C++ API should use this specialization.

[3] Chernishev et al., "A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store."

Template	Specialization, and when it is selected	Description and usage context
	`R` is `compact_sequence_stdc_instance<...>`	Holds only reference to the underlying object. Adapter to convert `compact_sequence_stdc_instance<...>` to `compact_sequence_range<R>`. `compact_sequence_stdc_instance<...>` is a type that comes from `compact_sequence_stdc<...>` - described below Used internally
`compact_sequence_stdc<...>` Represents standard layout C type cppreference.com contributors, C++ named requirements: StandardLayoutType that is used to type-erase `compact_sequence_range<R>` types i.e allow user to have one type for storing possibly multiple different sequences types. Metadata like alphabet type and storage unit width are stored inside the struct and specialized template dispatch lambda is called to convert it back to the `compact_sequence_range<R>` type. This is heavily used by Postgres code to store sequences as C structures.		Owns everything and stores fixed header that records struct (self) size in bytes (compatible with Postgres varlena), alphabet code, unit width, padding, and symbol count Used heavy by Postgres code as we cannot store template instances directly and need to hold them in one object that converts type parameters of templates to runtime values (with correct runtime dispatch). If you want to store sequences of different alphabets in one single container, you should use this.
	`compact_sequence_stdc_instance<Stdc,A,W>`, internal type obtained when dispatching the type at runtime	Carries nothing, this is primarily runtime tag Shouldn't be used directly, one should receive `compact_sequence_range<compact_sequence_stdc_instance<...>>` when using dispatch methods of the stdc template
`lazy_sequence<R, Scanner>` Represents sequence that can be loaded in fragments by some storage provider represented by type parameter `Scanner`	`Scanner` has static constexpr method `is_fragmented()` that returns false (this means that Scanner always holds entire sequence in memory and there's only one fragment). This is used as compatibility layer so you can directly create lazy sequence from compact sequence range instance. I have `resident_sequence_scanner<U>` for trivial implementation of non-fragmented scanners so you can write `lazy_sequence<R,resident_sequence_scanner<R>>` and don't have to implement scanner type on your own.	Holds single non-owning pointer to `compact_sequence_range<R>` Used as adapter to convert compact sequence range to lazy sequence. All primary pipeline operations should accept it as parameter

Template	Specialization, and when it is selected	Description and usage context
	`Scanner` is fragmented (`is_fragmented()` returns true)	Ows scanner, internal small cache for sequence fragments that are neither copyable or movable Allows iteration over data source of sequences that doesn't fit into memory at once. This is primary specialisation for loading sequences from PostgreSQL relations. The Postgres side implements large scanner type that tells the lazy sequence template how to fetch blocks of sequences
`bit_casted_view<Base, Out>` Only legal for few types of alphabets and storage widths.	Special non-owning type that is used to provide zero-cost changes to pitpacking. It allows to cast bitpacked sequences to different storage widths i.e cast sequence encoded on 32-bit int to be read as sequence encoded on 64-bit or 16-bit or 8-bit units. There are special requirements for alphabet to be able to do that (discussed below).	Holds only reference to underlying compact sequence range This is adapter type that provides zero-cost conversion between different specialisations of compact sequence range. Used for example by SIMD algorithms to have specific vector lane widths
`lazy_sequence_view<A>` Facade over every representation listed above that is typed-erased i.e it is templated on the alphabet only, so Postgres can directly accept the concrete type instead of template. It consists of two pointers: opaque backend state and a static `backend_ops` table that holds `bp_size`, four orientation-specific `read_at` slots (read forward or backward, write forward or backward), `try_resident_packed`, `copy_packed_sequence` and `worker_shareable`. Specialization here means which callback table is installed.	`owned_sequence_adapter<R, Scanner>`, created by `make_lazy_sequence_view` for a concrete `lazy_sequence<R,Scanner>`	Complete table over an owning lazy sequence. This is the table behind every lazy sequence that pipeline code receives, where Postgres provides pointers to actual functions.
	`compact_range_adapter<R>`, created by `make_lazy_sequence_view` for `compact_sequence_range<R>`	Used as convenient adapter to convert compact sequence range to lazy sequence view without worrying about scanner type and ownership. Useful for testing.
	`facade_adapter<A>`, created by `make_pg_sequence_call_owner` on the Postgres side	This struct holds Postgres Scanner state - it is either a detoasted flat value or a piece tree root together with its read session, because sequences need to be represented as trees of slice operations

Template	Specialization, and when it is selected	Description and usage context
	`resident_adapter<A>`, created by `admitted_packed_sequence::view()`	Postgres Scanner for sequence only holds descriptor for already materialized sequence
`retained_lazy_sequence_view<A>` Has only one form. This type pairs view mentioned above with opaque lifetime pointer. It's used in Postgres code for management of sequence lifecycle during DB operations	Constructed from a view, an opaque lifetime pointer and a `noexcept` release callback, so there is nothing to specialize	Used wherever a view has to outlive the expression that produced it. The Postgres read session allocates the owner in its own memory context and registers it there, so the retain is dropped even when Postgres unwinds with `longjmp` and skips C++ destructors. This means that this type is specific version of `std::unique_ptr` that allows us to provide custom lifecycle management during scans.
`admitted_packed_sequence<A>` Supporting type to propagate sequence along with memory permit (memory permits are used in BioseqDB to manage large memory regions for sequences)	Created during materialization by `materialize_packed_sequence`. In BioseqDB there's a little sequence access planner that is responsible for deciding if we can materialize it all at once and directly or it needs to be streamed. `plan_sequence_access` picks this type when a contiguous resident region and we cannot use existing materialization (for example we access specific nucleotide that doesn't align well with storage unit boundary, but the GPU algorithm requires continous region of sequence, so we need to copy it)	Owns the permit, the aligned buffer and its descriptor together, so accounted bytes are released with the object. Its `view()` returns an ordinary view over `resident_adapter<A>`, which is how SIMD and GPU kernels obtain contiguous packed input from a sequence that is tree-backed or fragmented

6.2. Packed symbols and a common C++ interface

The template-driven design allows usage of any arbitrary alphabet for sequences, though for now I only support DNA4 and DNA5.

Sequence is encoded using bitpacking - the symbols are converted into integers values with specified fixed number of bits per symbol, then several symbols are packed into one single storage unit. Storage unit can be integer of any physical size. This is generalisation of bitpacking strategies commonly used in other software like packed-seq Groot Koerkamp and Martayan, packed-seq: Bitpacked sequence trait and implementation Rust library developed by author of SIMD minimizers paper. packed-seq Rust library offers non-alphabet-generic packing Groot Koerkamp and Martayan, packed_seq crate documentation. My design was more inspired by SeqAn approach where packing container is available, but very limited in features SeqAn Project, SeqAn Manual.

SeqAn alphabet assigns each symbol an integer called its rank. For DNA4, the four ranks are $A = 0$, $C = 1$, $G = 2$, and $T = 3$. A rank is an encoding of a symbol. Each `compact_sequence_range<R>` type needs to know certain properties of the encoding that are held

(using CRTP) in `compact_sequence_range<R>::traits` type. In my storage symbols never spill to other storage units i.e to retrieve some symbol you always need to access at most one storage unit. The symbols in the storage unit are always packed in "little-endian" order i.e the first symbol is in the lowest bits of the storage unit and the last symbol is in the highest bits. The properties in traits type are either types or constexpr static expressions/functions:

- `alphabet_type` - type of the alphabet
- `storage_unit` - type of the underlying storage unit (usually `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`)
- `alphabet_size` - number of symbols in the alphabet
- `unit_width` - width in bits of the storage unit: sizeof(storage_unit) $\times 8$
- `bits_per_char` - stored bits per symbol. The default packed representation uses $\lceil \log_2(\text{alphabet_size}) \rceil$, giving two bits for DNA4 and three for DNA5.
- `letters_per_unit` - number of symbols that can be packed into one storage unit: $\left\lfloor \frac{\text{unit_width}}{\text{bits_per_char}} \right\rfloor$
- `unit_used_bits` - number of bits used in the storage unit (for alphabet sizes that are not a power of two, some bits must stay unassigned): letters_per_unit $\times$ bits_per_char
- `unit_leftover_bits` - number of bits leftover in the storage unit: unit_width$-$unit_used_bits
- `char_mask` - mask for the last symbol in the storage unit
- `rev_char_mask` - mask for the first symbol in the storage unit
- `used_storage_mask` - mask for the used bits in the storage unit
- Conversion functions for converting between storage unit and symbol: `letter_to_symbol`, `letter_to_char`, `letter_complement`
- Utilities for generating k-mer masks: `kmer_mask`

The figure below 6.1 shows an example of how I bitpack DNA4 sequence in storage units.

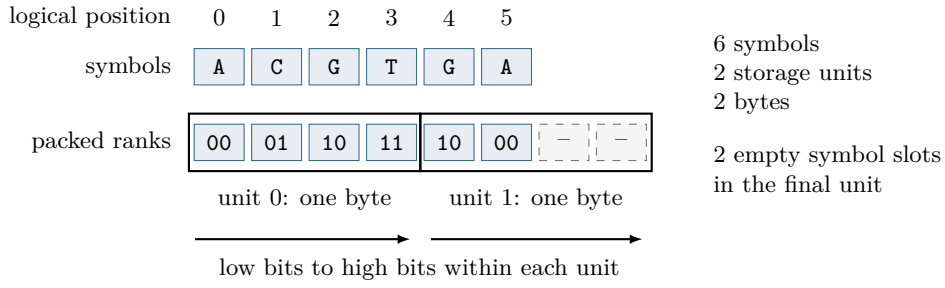


Figure 6.1: Packing `ACGTGA` with four DNA4 symbols per byte. The dashed slots are padding, which occurs at the end of the sequence as rarely sequence size is multiple of `compact_sequence_range<R>::traits::letters_per_unit` so they are not part of the sequence.

Template instance explosion

In BioseqDB I tried to allow user to specify any storage width that is power of two (8, 16, 32, 64) and alphabet (currently supported is DNA4 and DNA5). Because both of parameters are constexpr template parameters, this leads in some parts to creation of hundreds separate versions of pipeline. This slows downs compilation times and led to creation of absurdly large BMIs resulting in multi-gigabyte Clang .pcm files. Resulting binary still is optimised down to more convenient size, however I put lots of burden onto the compiler and linker. The goal of user-supplied storage widths was to observe how they affect performance and if they affect it at all. Currently in some parts of the pipeline I still cast the stored types to uint32_t, this happens because some algorithms like SIMD chaining require larger widths to store indices and scores. Extracting symbols directly from storage unit is a complex process involving lots of bitwise operations, so at this time most algorithms (SIMD/GPU) cannot operate directly on packed sequences as this would require lots of additional engineering effort. I added conversion helpers like one presented in the listing below 6.1 that convert requested ranges of sequence into non-bitpacked sequence that is easier to work with. This operation is done on demand and stored in separate memory pool region, so pipeline doesn't have to convert sequence from bitpacked to non-bitpacked format multiple times. Directly unpacking sequence inside pipeline at the latest point in time is potential opportunity for future improvements.

The following example uses actual code snippet from BioseqDB - in this case the caller owns `output`, which has at least $last - first$ bytes, and passes bounds satisfying $0 \le first \le last \le N$.

Listing 6.1: A consumer requests only the ranks it needs.

```
template<supports_compact_sequence_range R>
void read_fragment(const compact_sequence_range<R>& sequence,
                   std::size_t first, std::size_t last,
                   std::span<std::uint8_t> output) {
    // One rank per output byte, in forward order.
    sequence.template to_encoded_into<false>(output, first, last);
}
```

6.3. Slices keep logical coordinates

Postgres support array subscription operations natively, so I decided to support slicing as well in first version of BioseqDB.

A slice selects positions $[a, b)$: it includes a and excludes b. BioSeqDB uses zero-based positions, so $S[0]$ is the first symbol of the slice.

0-based indexing

This contradicts the fact that Postgres slices are 1-based, but it wasn't intentional decision. I had limited experience with Postgres array functionality and this realisation came only after most of the support was already implemented. What is more interesting that SeqKit - tool for operating on FASTA/Q files use 1-based indexing Shen, Sipos, and Zhao, SeqKit usage documentation: the subseq subcommand. This incompatibility might be something to reconsider in the future.

I provided utility functions for common operations like reversing and reverse-complementing slices as well, example available in listing 6.2.

Listing 6.2: Zero-based slices and sequence transforms in SQL.

```
WITH v(s) AS (
  VALUES ('ACGTAACCGTTA'::bioseq.seq('dna4', 32))      -- ACGTAACCGTTA
)
SELECT s[2:10]::text AS middle,                        -- GTAACCGT
       ((s[2:10])[1:5])::text AS nested,               -- TAAC
       s[-4:]::text AS suffix,                         -- GTTA
       bioseq.reverse(s[2:10])::text AS rev,           -- TGCCAATG
       bioseq.reverse_complement(s[2:10])::text AS rc -- ACGGTTAC
FROM v;
```

The bounds follow a small set of intuitive rules simiar to design of slices in Python Python Software Foundation, Python 3 Documentation: Common Sequence Operations:

- A missing lower bound means zero. A missing upper bound means the sequence length.
- A negative bound counts from the end (example: for length 12, -4 means 8) and bounds outside the sequence are clamped to its ends.
- An upper bound below the lower bound gives an empty slice, so `s[4:2]` has length zero.
- User can extract any element (symbol) from slice, which has separate SQL type `LETTER`. An out-of-range element, such as `s[12]` in this example, gives SQL `NULL`. An explicit null slice bound also gives `NULL` i.e $s[a:\text{NULL}]$ is equivalent to $s[\text{NULL}:a]$ and is also equivalent to $s[\text{NULL}:\text{NULL}]$ (all of them result in `NULL`)

The slicing can be implemented by old holding slice coordinates, without actually loading any sequences, by very simple coordinates arithmetic visible at figure below 6.2 For a slice view $V = S[a,b)$, view position j refers to $S[a+j]$. A second slice therefore combines its offsets directly:

$$V[c,d) = S[a+c, a+d).$$

The example's $S[2,10)[1,5)$ is $S[3,7)$, which reads `TAAC`. Reversal is represented by reversing start and end coordinates: For $W = \text{reverse}(S[a,b))$ we have:

$$W[j] = S[b-1-j], \qquad W[c,d) = \text{reverse}(S[b-d, b-c)).$$

Complementing operation leaves the coordinates unchanged, but alters binary flag representing if the slice is complemented or not.

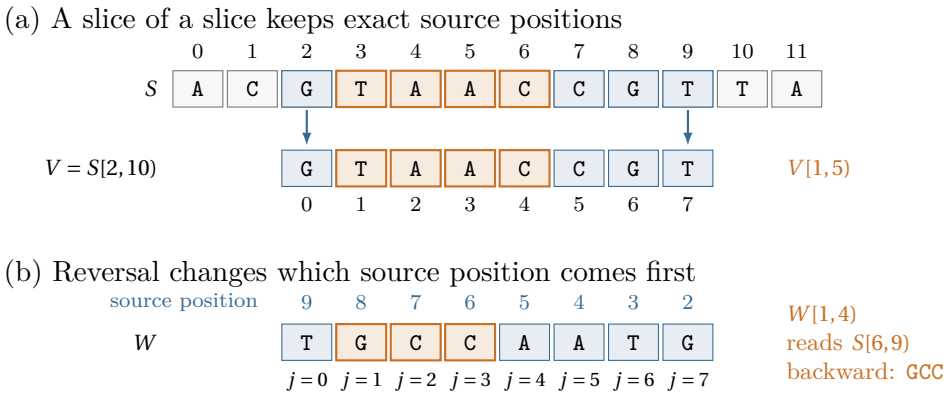


Figure 6.2: Blue cells form the first slice and orange cells show a second slice

6.4. Composing sequences without joining their payloads

The slice model I showed is very primitive. If we add ability to concatenate sequences we can represent any slice as a tree of pieces that represent operations on original sequence. In BioseqDB this tree refers to Postgres operators and datums.

The slice tree is constructed from the following elements:

- A source leaf describes interval in the source sequence.
- A literal leaf holds literal value referencing Postgres datum with additional metadata.
- A branch node that represents concatenation of two children. It holds total length, piece count (total number of leafs in subtree) and tree height. We collect total length to estimate how long the memory region in the pool should be to accomodate the materialized sequence.
- An edge that connects a node to its child. It holds reverse and complement flags, which affect the order of traversal and the interpretation of the leafs.

Figure 6.3 illustrates concatenation of three pieces including two source leaves and one literal leaf:

$$Z = S[2,6) \parallel \mathsf{reverse}(S[8,12)) \parallel \texttt{AC},$$

The result is `GTAAATTGAC`, with length $4+4+2=10$.

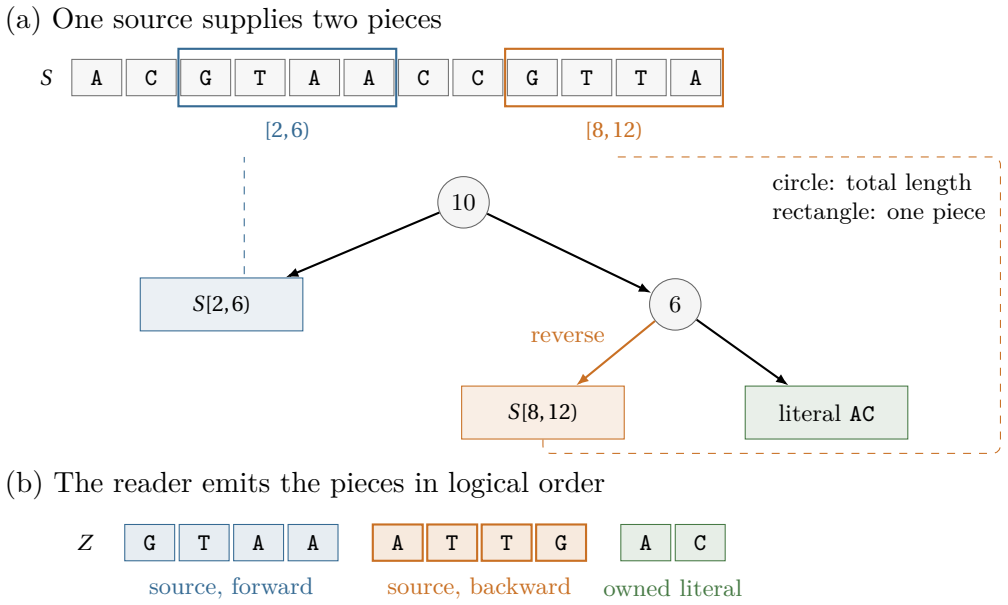


Figure 6.3: A piece tree describes how BioseqDB process slices of a sequence.

Each of the operations can easily be implemented by traversing the tree: concatenation joins roots and restores tree balance, slicing splits at the two bounds and leaves the middle part, reversing just as complementation flipps flags on the edges.

Reading actual slice content in BioseqDB is implemented using `lazy_sequence_view<A>` of table 6.1. It's built for the duration of a single call and the tree operations read only metadata from sequence locators (such as lengths). This means that we don't need to fetch large relations with sequence content and slicing is very performant. In the pipeline specific stages (like SIMD or GPU kernels) can request contiguous region of sequence and BioseqDB will materialize it into its own packed buffer first.

I implemented concatenation support for SQL text literals as well. This means that we can concatenate sequences with text directly without casting them, but this has it's own limitations:

- Slicing and reversal can operate on text without no known alphabet because they need only positions and order.
- Joining a known-alphabet sequence with a text literal validates the literal against that alphabet, so we require to go through entire literal (which shouldn't be a problem as we don't expect very long literals as this should be done via loading FASTA first into the table)
- Reverse-complement requires known alphabet with a complement rule for every symbol in that alphabet.
- Two different alphabets are not converted automatically to one another and this will result in an error.

6.5. Reading only what the algorithm requires

The storage can be provided in different forms as explained previously in the first section 6.1 In BioseqDB main implementation of Postgres-based storage is done via `lazy_sequence_`

`view<A>` and `retained_lazy_sequence_view<A>` mentioned in table 6.1. These types holds pointers to functions that implement the storage and represent the following operations:

- Report sizes of sequences
- Copy selected intervals into caller-owned memory, which in this case can be GPU-owned memory in GPU-accelerated pipeline stages
- Reading successive chunks
- Checking if the compatible interval was already materialized and can be borrowed (caching)

Internally I distinguish several access policies which are specified by the pipeline stage, they are the following choices:

1. Stream. If the algorithm accepts a stream, read the selected interval in bounded chunks and save it to reusable scratch memory (large memory pool)
2. Borrow. If the algorithm needs packed data, first check if's already in cache and safe to borrow.
3. Pack. In case if borrowing is not possible, algorithm needs to request new memory permit (this is routed through custom pool allocator). Caller is then responsible for this memory chunk.

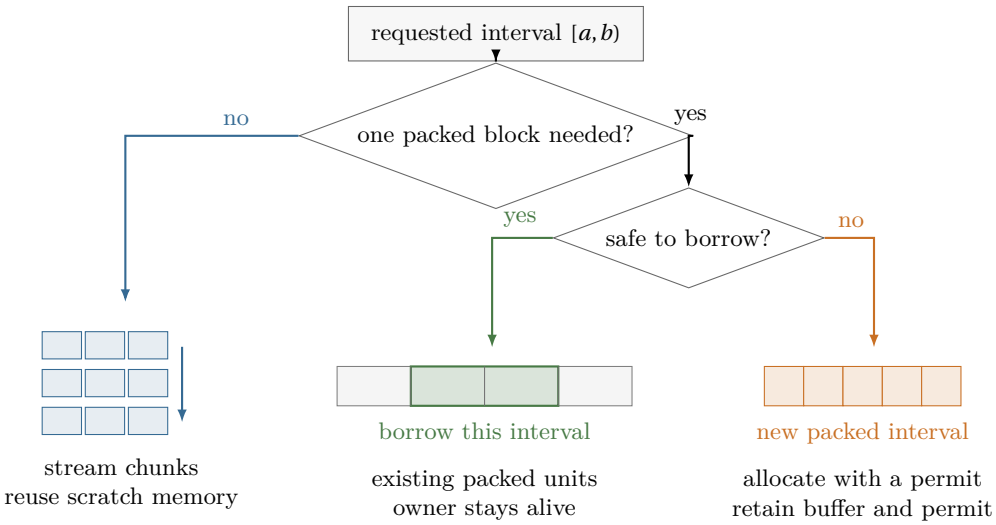


Figure 6.4: Simplified view of different access policies

Borrowing

Borrowing can fail if the region for example starts in the middle of storage unit in packed sequence, then we cannot reuse that memory chunk as it's misaligned

6.5.1. Copying ranks from a PostgreSQL sequence

The PostgreSQL source-provider view keeps its scanner and packed fragment slots in the read session. A rank request visits the requested symbols, looks up each symbol's fragment in that scanner and unpacks its rank. Repeated small requests can reuse packed fragments, but still repeat the callback and symbol-extraction work; they need not read the fragment again from disk or decode the whole fragment into ranks. Retaining a packed target for the alignment stage avoids these source-provider calls, although an algorithm requesting byte ranks must still unpack the selected interval.

At this point I need to describe lifecycles of various entities to illustrate how reading is performed. Figure 6.5 shows how what is happening under the hood during the scan:

- (a) Each row of a batch gets its own scanner with 16 fragment slots for caching. A miss looks in the decoded chunk cache (this is lower-layer cache for sequences chunks which may store multiple unrelated sequences and is discussed in later chapters), then in the columnar relation, and copies one fragment into a slot.

- (b) The incoming queries are grouped in batches. Each batch has it's own session, which is destroyed when batch processing is finished. The session is different entity to the chunk cache so relation isn't reopen when batch is finished and we keep large sequence chunks still in memory. At the start of the batch the backend first builds one packed copy in memory and distributes that copy to workers from the pool. It's important that in this architecture we only have one backend thread which is responsible for reading relation. This means that the actual pipeline is run by single thread that submits various work to worker pool. I needed to read the relation on a single thread to avoid race conditions on PostgreSQL's process-local state which is not thread-safe - PostgreSQL starts (forks) a new server process for each client connection, and even parallel SQL starts extra background worker processes rather than extra threads inside the same backend.[4] The same read could still be done in parallel by giving each reader its own backend: one way would be to fork a background worker with a database connection (which in this context is problematic as forking with pool running will result in segfaults) or open a second client session through libpq (libpq is thread-safe when each thread uses its own connection).[5] I did not take those paths for fragment reads during a query as it would complicate the implementation and this is something to be done in the future.

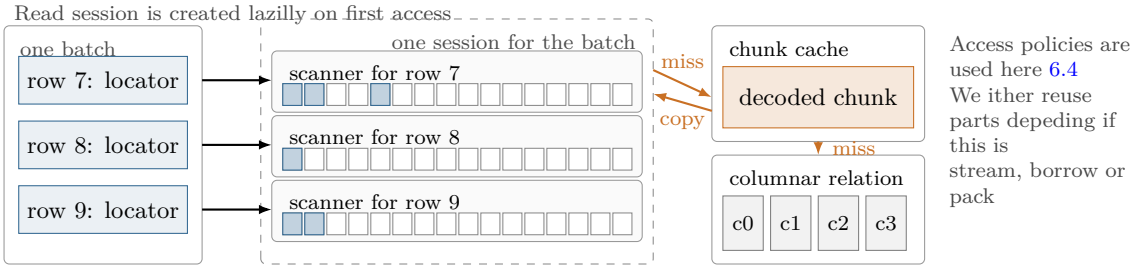
- (c) A view is built from two pointers - `state` is a piece tree together with the session that holds one scanner per source row and `ops` contains length, two flags and two function pointers. Providing function pointer serves as abstraction layer to divide non-Postgres code with actual Postgres implementation. Using templated types could be possible here, but they still would need type erasure. A retained view additionally holds a release callback (C-style destructor)

- (d) At every batch end, in this order: (1) release retained views, (2) finish each scanner so its slot copies go, (3) drop the session's memory. The worker pool and the snapshot naturally outlive the batch as discussed here. It might happen that we encounter an

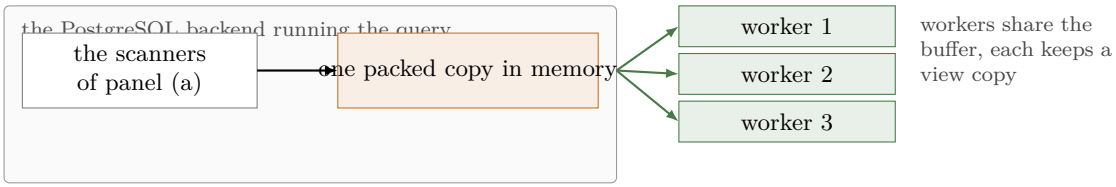
[4] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Architectural Fundamentals; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: How Parallel Query Works.

[5] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Background Worker Processes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Behavior in Threaded Programs.

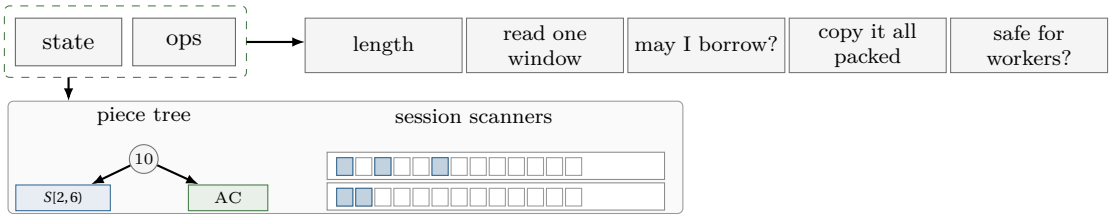
(a) One batch of rows gets reads session with multiple scanner. Each scanner has it's own slots for caching.



(b) Reading is single-threaded with distribution to multiple thread workers



(c) View copy sent to workers holds two base pointers (and release callback)



(d) Timeline of the scan

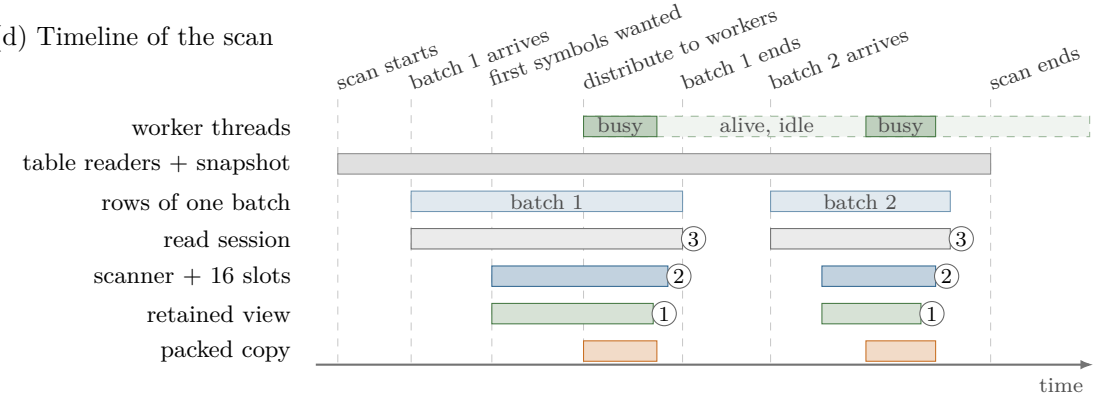


Figure 6.5: Illustration of structures used and operations performed on sequences during scan lifecycle

error and failing query skips C++ destructors, so steps 1–3 also run from a memory context callback (retained view release callback)

Panel (d) of Figure 6.5 has a gap between the end of batch 1 and the start of batch 2. In that gap the backend collects the next tuples from the relation (metadata and sequence locators, not entire sequences), applies the query's `WHERE` clauses, and places surviving tuples in the dispatch buffer. When enough tuples are collected, it submits the next batch. That collection still uses one relation-reading thread per PostgreSQL process. There's however support for `bioseq.scheduler.postgres_parallel.enabled` for any query that do not use alignment. The reasoning behind that is that only the thread pool wasn't designed around being multi-process safe and current statistics collection doesn't support it. The scans that do not use alignment are marked parallel-safe. This means that example shown on lsiting below 6.3 can be executed by postgres parallel scan.

Scans in details are covered in the next chapter 7.1 in detail.

Listing 6.3: Simple table read

```
SELECT id, bioseq.to_text(s[0:8]) FROM t;
```

Parallel scan

This is potential optimisation in the future. Statistics collection can be easily made multi-process safe by implementing reduce function, while thread pool design needs to be revisited and reimplemented.

Chapter 7

PostgreSQL API and custom scans

7.1. Designing SQL API for an aligner

In the previous chapter I presented how the sequence abstrations work and how they interact with the workers, we also had opportunity to look on how batching is done on figure 6.5. This chapter focuses on the main problem - how I incorporated this infrastructure into Postgres.

First, I want to discsuss how we can design SQL API for the aligner. Currently BioseqDB can be used in the following way:

Listing 7.1: Search once and return the fields of each hit.

```
CREATE TABLE targets (
  id bigint PRIMARY KEY,
  seq bioseq.seq('dna5', 64)
) USING bioseq;

SELECT t.id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.target_start(t.seq) AS start,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64));
```

The first `mmsearch` argument is the target sequence and the second is the query, - here query is supplied by `$1` (text parameter in prepared statement). The `score` alias means the dynamic-programming score of the current hit, CIGAR string describes the alignment and we also request position of the match in the sequence coming from target table.

One insteresting observation is that naturally each alignment can produce multiple hits, so this operation can return multiple rows for a single input row. We intuitively expect that there will be two kinds of selected expressions: Some of them will not depend on alignment (in this case it's primary key), so they will be the same for all these hits, while some of them like score and cigar will be different. Figure 7.1 illustrates that alignment should be set-returning function.

The first idea for implementation of such search was the simplest one - implementing alignment as callable Postgres function, which is very easy to do in C++. The function accepts query as raw SQL string:

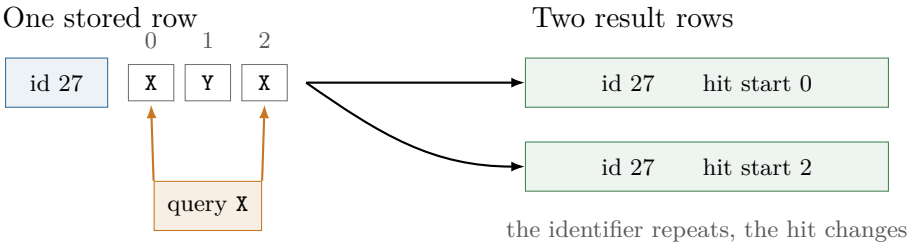


Figure 7.1: Searching `XYX` for `X` gives two occurrences, so the function needs to ”clone” metadata of tuple (id key in this case) and for each generated tuple fill-in the alignment hit position.

Listing 7.2: An early interface sketch passes SQL as text.

```
SELECT * FROM search_sql(
  'targets',
  'SELECT id, seq FROM reads WHERE id > 20'
);
```

This however introduces serveral problems:

- Which is the column we want to align? We can make a choice to hardcode column name to be `seq` at all times, but this limits flexibility heavily. Other choice is to allow passing the aligned column name as a parameter, which resolves this problem.
- How do optimiser know about inner query? As the inner query can be complex the optimiser doesn't know about it. We can call libpq inside the function, which can optimise the query separately, but then we should change it depending on the context: you can imagine very trivial case that the outer SELECT statement only selects primary key for some reason, so we don't have to do any alignment. In that case we would expect the function to do this optimisation, but this means that its implementation needs to do the same work as Postgres optimizer.

Listing 7.3: An outer condition cannot automatically filter a hidden scan.

```
-- Hypothetical SQL-string interface
SELECT h.*
FROM search_sql('targets', 'SELECT id, seq FROM reads') AS h
WHERE h.target_id = 27;
-- The function must itself restrict the hidden target scan.
```

The listing illustrates point stated by me: the function sees the target table through the string `'targets'`, but it's used in a context, where we will do only one alignment. If the function doesn't interpret its context, then it will perform alignment on entire table. The solution could be to pass this reponsibility to the user, but this generates huge inconvenience - for more complex queries, user would need to transfer SQL statements to and from the raw text pased to search function and manually be responsible for optimisation. In that case this should mean that when we forgot to do that the different is enormous - the statement will align two tables instead of one (could be orders of magnitude slower).

SQL already has a direct way to express one-to-many output. A set-returning function in a `LATERAL` join could return a record for every hit.[1]. The `LATERAL` clause lets that call refer to the current target row.[2]

Listing 7.4: A different API could return explicit hit records.

```
-- Alternative API sketch, not an existing BioSeqDB function
SELECT t.id, h.score, h.start, h.cigar
FROM targets AS t
CROSS JOIN LATERAL align_hits(t.seq, $1) AS h
WHERE t.id > 20;
```

This means that optimiser can work with queries, but user needs to do `CROSS JOIN LATERAL` for each alignment, which adds lots of boilerplate code. Additionally we loose ability to conditional evaluation in this case. Sometimes alignment may not require CIGAR which is more expensive operation as it requires running base alignment and the `SELECT` clause may only specify score. This prevents us from evaluating alignments differently based on what fields are needed.

To more easily indentify what parts of the alignment are needed (because otherwise we would need to track where the record type is used and how), I decided to separate returned record type into calls to functions. The function takes sequence as argument and returns some information about alignment. This concept is presented below.

Listing 7.5: Hidden state does not establish an evaluation order.

```
-- Hypothetical functions with shared hidden state
SELECT id, last_score(seq)
FROM targets
WHERE save_search(seq, $1)
  AND last_score(seq) > 100;
```

Now the fields are easy to track as we can directly look for a call to a function in the AST expression tree. However if the extension implemented only Postgres function this example couldn't work. We would need to pass some internal state between function calls - `save_search` would need to save some pointer to lazy execution object invisible to the user so it can be later called by `last_score`. This way we compute only what we need. Declaring these functions `VOLATILE` limits some planner transformations, but does not guarantee execution order between them[3] This means that it's not possible to reliably pass this hidden lazy object between functions.

We can see that example above is much cleaner than `LATERAL` join and easier to understand, but to implement it we need to tell Postgres how we want to execute this query - in which order expressions are evaluated and create our own tuples inside. Historically before PostgreSQL 9.5 only way this could be done was via foreign-data wrapper interface, which treated remote table as opaque object. Later Postgres version introduced custom scans and table access methods. There's overlap in all these interfaces, but the main advantage of TAMs and custom scans was that the logic wasn't only tied to one specific table and there was deeper and more cleaner integration with the executor.

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Set Returning Functions.
[2] PostgreSQL 17, `LATERAL` Subqueries.
[3] PostgreSQL 17, Function Volatility Categories.

1. PostgreSQL 9.1 added foreign tables and the foreign-data wrapper (FDW) API, a partial implementation of SQL/MED.[4] PostgreSQL 9.3 added `postgres_fdw`, the in-tree wrapper that queries another PostgreSQL server.[5]

2. PostgreSQL 9.5 added custom path and scan methods, so an extension can insert its own executor node into a plan.[6]

3. PostgreSQL 12 added table access methods (TAM) through `CREATE ACCESS METHOD …TYPE TABLE`. Heap remains the default TAM.[7]

Table 7.1 summarises the split.

Table 7.1: PostgreSQL interfaces that can supply rows to a query.

	FDW	Custom scan	TAM
First in core	9.1 API, 9.3 `postgres_fdw`	9.5	12
Attaches to	a foreign table	a plan node	a local table
Stores bytes here	no	no	yes
Typical work	fetch remote rows	replace a scan or join	layout, insert, vacuum

The BioseqDB implements the following components:

- Custom scans - allow to implement syntax suggested here with selective evaluation of the pipeline, efficient batching integrated with optimizer, so I rely on optimisations from Postgres as well. Theoretically custom scan can replace scans to any table, but I don not offer support for tables other than BioseqDB tables (created using `CREATE TABLE USING bioseq` TAM)

- Table access method - stores data in columnar format, so I can load large chunks of metadata and sequence in single row fetch

7.2. Integrating Postgres

7.2.1. Designing SQL API

A full text search interface (`pg_search`) from ParadeDB was an important motivation for the SQL API design of BioseqDB. ParadeDB is an extension written in Rust that uses custom text storage engine under the hood. I deeply inspected its source code to understand how we can implement our own syntax in BioseqDB. ParadeDB resolves similar problem as FTS can also return multiple hits for a single query.

ParadeDB score expression
score_from_relation 30

To return custom results instead of relying on Postgres evaluating the function (We don't want to let Postgres call actual `bioseq.score()` function, but rether provide it ourselves. ParadeDB has similar syntax and the same problem) ParadeDB replaces score expressions

[4] PostgreSQL Global Development Group, PostgreSQL 9.1 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Foreign Data.

[5] PostgreSQL Global Development Group, PostgreSQL 9.3 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: `postgres_fdw`.

[6] PostgreSQL Global Development Group, PostgreSQL 9.5 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Writing a Custom Scan Provider.

[7] PostgreSQL Global Development Group, PostgreSQL 12 Release Notes; PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Table Access Method Interface Definition.

with typed placeholders. These placefolders are filled from the current search result before projection. ParadeDB placeholder replacement inject_placeholders 493 Similar design was used in BioseqDB - described in Section 7.3.

Listing 7.6: ParadeDB pg_search examples

```
-- Some examples for pg_search
SELECT id, title, pdb.score(id) AS score
FROM documents
WHERE body ||| 'postgresql search'
ORDER BY score DESC, id;

SELECT id, pdb.snippet(body) AS excerpt
FROM documents
WHERE body &&& 'postgresql indexing';
```

Listing 7.6 shows how similar ParadeDB's problem is to my initial API designs.[8][9]

7.2.2. Plan lifecycle

To implement custom scan, we must understand different objects in Postgres:

1. Path: Describes possible inputs, estimated work, output row count, and ordering. The planner compares several paths and keeps the cheapest one[10].[11] The Genetic Query Optimizer is used only when the number of joins exceeds `geqo_threshold`. A single-table `BioseqScan` is ordinary cost comparison, not GEQO. The path is later converted into a plan. I register my own paths so PostgreSQL knows how to access the table.

2. Plan: In our case `CustomScan` contains child plans, expression trees, and descriptions of the search.

3. State: When plan is executed, postgres creates its state via my code. `CustomScanState` holds the input cursor, alignment buffers for sequences and metadata, and slots for generated hit. Postgres will call scan repeatedly to get new rows, so I use the state to hold any information that needs to be preserved between calls.

All types of Postgres API described in table earlier 7.1 mainly rely on callbacks (in Postgres called hooks). Extension fills the structure with function pointers and let postgres know which call is handled by which function. In this case PostgreSQL calls `PlanCustomPath` when it converts a selected custom path into a plan and later calls `CreateCustomScanState` to create the executor state[12] Figure 7.2 shows when PostgreSQL calls those hooks, when `BioseqScan` runs, and when the columnar table access method returns a tuple.

`ProcessUtility_hook` is used for utility statements such as `CREATE`, `DROP`, `COPY`, and `CREATE INDEX`. This is useful to forbid user from doing certain operations (Currently I only use it to log statements).

The TAM stores the user table in columnar format: A long sequence is not kept in that table row, only a locator is stored there. The sequence itself is stored in companion fragment

[8] Pigsty, pg_search: full text search for PostgreSQL using BM25.
[9] Neon, The pg_search extension.
[10] PostgreSQL Global Development Group, PostgreSQL Documentation: Planner/Optimizer.
[11] Conway, Inside the PostgreSQL Query Optimizer.
[12] PostgreSQL 17, Creating Custom Scan Paths and Creating Custom Scan Plans.

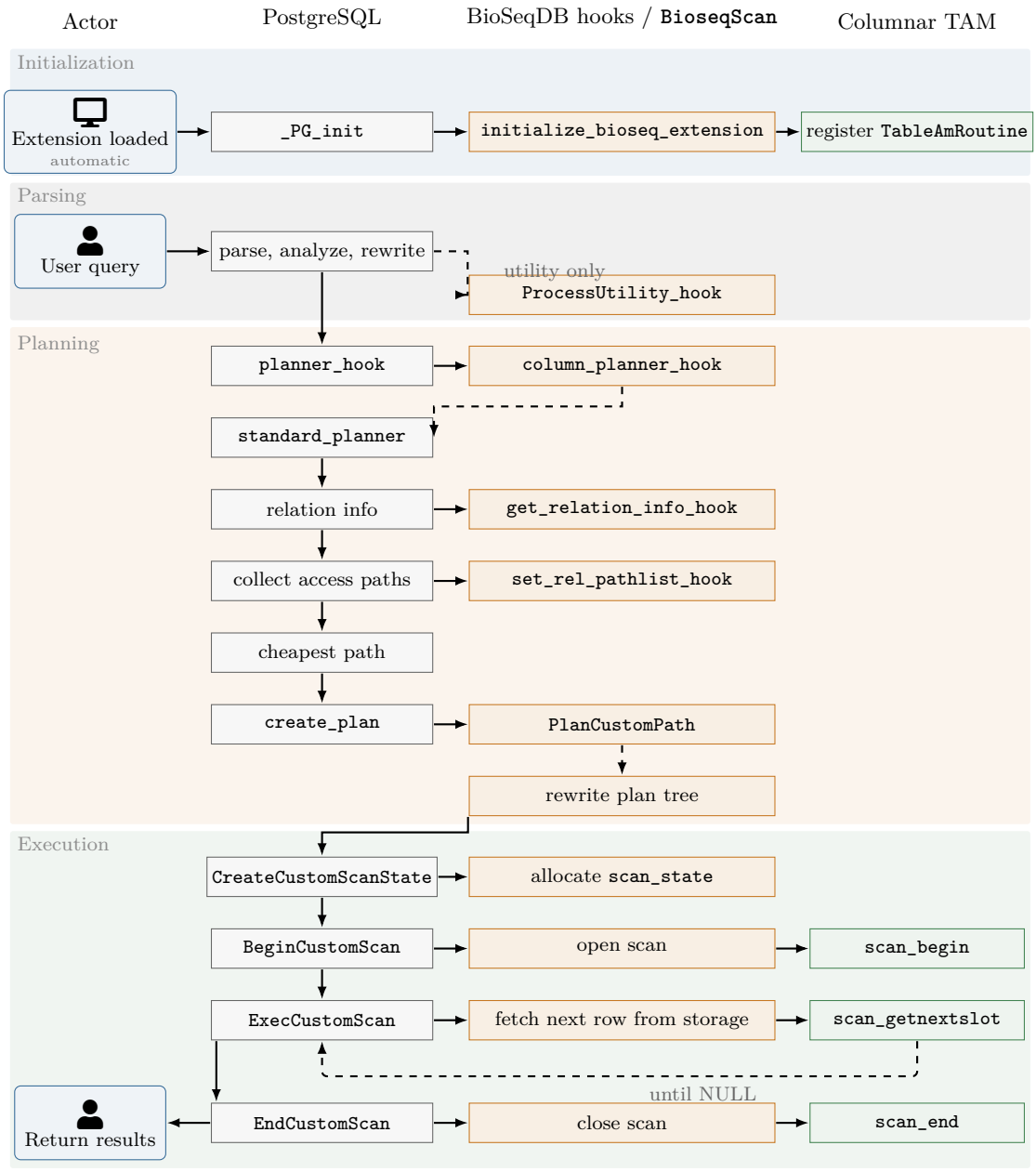


Figure 7.2: Lifecycle of `BioseqScan` and the columnar table access method. Bands mark exeuction states: initialization, parsing, planning, and execution. Boxes have various colours: Grey boxes are PostgreSQL, orange boxes are BioSeqDB hooks or the custom scan and green boxes are TAM callbacks. Arrows follow the order in which Postgres calls specific hooks. `column_planner_hook` calls `standard_planner`, then rewrites the finished plan, so we don't explicitly control GQO, but can affect it using costs functions, registering paths and even after plan is choosen, we have complete constrol over it (we are free to rewrite its parts). The scan is calling TAM to fetch rows from storage and returns them to Postgres until it returns NULL (no more tuples to fetch)

sub-tables, and all of them use the bioseq TAM as well. More on storage format is discussed in Chapter 8.1. I called the process of fetching fragment from locator "hydration" and TAM is responsible for it.

BioseqDB have control over the plan using `column_planner_hook`. This hook is called when PostgreSQL needs to perform planning and SQL query is already parsed. First BioseqDB hook validates `mmsearch` options, then it calls `standard_planner`. `get_relation_info_hook` and `set_rel_pathlist_hook` hooks, where we set options like disabling parallel scans or index-only scans and register custom access paths for cust scan, run inside that call. Then GQO is run and finds the best path that needs to be converted into a plan, so the `PlanCustomPath` is run. When `standard_planner` returns we are still in the `column_planner_hook`, so I'm free to rewrite plan tree freely. The rewrite mutor performs the following changes:

- Enable vectorized execution for simple aggregates Using modernized Citus/Hydra columnar vectorized optimisations I allow simple aggregates on metadta columns to be vectorized (processed in batches) for example: `COUNT(*)` or `SUM(foo_bar)`. If current node is Postgres `Agg` it will be replaced by custom aggregator `VectorAggNode`. If this plan has `VectorAggNode` as parent, then I return batches of rows instead of single tuples. Rewrite adds falgs to mark that the plan needs to change its output mode. Additionally Postgres `Agg`

- Wrap an index scans on columnar tables PostgreSQL can still choose a metadata index on a `USING bioseq` table and use external `IndexScan`. If that happens this index can read some tuples that shouldn't be red (columnar TAM allows removing rows. If that happens row is marked in bitmask, but index scan can go around it and still read the row, because it's physically there). To prevent that the `IndexScan` is replaced by custom `BioseqIndexScan` that hounours bitmask.

As entities like `bioseq.score(id)` are still ordinary functions, I discourage Postgres by using prohibitive execution costs[13]. The function implementation itself mostly just emits an error, because it shouldn't be ever called. Instead `ExecCustomScan` will using special Postgres expression nodes i.e `Const*` node to tell Postgres that the scan itself calculated the function's value and it shouldn't evaluate it by itself.

7.2.3. Different types of scans

When I talk about a custom scan I focus on `BioseqScan`, which is the primary way to read a `USING bioseq` table, but there are other types of custom scans used for different purposes in BioseqDB, some of them already mentioned earlier. All scans types are the following ones:

- Dedicated join support: `BioseqBatchJoin` is a custom scan with two children: one child is the target plan, the other is the query plan. Normally Postgres would use nested loop join that would read one target and one query and perform alignment for each pair, however we want to avoid this behaviour as target index can be loaded once (running `BioseqScan` for each pair would cause index for one table be loaded, then unloaded and loaded again for each query). This scan, dedicated for joins, prepares that target once (open the stored minimizer reader, or build index manually if not available) and runs the same logic as `BioseqScan` for each query in the batch. Dispatch buffer has separate API for joins, but they mostly effectively do same processing as for `BioseqScan` (collect queries in batches etc.)

[13] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Function Optimization Information.

- Slices across operations: `BioseqSliceProject` wraps a child plan when the query asks for a slice or other piece-tree operation, for example `SELECT seq[100:200]` or `mmsearch(t.seq[a:b], q.seq)`. It evaluates those expressions into expanded piece-tree Datums (Section 6.4) and sends them to Postgres as read-only expanded objects in a virtual slot. This means that the piece-tree is stored in the tuple itself and can be passed around other scans, joins etc. without translating it into a string yet: later `BioseqScan` or `BioseqBatchJoin` can align that slice lazily. When user requests a slice as in `SELECT seq[100:200]` and it doesn't go through other operation nodes, then it will be "hydrated" from piece-tree into actual byte sequence.

- Support for indexing: `BioseqIndexScan` is the wrapper around `IndexScan` described above, that allows to correctly read deleted columnar rows (support for bitmasks).

- Sorting by sequence: `BioseqBatchSortScan` is only for sorting by the sequence itself, for example `ORDER BY seq` or `ORDER BY seq DESC`. The planner will choose it only when there is exactly one `ORDER BY` key and that key has type `bioseq.seq`, and only when `bioseq.planner.sequence_sort_scan.enabled` is on (`ORDER BY seq, pk` and `ORDER BY seq::text` are still ordinary PostgreSQL sorts) This is experimental feature to allow faster sorting by sequence without the need to flatten entire sequence first. The custom scan buffers whole child rows, compares the sequence column as lazy views (not as printed text), and emits those same rows in the resulting permutation.

7.3. Dispatch buffer as glue between Postgres scan and alignment pipeline

I still need to explain how exactly `CustomBioseqScan` works with worker-architecture shown in Figure 6.5. The main abstraction here is the dispatch buffer which is reposible for transferring data between Postgres and workers.

Figure 7.3 shows how Panel (a) of Figure 6.5 looks in the context of Postgres scan. `BioseqScan` asks for the next row from TAM. This row is placed into dispatch buffer (locator only) and dispatch buffer holds all the structures presented before 6.5 - scanners. The dispatch buffer code then handles all view copies to the workers.

The collect loop is the gap between batch 1 and batch 2 in panel (d) of Figure 6.5. `BioseqScan` may call the user-table TAM many times before it returns to fill in the dispatch buffer. The backend thread that owns the scanners performs fragment reads, while workers only see the view copy described in panel (b).

7.3.1. Returning results back to Postgres

Now I not only need to build the tuple but somehow explain to Postgres that certain values correspond to requested values of function calls like `bioseq.dp_score(t.seq)`. PostgreSQL evaluates output expressions through an expression tree. Before returning its first alignment hit, the executor walks this tree and replaces the function calls with `Const` nodes. It writes the hit's values and null flags into these nodes and rebuilds the output-expression evaluator so that it reads the changed fields. This makes Postgres know that we already computed these values. The idea was created by analyzing how ParadeDB provides its search results

ParadeDB current-result score
exec_custom_scan 1784

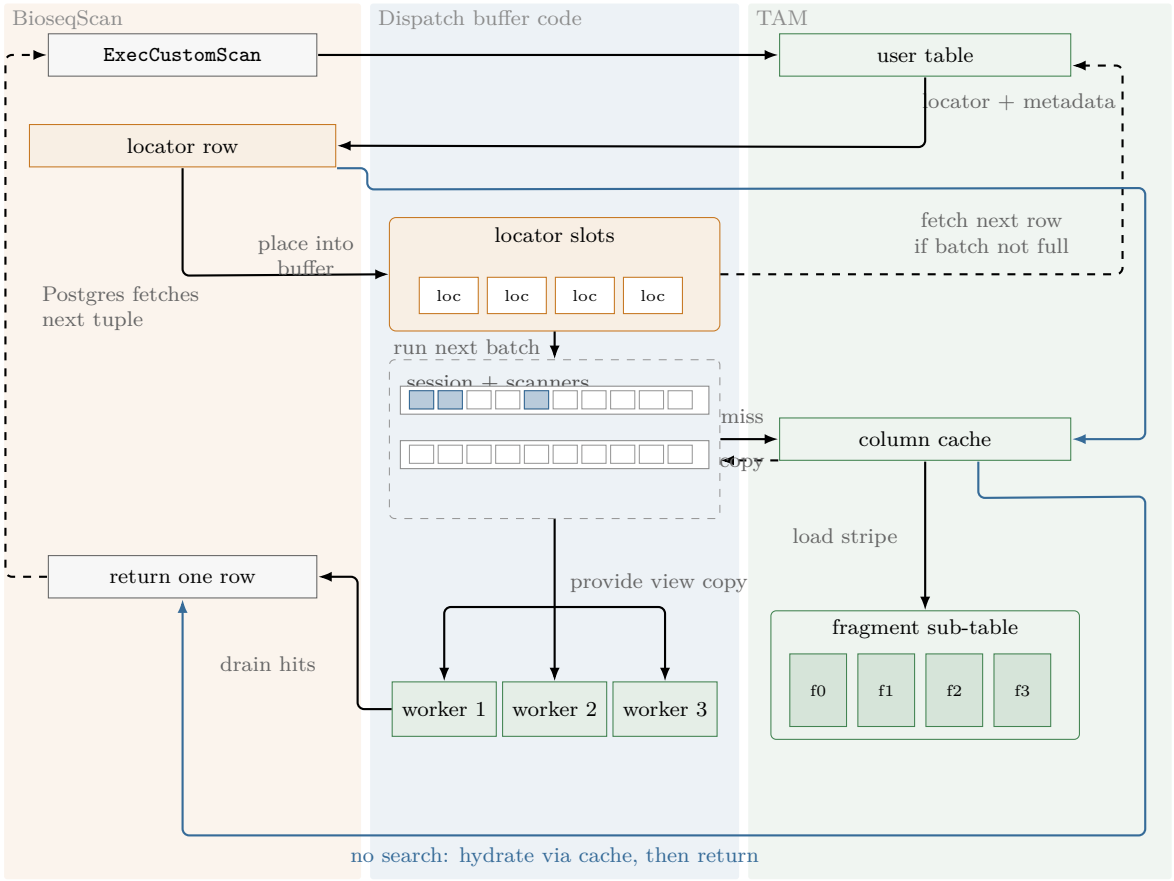


Figure 7.3: Inside `ExecCustomScan`, the following things happen: `BioseqScan` holds dispatch buffer and fetches a locator handles it to the buffer logic and retrieves results from it. The dispatch buffer stores locators, then builds a session that holds scanners. A scanner miss reads the fragment sub-table through the TAM column cache and copies the fragment into a slot. Workers receive a view copy from the scanners. Black arrows show the normal alignmen execution path, however it might happen that we don't perform alignment and just want to return the sequences itself. In that case the flow uses blue arrows (there's no `mmsearch` in the query) and `BioseqScan` hydrates the sequence row through cache and just returns it without interacting with dispatch buffer. Dashed arrows repesent loops here.

Algorithm 20 How to return tuple

1. Restore the original columns.
2. For every requested BioseqDB function like `score(seq)`, `target_start(seq)`, `cigar(seq)` and so on, write its field from this hit into the corresponding typed value node.
3. Rebuild the evaluator, then evaluate the output expressions.
4. Return the resulting SQL row

7.3.2. Supporting joins

A join can lift a helper such as `bioseq.score(t.pk)` out of the scan that produced the hit and an outer join can also emit a null row when one input has no match.

The `Const` rewrite from the previous subsection is nice execution trick inside `BioseqScan`, but the join planner does not see those constants. It still sees an ordinary function call, so PostgreSQL may copy that call onto the other input of a `LEFT JOIN`, or evaluate it again in a `Join` or `Aggregate` node.

PostgreSQL's `PlaceHolderVar` exactly to resolve this issue, however it's suprisingly low volume of documentation about it. I only found old mailing list archive, where it's described as an anti-COALESCE: the expression keeps its value on a matching row and is forced to `NULL` on a missing row.[14] The `PlaceHolderVar` records the relation at which the expression must be computed and the value then rises through the plan like a column. When an outer join nulls that relation, the placeholder becomes `NULL` (a `Const` cannot do this as it's always the same value).

PostgreSQL expression placement
PlaceHolderVar 2750

Without that wrapper the following could happen:

- The inner scan of a `LEFT JOIN` receives `bioseq.cigar(t1.pk)` even though the helper belongs to `t1` and that scan rewrites a function it does not own.
- A `Join` or `Aggregate` node calls the C++ stub. As I mentioned the function does not compute an alignment (the actual value is only scan-provided), so we get an error.
- Mishandling of `NULL` values:An unmatched outer-join row keeps a filled score instead of SQL `NULL` or `ORDER BY bioseq.score(t.pk)` after a join can sort on all-null keys.

ParadeDB uses the same mechanism for `pdb.score` and its planner function wraps the call in a `PlaceHolderVar` when the query has a join or an aggregate, so the custom scan keeps emitting the score and upper nodes do not re-run the function.

ParadeDB score placement
placeholder_support 314

In BioSeqDB I do this using `SupportRequestSimplify` hook as `bioseq_placeholder_support`, where we kip placeholders when there's no joins or aggregates.

7.3.3. Dispatch buffer internals

The main component I want to discuss here is the dispatch buffer: a container abstraction reponsible for retrieving, storing, distributing and collecting of the tuples with results. This requires managing memory and lifetime of Postgres tuples (both incoming from table and outgoing to the user).

Handling tuples

Postgres offsers `TupleTableSlot*` type, which represents reusable holder for a tuple. The Postgres executor keeps one scan slot on the expression node and asks the table access method to fill that same holder on every fetch.

This holder consist of a tuple descriptor, current column values, null flags, and a small operations (functions pointers) table that knows how to read or copy the physical layout currently stored inside it (PostgreSQL Global Development Group, PostgreSQL 17 source: tuple table slots). The tuple descriptor is the struct holding information about the columns schema (how many attributes it has, names of the attributes and their types with additonal metadata).

[14] Lane, Re: what exactly is a PlaceHolderVar?

PostgreSQL offsers the following types of slots Builtin tuple table slot types TTSOpsMinimalTuple 38:

- Buffer heap tuple (`TTSOpsBufferHeapTuple`). The slot points at a row that still lives in a shared buffer page.
- Heap tuple (`TTSOpsHeapTuple`). The slot owns a `palloc` copy of a full on-disk row with system columns (added by Postgres) (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout).
- Minimal tuple (`TTSOpsMinimalTuple`). The slot owns a compact copy of the user columns only without system columns added by Postgres. Sort, hash, and materialize nodes use this type because it is smaller than a heap tuple.
- Virtual tuple (`TTSOpsVirtual`). Instead of storing entire row it stores arbitrary data in array (usually pointers to the actual values, so we can reference existing tuple).

The TAM has one slot that is used to return tuples and it's overwriten with each tuple fetch (`getnextslot` call). This means that dispatch buffer has to store the tuple somewhere. In a dispatch buffer method called `submit_scan` I allocate a fresh slot with `TTSOpsMinimalTuple`, copy the current user columns with `ExecCopySlotMinimalTuple`, and store it with `ExecStoreMinimalTuple`. The third argument is `shouldFree=true`, so the copied slot owns the bytes and will `pfree` them when the buffer later drops the slot.

Sequence tuples in the dispatch buffer needs to represent lazy-loadable seqences, so during their lifetime they can become of of the 3 types (I need to distinguish them, because in code they are all just minimal tuples, but semantically express different concepts):

- Locator: This object is what is usually stored in the user table and represents entire sequence stored in separate internal table. This is "unhydrated" sequence.
- Expanded: Postgres has a flag `VARATT_IS_EXTERNAL_EXPANDED` (I call this in a code "expanded recipe") to mark the value is not a flat byte array yet, but complex structure in memory. This happens when sequence holds piece-tree (slice graph) 6.4, so we have multiple slice operations to perform if we want to convert this value to flat sequence. This is also different form of "unhydrated" sequence.
- Varlena: Representing "hydrated" sequence fragment, so the slices or locator were converted to some flat byte array.

The alignment pipeline uses first two representations directly and as mentioned before when discussing sequence abstractions 6.1 the sequence will be kept as a locator or piece-tree until the actual data is requested 7.3. Then I load requested bytes and return them directly. However in some cases we need to convert the tuple into 3rd form. If user requests slice projection directly it goes through separate branch of BioseqDB handling where that conversion is performed:

Listing 7.7: Direct projection of a sequence slice.

```
SELECT seq[2:3] FROM table;
```

Buffer API

Table 7.2 lists the most important methods of the dispatch buffer that are used during `ExecCustomScan` call. `Snapshot` here means the set of row versions this statement is allowed to see (this is mechanism for MVCC support in PostgreSQL) (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control) and is discussed later.

Table 7.2: Dispatch-buffer methods used by the Bioseq custom scans.

Method group	When the custom scan calls it	What it does
`configure_params`, `replace_executor_snapshot`, `bind_sequence_source_provider`	`BeginCustomScan` after the scan has a relation and a snapshot. PostgreSQL parallel workers (only for queries that do not use alignment 6.3) restore a copy of that snapshot and call `replace_executor_snapshot`. The worker must use its own registered `Snapshot` pointer for later fragment reads.	Stores aligner settings, the user table identity, the borrowed snapshot, and the statement-owned sequence source that scanners will read.
`configure_mmsearch`, `configure_mmsearch_per_row`, `configure_mmsearch_per_row_evaluated`	`BeginCustomScan` once the planner-private list has named the search, or on `ReScan` when a join parameter changes the query.	Records which columns are the target and (if needed) the per-row query, copies a constant query, and turns the buffer into the writing stage.
`submit_scan`, `submit_scan_evaluated`	Writing stage of `ExecCustomScan`, after a TAM row passed the filter conditions. The "evaluated" method is called when parameter in `mmsearch` is an expression that needs to be evaluated.	Copies that row into a buffer-owned MinimalTuple 7.3.3 slot and records its row number, primary key, and sequence locator (or evaluated Datums). Does not run alignment yet.
`ready_to_dispatch`, `has_finished_writing`, `mmsearch_active`	The `ExecCustomScan` loop, after each submit and after each drained hit.	Returns flags indicating whether the physical buffer has reached 8192 rows, whether the current batch has already been aligned, and whether this scan is an `mmsearch` scan at all.
`run_batch_and_build_cursors`	When the buffer is full, or when the TAM source is exhausted.	Streams the buffered locators into the alignment pipeline, then builds an output cursor for retrieving hits.
`read_next_output`	Reading stage of `ExecCustomScan`, once writing has finished.	Returns the next hit together with a pointer to the copied scan slot. The scan then fills the match-field `Const` nodes from Section 7.3 and projects.
`clear_slots`	After a full 8192-row batch has been drained while the TAM source still has rows, and on `ReScan` or `EndCustomScan`.	Frees every copied MinimalTuple slot and the payload arena so the next physical batch can start empty.
Support for joins: `configure_mmsearch_join`, `begin_join_target`, `prepare_join_target_queries`, `set_join_query_batch`, `run_join_query_batch_and_build_cursors`, `finish_join_target`, `reset_join_runtime_for_rescan`, `abandon_join_runtime_after_pg_error_noexcept`	Methods called only by `BioseqBatchJoin` custom scan for custom join (discussed in next section 7.4). PostgreSQL `ReScan` restarts a plan node from its first row, often with new parameters (for example the inner side of a nested loop for the next outer row, or a cursor rewind). `ReScanCustomScan` therefore calls the reset helper so leftover join targets and hits are dropped.	Holds one join target in the buffer, streams inner queries in batches of at most 8192, runs alignment against that stored target, then drops the target. The reset and abandon helpers clear join state without asking the caller to redo `submit_scan`.

The code in `ExecCustomScan` pulls one TAM row, runs filtering conditions (these can reject the row based on metdata) and on success calls `submit_scan` to let buffer copy the tuple. When the copies in buffer reach the set capacity `dispatch_batch_capacity` $8192 = 2^{13}$, `ready_to_dispatch` flag changes to true so the scan calls `run_batch_and_build_cursors`. This method is also invoked when there are no more tuples in TAM source.

`run_batch_and_build_cursors` is where the batch starts and scanners and workers from Figure 7.3 run. It groups the locators already stored in the MinimalTuple copies, lets scanners fetch fragment bytes through the TAM under the Postgres table snapshot object and then distributes the views to workers.

Then the scan needs to read results. Reading stage calls `read_next_output` on every later `ExecCustomScan` to retrieve the next result (this means that in this semantic first call to `ExecCustomScan` takes very long as it needs to align first batch, while later calls return tuples one-after-another from the output cursor until they are exhausted and process repeats). This is the place where we need to set values of nodes and project the tuple to let Postgres construct final results 7.3. After Postgres consumes that projected tuple, the next call drops the previous result slot and asks for the next hit. When the cursor is empty and the TAM source is not yet exhausted, the node calls `clear_slots` and returns to the writing stage (this corresponds to finishing batch and starting collecting rows again)

Execution statistics. Each completed call to `run_batch_and_build_cursors` writes a `pipeline_execution` aggregate onto the buffer, where I collect debugging statistics. `EXPLAIN ANALYZE` reads then through `execution_stats()`. Table 7.3 presents some of them as the list is long and not every one is relevant to the common user.

Table 7.3: Grouped execution statistics collected by the dispatch buffer.

Group	Fields (batched)	What they tell you
Backend choice	requested versus actual backend, backend declines, backend fallbacks	Whether the scan asked for GPU or CPU, and whether the runtime had to use fallback (hardware unavailable or alogirhtm is not supported).
Alignment jobs	attempted, completed, failed	How many target and query pairs the pipeline tried, how many finished.
Target index	target rows, persisted index rows, missing persisted rows, recomputed rows	Whether minimizer indexes were read from storage or rebuilt for this batch.
GPU dispatch	seed, chain, backtrack and other stages	How much of the aligner actually ran on the device
Stage timings	per-stage nanosecond counters (query sketch, index build, seed, chain, extend, …)	Which pipeline stage dominated wall time on this statement.
Integrity flags	mixed environment, mixed persisted identity, saturated counters	Additional flags indicating overflow or aggregation of counters and ”mixing” which means that different stages of pipeline run in different modes (incompatible k/w or GPU/CPU mixing) - this is used for debugging

7.3.4. Memory admission before dispatch

Before overlapping work, BioseqDB requests a process-local reservation for its input and temporary buffers. The check considers available memory, the process footprint and existing reservations. A rejected request returns immediately. The caller must reduce the work group, use a supported serial path or report an error. The admission check itself does not wait for memory to become available. A reservation accounts for intended use, but does not reserve physical pages in the operating system.

7.3.5. MVCC support

PostgreSQL holds its current input row in temporary storage called a tuple slot, but another transaction may also update that row. `Snapshot` in Postgres is set of row versions some statement is allowed to see as it needs to perform visibility checks before doing anything (PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Introduction to Concurrency Control). `configure_params` borrows this snapshot object so later fragment reads and bulk fetches observe the same MVCC view as the original TAM fetch. Let's see if this is compatible with the BisoeqDB model:

- Copying values: Dispatch buffer always copies the original tuple into an owned MinimalTuple slot, so Postgres won't overwirte this data.
- Saving Postgres snapshot object: PostgreSQL uses a snapshot to decide which row versions a command can see. The sequence saves scan's Snapshot and checks a stored reference against the row (visibility check). Stored source fragments are immutable. An update publishes a new version, while an older snapshot can still read its earlier version. Delaying a read does not switch it to the latest committed sequence.[15]
- Saving row numbers: While the primary key is important for user, the row internally holds a row number that is used to indentify results.
- Opening relation: In scan I use `ACCESS SHARE` relation lock to prevent conflicting operations such as dropping or rewriting a relation while it is in use (concurrent updates and deletes are still possible, but avoided due to snapshot mechanism)[16]

I use existing Postgres mechanisms to support MVCC so it didn't require any additional custom code.

7.3.6. Filter placement

As this mentioned here, before saving tuple into the buffer, scan checks if the filter predicate on non-alignment dependent values evaluates to true. This is done to avoid aligning rows that will be discarded anyway. In BioseqDB there are two types of filter conditions:

- A row condition such as `t.id > 20` depends only on stored columns, so is evaluated before alignment. Sometimes columns that are comparable (like Postgres primitive types) can allow min/max chunk prunning to be used. This is special strategy to skip entire columnar chunks with data 2

[15] PostgreSQL 17, Transaction Isolation.
[16] PostgreSQL 17, Explicit Locking.

- A hit condition This is special case of alignment prefilter. I have a separate stage in the pipeline that can be invoked to run cheaper estimation of edit distance for each sequence. If I detect special predicate of form `bioseq.edit(t.seq) < 10` I run pipeline with prefilter enabled. This is different to the case above, because it still requires batching tuples and running pipeline, but it allows for faster termination. If the distance estimation yields false, then the rest of stages are skipped (there's no seeding, chaining, backtracking etc.) so we avoid even the cost of fetching the minimizers from the saved index. This results in great speedups, but at this time is yet an experimental feature using code from libraries that weren't developed by me.

Listing 7.8: Filter input rows and then filter alignment hits.

```
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM targets AS t
WHERE t.id > 20
  AND mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND bioseq.edit(t.seq) < 10
LIMIT 5;
```

Potential features

Currently I do not optimise limits so `LIMIT 5` will still evaluate all tuples in batch even after more than 5 results are found. I also do not provide any optimisations for score ordering, so `ORDER BY score DESC LIMIT 5` doesn't result in any specialised algorithm. However there are some very speicific cases that would be interesting to optimise. We can use faster chaining if we know that there's only one good hit per sequence. This isn't expressable in SQL with this API, but you can imagine providing custom limit per one row: `WHERE bioseq.count(t.seq) = 1`.

7.4. Joins support

Asssime we have a search between two tables `reads` and `targets`:

Listing 7.9: Search sequence pairs from two tables.

```
SELECT t.id AS target_id, q.id AS query_id,
       bioseq.dp_score(t.seq) AS score,
       bioseq.cigar(t.seq) AS cigar
FROM targets AS t
JOIN reads AS q ON mmsearch(t.seq, q.seq)
WHERE t.id > 20 AND q.id <= 100;
```

There are several possible plans for searches:

- Search before a join: A base `BioseqScan` can search a target against a constant query. This is the most straightforward operation, but isn't possible for the described join.
- Search with an outer parameter: Postgres can create a nested-loop plan that used `BioseqScan` inside. This was discussed in scan list 7.2.3 and causes the target to be loaded multiple times, which reduces performance heavily.

- Search at the join: `BioseqBatchJoin` replaces the nested-loop plan and allows to keep the target index loaded only once, while iterating over queries. This is the most efficient plan for the described join.

Interesting thing happens if we want to make two searches for the same row. Consider the example below:

Listing 7.10: Two searches with one ambiguous target binding.

```
-- Unsupported: this will cause an error
SELECT t.id, bioseq.dp_score(t.seq)
FROM targets AS t
WHERE mmsearch(t.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(t.seq, $2::bioseq.seq('dna5', 64));
```

In this case due to my choise of SQL API, we don't know which score `bioseq.dp_score(t.seq)` refers to as we have two potential results with different scores. To resolve this issue we can do a self-join on the same table:

Listing 7.11: Give each independent search its own target binding.

```
-- Supported: this is valid use-case within BioseqDB
SELECT a.id,
       bioseq.dp_score(a.seq) AS score_a,
       bioseq.dp_score(b.seq) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64));
```

Now, let a_i be hit i of the search bound to alias `a`, and let b_j be hit j of the search bound to alias `b`. Figure 7.4 shows their combinations for one target row. We can see that natural extension of the used syntax would be to generate cross-product of hits. It's worth noting that currently we are not able to express search-specific limits 7.3.6 meaning that there's no optimisation for example if we wanted to have only best-hit per each sequence (only tuple with best score on both searches). In that case all tuples needs to be evaluated and user can construct a query manually to select the highest-scoring hits:

Listing 7.12: Keep one highest score from each of the two searches, per stored row.

```
SELECT a.id,
       MAX(bioseq.dp_score(a.seq)) AS score_a,
       MAX(bioseq.dp_score(b.seq)) AS score_b
FROM targets AS a
JOIN targets AS b ON a.id = b.id
WHERE mmsearch(a.seq, $1::bioseq.seq('dna5', 64))
  AND mmsearch(b.seq, $2::bioseq.seq('dna5', 64))
GROUP BY a.id;
```

The intereesting part is that self-joins in relational databases are commonly used for hierarchical structures (trees), detect duplicates, extract consecutive elements (and other comparison operations). In this case it's used to create new alias for the column without any additional operations.

There is however one interesting exception that is a bug in the BioseqDB code (initially I supported conflicting calls under one alias and there is leftover branch of the code that still

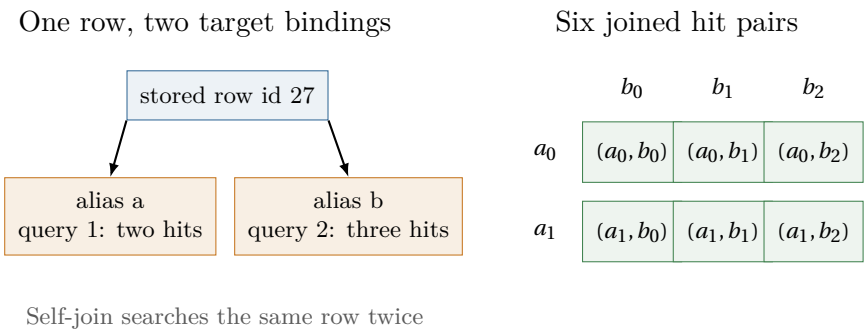


Figure 7.4: Two hits from the first search and three from the second produce six combinations. Additional hit conditions may select subset of this cross-product matrix, but this is responsibility of the user as I don't provide any specialised optimisation for example to extract only one best-hit for each search.

supports it). Users should be aware that there still can be some undiscovered edge cases that are not correctly detected. I initially planned to support same-alias searches if no feature is used (we don't specify CIGAR, score etc.), but this results in usability issue that adding these properties suddenly requires user to rewrite the query completely. For this reason I later decided to disable any conflicting alias searches even if we don't specify any properties in the `SELECT` list.

Listing 7.13: Low priority bug that still exists in the code

```
-- This shouldn't be possible, but currently works (bug)
SELECT t.id, bioseq.dp_score(t.seq) AS score
FROM queries AS q, targets AS t
-- index_read_requirement is a special debug parameter to disable BioseqDB from
    loading index
-- for some reason enabling it allows you to trigger specific BioseqJoin path
    that doesn't correctly detect the conficting calls
-- The queries table needs to be normal table, not defined using "USING bioseq"
WHERE mmsearch(t.seq, q.seq, aligner_options(index_read_requirement => 1))
  AND mmsearch(t.seq, 'ACGTACGTACGTACGT'::bioseq.seq('dna5', 64));
```

7.5. Caching results

As mentioned before I provide layers of caching:

- Chunk (strip) cache that stores rows of columnar stored (compressed blobs that can potentially store multiple sequences) 7.3. Described more in detail in next chapters 8.1
- Lazy interface cache implemented by scanner slots 7.3
- I also provide long-term experimental cache layer for alignments results described here.

An alignment query may run for hours while returning a small result. If two sessions submit the same request, running the alignment twice wastes work. I created separate small `bioseq_cache` extension that is provided along with BioseqDB.

> **Experimental support**
>
> This chapter doesn't go into details of chache implementation, because it's still experimental. I didn't perform enough testing to be confident enough about all edge cases correctness as I needed to be very careful how to lock queries and split work (it needs comparing expression trees to detect query identity or overlap, serializing them to store them, parallel-safe coordination of locking). The subextension isn't loaded by default and user need to execute `CREATE EXTENSION bioseq_cache` manually and restard Postgres server. There are some problems with this approach as the MVCC Postgres visiblity can change during execution (there could be insert when alignment is running). This problem wasn't addressed so the user would see old version of the data. That's why I decided to disable caching now. Moreover the "excact results" setting is fully supported now and can be enabled if user loads extension, while "reusable fragments" have most of the code ready except that it was only tested on scalar types (INTs, FLOATs etc.) and sequence support will require some alterations (planner will fallback to "exact results" anyway)

The cache subextension coordinates requests and stores completed results. The extension provides two forms of reuse:

- Exact results: If user executes two identical queries at the same time, we provide special custom scan in plan that waits for currently running job and loads results from it. It needs to happen during planning stage - planner needs to detect that alignment pipeline is running and then during execution stage block execution until results are available.
- Reusable fragments: This is more complicated part of the code that doesn't completely block execution of the query. If similar query is currently running, planner will try to alter current query to still execute it, but try to cover region that is not covered by running jobs.

producer is the query runner that currently executes alignment job. In that timeframe another Postgres-spawned process with other user query waits for results. Submission of partial results would be possible, although very complicated to implement correctly. sealing here means that we store last tuple, collect some metadata like tuple counts and create new UUID for the result. Figure 7.5 shows what happens during the execution of two queries. The cache has expiration TTL time after which the results cannot be reused anymore.

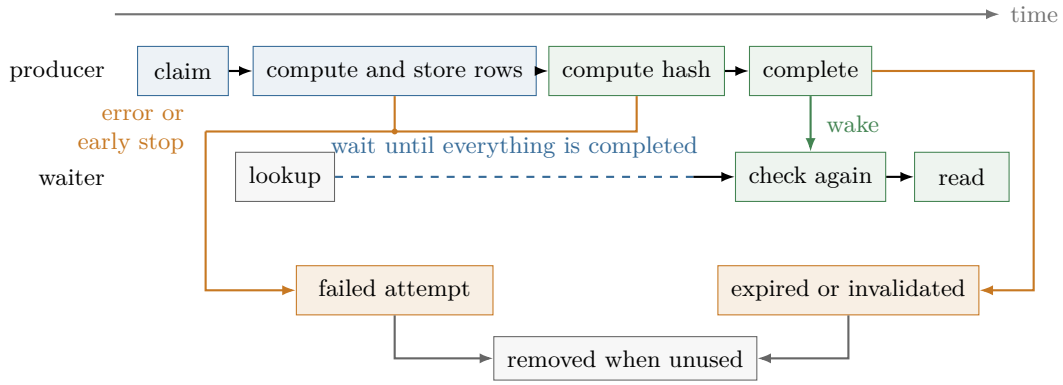


Figure 7.5: Consumer just waits for all results from producer and then reads them.

There's one lock that is taken by any consumer during execution just to access shared state. The consumer sleeps and preiodically acquires and releases the lock to check if it can continue execution (polling). If the producer fails or is cancelled I wake all consumers that registered for this job and then each of them will try manually run the query (so the consumer won't fail due to producer error or cancellation).

The interesting problem is that matching SQL text alone is insufficient. I needed to include analyzed query, parameter values, result types, relevant execution settings, and the database context objects to make sure we reliably detect identical queries. Additionally I keep track of query dependencies (all relations) and if the relation changes (row insert) during processing, then cache will be invalidated. This means that insert during consumer's wait time will cause the consumer to execute query by themselves (although this require more testing to confirm correctness)

7.5.1. Reusing fragments of queries

Suppose that table is kept unchanged, but user slightly changes some metadata condition (for example instead of aligning all sequences with dates between some range, we extend the range to include few more dates). For indentiy comparison I store the analyzed query plan as well so we can say what filter conditions were used for that cache entry.

The general idea of describing what is cached, reading useful part and computing remainer was very loosely inspired by semantic caching paper[17]

For example, select input rows with $40 \le id < 100$. Completed sources cover $0 \le id < 60$ and $50 \le id < 80$. We read $[40, 60)$ from the first source, $[60, 80)$ from the second, and compute $[80, 100)$. Let's define it formally via predicates:

- $T(t)$: t is an input tuple
- $A(t, h)$: h is tuple with alignment of tuple t
- $p(t)$: the new SQL predicate P evaluates to `TRUE` on t
- $c_i(t)$: cached source i has tuple with alginment for which the original predicateP_i evaluates to `TRUE` on t.

[17] Dar, Franklin, Jónsson, Srivastava, and Tan, Semantic Data Caching and Replacement, VLDB 1996, pp. 330–341.

Order the selected cached sources from 1 to m, source i supplies part B_i, while the reminder we have to compute is D:

$$B_i = \{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t) \wedge c_i(t) \wedge \forall j < i : \neg c_j(t)\}, \tag{7.1}$$

$$D = \{t \mid T(t) \wedge p(t) \wedge \forall i \in \{1,\ldots,m\} : \neg c_i(t)\}. \tag{7.2}$$

Each requested hit is returned from the first valid source covering its input, or computed from D:

$$\{(t,h) \mid T(t) \wedge A(t,h) \wedge p(t)\} = B_1 \uplus \cdots \uplus B_m \uplus \{(t,h) \mid t \in D \wedge A(t,h)\}. \tag{7.3}$$

Here $\uplus$ joins disjoint sets of occurrences. This simple formal description can be implemented as large `UNION ALL` operation of the entries in cache. Figure 7.6 shows the same example, including the overlap between the two sources.

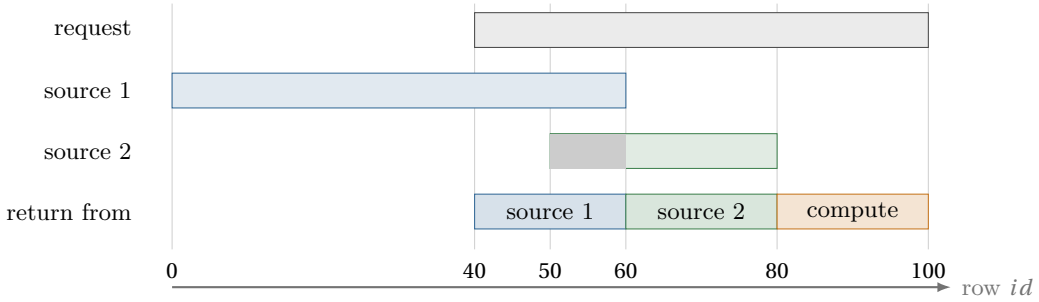


Figure 7.6: Example of resuable query with one predicate on the primary ID column.The new request uses source 1 for $[40,60)$, source 2 for $[60,80)$, and computes $[80,100)$. The gray overlap in source 2 is omitted because source 1 already contains those occurrences. Cached rows outside the new request are also omitted.

With both sources valid, the missing inputs have this equivalent SQL description:

Listing 7.14: Input rows on which the expensive operation is still needed.

```
SELECT id, seq
FROM sequences
WHERE id >= 40 AND id < 100 -- New query we want to compute
  AND (id >=  0 AND id < 60) IS NOT TRUE -- Minus source 1 results
  AND (id >= 50 AND id < 80) IS NOT TRUE; -- Minus source 2 results
```

There are few important details about the implementation:

- I need to correctly handle NULLs For a nullable predicate, `NOT P` also becomes unknown when P is unknown. `P IS NOT TRUE` includes both false and unknown, so it keeps every input excluded from the earlier cache.[18]
- Handle source failures If source 1 fails, c_1 is falsethen D should become $[40,50) \cup [80,100)$ so on error we don't abort execution and just adjust computed region (currently this is realised by adding barrier for each query and computing D at the end)
- Column differences There could be a case in which source supplies only subset of columns. I don't optimise this scenario as computing new columns still requires running entire pipeline again, so we assume here that the column list is always the same.

[18] PostgreSQL 17, Comparison Functions and Operators, Table 9.2.

Chapter 8

Physical sequence storage in PostgreSQL

8.1. Store sequences together with their metadata

I already mentioned before for example when discussing sequence abstractions 6.1 that I usually operate on locators, that just point to location where actual sequence is stored.

PostgreSQL already has mechanism for that: it stores large values out of line (outside actual row) through TOAST.[1] This mechanism however, has it's limitations - it's storage is limited to 1GB per row, which is an effect of maximum width of integer storing row's OID (object indentifier is internal numbering scheme in Postgres). If we assume that DNA4 sequence is bit-packed and it's entire human genome (lengths of around 3.1 - 3.2 Gbps) this gives us conservatively $3 * 10^9 * \frac{2}{8}B = 750MB$ of storage per row. If we want to store both pairs of chromosomes we can pesimisitcally require $1.6GB$ of storage per row. In most cases researchers split the sequence into smaller chunks, but this is manual process requiring additional tools.

Moreover, most of the workflows are OLAP-type.: user is expected to do bulk inserts, do not perform frequent updates and deletes when running alignment. Usual TOAST wouldn't take advantage of that specific workflow assumption as we cannot read TOAST in bulk.

That's why I decided to implement custom TOAST-like mechanism in BioseqDB and use columnar storage appropiate for OLAP workflows:

- User table holds all the data in columnar format, where the sequence column itself is replaced with small locator object that points to sequence fragment.
- Sequence fragments are stored in multiple sharded tables that are normally invisible to the user. All of these tables also use columnar storage, so short sequences are treated the same way as metadata in columnar context - they will be merged into one batch. I call these batches of rows "strips" in the code.
- Persistent indexes are separate entities stored as tables that are normally not visible to the user as well. They are responsible for providing pre-computed objects for sequences. There's small abstraction layer in BioseqDB which allows to extend it by adding different types of indexes that provide different information.

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: TOAST.

Adapted from Citus and Hydra Columnar

BioSeqDB's columnar layer adapts code from both Citus Columnar and Hydra Columnar.[a] Hydra extended the Citus code with features including very primitive vectorized SQL execution and a cache of stripes. In BioSeqDB I modernized the adapted C code by rewriting it completely into C++26 modules with automatic code generation and added support for newer PostgreSQL 19.0 (which in case of Hydra original codebase was not supported). Only small of the original code was left untouched. This code was used as a low-level storagebase, where concepts like sequence locators, fragment readers, and sequence indexes are purely my innovations.

[a] Citus Data contributors, Citus Columnar storage implementation; Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.

The linked Hydra columnar documentation was unavailable as the theam building it moved to create a different database system. Although I was basing my implementation in the Hydra codebase, understanding the chunk operations required multiple manual investigations to understand it completely.[2] Current BioseqDB storage design is an effect of this process.

It's worth noting that I mention both Citus[3] and Hydra sources here as Hydra was fork of Citus that diverged at some point to later be abandoned. Citus columnar is still actively maintained but it lacks features like row deletion, vectorization and it's very simple columnar interface. Meanwhile Hydra offered these features, but due to being abandoned didn't correctly build for newer Postgres version and lacked bugfixes done by Citus team since the forking moment. Modernizing the Hydra codebase thus involved not only manually inspecting the code, but also tracking commits between Hydra and Citus. This work thus was complicated and very time-consuming involving also detection of new bugs in Hydra code that were undected after retirement of the project.

8.2. Columnar storage

ClickBench is a benchmark for analytical DBMS and it also reported results for both Hydra and Citus. Figure 8.1 uses historical benchmark submissions on AWS machines.[4] The benchmark consisted of around 43 queries over a dataset of around 100 million rows (3 runs with minum of second two taken as into the sum).

Both columnar submissions store about 19 GB, compared with PostgreSQL's 77.8 GB. This was due to support of compression on stored columns. Hydra[5] had much better performance due to introduction of vectorized SQL execution for aggregates and column cache that wasn't present in Citus.[6] These results do not translate directly to BioseqDB as the benchmarks tests some arbitrarily-built queries, however it motivated me to put big effort into modernizing Hydra codebase and incorporating it into BioseqDB as mentioned in previous section 8.1.

[2] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[3] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."
[4] ClickHouse contributors, ClickBench: analytical database benchmark.
[5] Hydra contributors, Hydra Columnar: PostgreSQL columnar storage extension.
[6] Cubukcu et al., "Citus: Distributed PostgreSQL for Data-Intensive Applications."

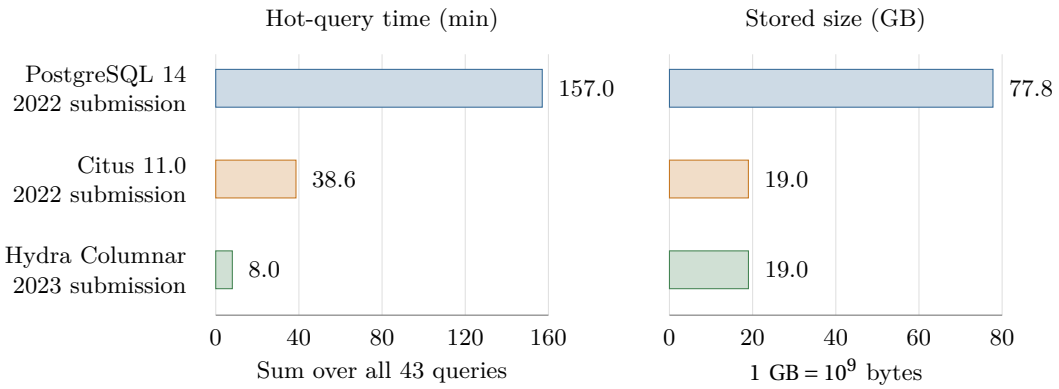


Figure 8.1: Historical ClickBench, calculated from the repository snapshot of 20 September 2023. Each submission uses `c6a.4xlarge` machine with 16 vCPUs, 32 GiB RAM, and 500 GB of GP2 storage. PostgreSQL and Citus results are dated 1 July 2022, and Hydra results 19 September 2023. The times use the sum of shorter of runs two and three for every query. Note that these are "best-fit" results so the configuration was different. It's good however for quick comparison of performance.

8.2.1. Columnar layout

The columnar layout has three levels:

- A stripe is a batch of chunks coresponding to set of rows. Normally one stripe holds maximum is 150,000 rows.
- A chunk group is a group of chunks for the same rows inside a stripe. By default one chunk group holds maximum of 10,000 rows.
- A column chunk is the most basic entity that just holds values for one group as a one column. It also stores aggregated values (count, min, max) to enable faster aggregate operations (vectorization) and the bitmask (marking which values are NULLs).

Stripe and chunk-group sizes are options per table, where larger ones allows more efficient compression and smaller ones allows more fine-grained read access.[7]

Figure 8.2 shows this model for the six rows. By default Postgres row-based storage (heap tuple) stores `id`, `sample_id`, and the sequence locator together. The stripe is columnar representation of these rows. Each column occupies its own sequence of chunks. A chunk group is only an abstract concept for chunks that are related to the same rows. The last group of a stripe can be shorter if total row count isn't a multiple of chunk group size.

[7] Davis, Citus 10 brings columnar compression to Postgres.

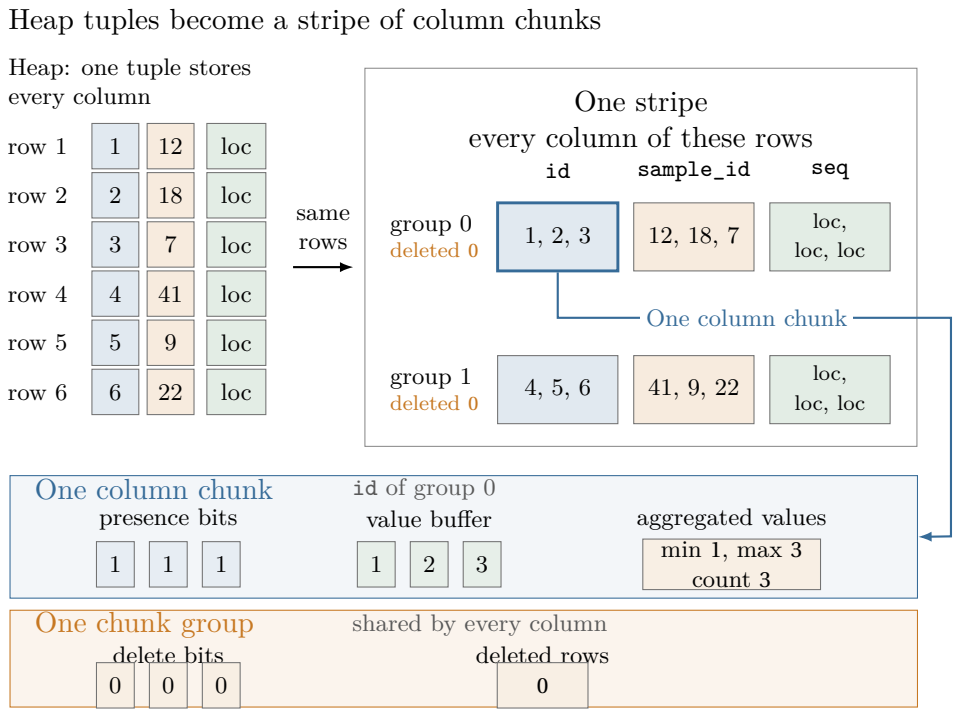


Figure 8.2: The same six rows as heap tuples and as one stripe. Matching colours mark the same column. The blue callout is one column chunk: presence bits, present values, min/max, and slot count. The orange callout is the chunk group: a deletion mask and `deleted_rows`

When I want to store row, each time the writer append this row to the current chunk group (in memory). After sufficient number of row is reached (max chunk group size) the code serializes each column chunk to a stripe buffer that still is in memory If the number of rows per stripe is reached (max stripe size) then stripe is "flushed", which means writing the stripe. Commonly the maximums of stripe row counts will be hit, but there's additional check if the current chunk group for specific column buffer exceeds $1GB$ limit, then it forces flush of the entire stripe. This means that if we store large BLOB along very small column, the BLOB will force the stripe to end i.e stripes always need to account for the same number of rows in each column (this also implies that the decision to store sequences via locators increases efficiency of that storage model as more tuples with non-sequence fields can be packed into single stripe). Additionally there's hard limit on maximum size of $256MiB$ for a single value in row.

Storing large BLOBs

Users should avoid storing large BLOBs along small columns - either store BLOBs of similar sizes together or only store smaller payloads in the same table. This is potential optimisation to be implemented in the future, although it's very low priority.

The figure 8.3 shows 3 main stages: we save tuples first to accumulate enough of them, then serialize them to a buffer and when stripe's buffer eaches $1GB$ or the strip max row count, only then it's flushed. Algorithm 21 presents pseudocode for that process as well.

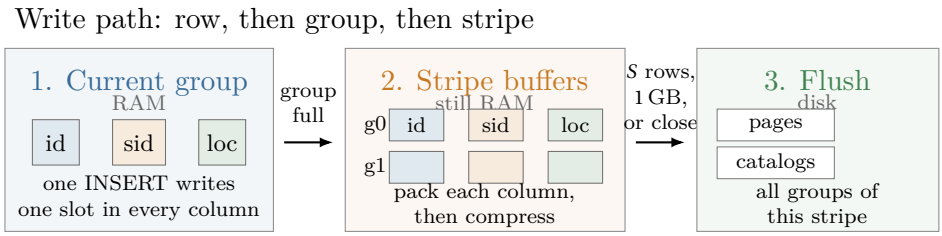


Figure 8.3: How we pack chunks groups into stripe buffers, then to flush them if necessary.

Algorithm 21: Appending rows

Require: columns C, group limit G (default 10,000), stripe limit S (default 150,000), comparators Compare_c (possibly NULL)

$G_{\max} \leftarrow \lceil S/G \rceil$
$s \leftarrow \text{NULL}$ open stripe, or none

procedure AllocateNewStripe
  $s.n \leftarrow 0$ rows already in this stripe
  $s.P \leftarrow 0^{|C| \times G}$ presence bits of the current group
  $s.B \leftarrow \varepsilon^{|C|}$ serialized present values of the current group
  $s.K \leftarrow \{\min : \bot, \max : \bot, cnt : 0\}^{|C| \times G_{\max}}$ skip min, max, and slot count of every column chunk
  $s.\text{chunk} \leftarrow \{P : \varepsilon, V : \varepsilon\}^{|C| \times G_{\max}}$ packed streams of every serialized group
  $s.N \leftarrow 0^{G_{\max}}$ row count of each serialized group
  return s

procedure WriteRow(v)
  if $\text{IsOpen}(s) \wedge \exists c \in C : \text{Size}(s.B_c \| v_c) > 1\,\text{GB}$ then
    FlushStripe(s)
  if $\neg\text{IsOpen}(s)$ then
    $s \leftarrow$ AllocateNewStripe
  $g \leftarrow \lfloor s.n/G \rfloor, \quad r \leftarrow s.n \bmod G$
  for all $c \in C$ do
    if $\text{IsNull}(v_c)$ then
      $s.P_{c,r} \leftarrow 0$
    else
      if $\text{Align}(v_c) \geq 256\,\text{MiB}$ then
        error
      $s.P_{c,r} \leftarrow 1$
      $s.B_c \leftarrow s.B_c \| \text{Serialize}(v_c)$
      if $\text{Compare}_c \neq \text{NULL}$ then
        $s.K_{c,g}.\min \leftarrow \min_{\text{Compare}_c}(s.K_{c,g}.\min, v_c)$
        $s.K_{c,g}.\max \leftarrow \max_{\text{Compare}_c}(s.K_{c,g}.\max, v_c)$ $\bot$ is the identity
    $s.K_{c,g}.cnt \leftarrow s.K_{c,g}.cnt + 1$
  if $r = G - 1$ then
    SerializeGroup(s, g, G)
  $s.n \leftarrow s.n + 1$
  if $s.n \geq S$ then
    FlushStripe(s)

procedure SerializeGroup(s, g, k)

35: $s.N_g \leftarrow k$
36: for all $c \in C$ do
37: $s.\text{chunk}_{c,g}.P \leftarrow \text{PackBits}(\langle s.P_{c,r} \rangle_{r=0}^{k-1})$
38: for all $c \in C$ do
39: $s.\text{chunk}_{c,g}.V \leftarrow \text{Compress}(s.B_c)$ identity if incompressible
40: $s.B_c \leftarrow \varepsilon$

41: procedure FlushStripe(s)
42: $k \leftarrow s.n \bmod G$
43: if $k > 0$ then
44: SerializeGroup(s, $\lfloor s.n/G \rfloor$, k)
45: $m \leftarrow \lceil s.n/G \rceil$
46: for all $c \in C$ do
47: $\text{Write}(\langle s.\text{chunk}_{c,g}.P \rangle_{g=0}^{m-1})$
48: $\text{Write}(\langle s.\text{chunk}_{c,g}.V \rangle_{g=0}^{m-1})$
49: WriteCatalog(s)
50: $s \leftarrow \text{NULL}$

51: procedure WriteCatalog(s)
52: $\text{Write}(\langle s.N, s.K, 0^{s.n} \rangle)$ Saves row count for group, aggegate stats and deletion mask

In's important that closing the writer also flushes any leftover stripe, so all in-memory buffers are saved to disk when transaction is committed.

As presented on figure 8.2 column chunk stores a presence bitmap, a value buffer of only the present values, and aggregated values used later for delettion and vectorized aggregation: the smallest and largest non-NULL values (when the type has a comparison function), and a row count (`value_count`) counting all values in the chunk including NULLs.

To support deletion, the chunk group stores `deleted_rows`, along with separate row mask representing which rows were deleted. `DELETE` only sets the matching mask bit in the chunk group's mask and increments `deleted_rows`. This allows BioseqDB to offer fast deletion without the need to rewrite the columns themselves. The wasted storage can be however reclaimed:

- `VACUUM (TRUNCATE)` may rewrite last stripes in table whose undeleted rows still fit in one stripe and whose decompressed size is below 1 GB. A single last stripe is rewritten only when more than 20% of its rows are deleted. The rewrite copies undeleted rows into a new stripe and drops the old ones.
- The internal tables storing sequence fragments cannot be rewritten as each locator needs to store the row number. Supporting VACUUM for sequence fragments would need to be dispatched to all tables including user table. Currently VACUUM is only supported on the user table.
- `VACUUM FULL` can rewrite the whole table to reclaim wasted space.

8.2.2. Smart reads

In BioseqDB it's unnecessary to read all chunks all the time. The two optimisations are:

1. Projection means that only selected columns from `SELECT` query are red. This is trivial optimisation.

2. Min/max pruning is more interesting one. For an ordered column, the writer records the smallest and largest non-NULL values in each chunk (in aggregates fields seen in figure 8.2). The BioseqDB TAM presented in figure 7.2 can determine if none of the values in the chunk group satisfy the filter. If that happens, entire chunk group is skipped. For example, a group with sample identifiers from 1 to 100 cannot satisfy `BETWEEN 500 AND 600`.

Figure 8.4 shows both optimisations. Group that satisfies both of these conditions still needs to apply filters to each row individually, but I save time but not touching groups that are not revelant to the search. The figure assumes query `SELECT id FROM table WHERE sample_id BETWEEN 500 AND 600` is executed.

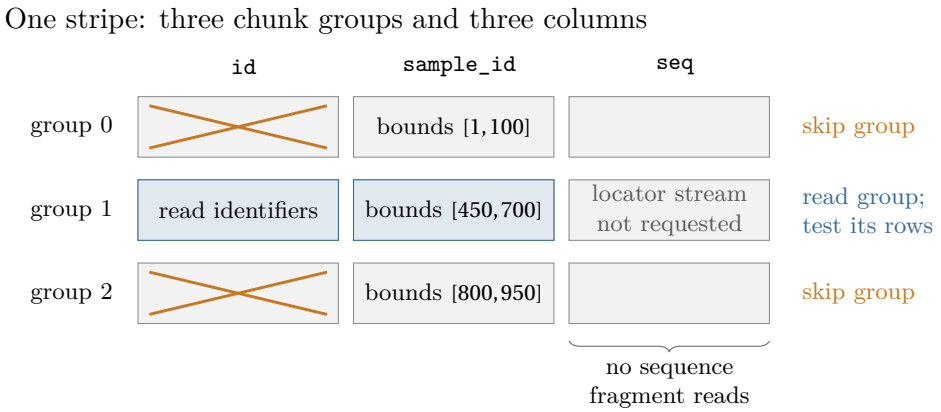


Figure 8.4: Column selection and group pruning for simple query `SELECT id FROM table WHERE sample_id BETWEEN 500 AND 600`. Blue cells are read, orange crosses mark groups whose aggregates exclude all the rows. Unrequested sequence locators remain unread, because they were not specified in the `SELECT` clause. The drawing shows only logical representation of chunks, not actual disk layout.

Prunning for sequence columns

Sequence columns with locators are excluded for min/max pruning, but it's possible to implement that feature theoretically. I decided not to do it, because storing min/max values would be simple as locator isn't big object, however computing these values for very large sequences will be computionally expensive and would negatively impact table inserts.

8.2.3. Compression

Figure 8.5 shows in detail how presence bitmap is computed. If the NULLs are very frequent, I will save the space by not storing them.

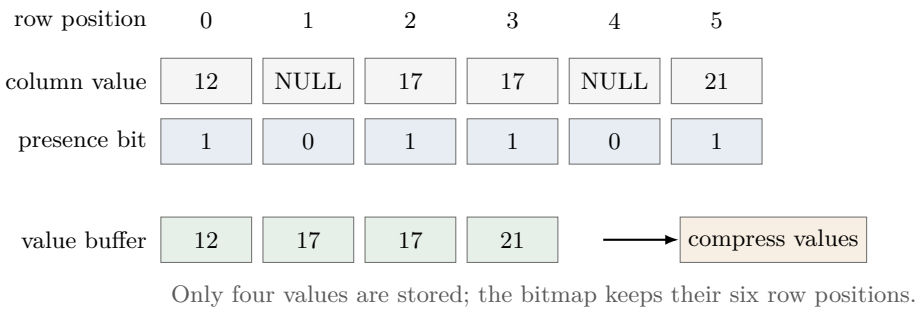


Figure 8.5: Presence bitmap is useful to enable better NULL compression

The Hydra code that BioseqDB columnar TAM was based on supported PGLZ, LZ4, or Zstandard, depending on the build and table options. In BioseqDB all compression is disabled by default as I would like to benchmark in the future best (and more modern) compression algorithms decicated for example to sequence storage. Abadi et al. show that the choice of compression method depends on both the data and the query workload,[8] so I leave it as future extension.

8.3. Raw byte storage

After stripe is serialized and flushed it needs to be written to disk - function $Write$ and $WriteCatalog$ in the pseudocode 21

- Physically first written is the presence bitmap, then the value buffer for a column, then each column follow. These are placed in the user relation database pages.
- Catalog (row counts, aggregates and deletion mask) is written separately from the stripe data and it's placed in usual heap rows i.e BioseqDB introduces separate relations for storing chunks information.

Catalog is represented by four relations inside the BioseqDB:

[8] Abadi, Madden, and Ferreira, "Integrating compression and execution in column-oriented database systems."

Listing 8.1: Metadata catalogs for flushed stripes.

```
CREATE TABLE bioseq_internal.stripe (
  storage_id bigint,
  stripe_num bigint,
  file_offset bigint,
  data_length bigint,
  column_count integer,
  chunk_row_count integer,
  row_count bigint,
  chunk_group_count integer,
  first_row_number bigint
);

CREATE TABLE bioseq_internal.chunk_group (
  storage_id bigint,
  stripe_num bigint,
  chunk_group_num integer,
  row_count bigint,
  deleted_rows bigint
);

CREATE TABLE bioseq_internal.chunk (
  storage_id bigint,
  stripe_num bigint,
  attr_num integer,
  chunk_group_num integer,
  minimum_value bytea,
  maximum_value bytea,
  value_stream_offset bigint,
  value_stream_length bigint,
  exists_stream_offset bigint,
  exists_stream_length bigint,
  value_compression_type integer,
  value_compression_level integer,
  value_decompressed_length bigint,
  value_count bigint
);

CREATE TABLE bioseq_internal.row_mask (
  id bigint,
  storage_id bigint,
  stripe_id bigint,
  chunk_id integer,
  start_row_number bigint,
  end_row_number bigint,
  deleted_rows integer,
  mask bytea
);
```

When writing stripe data I need to take care about page boundaries. Let BLCKSZ be the PostgreSQL page size in bytes (this is compile-time constant provided by Postgres), H the page-header size (technically sizeof(PageHeaderData)),[9] and $P = \text{BLCKSZ} - H$ the remaining

[9] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Database Page Layout.

page capacity. If $x \geq 0$ is the total offset of the x-th byte, then page coordinates are defined as:

$$\mathbf{block}(x) = 2 + \lfloor x/P \rfloor, \qquad \mathbf{position}(x) = H + (x \bmod P).$$

BioseqDB will write multiple pages for each stripe, so I need to take additional care about buffers going through multiple page boundaries as seen on figure 8.6.

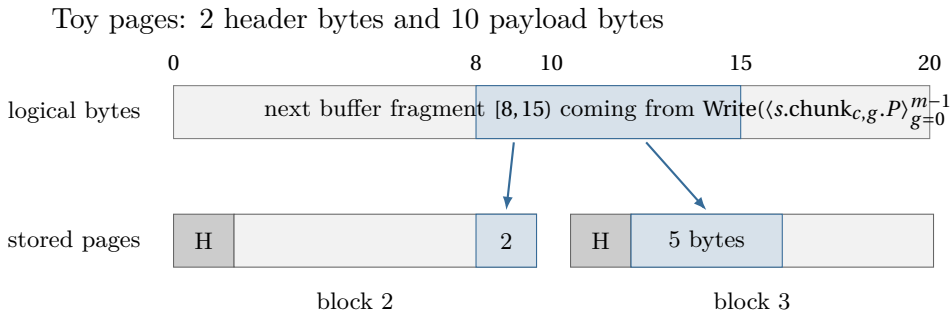


Figure 8.6: I use logical offsets $\mathbf{block}(x)$ and $\mathbf{position}(x)$ to hide page boundaries. Diagram shows that we want to write some fragment of in-memory buffer $[8, 15)$ to the disk, but it needs to be split across multiple pages. In reality the dimensions are larger than just 20 bytes, but it illustrates the idea.

The original idea comes from Citus page writer that it's avaialble in their public repository

Citus columnar page mapping
columnar_storage.c 118

To allow concurrent access (queries during inserts) I need to note that the writer does the following things:

- Reservation: Identifiers for chunks and groups is done under the relation extension lock.
- Write: Writes to pages are done using PostgreSQL buffer locks and generic write-ahead logging (WAL).[10]
- Record completed data: The logic uses Postgres' Snapshots so data will be visible only after the transaction is committed.

8.4. Locator implementation

Sequence locator are implemented as 40-byte struct in C++. The struct must also include the Postgres header so it's effectively 56 bytes in total. The Postgres header stores alphabet code and base storage unit as `TYPMOD` field (custom field for storing type modifier). I don't provide small-sequence optimisation in BioseqDB, because that would complicate the implementation (instead of storing locator for small sequences we can potentially store sequence directly in the user table. This would require storing it's padding so potential maximum sequence length that could be stored inline would be $(40B - 1B) * 8/2 = 156$ symbols for DNA4).

[10] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Generic WAL Records.

Listing 8.2: The 40-byte locator stored after the 16-byte sequence header.

```
struct sequence_locator {
  std::uint8_t  payload_version;   // format version used by me to change the locator format
    (currently it's just set to 3)
  std::uint8_t  bucket_index;      // length bucket 0..34
  std::uint16_t reserved_16;       // pad so shard_index is 4-byte aligned, must be 0
  std::uint32_t shard_index;       // fragment relation inside the bucket
  std::uint32_t fragment_count;    // consecutive fragment rows
  std::uint32_t reserved_32;       // pad so logical_length is 8-byte aligned, must be 0
  std::uint64_t logical_length;    // sequence length in symbols
  std::uint64_t first_fragment_row;// row of fragment 0
  std::uint64_t source_epoch;      // immutable source version, must be > 0
};
```

- the internal tables are divided into groups by sequence length and these groups are divided into shards (so I record both of these value to identify sequence)
- the number of row for sequence first fragment and number of fragments (i.e number of rows occupied by sequence)
- sequence length in symbols (logical length)
- version of the sequence in case it's changed (source epoch)
- version of locator format (not relevant for users)

Locator format optimisation

Currently locators size could be reduces as I don't need to store version (it was used for developent) and I waste too much space for padding. It's potential oppornunity to optimsie in the future (it wasn't a priority as locators occupy overall small space in comparison to sequences themselves).

Each sequence as mentioned in section desribing sequence interfaces 6.2 has it's alphabet type and base storage unit. Base store unit is configurable from SQL. User can declare any power of 2 that is width of supported machine integer:

Listing 8.3: Examples of different base storage units

```
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 64);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 32);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 16);
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 8);
-- Example for DNA5 (floor(64/3) = 21 symbols per unit)
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna5', 64);
-- This is illegal as there's no uint128_t. 128-bit unit is just equivalent to
    two 64-bit units.
SELECT 'ACTGTGCAACCCCTG'::bioseq.seq('dna4', 128);
```

Sequence fragments

Sequences are divided into fragments to enable lazy loading. A fragment is 1024 packed storage units, so actual bytes occupied depend by the width of storage supplied by the user. The maximum possible fragment size is for 64-bit storage unit and it's $1024 * 8B = 8192B$ (note that this goes beyond default postgres page size of $8192B$, because page needs to include the page header as well)

Fragment abstraction is a separate layer above columnar storage. Shards for fragments are internally held as `CREATE TABLE ... (seq bioseq.seq(...)) USING bioseq`, so each fragment is one TAM row in that single sequence column.

Storage considerations

Fragments always storing 1024 storage units which works well for long sequences, however if the sequence is short then only small number of storage units will be occupied, while the rest is filled with zeros. Compression (default is pglz, but zstd is avialable as well and others for manual building) will take care of this problem and shrink the fragment. However it still has measureable effects on processing in the pipeline as BioseqDB will waste space in memory for very short sequences. This along with locator format optimisation is a task for future work. If we assume occupancy rate of 50% then minimal sequence size for 32-bit storageshould be $1024 * \frac{32}{2} * 0.5 = 8192$ symbols and for smallest possible storage: $1024 * \frac{8}{2} * 0.5 = 2048$ symbols.

8.4.1. Two levels of sharding

Postgres tables have limits on the number of stored rows, so in the most extreme cases the amount of sequence data can be more than one table can hold, especially if we store many full human genomes. A (single copy i.e haploid) human genome is about 3.1 to 3.2 billion bases (3.2×10^9 symbols and the 32-bit DNA4 unit from the figure above). PostgreSQL documents a maximum table size of 32 TB at $8192B$ default page size.[11] That bound is a result of internal Postgres 32-bit page index.

$$
\begin{aligned}
F &= 1024 \times \frac{32}{2} = 16384 && \text{symbols per fragment} \\
n_{\text{frag}} &= \left\lceil \frac{3.2 \times 10^9}{F} \right\rceil = 195313 && \text{fragment rows per genome} \\
B &= 16 + 1024 \times 4 = 4112 && \text{bytes per fragment row} \\
P &= 8192 - 24 = 8168 && \text{usable bytes per page} \\
n_{\text{page}} &= \left\lceil \frac{n_{\text{frag}} B}{P} \right\rceil = 98327 && \text{pages per genome} \\
n_{\text{seq}} &= \left\lfloor \frac{4294{,}967{,}294}{n_{\text{page}}} \right\rfloor = 43680 && \text{genomes in one table}
\end{aligned}
\tag{8.1}
$$

Without compression, one fragment table can therefore hold about 43,680 human-genome sequences before it runs out of pages. Practically compression rates for human genomes were already measured Kryukov et al., "Nucleotide Archival Format (NAF) enables efficient lossless reference-free

[11] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: Appendix K. PostgreSQL Limits.

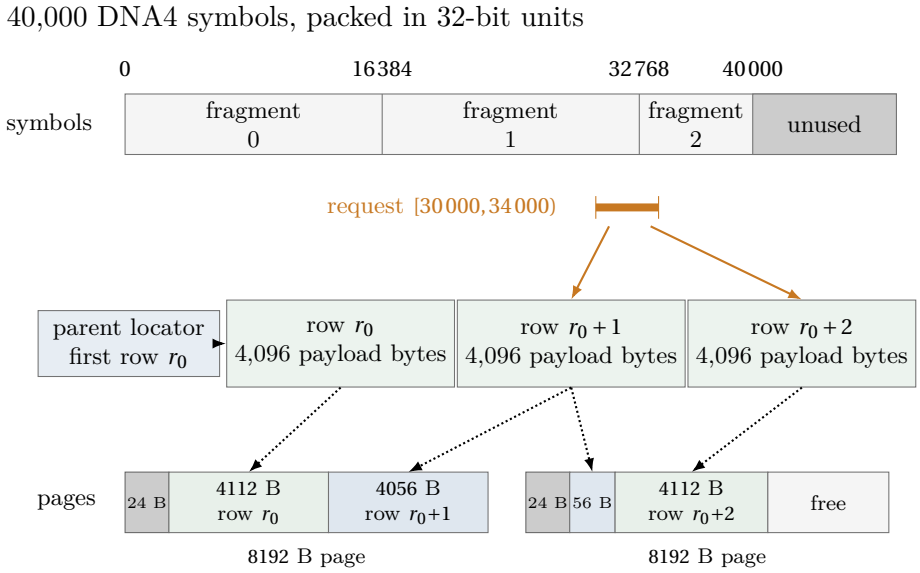


Figure 8.7: A 40000-symbol DNA4 value with 32-bit units uses $1024 \times 32/2 = 16384$ symbols per fragment, so two full fragments and one partial fragment. Request $[30000, 34000)$ reads fragments 1 and 2. Each stored row is 4112 bytes (4096 payload plus the 16-byte sequence header). The TAM concatenates those in column buffers via writer code 21 and during flush, writes them on 8192-byte pages, skipping the 24-byte `PageHeaderData` (8168 usable bytes). One fragment fits in a page, but two concatenated fragments do not ($8224 > 8168$), so fragment 1 continues after the next header. The compression will shrink the last fragment as it's mostly zeroed, but the illustration assumes no compression happened.

compression of DNA sequences" and zstd is capable of compressing a human genome to effectively 790 MB (7.9×10^8 bytes). With that size, each genome occupies 96,719 pages, so the same limit allows 44,406 genomes in one table.

Table limits

I present this calculation to present that hitting the table limit isn't very improbable. There exist datasets bigger than this limit. To prvent users from hitting limit and requiring them to manually spli tables, I decided to provide internal sharding.

The internal sharding allows fragments to spill over into multiple tables. If the underlying table achieves $1TB$ limit (which is lower than theoretical 32 Postgres limit), fragements are stored in newly created columnar relations. The sharding is two-level:

- length bucket: groups sequences by their logical number of symbols - precisely $max(0, min(\lfloor log_2(|s|) \rfloor - 8, 34))$ is the bucket index.
- shard: inside one length bucket BioseqDB can create multiple tables when existing tables reach $1TB$ limit (potentially to store extremally large datasets)

The current layout has 35 buckets. The first covers lengths $[0, 2^8)$ and each next bucket covers $[2^i, 2^{i+1})$ for $i = 8..40$. The largest bucket is designated to store sequences longer than 2^{41} symbols.

Sharding limitations

It's very important to note that current during stream access policy 6.5 the sequence size is not known until its end is reached, thus my code in BioseqDB decides to store the sequence in the largest bucket (34) even when the sequence is short. This code path is utilised by IO submodule that loads sequences from FASTA file. This means that each sequence loaded from FASTA currently will be stored in the largest bucket. I plan to resolve this problem in the future - solution could be adding ability to IO submodule to search for first sequence end and assume all sequences are similar sizes. Alternatively I could write one sequence and then adjust the bucket after size is known. Both of these solutions however add additional overhead and complexity to the FASTA loader module. Additionally length-based sharding can allow better utilisation of fragments memory (so fragments are smaller even without compression 8.4 and they require fewer bytes when allocated in RAM)

8.5. MVCC and deletion for sequence fragments

I discussed MVCC support when talking about Postgres Snapshots in chapter about BioseqDB custom scans 7.3.5. In custom scan case support is rather simple and provided by the Postgres engine itself. It only required me to correctly use it. Storing fragments is entirely separate problem to visibility in columnar storage, because multiple queries can operate on locators, while some other process can modify sequence data in the same time.

Deleting sequence require doing two operations:

- Deleting row for fragment, which under the hood will mark the row as deleted in the row-mask metadata.

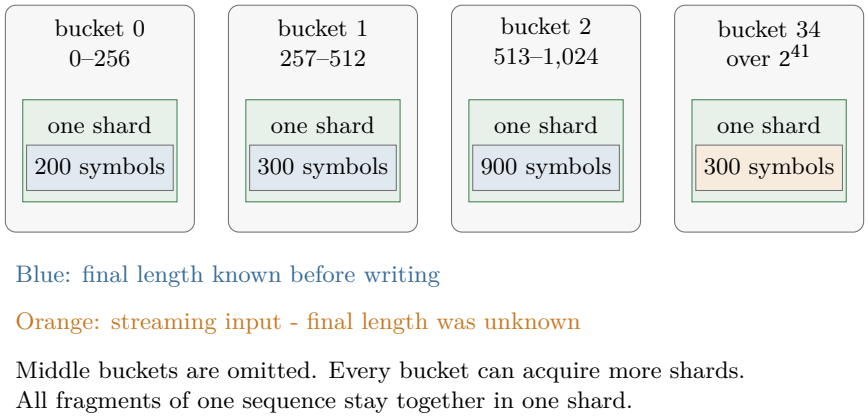


Figure 8.8: Length-based buckets are choosen based on the sequence length log 2, however sequences coming from FASTA/FASTQ have no known length and are streamed, thus internal code always store them in the largest bucket (34). There are multiple tables (called shards) per one bucket depending if we reach $1TB$ limit.

- As fragment is separate abstraction layer than columnar storage, I also store versions of rows in locators, so deletion just needs to invalidate that version (epoch)

Each epoch has stored separate row in the catalog table (so it's one entry per sequence version):

Listing 8.4: Heap catalog for one immutable sequence version. Invalidation updates `state`.

```
CREATE TABLE bioseq_internal.sequence_source_version (
  source_epoch bigint PRIMARY KEY, -- immutable version id, must be > 0
  user_relid oid,                  -- parent user table
  database_oid oid,                -- parent database
  tablespace_oid oid,              -- parent tablespace
  main_relfilenode oid,            -- parent file node
  main_storage_id bigint,          -- parent TAM storage generation
  column_attno smallint,           -- 1-based sequence column
  locator_version smallint,        -- locator format, currently 3
  bucket_idx smallint,             -- length bucket 0..34
  shard_idx integer,               -- shard inside the bucket
  fragment_relid oid,              -- fragment table, NULL if empty
  fragment_storage_id bigint,      -- fragment TAM storage generation
  first_sub_row_no bigint,         -- row of fragment 0
  num_fragments bigint,            -- consecutive fragment rows
  total_bp_size bigint,            -- sequence length in symbols
  alphabet_code smallint,          -- packed alphabet
  storage_unit_byte_width smallint,-- unit width in bytes
  padding_letters integer,         -- unused letters in last packed unit
  source_content_checksum bigint,  -- CRC-64 of decoded symbols
  state smallint                   -- 0 unpublished, 1 ready, 2 invalid
);
```

The specific epoch's `state` follows very simple linear transitions:

- Unpublished (0). An insert has allocated an epoch (sequence version) and written fragments. Locator is then filled with state 0 which is default one. It means that the

locator in the user table is not yet written (so the fragment is there, but there's no saved locator that points to it)

- Ready (1). Locator was written to a table, so I update the epoch state to mark that this sequence version is used somewhere (not doing that might lead to dangling fragments that occupy storage but have no existing locator). This state is crucial to prevent a situation in which locator write fails or is rolled back, but fragment was written.
- Invalid (2). Locator cannot read this row version anymore, the underlying fragment shouldn't be accessed. This happens when user either calls `DELETE` or `UPDATE`.

Note that the catalog contains far more information that is neccessary. I decided to save CRC hashes to make sure the fragment is not malformed during operations or shard coordinates shared by the locator itself. As each row version requires separate entry this limits the maximum number of sequences stored in the BioseqDB, however entries are cheap so I didn't incorporate it into estimate calculations 8.1.

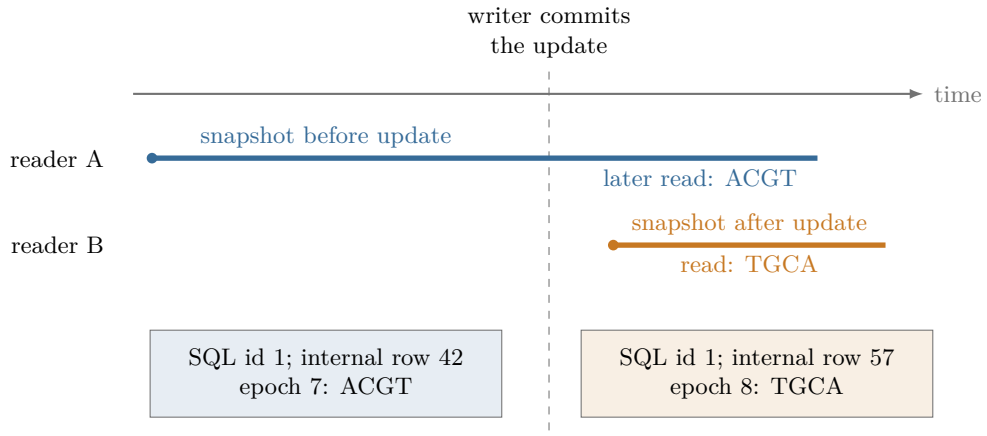


Figure 8.9: Postgres snaphot mechanism allows to check visivility of rows in MVCC model, but it's not sufficient on its own. Let's assume that reader A and reader B are reading the same sequence before and after other process commited update. Using fragments epochs versioning system means that read A will see older locator via snapshot, while reader B will get newer locator row with changed epoch. Both of them can complete reads as both epochs are still valid.

Listing 8.5 gives SQL code for this example:

8.5.1. BioseqDB all cache layers

The optional column cache stores decompressed stripe chunks. It is separate from the scanner-owned packed fragment slots in Figure 6.5.

The setting `bioseq.column_cache.enabled` controls the decompressed cache. The original benchmark campaign disabled it, whereas binary releases enable it by default. It stores decoded chunks inside one PostgreSQL backend process. Its key contains relation, stripe, chunk group, and column identifiers (the configured capacity when enabled defaults to 200 MiB)

The cache can return borrowed pointer to a decompression buffer for a chunk, which allows to prevent additional chunk loads wasting time on decompression multiple times. As

Listing 8.5: A repeatable-read transaction keeps the old sequence version. Initially, row id 1 begins with ACGTACGT.

```
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT

-- Session B, while A remains open
BEGIN;
UPDATE reads
SET seq = repeat('TGCA', 4096)::bioseq.seq('dna4', 32)
WHERE id = 1;
COMMIT;

-- Session A
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- ACGTACGT
COMMIT;
SELECT bioseq.to_text(seq[0:8]) FROM reads WHERE id = 1;
-- TGCATGCA
```

I discussed all methods of caching within BisoeqDB already, I present table with comparison of all cache layers:

Table 8.1: Stored bytes and reusable memory have different owners.

Object	Retained data	Scope
PostgreSQL buffer	Relation page bytes	Internal buffer manager in Postgres pins each page and loads/unloads pages on demand so multiple accesses to the same page are efficient
Column cache 8.5.1	Decoded column value buffer	Can cache decompressed chunks, so loading and decompression of the same chunks can be efficient
Fragment slots 6.5	Copy of one packed fragment	Scanner-owned packed copies reused within a read session, independently of the column-cache setting.
Query-result cache 7.5.1	Completed SQL result rows	Separate `bioseq_cache` subextension can store and share results after pipeline execution is finished so same or similar queries can be efficient (experimental and disabled by default)

Chapter 9

Persistent sequence indexes

9.1. Keep the lookup work between queries

In previous chapter 8.1 I describe how columnar and sequence storage layers are implemented in BioseqDB. There's however also aspect for storing indexes for sequences. We would like to be able to store computed minimizers (seeds/sketches) 3.1 in a way that enables performant computations of alignments. They should be computed once when sequence is loaded from file or updated and then reused for subsequent select queries.

I want to discuss existing aligners' indexing first.

9.2. What existing sequence indexes store

9.2.1. Indexing: minimap2

After computing minimizers they need to be grouped by their hash, minimap2 choose to use the lowest b hash bits (by default $b = 14$) to choose one of 2^b buckets.[1] Minimap2 builds a hash table per bucket whose keys are the remaining hash bits. This buckets with hasmaps are serialized into `.mmi` file. I performed in-depth analysis of how the `.mmi` format is specified based on available Github old issues[2] and original code from minimap2's `mm_idx_dump` and `mm_idx_load` functions.

Writing and loading an index part
mm_idx_dump, mm_idx_load 475

The index file is concatenation of blocks presented in the Figure 9.1:

- Magic bytes The bytes are `MMI` followed by `0x02` (`MM_IDX_MAGIC`).
- Metadata Minimap2 saves w, the k-mer length k, used bitcount b, the number of sequences n_{seq} and a flag word (for various minimap2-specific bitflags)
- Sequence table For each of the n_{seq} sequences the file stores 1-byte name length ℓ followed by ℓ ASCII characters representing the sequence name (header taken from original FASTA file), and a 32-bit sequence length
- Buckets The buckets store the hash-map, where key is remaining hash and bit indicating if there's single value for that key or not. The hash-map is stored as two components however. First is the value list p which is used when there are more than 1 value per

[1] Li, "Minimap2: pairwise alignment for nucleotide sequences"; Li, minimap2.

[2] minimap2 contributors, How to interpret the mmi index file? Issue 820; Lanius, parse_mmi.py: Python parser and lookup functions for minimap2 indexes.

key. Second is the hash array itself, which stores key/value pairs. If there are more than one value per key (last bit of key set to 1), then the value represents offset and count into the value list p The values, either as values in the key/value pairs or entries in p store sequence id, position and strand bit.

- Bit-acked bases (optional) If the `MM_I_NO_SEQ` flag is unset manually by user, bit-packed reference bases are saved at the end.

The depiction of the index file layout is presented in the Figure 9.1 below.

The idea of storing hash-map as two lists inside bucket is also presented in the Figure 9.2 below to allow easier understanding of the index concept.

9.2.2. Indexing: mmseqs2

There are alternative approaches in literature to building an index. MMseqs2 stores one index file that contains counts and offsets, and then writes larger index files that contain all occurence lists instead of bucketing them like minimap2 does. It's worth noting that 2017 MMseqs2[3] paper uses spaced k-mers which wasn't the focus of this thesis, so I didn't investigate more how the alignment is precisely done.

MMseqs2 offers great performance in clsutering, which is different problem to alignment, however connected as it involves estimation of edit distances for large datasets. The software also allows sets of taxonomy operations and alignment-based searches as well.[4][5][6][7]

Count and fill the MMseqs2 index
IndexBuilder::fillDatabase 56

Read a word's entry range
getDBSeqList 178

Keep distinct word codes within one target
addSequence 350

I illustrate the idea of MMseqs2 index on a small example in the Figure 9.3 below, where I just assume that we use 2-letter k-mers of natural language for simplicity.

9.3. BioseqDB indexing abstractions

Indexes are stored separately in BioseqDB 8.1 to all internal sequence fragmented tables 8.8 and they are separate entities to the usual indexes in Postgres which provide generic access paths to the tables.

There is thin layer of abstraction over the indexes, so in the future implementation of other types of indexes is easy. The storage abstraction requires implementer only to specify the type and depending on it we only need to provide logic to write the index chunks. One problem with current state of BioseqDB is that the index reading is not abstracted, so we encapsulate details of implementation, but they leak into pipeline (as the pipeline currently needs to understand how we store minimizers).

[3] Steinegger and Söding, "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets"; Steinegger and Söding, "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets."

[4] Steinegger and Söding, "Clustering huge protein sequence sets in linear time."

[5] M. Mirdita et al., "Fast and sensitive taxonomic assignment to metagenomic contigs."

[6] Milot Mirdita, Steinegger, and Söding, "MMseqs2 desktop and local web server app for fast, interactive sequence searches."

[7] Kallenborn et al., "GPU-accelerated homology search with MMseqs2."

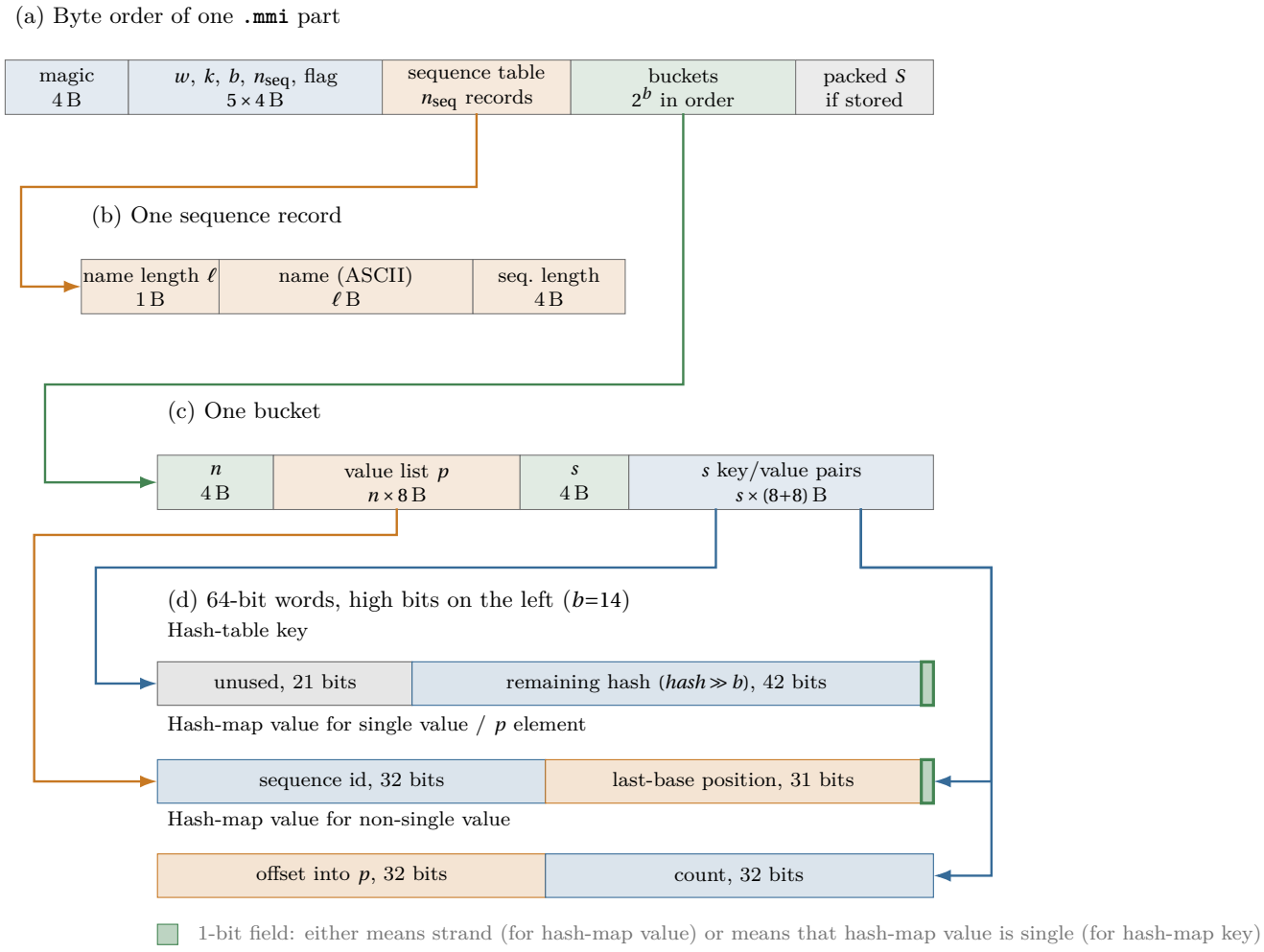


Figure 9.1: Interpretation of one `.mmi` index. For very large datasets index files are split into multiple parts where each of them follows the same format. Minimap2 needs to store sequence ID (RID 3.1), because the index parts are generated for a set of sequences, not just one, so each part needs to identify which reference sequence the minimizer comes from. Arrows represent how specific elements of arrays look like in close inspection. Panel (a) depicts high-level structure of the index 9.2.1. Panel (b) presents records storing sequence information. Panel (c) shows each bucket of minimizers, which stores its own hash-map via two lists. Panel (d) shows how elements of p looks like and key/values in the hash-map list. Hash map key is always remaining 42 bits with bit indicating that there's single value for that key. If the bit is set, then value in the key/value pair is (sequence id, position, strand bit), if not, then value points via offset/count pair into the list p. The list p stores the same (sequence id, position, strand bit) values.

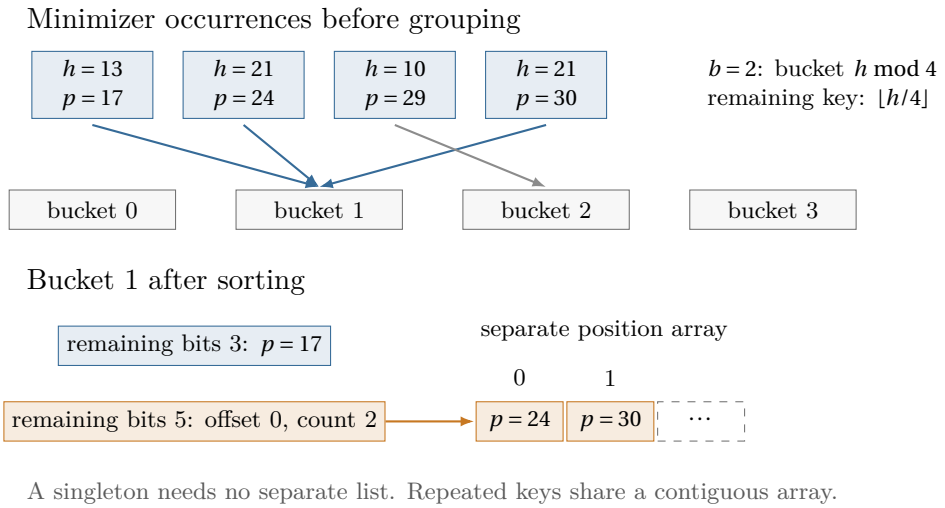


Figure 9.2: A simplified minimap2 bucket: hashes 13 and 21 have the same lowest two bits, but different remaining keys, so they land in bucket 1 together. The two occurrences of hash 21 become adjacent in the bucket's position array.

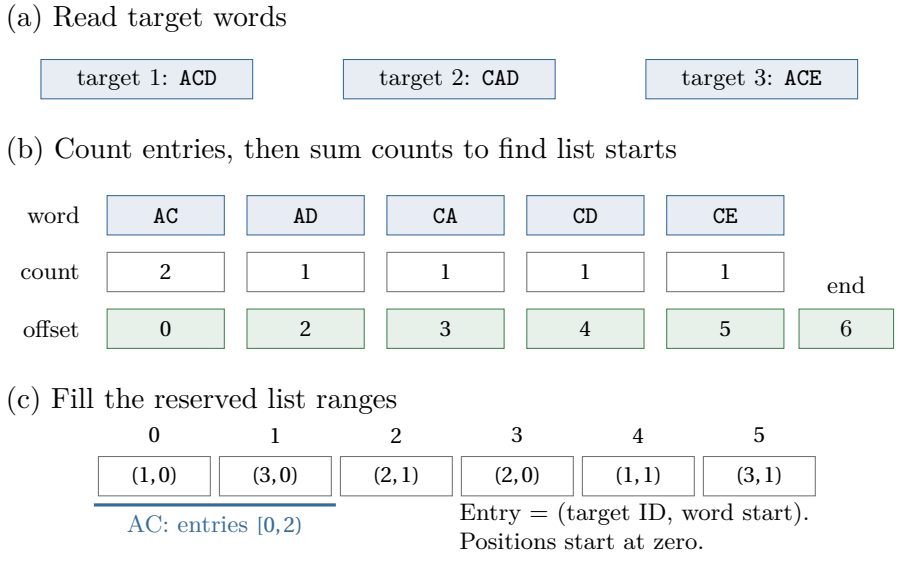


Figure 9.3: A small example of MMseqs2's lists construction, where word/count/offsets triplets are stored in single file, file the actual (very large) lists of entries are stored separately

Seed abstraction

In the future I want to improve curent implementation to use abstract notion of ”seed”. I already extensively use templates, so introducing new concept to represents types of seeds isn’t a problem. This abstraction will allow to fully decouple indexing from the pipeline. Pipeline stages then could automatically detect how to compute certain seeds (like minimizers) and how to read and store them. Current BioseqDB implementation only uses minimizers, so I decided not to decicate my time to this idea.

I assume here that index store chunks which are separate abstraction to the chunks in columnar tables. One chunk consists of bytes the index implementation needs to write or store and I assume the chunk cannot be larger than $960MB$ (due to Postgre’s TOASTing limits). The chunks are stored in internal tables that does not use columnar storage, they’re ordinary heap tables - I decided that columnar storage in this case won’t give any advantage (although I didn’t A/B test this approach). Additionally, index tables are create `UNLOGGED` to avoid unnecessary logging and to reduce the amount of data written to the disk.

There are several supported methods of storing index:

- **`CHUNKED_PER_FRAGMENT`**. One fragment 8.4 of a sequence can have multiple chunks in the index.
- **`INLINE_PER_SEQUENCE`**. One sequence has a single chunk that is stored as Postgres varlena. This means that entire data stored for one sequence cannot exceed $960MB$ if that happens BioseqDB will error. `symbol_counts` is BiosqeDB index that uses this layout. It stores how many times each symbol occurs in that sequence and the payload is so small that it nicely fits into one small row.
- **`CHUNKED_PER_TABLE`**. This is special layout that in its implementation exposes only simple methods to access random chunks of the table. The idea is that some indexing structures may want to implement custom strategies, for example trees of sequences or graphs of fragments that do not fit into other representations. I don’t use this layout at the moment, but it’s supported to provide extension point in the future.

Figure 9.4 shows what three of these layouts are representing.

In reality the fragment’s number can be viewed as a generic sub-key. This distorts the original meaning of **`CHUNKED_PER_FRAGMENT`**, because in relity it’s just ”per-sequence” but with extra key: ⟨query_key,
sub_key,
chunk_no⟩, while **`CHUNKED_PER_TABLE`** has ⟨chunk_no⟩. This idea is depicted below by real SQL schema of internal storage:

Listing 9.1: One heap row of a per-fragment index.

```
CREATE TABLE bioseq_internal.seqidx_mpf_... (
  query_key   BIGINT NOT NULL, -- which sequence in this shard
  sub_key     BIGINT NOT NULL, -- which item of that sequence (can be fragment
    number but doesn't practically have to)
  -- Then we store byte array defined by index implementation but split across
    chunks:
  chunk_no    INT    NOT NULL, -- 0, 1, 2, ... of that item's byte string
  chunk_bytes INT    NOT NULL, -- how many bytes this chunk holds
  blob        BYTEA  NOT NULL, -- the bytes of this chunk
```

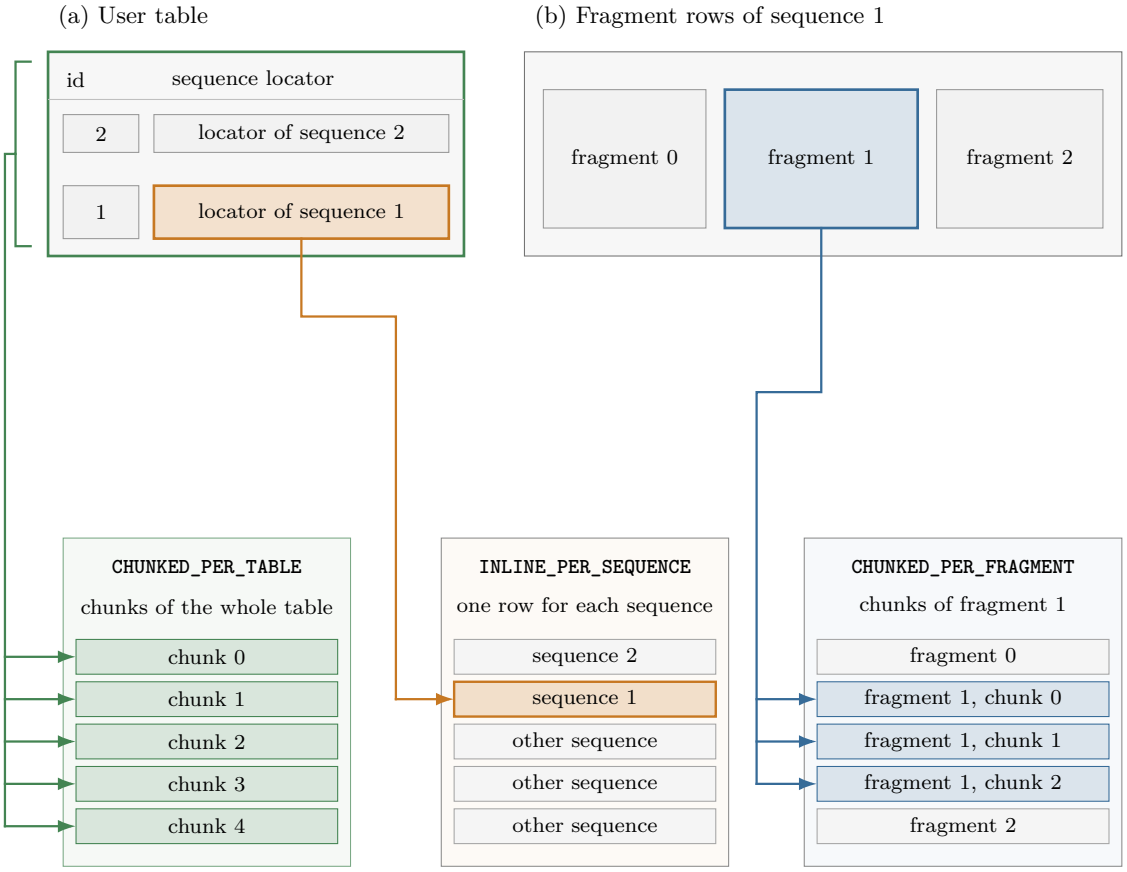


Figure 9.4: Each index is drawn as five rows. The green brace represents the fact that the index structure represents all sequences stored in the table. The orange arrow shows that one locator corresponds to exactly one chunk (one row in the index table). The blue arrow shows that one fragment can have multiple chunks. In case of `CHUNKED_PER_TABLE` we only store chunk number and for `CHUNKED_PER_FRAGMENT` I store fragment's number and chunk number.

```
  PRIMARY KEY (query_key, sub_key, chunk_no)
);
```

The payload of an index is one byte array. `CHUNKED_PER_FRAGMENT` stores that array as rows of Listing 9.1. When the array is longer than one chunk, it is divided into chunk 0, chunk 1, and the following chunks. Those rows keep the same `query_key` and the same `sub_key`. A reader orders them by `chunk_no` and concatenates `blob`.

Storage details

It's important to note that for minimizers per fragment MPF the sub-key number is the fragment number. I use the same column to store another number when the index is not split by fragment. Technically this means that **`CHUNKED_PER_FRAGMENT`** should have its name changed to something more appropriate.

In code `pg_supported_indexes` is a special templated that connects those implementation classes to the registered indexes. This is frequent practice used by me - encoding implementation vs. abstraction pairs as tuples in type lists created using template metaprogramming, so if extension is required we only need to write new type and add it into the `pg_supported_indexes` template list. Checks if implementation is valid are done via C++26 concepts 9.2:

Listing 9.2: Minimizers per fragment require fragment-chunk storage.

```
template <class p_storage>
struct minimizers_per_fragment_index {
  static_assert(is_index_chunked_storage_for<
      p_storage, sequence_index_layout::fragment_chunks>);
  static constexpr sequence_index_layout index_type =
      sequence_index_layout::fragment_chunks;
  using storage_type = p_storage;
};
```

9.4. BioseqDB supported indexing

There are 3 implementations of indexes inside BioseqDB:

- **`minimizers_per_fragment`** (shortened to `mpf`) stores minimizers calculated per one fragment and uses **`CHUNKED_PER_FRAGMENT`** layout.
- **`minimizers_per_sequence`** (shortened to `mps`) stores minimizers calculated per sequence, but as the number of minimizers grows they would not fit in a single chunk, thus one may consider using **`CHUNKED_PER_TABLE`**. However, as mentioned, the sub-key 9.3 is a number, so in MPS I use this number as a hash-group id, not as a hash. The rows stay in the **`CHUNKED_PER_FRAGMENT`** layout even though these numbers are not fragment numbers, as seen on the figure below 9.5.
- **`symbol_counts`**: stores counter of occurences of each symbol in a sequence, uses **`INLINE_PER_SEQUENCE`** layout, because the counter struct is very small, so Postgres inlines the varlena into the row.

The main idea behind MPF and MPS was:

- MPF: I can keep small hashmaps separated so random queries during seeding stage, where we need to find common minimizers between query and target, can hit CPU caches more often as the accesses are localized.
- MPS: The most frequent minimizers are trimmed by minimap2 and MPF trimming is done per fragment, so it's less reliable and can remove minimizers that are locally frequent, but globally rare reducing total number of minimizers and speeding up processing at the cost of potentially missing some matches. Additionally, MPF needs to perform multiple hash lookups if we want to determine all locations of minimizer's occurences so there's also performance penalty. This penalty can be also mitigated by the fact that MPF is easier to parallelize. Thus we can see there are several important trade-offs and it's not clear if the implementation of MPS should be more or less performant in theory. In case of BioseqDB as it would be very inneficient to store all minimizers individually, I decided to still generate minimizers per fragment, but store them and merge later during process that can potentially spill to disk (if the entire index fits into RAM then no spillage occurs, but if it doesn't there are heavy performance penalties). Additionally I must note that MPS uses the trick described here 9.3, so the sub-key is a hash-group number, not fragment number. This implies that if the query is short there will be not many groups to check, and as each group is held as heap rows, then MPS can be very fast, because it reads only the list of one group, not the entire index.

The figure 9.5 below shows 6 minimizer occurrences in a sequence, and panel (c) shows the byte strings that store them. Panels (a) and (b) show the same six minimizers in two groupings. Panel (c)and later illustrate the encoding of the strctures in binary strings (every rectangle onward is the binary string `blob` of one heap row from Listing 9.1), where the label on the left is that row's `query_key` and `sub_key`. If a string is longer than one chunk, later chunks continue it under the same two numbers.

Binary coding of non-grouped MPF

There are two types of MPF index depedning on BioseqDB setting flag: `bioseq.aligner.grouped_soa_generation.enabled`. The non-grouped version is depicted in panel (c). The sub-key is just the fragment number. The entry intself contains two arrays similar to what minimap2 9.1 does. The first array with `controls` contains hash value, start position in `positions` array and count of how many positiosn have that same hash. `positions` just represent minimizer positions. Usually in headers there's additional metadata and some of it is redundant as the exact shapes evolved during my implementation. There are versions and special magic values to potentially error if index was corrupted.

Binary coding of grouped MPF

I tried to improve MPF by doing something more sophisticated. The MPF can changed format based on grouped generation setting flag. Panel (d) shows how grouped MPF looks like:

- This is similar to MPF buit instead of storing array of structures (AoS) we store structure of arrays (SoA) so hash, start, and count are now three arrays in a row.
- I use special bit i.e 2^{61} to store positions array separately in separate row.

- There also is one special directory row using unique `sub\_key` value. The directory is list of $128B$ entries mapping fragment number to specific `sub\_key` value.
- Each directory row $128B$ entry constains useful metadata like min(hash), max(hash) so some rows doesn't have to be read at all if they don't contain hash values we are interested in.
- Using this `sub\_key↔fragment` mapping allows to potentially split fragment hash/start/count entry into multiple ones. However this is not used in the code currently and effectively `sub_key` = `fragment` for all entries.
- The directory item also contains information that is redundant for example `controls\_first` is number of first chunk storing hash/start/count binary string. This is always `0` as the binary strings are split into chunks, which are zero-indexed. The same logic applies to other fields and the entry size could be massively reduced.
- Grouped-MPF is very generic format and it also supports variable-width entries. The width of hash/start/count/positions can be adjusted independently for example for very short sequences we could use $2B$ `position` array and for very low minimizer density we could use $2B$ for `start\_array` or for very unrepetitive sequences where unique minimizers are very common, we could use $2B$ for `count\_array`. Practically, current code uses currently same widths for most of the fields and hash is adjusted depending on k and sequence length from minimizer type 3.17.

On the figure Panel (d) I didn't depict all fields in the MPF grouped directory as not all fields are relevant. Table 9.1 below lists all fields that are stored:

Binary coding for MPS

The MPS index stores the same rows as non-grouped MPF, but this time we also have additional one directory row. Also the fragment keys are just not reffering to fragments anymore, but they represent groups of hashes similarily to minimap2 9.1.

The directory contains metadata information like how many distinct hashes are there in total and for every group it stores min(hash), max(hash) to allow us to quickly skip groups if certain hash values do not exist there.

MPS index building needs to spend additional work on sorting and merging so later queries can skip unrelated hash ranges, while MPF just keeps smaller hashmaps based on fragments. The current default BioseqDB active minimizer index is MPF.

9.4.1. Index versioning

Internal sequence storage tables use `epoch` to provide sequence versioning 8.2, indexes don't have to use separate versioning field. I decided not to store epoch in the index table 9.1, but instead `query_key` is number of the row inside the table shard. They are stable with respect to `INSERT`s, `DELETE`s and `UPDATE`s as update will create new sequence version. Special handling is only required when tables are rewritten using `VACCUMFULL`: vacuum full would cause the rows to change numbering, however the indexes tables contain a hash of database, table space and Postgres `relfilenode`, so if the table is rewritten (`relfilenode` will change) BisoeqDB won't detect the old index. This is the most naive approach, but provides basic protection against table rewrites.

Table 9.1: Fields of one 128-byte grouped MPF directory entry.

Bytes	Size	Field
0–3	4	partition number (this is an index of entry in the directory row's array)
4–7	4	fragment
8–15	8	min(hash)
16–23	8	max(hash)
24–31	8	sub-key (in theory was meant to use to allow dividing entry for fragment into multiple rows, but currently it's always equal to frament number)
32–35	4	first chunk of the control byte string (chunks are zero-indexed so it's always 0 and should be removed in the future)
36–39	4	how many control chunks (counts in how many chunks hash/start/count binary array will use)
40–43	4	first chunk of the position array (chunks are zero-indexed so it's always 0 and should be removed in the future)
44–47	4	how many position chunks (counts in how many chunks position binary array will use)
48–55	8	how many controls (number of items in each of hash/start/count arrays)
56–63	8	how many positions (number of items in the position array)
64–71	8	byte size of the control byte string
72–79	8	byte size of the position array
80–87	8	checksum of the control byte string
88–95	8	checksum of the position array
96	1	hash width: 2, 4, or 8
97	1	start-array width: 2, 4, or 8
98	1	count width: 2, 4, or 8
99	1	position width: 2, 4, or 8
100	1	Used to represent differnt orders of hash/start/count arrays. Now meaningless and fixed to 1
101	1	Used to reresent how many positions there are per one hash entry. It's always 1, so this is meaningless.
102	1	Meaningless now, fixed to 2
103	1	Number of extra entries in start aray at the end. Now it's meaningless and fixed to 1 (and we don't need to store this one extra item at all)
104–127	24	zeros

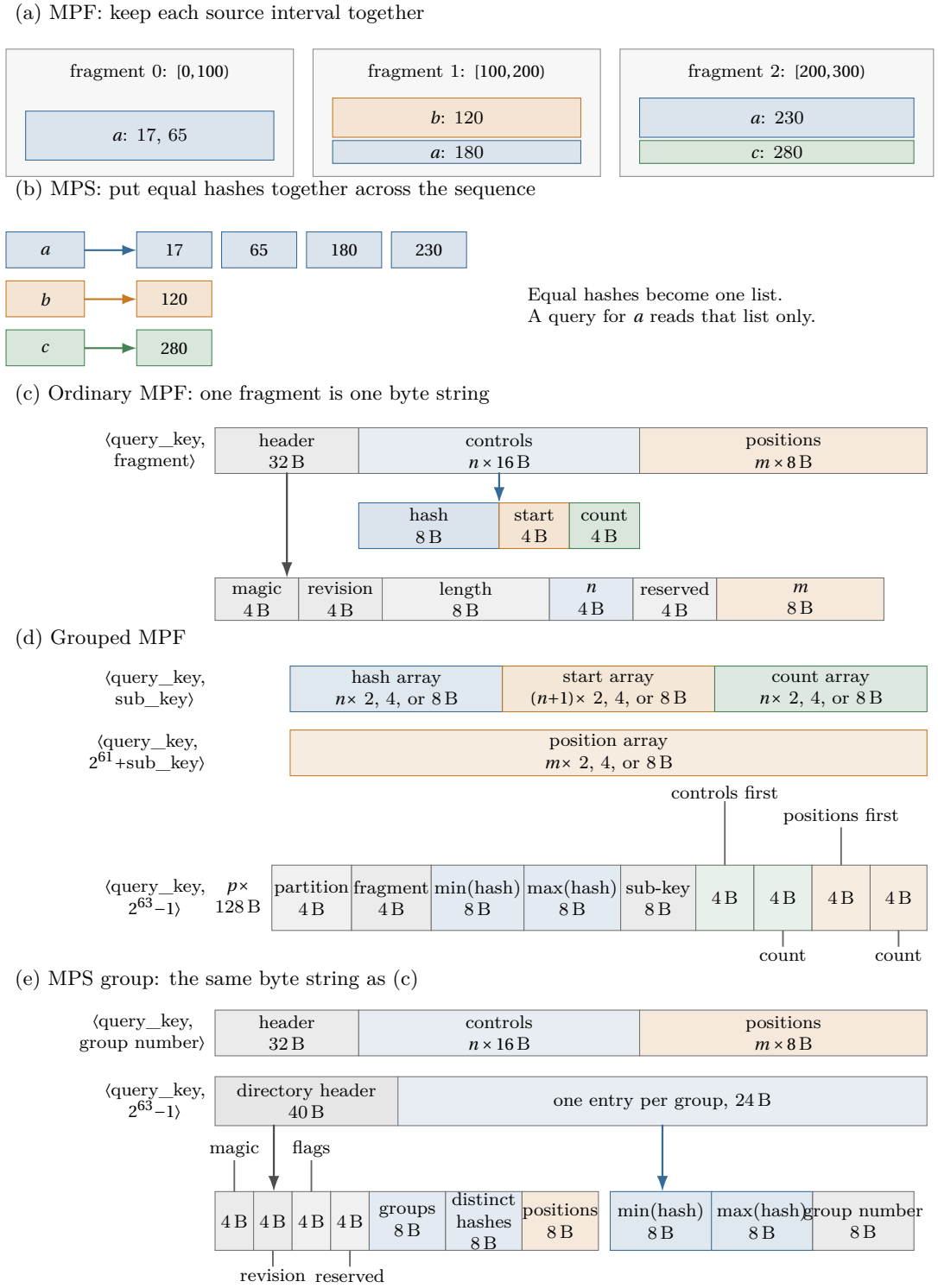


Figure 9.5: Each row with two key in brackets ⟨query_key, sub_key⟩ represent binary string that will be split into multiple chunks that could be find by complex key ⟨query_key, sub_key, chunk_number⟩. There's always one special sub-key for directory for grouped MPF and MPS, these formats also store two rows per one sub-key.

9.4.2. Symbol count index

I implemented one index type that does not store minimizers at all: `symbol_counts` index keeps a very small symbol histogram for one sequence. Each symbol has 64-bit counter (for DNA4, the counters are A, C, G, T so sequence `AACGTA` gives $(3, 1, 1, 1)$). The counts are stored separately because as we use heap rows the value will be stored inline in Postgres table. This allows very fast access and usually users don't need to count symbols.

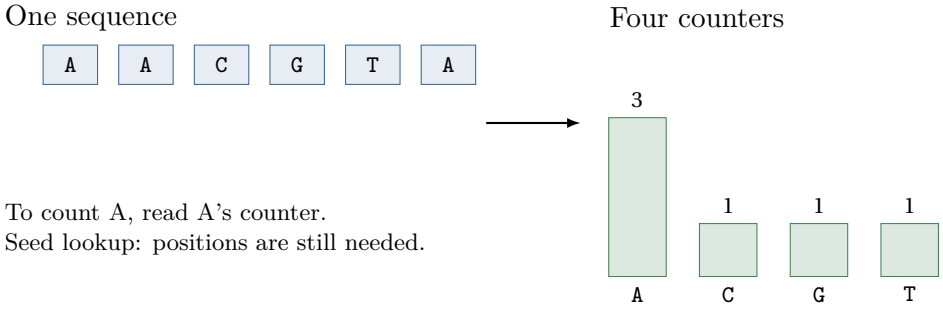


Figure 9.6: Hitsogram counter for symbols is stored separately so BioseqDB access it only when specified in the query

Listing 9.3: Request a symbol count for each stored sequence.

```
SELECT id, bioseq.symbol_count(seq, 'A') AS a_count
FROM reads;
```

Using this small metadata index allows me to never access the sequence content at all, meaning the read is very fast. If the index entry is not found, then BioseqDB will traverse entire sequence. The primary benefit is that this can be computed when loading FASTA or inserting rows autmatically so the user pays the cost only once (or later when infrequent `INSERT` or `UPDATE` happen)

9.5. Building MPS

Building MPF index is rather simple as we traverse each fragment and can separately construct hashmaps with minimizers (and hashmaps for fragmenst are relatively small compared to all RAM capacity). Constructing MPS is more complex as I need to somehow store all minimizer globally and this can potentially be bigger than entire RAM capacity. Index builder could collect every $(hash, position, strand)$ record and sort the complete vector, but for maps bigger than RAM this couldn't be done efficiently, instead current code does similar thing to MPF with a slight differences:

- To reduce memory access costs as much as possible MPF treats each fragment as entirely separate sequence, thus minimizers on edges won't be generated (the first and last $k + w - 2$ symbols). This reduces minimizer density at edges but means that we access only one fragment to generate index entry.
- For MPS I use subtly different approach and I collect all minimizers. This means that during parallelized fragment read, the reader need to read not only own fragment, but also parts of fragment to the right and left. This is depicted as "context" on the figure below 9.7.

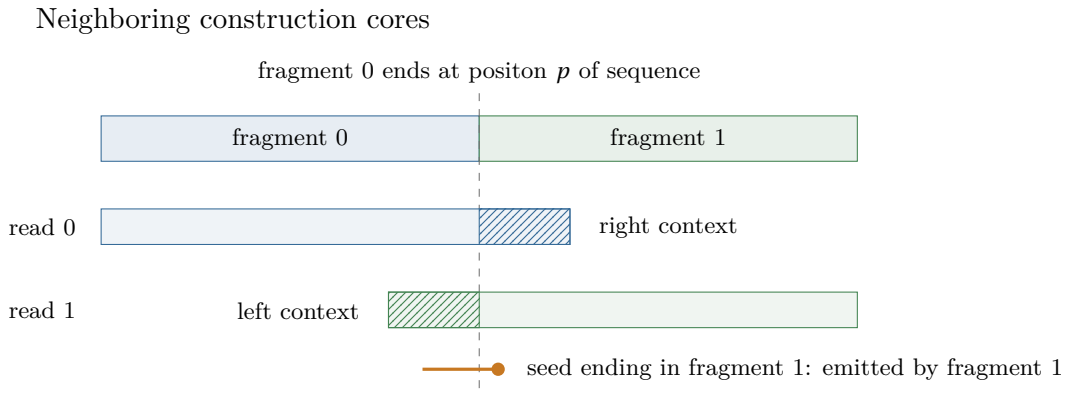


Figure 9.7: Each reader to generate minimziers for one fragment in MPS read also neighbouring fragments so first and last minimizers in fragment can be generated. Size of "context" is $w+k-2$ symbols.

MPS does the following steps to build entire index:

1. Read fragments and for one fragment compute minimizers, sort them by hash and remove duplicates similarily to MPF.
2. Write this computed MPF-like part into temporary file storage.
3. After all fragments have temporary files, then select fixed number of them and merge them into one sorted temporary file.
4. Continue this merging procedure until one temporary file remains.
5. Read the final temporary file and build directory row inside storage.

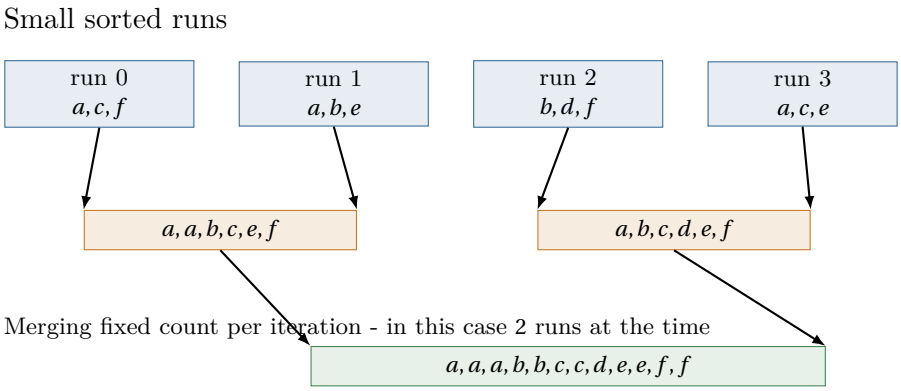


Figure 9.8: Merging of hash entries (hash value and minimizer position)until one final entry is left and can be stored permamently in index storage. Letters represent different hash values.

The spill path uses PostgreSQL's `BufFile` temporary-file interface. The sorter retains sorted runs in RAM when memory admission succeeds and spills them when it cannot reserve that memory. It can also move a retained run to a temporary file to make room for later

work. Thus the merge structure in Figure 9.8 does not require every run to be written to disk.

9.6. Selective reads

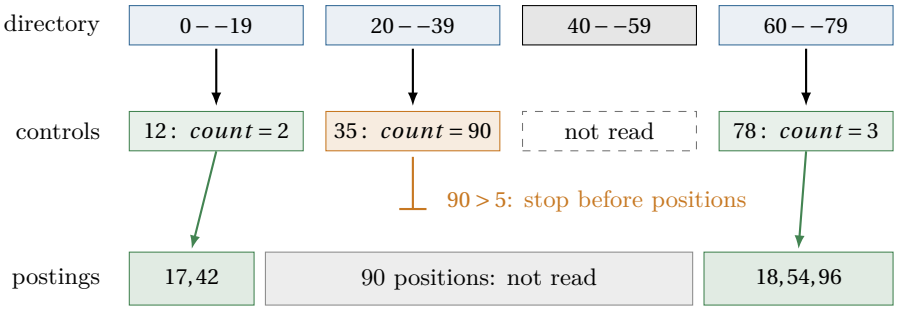
Main benefit of MPS is that each group can be a single row within Postgres (if n, m are not causing the array to spill over multiple chunks 9.4). This means that minimap2 has to read and check every entry in index, while BioseqDB can only read entiore directory row and using $min(hash)$ and $max(hash)$ values to decide if we should read certain groups of minimizers.

The similar logic applies to grouped MPF index as well, but in case of grouped MPF we can also check count array and if the count is very high, then we can abandon readind reparate position array at all (the most frequent minimizers should be filtered out as they are not good candiates for anchors).

The hash-major MPS reader performs the following steps:

1. Read and check the complete directory, then find hash ranges that intersect with query hashes.
2. Read the selected controls to find each key's exact global occurrence count
3. Apply the occurence count threshold: if the count is high, we don't process this hash
4. Read the chunks selected by this filtering and read positions form them

Query hashes {12,35,78}; occurrence limit 5



Query filters out the most frequent minimizers according to configured threshold and skips min-max ranges of hashe

Figure 9.9: Two decisions save different reads. All directory entries are read; blue ranges select the controls. Counts reject a frequent hash before its long list is loaded. Positions and thresholds are illustrative.

9.7. Other index considerations

Indexes can automatically update themselves on index, however currently enabled by default is grouped MPF index, which due to bugs cannot correctly return minimizers which leads to recomputing them when no explicit index read policy is specified and it doesn't support update on insert. More details are presented in the Appendix 1, where I also present concrete examples of how the BioseqDB should be used.

This bug was caused by the fact that grouped MPF not only produces relational index, but also saves that compressed relation chunks into separate file that is used to enable faster reads during JOINs. The reader by default rejects ordinary single table queries, so you need to change the GUC to disable grouped-MPF (and use regular MPF instead). Saving file enables the JOIN to concurrently access stored data which is a cheap workaround around the decision to only use single Postgres reader process.

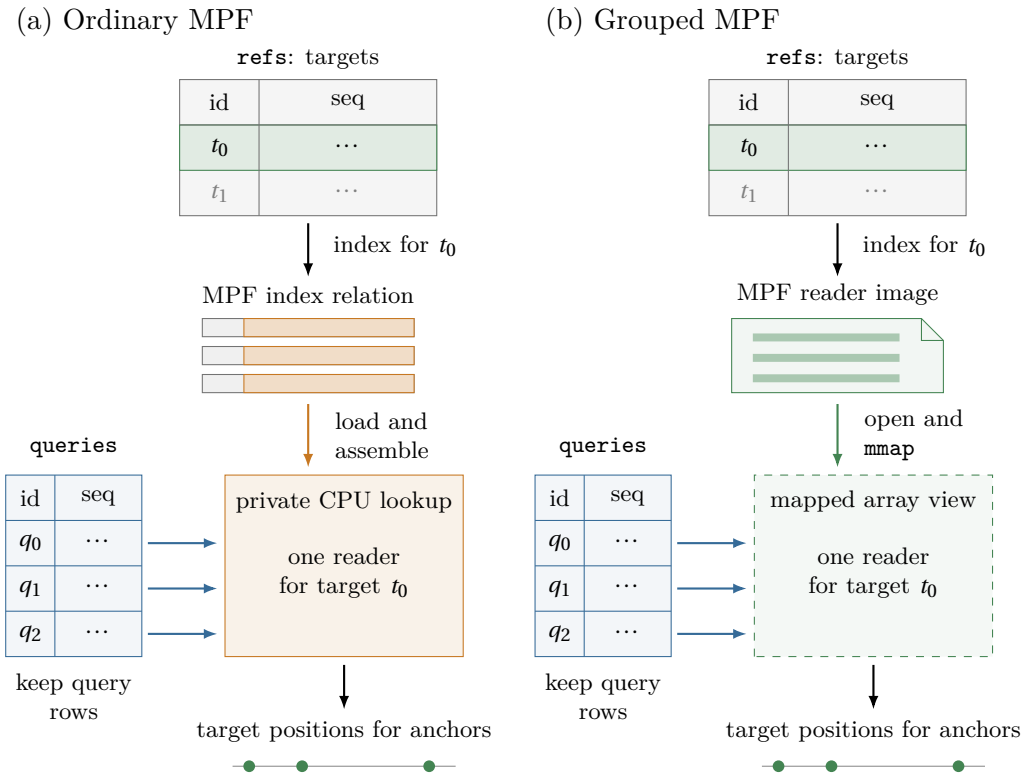


Figure 9.10: For large joins grouped-MPF uses saved direct binary dump of relational index, which speeds up the alignment by reducing time to sequentially read the backing table

All indexes in BioseqDB can be automatically updated on insert (but as noted earlier, grouped-MPF will produce error on insert due to buggy implementation)

Table 9.2: Maintenance policies

Policy	Meaning
`UPDATE_ON_INSERT`	Automatically compute and store minimizers for new sequences
`UPDATE_MANUALLY`	User needs to manually call `bioseq.update_index`

9.7.1. Reading saved minimizers

The query and target must agree on their minimizer scheme and parameters otherwise the reader will refuse to read the index or emit the error. By default read requirement allows both reading the index and in case if does not exist or is incompatible, recomputing all minimizers. This behaviour however has some noticable bugs 2.14.

Table 9.3: Default policies in BioseqDB

Index	Maintenance policy	Default read role
`minimizers_per_fragment`	`UPDATE_ON_INSERT`	Active minimizer index
`minimizers_per_sequence`	`UPDATE_MANUALLY`	Not selected
`symbol_counts`	`UPDATE_ON_INSERT`	Used by supported symbol-count operations

Repeated lookups on wider hashes For supported 32-bit hashes, grouped MPF provides a sequence-wide direct lookup from a query hash to its position list. Wider hashes use the directory row, which searches fragment metadata. This means that queries can therefore read the same data again and again

Index in RAM Mapping a reader file does not load all its pages into RAM. First accesses can require disk reads, while later accesses may use the operating-system page cache. Restarting PostgreSQL discards process-local state but does not guarantee an empty operating-system cache.

Chapter 10

Benchmarking and performance evaluation

10.1. Benchmarking and performance evaluation

10.1.1. What the results show

BioseqDB was evaluated on JOIN between query and reference tables where reference table contained only one sequence and reference table contained 1024 randomly generated sequences. The cases where divided by me into two groups:

- Index: build the reference index first, restart PostgreSQL, then measure the query, including access to the stored index.
- No index: measure construction of a temporary reference index and the query together. The query batch shares that temporary index.

Overall the main outcomes are as follows:

1. BioseqDB is the first tool to my knowing that integrates database with aligner software
2. Memory admission system and custom grouped-MPF index allow to achieve speedups for already built index by 3x to 8x compared to minimap2 in typical cases and 100x where heavy swapping occurs for long sequences
3. BioseqDB for preexisting index beats all common aligners and doesn't require any data preparation (other tools required customized scripts to split the long sequences into chunks)
4. BioseqDB without index is usually slower than minimap2, but faster than other similar aligner tools (this is acceptable due to nature of the typical use cases)
5. BisoeqDB exposed failures in terms of very inneficient GPU implementation and anomalous implementations of some of other coinfigurations (like scalar)

What an indexed minimizer comparison measures

Scalar and SIMD-ref have similar indexed times in this example, where they reuse the same kind of reference index and share most of the query pipeline.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$
MPF / Scalar Scalar-DP / Block	3.8s ▲3.0× 3.1s ▲3.3×	3.5s ▲3.1× 2.7s ▲3.2×
MPF / SIMD-ref Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.5×	3.4s ▲3.2× 2.6s ▲3.3×

Table 10.1: Two minimizer implementations with MPF, Scalar-DP and Block held fixed. Source: Table 10.23, rows R4, R3.

An indexed row does not measure reference-index construction

The benchmark requires a compatible persisted reference index. It builds that index before restarting PostgreSQL, and does not rebuild its minimizers inside the measured query. The query sequences themselves have no persisted minimizer index: their minimizers are extracted to search the reference. Changing the minimizer therefore does not measure reference construction again, so the results above measure the same thing for the ”index” case.

In each small table, the upper duration is A: the complete query measurement, and the lower duration is B: the A time with substracted time for recorded CIGAR/MAPQ. This means that in BioseqDB B represents all the time excluding final alignment. You can find complete raw result tables in Benchmark results Section 10.1.2.

The measurements were performed on AWS machines of various classes always of size xlarge and local machine with MacOS ARM64. The exact specifications and measurement methotolopgy can be found in the benchmark section as well.

The minimap2 baseline is always gray in the tables, while preferred configuration (grouped-MPF with all SIMD routes) is marked with yellow colour.

Paying for the reference once

The preferred BioseqDB configuration beats minimap2 when the reference index is available, but loses when index construction is included in this example.

Configuration	M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7I / No index 2 Gbp / DNA4 $k = 15, w = 10$
minimap2	10.7s ▼3.2× 8.6s ▼3.5×	56.6s ▲2.1× 54.6s ▲1.9×
MPF / Bounded SIMD-DP / Block	3.5s ▲3.0× 2.7s ▲3.2×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×

Table 10.2: The preferred configuration on M7I, with and without a prepared reference index. Source: Table 10.23, rows R1, R10.

BioseqDB can then read its prepared lookup structures, as shown in Figures 9.10 and 9.9, which is great success. User can usually spend more time once waiting for data load to benefit from faster queries.

Specialized kernel failure

The scalar MPF configuration takes much longer on the smaller reference with longer k-mers than on the larger reference with shorter k-mers.

Configuration	M7A / Index 4 Gbp / DNA4 $k = 15, w = 10$	M7I / Index 4 Gbp / DNA4 $k = 15, w = 10$
MPF / Scalar	5.4s	4.9s
	▲ 6.7×	▲ 7.7×
Scalar-DP / Block	4.6s	4s
	▲ 7.6×	▲ 8.9×
MPF / Bounded	9.9s	13s
	▲ 3.6×	▲ 2.9×
SIMD-DP / Block	9s	11.9s
	▲ 3.9×	▲ 3.0×

Table 10.3: Indexed DNA4 with the shorter k-mer profile. Source: Table 10.21, rows R4, R23.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 20, w = 15$	M7I / Index 2 Gbp / DNA4 $k = 20, w = 15$
MPF / Scalar	5m 39.5s	12m 20.4s
	▼ 17.3×	▼ 40.7×
Scalar-DP / Block	5m 39s	12m 19.8s
	▼ 18.2×	▼ 44.8×
MPF / Bounded	3s	2.6s
	▲ 6.6×	▲ 6.9×
SIMD-DP / Block	2.3s	1.9s
	▲ 8.1×	▲ 8.6×

Table 10.4: Indexed DNA4 with the longer k-mer profile, despite a smaller reference. Source: Table 10.24, rows R38, R2.

Smaller hashes can be directly fetched from MPF index, while for longer the directory row is used 9.7.1 9.7.1, this however doesn't explain the difference. One possible explanation is that hash for scalar is too wide and it forces these lookups, because bounded SIMD algorithm limits the width of hashes. Alternatively this could be a problem with how the algorithm uses memory admission system and going into SWAP usage, but such drastic change is unlikely. What I think is the most likely, is the k=15, w=10 has certain optimisations I hardcoded first. When migrating algorithms to be generic I could do it in a way that the performance is heavily deraded for non-default caase.

Nevertheless I'm still accepting these results as k=15, w=10 are the most common (and default minimap2) used parameters.

RMQ/SIMD chaining performance

The preferred SIMD configuration is competitive, but the fastest chaining choice changes with the testing case and the machine. Suprisingly on MacOS where the most of the development was done SIMD-DP is the fastest one. I didn't measure region-specific timings (forward chaining/backward chaining) in this benchmark, so in the future I could spend more time investigating how algorithms compare in terms of memory accesses and other datasets.

Workload	MPF / Bounded Scalar-DP / Block	MPF / Bounded SIMD-DP / Block	MPF / Bounded RMQ / Block
M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	3.6s ▲3.1× 2.8s ▲3.5×	3.7s ▲3.0× 3s ▲3.4×	3.7s ▲3.0× 2.9s ▲3.4×
M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$	3.4s ▲3.1× 2.6s ▲3.3×	3.5s ▲3.0× 2.7s ▲3.2×	3.5s ▲3.0× 2.7s ▲3.2×

Table 10.5: Chaining choices with MPF, bounded minimizers and Block fixed. Source: Table 10.23, columns C1, C2.

Workload	MPF / Bounded Scalar-DP / Block	MPF / Bounded SIMD-DP / Block	MPF / Bounded RMQ / Block
M7A / Index 4 Gbp / DNA4 $k = 15, w = 10$	5.3s ▲6.8× 4.6s ▲7.7×	9.9s ▲3.6× 9s ▲3.9×	5.4s ▲6.8× 4.6s ▲7.6×
M7I / Index 4 Gbp / DNA4 $k = 15, w = 10$	5.5s ▲6.9× 4.6s ▲7.8×	13s ▲2.9× 11.9s ▲3.0×	9.9s ▲3.9× 9.1s ▲4.0×

Table 10.6: The same choices on the larger DNA4 reference. Source: Table 10.21, columns C1, C2.

Workload	MPF / Bounded Scalar-DP / Block	MPF / Bounded SIMD-DP / Block	MPF / Bounded RMQ / Block
Local / Index 2 Gbp / DNA4 $k = 20, w = 15$	2.4s ▲12.0× 1.9s ▲13.8×	2s ▲15.0× 1.3s ▲20.4×	9.5s ▲3.1× 9s ▲2.9×

Table 10.7: A local case in which SIMD-DP wins and RMQ loses. Source: Table 10.20, column C7.

Ambiguous bases problem

The local DNA5 slowdown is much larger in the complete no-index measurement than in its index measurement.

Configuration	Local / Index 2 Gbp / DNA4 $k = 15, w = 10$	Local / Index 2 Gbp / DNA5 $k = 15, w = 10$	Local / No index 2 Gbp / DNA4 $k = 15, w = 10$	Local / No index 2 Gbp / DNA5 $k = 15, w = 10$
MPF / Bounded	2.5s	5.4s	1m 54.3s	5m 17.3s
	▲ 11.5×	▲ 2.8×	▼ 2.6×	▼ 8.2×
SIMD-DP / Block	1.7s	4.2s	1m 45.5s	1m 54.9s
	▲ 15.6×	▲ 3.1×	▼ 2.5×	▼ 3.0×

Table 10.8: The preferred local configuration on DNA4 and DNA5. Source: Table 10.20, row R2.

”N” bases change the workload at several stages. The bounded DNA5 sketcher includes N in the k-mers it hashes and it involves entirely different bit-packed representation.

Anomalous A/B difference

This MPS configuration loses to minimap2 on complete time even though its B time is lower.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 20, w = 15$	M7I / Index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	19.6s	18.2s
	▼ 6.7×	▼ 6.9×
	18.6s	16.5s
	▼ 8.4×	▼ 8.6×
MPS / Scalar	43.8s	38s
	▼ 2.2×	▼ 2.1×
SIMD-DP / Block	8.6s	8.7s
	▲ 2.2×	▲ 1.9×

Table 10.9: An indexed comparison whose direction changes between A and B. Source: Table 10.24, rows R1, R18.

I previously had problem with implementation of divergence-sort in Section 3.9.2 but divergence is not substracted in B. Such big difference may be caused by either problems with measurement methodology (because alignment is massively parallel and calculating B is not correct) or most of the slowdown happens in CIGAR work due to special treatment of N bases in DNA5.

Block aligner is usually faster than KSW2

Block Aligner improves complete time compared to KSW2.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7I / Index 2 Gbp / DNA4 $k = 15, w = 10$	M4 / Index 2 Gbp / DNA4 $k = 15, w = 10$
MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×
MPF / Bounded SIMD-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.9s ▲2.3× 4.3s ▲4.4×

Table 10.10: Changing only the requested base aligner in the preferred configuration. Source: Table 10.23, rows R10, R14.

Block Aligner follows an adaptive block through the scoring matrix, whereas KSW2 uses its banded dynamic programming and stopping rules 5.1.

MPF is preffered over MPS

MPF is both faster to query and faster to construct.

Configuration	M7A / Index 2 Gbp / DNA4 $k = 15, w = 10$	M7A / No index 2 Gbp / DNA4 $k = 15, w = 10$
MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×
MPS / Bounded SIMD-DP / Block	6.7s ▲1.7× 6s ▲1.7×	2m 20.8s ▼2.7× 1m 57.4s ▼2.3×

Table 10.11: MPF and MPS with the same minimizer, chaining and alignment choices. Source: Table 10.23, rows R10, R27.

This happens because MPS overall offers no benefit because it construction requires merge sort with potential disk spilling and querying it is more complicated than in case of MPF 9.5

Configuration	M7A / Index 4 Gbp / DNA5 $k = 20, w = 15$	M7I / Index 4 Gbp / DNA5 $k = 20, w = 15$
MPF / Bounded SIMD-DP / Block	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×
MPS / Bounded SIMD-DP / Block	8m 5.8s ▼14.3× 38.1s ▼16%	5m 56.7s ▼10.4× 35.4s ▼8%

Table 10.12: The large DNA5 MPS slowdown appears on several machine classes. Source: Table 10.22, rows R2, R7.

In the example here we can see that likely the k=20, w=15 contains more unique minimizers than k=15, w=10 so the disk spillage happened during measurement and the query needed to check more unique minimizers, which caused multiple random accesses to large index.

BioseqDB memory admission system

The minimap2 baseline itself slows sharply on C7G.

Configuration	M7A / Index 4 Gbp / DNA4 $k = 20, w = 15$	M7I / Index 4 Gbp / DNA4 $k = 20, w = 15$	M4 / Index 4 Gbp / DNA4 $k = 20, w = 15$	C7G / Index 4 Gbp / DNA4 $k = 20, w = 15$
minimap2	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×
MPF / Bounded SIMD-DP / Block	9.2s ▲18.3× 8.5s ▲19.0×	11.4s ▲15.0× 10.7s ▲15.3×	6.8s ▲43.2× 5.1s ▲54.6×	39.1s ▲38.5× 38s ▲39.5×

Table 10.13: The same indexed workload across the AWS machine classes. Source: Table 10.22, rows R1, R2.

The C7G minimap2 record reports severe paging during measurement. C7G has less physical RAM than the other AWS classes, so the index with more unique minimizers causes excessive paging.

M4 is slower in many comparisons, but is not uniformly the slowest machine. Its older CPU and lower EBS (AWS disk) bandwidth make it slow, but in cases like presented here C7G can also undergo reduced performance[1] The benefit of BioseqDB is that we actively avoid using SWAP by making sure the memory is sufficient for all allocations of commom memory arenas.

[1] Amazon Web Services, Amazon EBS-optimized instance types.

Index query slower than no-index query

For this case of C7G measurement benchmark reported that indexed query is slower than no-index query.

Configuration	C7G / Index 2 Gbp / DNA4 $k = 20, w = 15$	C7G / No index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	6m 29.2s ▼ 92.8× 5m 56.9s ▼ 184.9×	4m 51.6s ▼ 2.9× 4m 46.6s ▼ 3.0×
MPF / SIMD-ref SIMD-DP / Block	13m 2.2s ▼ 2.0× 13m 1.5s ▼ 2.2×	2m 49.5s ▲ 1.7× 2m 21.9s ▲ 2.0×

Table 10.14: The apparently reversed index benefit for MPF and SIMD-ref. Source: Table 10.24, rows R1, R48.

This means that BioseqDB requires further optimisation to enable adappitve decisioning whether to use index of not. In this case despited repeated measurements the machine could go period of temproarily reduced disk IOPS, which caused disk reads to be slower than computing index from scratch (in second case the pipeline doesn't have to interface with disk as much)

Outliers

Several wide-hash MPF were unable to complete in time.

Workload	MPF / Scalar Scalar-DP / Block	MPF / SIMD-ref RMQ / Block
C7G / Index 4 Gbp / DNA4 $k = 20, w = 15$	3h 12m 16.6s ▼ 7.7× 3h 12m 14.4s ▼ 7.7×	2h 47m 50.9s ▼ 6.7× 2h 47m 48.7s ▼ 6.7×

Table 10.15: Examples of the longest indexed observations. Superscript s denotes a single sample. Source: Table 10.22, column C4.

It's likely that this is a bug in memory admission system which stalled entire pipeline. Other machines didn't report such bad timings.

GPU native algorithms are slower than CPU

Unfortunatelly BioseqDB was my first GPU implementation in Vulkan. I tested it both on local machine with integrated GPU and on remote Vast.ai rented GPU machine. In both of the cases there were issues with allocating to much memory on GPU or using CPU host memory instead of GPU. It's worth considering G3SA approach with different order of chaining scores computations and also consider one integerated kernel to perform all pipeline stages on GPU without frequent host-device transfers.

Configuration	Local / Index 2 Gbp / DNA4 $k = 15, w = 10$	Local / Index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	28.6s ▼ 11.5× 25.8s ▼ 15.6×	29.3s ▼ 15.0× 26.2s ▼ 22.9×
MPF / Vulkan Vulkan-DP / Block	6m 46.4s ▼ 14.2× 6m 34.1s ▼ 15.3×	3m 11.5s ▼ 6.5× 3m 2.2s ▼ 6.9×

Table 10.16: The local integrated GPU with Vulkan seeding and chaining. Source: Table 10.20, rows R1, R67.

The GPU faster by serveral orders of magnitud don't translate into better performance:

Configuration	Vast.ai / Index 2 Gbp / DNA4 $k = 15, w = 10$	Vast.ai / Index 2 Gbp / DNA4 $k = 20, w = 15$
minimap2	14.1s ▲ 7.7× 13s ▲ 4.5×	1m 34.6s ▶ 4% 1m 33.6s ▼ 2.1×
MPS / Vulkan Vulkan-DP / Block	1m 48.7s ▼ 7.7× 58.9s ▼ 4.5×	1m 38.5s ▶ 4% 47s ▲ 2.0×

Table 10.17: The MPS Vulkan route and the same-host minimap2 baseline on an RTX 5090. Source: Table 10.30, rows R1, R3.

GPU capacity failures

I tried putting parts of MPS index directly on the GPU, however the buffers size is limited and current Bulkan code fails on measured cases for 4Gbp sequences.

Configuration	Vast.ai / Index 4 Gbp / DNA4 $k = 15, w = 10$	Vast.ai / Index 4 Gbp / DNA4 $k = 20, w = 15$
MPF / Vulkan Vulkan-DP / Block	7m 7.5s ▼21.9× 6m 8.1s ▼20.4×	5m 48s ▼8.2× 4m 48s ▼6.9×
MPS / Vulkan Vulkan-DP / Block	×[2]	×[1]

Table 10.18: Completed MPF observations and MPS preparation failures on the RTX 5090. Source: Table 10.30, rows R5, R3.

10.1.2. Measurement procedure and reading the full tables

The following tables present results comparins BioseqDB and minimap2 on one Macos machine and 4 AWS machines.

- Index. A persisted index is constructed before measurement. Each repeat starts a fresh process and measures reading the index and mapping the queries. Construction of the persisted index is excluded. Operating-system page caches are not purged, so this is process-cold execution, but not a guaranteed cold disk read (I decided to measure it this way, because minimap2 still has a chance to use system page cache).
- No index. No pre-existing user index is available.
- Two measurements per cell. The first duration, A, is the nearest-rank P50 of the complete measured interval. The second, B, is the nearest-rank P50 after subtracting the union of instrumented CIGAR and MAPQ in each repeat

By default chunk cache for decompressed columns is disabled. Each repeat runs 1,024 queries against one reference sequence:

- References contain exactly 2×10^9 or 4×10^9 base pairs The parameter pairs are $(k, w) = (15, 10)$ and $(20, 15)$,
- Each query and target is randomly generated. DNA4 uses same probability for each base (25%). The DNA5 generator assigns probability 1% to N and 24.75% to each base.
- Query samples length is 20,000 bp and standard deviation 500 bp (before applying insertions and deletions), it's reverse-complemented with probability 0.5.
- Substitution probability is 0.001 and for insert/deletions it's 0.0005.
- Each query originates from one known sampled reference interval and later substitutions/insertions/deletions are applied to it.

BioseqDB uses task-parallel (most parallel-friendly mode) execution with prefiltering disabled and both CIGAR and MAPQ projection enabled.

AWS machines were missing GPUs so all Vulkan rows are grouped together and marked as not available.

Reading comparisons and highlights. Each cell places the A duration above its comparison, followed by the B duration and its own comparison. Durations are rounded to tenths of a second for display. All comparisons and minima use the original integer nanoseconds.

- Minimap2 (light-gray) as the baseline is first row in each table.
- All of the rows apart from minimap2 are ordered top-to-bottom by the minimum A time for the given "index" configuration.
- A BioseqDB value is compared with minimap2 for the same configuration (green/red triangles), while the minimap2 row compares with the fastest measured eligible BioseqDB effective path in that same column
- Pale yellow identifies the preferred BioseqDB configuration: MPF / Bounded / SIMD-DP / Block. This is the author's (my) selected configuration.
- A dot means there is no measurement, while a horizontal dash with ^{g} identifies that GPU configuration wasn't available (no GPU on AWS machines)

Table 10.19: Machine specs

Property	Local	M7A	M7I	M4	C7G
Machine	MacBook Pro	m7a.xlarge	m7i.xlarge	m4.xlarge	c7g.xlarge
CPU	Apple M1 Pro	AMD EPYC 9R14	Xeon Platinum 8488C	Xeon E5-2686 v4 @ 2.30GHz	Neoverse-V1
Architecture	arm64	x86_64	x86_64	x86_64	aarch64
OS version / kernel	macOS 15.6 / Darwin 24.6.0	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws	Linux / 7.0.0-1012-aws
OS build / distribution	24G5054d	Distribution not recorded	Distribution not recorded	Distribution not recorded	Distribution not recorded
Logical / physical cores	8 / 8 (6P + 2E)	4 / 4	4 / 2	4 / 2	4 / 4
Physical RAM	16 GiB unified	16 GiB	16 GiB	16 GiB	8 GiB
GPU	Apple M1 Pro, 14 cores	None available	None available	None available	None available
Storage	APPLE SSD AP0512R	EBS gp3	EBS gp3	EBS gp3	EBS gp3
Disk capacity	500.28 GB	500 GiB	500 GiB	500 GiB	500 GiB
Provisioned disk I/O	Not recorded	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s	6,000 IOPS / 500 MiB/s

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	17.7s ▲2.2× 17s ▲2.2×	21.5s ▲15% 20.2s ▲11%	14.6s ▲8.0× 13.8s ▲8.2×	21s ▲16% 19.8s ▲11%	2.5s ▲11.5× 1.7s ▲15.6×	5.4s ▲2.8× 4.2s ▲3.1×	2s ▲15.0× 1.3s ▲20.4×	5.3s ▲1.6× 3.9s ▲2.1×	3m 48.1s ▼1.3× 3m 35s ▼28%	8m 57.6s ▼3.9× 3m 49.2s ▼1.7×	2m 52s ▶ 2% 2m 39.3s ▲7%	8m 13.1s ▼8.8× 2m 54.5s ▼3.2×	1m 54.3s ▼2.6× 1m 45.5s ▼2.5×	5m 17.3s ▼8.2× 1m 54.9s ▼3.0×	1m 28.3s ▼1.7× 1m 16.8s ▼1.6×	4m 45.7s ▼9.9× 1m 26.6s ▼3.1×
R3	MPF / Bounded Scalar-DP / KSW2	·	36.5s ▼1.4× 34.8s ▼1.5×	27.1s ▲4.3× 25.6s ▲4.4×	30.3s ▼22% 28.7s ▼29%	·	5.8s ▲2.6× 4.2s ▲3.1×	2.4s ▲12.3× 1.1s ▲22.9×	5.4s ▲1.6× 3.8s ▲2.2×	·	·	·	·	·	·	·	·
R4	MPF / Bounded Scalar-DP / Block	·	34.8s ▼1.4× 33.5s ▼1.5×	26s ▲4.5× 25.4s ▲4.5×	30.7s ▼23% 29.5s ▼1.3×	·	5.4s ▲2.8× 4.3s ▲3.0×	2.4s ▲12.0× 1.9s ▲13.8×	5.1s ▲1.7× 3.9s ▲2.1×	·	·	·	·	·	·	·	·
R5	MPF / Bounded SIMD-DP / KSW2	20.8s ▲1.9× 19.1s ▲1.9×	16.9s ▲1.5× 15.3s ▲1.5×	19.8s ▲5.9× 18.2s ▲6.2×	30.1s ▼21% 28.5s ▼28%	3.7s ▲7.8× 2.4s ▲10.7×	6s ▲2.5× 4.5s ▲2.9×	2.5s ▲11.7× 1.2s ▲22.6×	5s ▲1.7× 3.6s ▲2.3×	3m 32.7s ▼23% 3m 17.7s ▼17%	·	2m 44.9s ▲6% 2m 32.2s ▲11%	·	1m 52.7s ▼2.6× 1m 40s ▼2.4×	5m 12.1s ▼8.0× 1m 46.7s ▼2.8×	1m 26.1s ▼1.6× 1m 13.9s ▼1.5×	4m 41.4s ▼9.8× 1m 21.9s ▼2.9×
R6	MPF / Bounded RMQ / KSW2	·	34.8s ▼1.4× 33.2s ▼1.5×	27.4s ▲4.3× 25.9s ▲4.4×	31.7s ▼27% 30.1s ▼1.4×	·	5.8s ▲2.6× 3.9s ▲3.3×	2.8s ▲10.6× 1.2s ▲21.1×	5.4s ▲1.6× 3.7s ▲2.2×	·	·	·	·	·	·	·	·
R7	MPF / Bounded RMQ / Block	·	34s ▼1.3× 32.7s ▼1.4×	4s ▲29.1× 3.4s ▲32.9×	15.4s ▲1.6× 14.2s ▲1.6×	·	5.9s ▲2.6× 4.6s ▲2.8×	9.5s ▲3.1× 9s ▲2.9×	14s ▼1.6× 12.7s ▼1.6×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —^{g} = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R8 BioseqDB	MPS / Bounded Scalar-DP / Block	·	·	·	·	4.1s ▲7.0× 3.2s ▲8.1×	3m 47.5s ▼14.9× 21.8s ▼1.7×	4.5s ▲6.5× 3.7s ▲7.1×	3m 29.2s ▼24.1× 13.3s ▼1.6×	·	·	·	·	·	·	·	·
R9	MPS / Bounded RMQ / Block	·	·	·	·	4.3s ▲6.7× 3.2s ▲8.0×	3m 45.3s ▼14.8× 21.8s ▼1.7×	4.4s ▲6.6× 3.8s ▲7.0×	3m 30s ▼24.2× 13.4s ▼1.6×	·	·	·	·	·	·	·	·
R10	MPS / Bounded SIMD-DP / Block	·	·	·	·	4.3s ▲6.7× 3.3s ▲7.9×	3m 45.2s ▼14.7× 22.3s ▼1.8×	4.5s ▲6.6× 3.8s ▲6.9×	3m 27.4s ▼23.9× 14.1s ▼1.7×	3m 53.6s ▼1.4× 3m 40.3s ▼1.3×	8m 50.7s ▼3.9× 3m 46s ▼1.7×	2m 51.3s ▶ 2% 2m 38.1s ▲8%	7m 58.5s ▼8.6× 2m 48.9s ▼3.1×	1m 59.6s ▼2.8× 1m 49s ▼2.6×	5m 15.7s ▼8.1× 1m 54.7s ▼3.0×	1m 28s ▼1.7× 1m 18.3s ▼1.6×	4m 46.9s ▼10.0× 1m 25.3s ▼3.0×
R11	MPS / Two-stack Scalar-DP / Block	·	·	·	·	4.3s ▲6.6× 3.4s ▲7.7×	3m 32.4s ▼13.9× 16.6s ▼1.3×	11.8s ▲2.5× 4.5s ▲5.8×	3m 21.7s ▼23.2× 8.6s ▼5%	·	·	·	·	·	·	·	·
R12	MPS / Two-stack RMQ / Block	·	·	·	·	4.3s ▲6.6× 3.4s ▲7.7×	3m 35.1s ▼14.1× 17.4s ▼1.4×	11.8s ▲2.5× 4.6s ▲5.7×	3m 22s ▼23.3× 8.6s ▼6%	·	·	·	·	·	·	·	·
R13	MPS / SIMD-ref RMQ / Block	·	·	·	·	4.3s ▲6.6× 3.5s ▲7.4×	3m 51.3s ▼15.1× 27.3s ▼2.1×	12.9s ▲2.3× 5.8s ▲4.5×	3m 38.6s ▼25.2× 20.6s ▼2.5×	·	·	·	·	·	·	·	·
R14	MPS / SIMD-ref Scalar-DP / Block	·	·	·	·	4.4s ▲6.6× 3.5s ▲7.5×	3m 50.5s ▼15.1× 26.1s ▼2.1×	12.5s ▲2.3× 5.6s ▲4.7×	3m 39.6s ▼25.3× 20.2s ▼2.5×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R15 BioseqDB	MPS / Scalar SIMD-DP / Block	·	·	·	·	4.4s ▲6.6× 3.5s ▲7.5×	3m 46.2s ▼14.8× 22.6s ▼1.8×	13.3s ▲2.2× 5.5s ▲4.7×	3m 35s ▼24.8× 16.3s ▼2.0×	·	·	·	·	·	·	·	·
R16	MPS / SIMD-ref SIMD-DP / Block	·	·	·	·	4.4s ▲6.6× 3.5s ▲7.4×	3m 53.3s ▼15.3× 27s ▼2.1×	12.7s ▲2.3× 5.6s ▲4.7×	3m 41.1s ▼25.5× 21s ▼2.6×	·	·	·	·	·	·	·	·
R17	MPS / Two-stack SIMD-DP / Block	·	·	·	·	4.4s ▲6.5× 3.5s ▲7.5×	3m 39s ▼14.3× 18.2s ▼1.4×	13.3s ▲2.2× 4.5s ▲5.9×	3m 26s ▼23.7× 9.6s ▼18%	·	·	·	·	·	·	·	·
R18	MPS / Scalar Scalar-DP / Block	·	·	·	·	4.4s ▲6.5× 3.5s ▲7.5×	3m 47.4s ▼14.9× 22.4s ▼1.8×	12.5s ▲2.4× 5.6s ▲4.7×	3m 34.3s ▼24.7× 14.9s ▼1.8×	·	·	·	·	·	·	·	·
R19	MPS / Scalar RMQ / Block	·	·	·	·	4.4s ▲6.5× 3.5s ▲7.5×	3m 43.8s ▼14.6× 21.9s ▼1.7×	12.9s ▲2.3× 5.8s ▲4.5×	3m 35.1s ▼24.8× 15.1s ▼1.9×	·	·	·	·	·	·	·	·
R20	MPS / Bounded Scalar-DP / KSW2	·	·	·	·	5s ▲5.7× 3.2s ▲8.1×	3m 48.5s ▼15.0× 22s ▼1.7×	5.5s ▲5.3× 3.9s ▲6.7×	3m 32s ▼24.4× 13.3s ▼1.6×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.20: (A): Local machine

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R21 BioseqDB	MPS / Bounded RMQ / KSW2	·	·	·	·	5.1s ▲5.6× 3.2s ▲8.1×	3m 47.8s ▼14.9× 22.2s ▼1.7×	5.3s ▲5.6× 3.7s ▲7.1×	3m 31.9s ▼24.4× 13.4s ▼1.6×	·	·	·	·	·	·	·	·
R22	MPS / SIMD-ref Scalar-DP / KSW2	·	·	·	·	5.1s ▲5.6× 3.4s ▲7.5×	3m 51.5s ▼15.2× 25.9s ▼2.0×	14.7s ▲2.0× 5.7s ▲4.6×	3m 44.9s ▼25.9× 21.1s ▼2.6×	·	·	·	·	·	·	·	·
R23	MPS / Bounded SIMD-DP / KSW2	·	·	·	·	5.2s ▲5.5× 3.2s ▲8.1×	3m 47s ▼14.9× 21.5s ▼1.7×	5.3s ▲5.5× 3.8s ▲7.0×	3m 32.1s ▼24.4× 13.2s ▼1.6×	3m 54s ▼1.4× 3m 39s ▼30%	·	2m 52.6s ▶1% 2m 36.8s ▲8%	·	2m 3.1s ▼2.8× 1m 50.4s ▼2.6×	5m 13.3s ▼8.1× 1m 50.9s ▼2.9×	1m 30.6s ▼1.7× 1m 17.7s ▼1.6×	4m 44.7s ▼9.9× 1m 25.3s ▼3.0×
R24	MPS / Two-stack RMQ / KSW2	·	·	·	·	5.2s ▲5.5× 3.4s ▲7.6×	3m 34.7s ▼14.1× 16.7s ▼1.3×	16.7s ▲1.8× 4.6s ▲5.7×	3m 27.7s ▼23.9× 8.4s ▶3%	·	·	·	·	·	·	·	·
R25	MPS / Two-stack Scalar-DP / KSW2	·	·	·	·	5.2s ▲5.5× 3.4s ▲7.6×	3m 34.2s ▼14.0× 16.5s ▼30%	21.8s ▲26% 4.8s ▲5.5×	3m 28.2s ▼24.0× 8.6s ▼6%	·	·	·	·	·	·	·	·
R26	MPS / Two-stack SIMD-DP / KSW2	·	·	·	·	5.2s ▲5.5× 3.4s ▲7.6×	3m 40.4s ▼14.4× 17.5s ▼1.4×	16.6s ▲1.8× 4.5s ▲5.8×	3m 28s ▼24.0× 8.5s ▼5%	·	·	·	·	·	·	·	·
R27	MPS / SIMD-ref RMQ / KSW2	·	·	·	·	5.2s ▲5.5× 3.5s ▲7.3×	3m 52.6s ▼15.2× 27.3s ▼2.2×	14.8s ▲2.0× 5.7s ▲4.6×	3m 42.9s ▼25.7× 20.5s ▼2.5×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.20: (A): Local machine

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R28 BioseqDB	MPS / Scalar SIMD-DP / KSW2	·	·	·	·	5.3s ▲5.4× 3.4s ▲7.6×	3m 46.1s ▼14.8× 21.5s ▼1.7×	17.6s ▲1.7× 5.5s ▲4.7×	3m 36.6s ▼24.9× 15.3s ▼1.9×	·	·	·	·	·	·	·	·
R29	MPS / SIMD-ref SIMD-DP / KSW2	·	·	·	·	5.3s ▲5.4× 3.5s ▲7.4×	3m 52.8s ▼15.2× 26.4s ▼2.1×	14.7s ▲2.0× 5.6s ▲4.7×	3m 42.9s ▼25.7× 20.3s ▼2.5×	·	·	·	·	·	·	·	·
R30	MPS / Scalar RMQ / KSW2	·	·	·	·	5.3s ▲5.4× 3.4s ▲7.6×	3m 49.4s ▼15.0× 21.8s ▼1.7×	14.6s ▲2.0× 5.7s ▲4.6×	3m 35.5s ▼24.8× 15.1s ▼1.9×	·	·	·	·	·	·	·	·
R31	MPS / Scalar Scalar-DP / KSW2	·	·	·	·	5.3s ▲5.4× 3.4s ▲7.6×	3m 47s ▼14.9× 22s ▼1.7×	16.3s ▲1.8× 5.6s ▲4.7×	3m 35.8s ▼24.9× 15s ▼1.8×	·	·	·	·	·	·	·	·
R32	MPF / Two-stack SIMD-DP / Block	·	·	·	·	·	5.4s ▲2.8× 4.2s ▲3.0×	1m 45s ▼3.6× 1m 44.4s ▼4.0×	59.1s ▼6.8× 58.5s ▼7.2×	·	·	·	·	·	·	·	·
R33	MPF / Scalar SIMD-DP / Block	·	·	·	·	·	6.1s ▲2.5× 4.7s ▲2.7×	1m 52.3s ▼3.8× 1m 51.9s ▼4.3×	3m 2.6s ▼21.0× 3m 1.6s ▼22.3×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R34 BioseqDB	MPF / Scalar RMQ / Block	·	·	·	·	·	8s ▲1.9× 6.7s ▲1.9×	1m 50s ▼3.7× 1m 49.6s ▼4.2×	3m 3.4s ▼21.1× 3m 2.4s ▼22.4×	·	·	·	·	·	·	·	·
R35	MPF / Scalar Scalar-DP / KSW2	·	·	·	·	·	8.5s ▲1.8× 7s ▲1.8×	1m 43.2s ▼3.5× 1m 41.9s ▼3.9×	3m 8.6s ▼21.7× 3m 7.3s ▼23.0×	·	·	·	·	·	·	·	·
R36	MPF / Scalar SIMD-DP / KSW2	·	·	·	·	·	9s ▲1.7× 7.5s ▲1.7×	1m 43s ▼3.5× 1m 41.7s ▼3.9×	3m 8.9s ▼21.8× 3m 7.6s ▼23.1×	·	·	·	·	·	·	·	·
R37	MPF / Scalar Scalar-DP / Block	·	·	·	·	·	10.1s ▲1.5× 8.9s ▲30%	1m 48.8s ▼3.7× 1m 48.4s ▼4.1×	3m 4.8s ▼21.3× 3m 3.8s ▼22.6×	·	·	·	·	·	·	·	·
R38	MPF / Two-stack SIMD-DP / KSW2	·	·	·	·	·	10.4s ▲1.5× 8.8s ▲1.4×	1m 21.3s ▼2.8× 1m 19.5s ▼3.0×	1m 1.1s ▼7.0× 58.6s ▼7.2×	·	·	·	·	·	·	·	·
R39	MPF / SIMD-ref RMQ / Block	·	·	·	·	·	10.5s ▲1.4× 9.3s ▲26%	1m 40.4s ▼3.4× 1m 40s ▼3.8×	1m 36.1s ▼11.1× 1m 35.1s ▼11.7×	·	·	·	·	·	·	·	·
R40	MPF / SIMD-ref SIMD-DP / Block	·	·	·	·	·	10.8s ▲29% 9.6s ▲25%	1m 42.9s ▼3.5× 1m 42.6s ▼3.9×	1m 57.8s ▼13.6× 1m 56.7s ▼14.4×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.20: (A): Local machine

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R41 BioseqDB	MPF / SIMD-ref SIMD-DP / KSW2	·	·	·	·	·	10.9s ▲29% 9.3s ▲27%	1m 44.7s ▼3.6× 1m 43.3s ▼3.9×	1m 37.2s ▼11.2× 1m 35.9s ▼11.8×	·	·	·	·	·	·	·	·
R42	MPF / Two-stack Scalar-DP / Block	·	·	·	·	·	10.9s ▲28% 9.7s ▲24%	1m 21s ▼2.8× 1m 20.6s ▼3.1×	57.4s ▼6.6× 56.7s ▼7.0×	·	·	·	·	·	·	·	·
R43	MPF / Two-stack Scalar-DP / KSW2	·	·	·	·	·	11.4s ▲25% 9.9s ▲22%	1m 21.1s ▼2.8× 1m 19.3s ▼3.0×	58.7s ▼6.8× 56.2s ▼6.9×	·	·	·	·	·	·	·	·
R44	MPF / Scalar RMQ / KSW2	·	·	·	·	·	12s ▲22% 10.5s ▲17%	1m 44.5s ▼3.6× 1m 43.2s ▼3.9×	3m 0.8s ▼20.8× 2m 59.5s ▼22.1×	·	·	·	·	·	·	·	·
R45	MPF / Two-stack RMQ / KSW2	·	·	·	·	·	12.4s ▲19% 10.8s ▲15%	1m 12.2s ▼2.5× 1m 10.5s ▼2.7×	59.7s ▼6.9× 57.2s ▼7.0×	·	·	·	·	·	·	·	·
R46	MPF / Two-stack RMQ / Block	·	·	·	·	·	12.7s ▲17% 11.5s ▲10%	1m 35.2s ▼3.2× 1m 34.5s ▼3.6×	58s ▼6.7× 57.2s ▼7.0×	·	·	·	·	·	·	·	·
R47	MPS / Two-stack Vulkan-DP / Block	·	·	·	·	26.2s ▲8% 19s ▲26%	3m 49.5s ▼15.0× 33.7s ▼2.6×	13.1s ▲2.2× 5.8s ▲4.5×	3m 23.1s ▼23.4× 9.9s ▼22%	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R48 BioseqDB	MPS / Two-stack Vulkan-DP / KSW2	·	·	·	·	27.6s ▶ 4% 18.6s ▲28%	3m 51.3s ▼15.1× 33.5s ▼2.6×	18.1s ▲1.6× 5.8s ▲4.5×	3m 29.5s ▼24.1× 9.9s ▼21%	·	·	·	·	·	·	·	·
R49	MPS / Bounded Vulkan-DP / Block	·	·	·	·	25.5s ▲11% 18.2s ▲29%	4m 7.7s ▼16.2× 43.2s ▼3.4×	19.8s ▲1.5× 12.9s ▲2.0×	3m 47.2s ▼26.2× 29s ▼3.6×	·	·	·	·	·	·	·	·
R50	MPS / Scalar Vulkan-DP / Block	·	·	·	·	28.8s ▶ 1% 18.6s ▲28%	4m 7s ▼16.2× 44.1s ▼3.5×	20.9s ▲29% 13.8s ▲1.9×	3m 50.9s ▼26.6× 31.3s ▼3.8×	·	·	·	·	·	·	·	·
R51	MPS / SIMD-ref Vulkan-DP / Block	·	·	·	·	25.2s ▲12% 17.9s ▲1.4×	3m 59.3s ▼15.7× 35.3s ▼2.8×	21.4s ▲27% 14.2s ▲1.8×	3m 57.1s ▼27.3× 37.5s ▼4.6×	·	·	·	·	·	·	·	·
R52	MPS / Bounded Vulkan-DP / KSW2	·	·	·	·	28.4s ▶ 0% 19.2s ▲26%	4m 4.9s ▼16.0× 42.5s ▼3.3×	23s ▲21% 14s ▲1.9×	3m 50.4s ▼26.5× 28.8s ▼3.5×	·	·	·	·	·	·	·	·
R53	MPS / SIMD-ref Vulkan-DP / KSW2	·	·	·	·	28.4s ▶ 0% 18.2s ▲30%	4m 1.2s ▼15.8× 35.5s ▼2.8×	23.8s ▲19% 14.5s ▲1.8×	3m 57.5s ▼27.4× 37.7s ▼4.6×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.20: (A): Local machine

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R54 BioseqDB	MPS / Scalar Vulkan-DP / KSW2	·	·	·	·	31s ▼9% 18.4s ▲29%	4m 7.9s ▼16.2× 43.3s ▼3.4×	24.2s ▲18% 15s ▲1.7×	3m 55.7s ▼27.1× 32.7s ▼4.0×	·	·	·	·	·	·	·	·
R55	MPF / Bounded Vulkan-DP / KSW2	57.1s ▼1.4× 40.2s ▼9%	5m 53.3s ▼14.0× 51.2s ▼2.3×	1m 3.5s ▲1.8× 46.7s ▲2.4×	6m 14.8s ▼15.1× 1m 3.1s ▼2.8×	42s ▼1.5× 27.9s ▼8%	3m 59.2s ▼15.7× 30.8s ▼2.4×	32.3s ▼10% 17.8s ▲1.5×	3m 47.2s ▼26.2× 23.7s ▼2.9×	·	·	·	·	·	·	·	·
R56	MPF / Bounded Vulkan-DP / Block	44.5s ▼12% 29.5s ▲20%	6m 10.7s ▼14.7× 1m 8.5s ▼3.0×	1m 0.8s ▲1.9× 45.9s ▲2.5×	6m 29.4s ▼15.6× 1m 21.6s ▼3.7×	37.9s ▼1.3× 25.3s ▶ 2%	4m 4.2s ▼16.0× 38.6s ▼3.0×	37s ▼26% 24.7s ▲6%	3m 48.2s ▼26.3× 23.4s ▼2.9×	·	·	·	·	·	·	·	·
R57	MPF / SIMD-ref Vulkan-DP / Block	·	·	·	·	39s ▼1.4× 26.4s ▶ 2%	3m 49.5s ▼15.0× 24s ▼1.9×	13m 49.2s ▼28.3× 13m 37.6s ▼31.2×	14m 54.7s ▼103.0× 11m 32.8s ▼85.2×	·	·	·	·	·	·	·	·
R58	MPF / Scalar Vulkan-DP / Block	43.3s ▼9% 28.9s ▲21%	·	·	·	39s ▼1.4× 26.5s ▶ 3%	4m 5.4s ▼16.1× 39.5s ▼3.1×	12m 37.6s ▼25.8× 12m 26.5s ▼28.5×	27m 27.3s ▼189.7× 24m 4.7s ▼177.7×	·	·	·	·	·	·	·	·
R59	MPF / Two-stack Vulkan-DP / KSW2	·	·	·	·	40.9s ▼1.4× 27.2s ▼5%	4m 6.8s ▼16.2× 36.8s ▼2.9×	13m 56.9s ▼28.5× 13m 25.1s ▼30.7×	10m 19.9s ▼71.4× 6m 50.6s ▼50.5×	·	·	·	·	·	·	·	·
R60	MPF / Scalar Vulkan-DP / KSW2	59.4s ▼1.5× 42s ▼14%	·	·	·	40.9s ▼1.4× 26.8s ▶ 4%	3m 59.5s ▼15.7× 31.6s ▼2.5×	13m 53.7s ▼28.4× 13m 39.4s ▼31.3×	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.20: (A): Local machine

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R61 BioseqDB	MPF / SIMD-ref Vulkan-DP / KSW2	·	·	·	·	41.8s ▼1.5× 27.5s ▼6%	3m 50.7s ▼15.1× 23.9s ▼1.9×	13m 53s ▼28.4× 13m 39.1s ▼31.2×	15m 18.3s ▼105.7× 11m 51.5s ▼87.5×	·	·	·	·	·	·	·	·
R62	MPF / Two-stack Vulkan-DP / Block	·	·	·	·	43.6s ▼1.5× 31.6s ▼22%	4m 5.1s ▼16.0× 36.3s ▼2.9×	12m 53.5s ▼26.4× 12m 25.3s ▼28.4×	10m 19.7s ▼71.4× 6m 54.6s ▼51.0×	·	·	·	·	·	·	·	·
R63	MPF / SIMD-ref Scalar-DP / Block	·	·	·	·	·	·	1m 34.2s ▼3.2× 1m 34s ▼3.6×	1m 37.5s ▼11.2× 1m 36.6s ▼11.9×	·	·	·	·	·	·	·	·
R64	MPF / SIMD-ref RMQ / KSW2	·	·	·	·	·	·	1m 44s ▼3.5× 1m 42.7s ▼3.9×	1m 34.8s ▼10.9× 1m 33.5s ▼11.5×	·	·	·	·	·	·	·	·
R65	MPF / SIMD-ref Scalar-DP / KSW2	·	·	·	·	·	·	1m 44.3s ▼3.6× 1m 43s ▼3.9×	1m 39.6s ▼11.5× 1m 38.4s ▼12.1×	·	·	·	·	·	·	·	·
R66	MPF / Vulkan RMQ / Block	·	·	·	·	5m 41.1s ▼11.9× 5m 31.7s ▼12.8×	8m 50.1s ▼34.7× 5m 27s ▼25.7×	3m 9.8s ▼6.5× 3m 1.4s ▼6.9×	6m 26.4s ▼44.5× 3m 9.2s ▼23.3×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R67 BioseqDB	MPF / Vulkan Vulkan-DP / Block	9m 36.5s ▼14.5× 9m 22.3s ▼15.3×	13m 49.8s ▼32.9× 8m 45.2s ▼23.2×	4m 57.7s ▼2.5× 4m 42.7s ▼2.5×	10m 50.3s ▼26.1× 5m 42.9s ▼15.4×	6m 46.4s ▼14.2× 6m 34.1s ▼15.3×	8m 46.4s ▼34.5× 5m 22.9s ▼25.4×	3m 11.5s ▼6.5× 3m 2.2s ▼6.9×	6m 30.8s ▼45.0× 3m 13.4s ▼23.8×	9m 50.5s ▼3.4× 9m 37.9s ▼3.4×	14m 21.8s ▼6.3× 9m 10.7s ▼4.1×	4m 10.1s ▼1.4× 4m 1s ▼1.4×	9m 51.7s ▼10.6× 4m 35.5s ▼5.0×	6m 16.9s ▼8.7× 6m 6.9s ▼8.6×	9m 20.4s ▼14.4× 5m 57.2s ▼9.4×	3m 10.7s ▼3.6× 3m 0.2s ▼3.7×	6m 41.2s ▼14.0× 3m 21.5s ▼7.1×
R68	MPF / Vulkan RMQ / KSW2	·	·	·	·	5m 51.9s ▼12.3× 5m 40.4s ▼13.2×	8m 46.4s ▼34.5× 5m 22.2s ▼25.3×	3m 12.6s ▼6.6× 2m 58.9s ▼6.8×	6m 29.8s ▼44.9× 3m 8.7s ▼23.2×	·	·	·	·	·	·	·	·
R69	MPF / Vulkan Scalar-DP / KSW2	·	·	·	·	5m 38.2s ▼11.8× 5m 28s ▼12.7×	8m 37.9s ▼33.9× 5m 16s ▼24.9×	3m 25.8s ▼7.0× 3m 14.4s ▼7.4×	6m 24.9s ▼44.3× 3m 8.7s ▼23.2×	·	·	·	·	·	·	·	·
R70	MPF / Vulkan SIMD-DP / KSW2	·	·	·	·	5m 44.6s ▼12.1× 5m 34.2s ▼12.9×	8m 41.2s ▼34.1× 5m 20s ▼25.2×	3m 33.4s ▼7.3× 3m 19s ▼7.6×	6m 39.3s ▼46.0× 3m 17.5s ▼24.3×	·	·	·	·	·	·	·	·
R71	MPF / Vulkan Scalar-DP / Block	·	·	·	·	5m 40.3s ▼11.9× 5m 29.9s ▼12.8×	8m 42.9s ▼34.2× 5m 22.1s ▼25.3×	3m 41.9s ▼7.6× 3m 31.8s ▼8.1×	6m 25.4s ▼44.4× 3m 9.3s ▼23.3×	·	·	·	·	·	·	·	·
R72	MPF / Vulkan Vulkan-DP / KSW2	·	·	·	·	6m 5.5s ▼12.8× 5m 53.5s ▼13.7×	8m 57.9s ▼35.2× 5m 38.3s ▼26.6×	3m 46.3s ▼7.7× 3m 33.1s ▼8.1×	6m 39s ▼45.9× 3m 23.4s ▼25.0×	·	·	·	·	·	·	·	·
R73	MPF / Vulkan SIMD-DP / Block	·	·	·	·	5m 43.9s ▼12.0× 5m 34.1s ▼12.9×	8m 37.5s ▼33.9× 5m 14.9s ▼24.8×	4m 42.2s ▼9.6× 4m 30.5s ▼10.3×	6m 35.3s ▼45.5× 3m 19.5s ▼24.5×	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.20: (A): Local machine

Engine	Execution	Index								No index							
		4 Gbp				2 Gbp				4 Gbp				2 Gbp			
		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15		k=15, w=10		k=20, w=15	
		DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5	DNA4	DNA5
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	39.7s ▼2.2× 36.8s ▼2.2×	25.2s ▼1.5× 22.7s ▼1.5×	1m 56.9s ▼29.1× 1m 53.2s ▼32.9×	24.9s ▼1.6× 22.2s ▼1.6×	28.6s ▼11.5× 25.8s ▼15.6×	15.3s ▼2.8× 12.7s ▼3.3×	29.3s ▼15.0× 26.2s ▼22.9×	8.7s ▼1.7× 8.1s ▼2.3×	2m 52.4s ▲19% 2m 48.5s ▲15%	2m 17.1s ▲3.9× 2m 15.9s ▲1.7×	2m 55s ▼6% 2m 51s ▼12%	55.9s ▲8.6× 55.1s ▲3.1×	43.3s ▲2.6× 42.5s ▲2.4×	38.9s ▲8.0× 38.2s ▲2.8×	53.1s ▲1.6× 48.2s ▲1.5×	28.7s ▲9.8× 28.2s ▲2.9×
R74 BioseqDB	MPS / Vulkan RMQ / KSW2	·	·	·	·	1h 56m 53.5s ▼245.5× 1h 56m 41.7s ▼271.1×	·	·	·	·	·	·	·	·	·	·	·
R75	MPS / Vulkan RMQ / Block	·	·	·	·	1h 56m 56.6s ▼245.6× 1h 56m 45.8s ▼271.2×	·	·	·	·	·	·	·	·	·	·	·
R76	MPS / Vulkan SIMD-DP / KSW2	·	·	·	·	1h 56m 58.7s ▼245.7× 1h 56m 45.6s ▼271.2×	·	·	·	·	·	·	·	·	·	·	·
R77	MPS / Vulkan Scalar-DP / Block	·	·	·	·	1h 57m 18.3s ▼246.4× 1h 57m 8.7s ▼272.1×	·	·	·	·	·	·	·	·	·	·	·
R78	MPS / Vulkan SIMD-DP / Block	·	·	·	·	1h 57m 20.2s ▼246.4× 1h 57m 9.8s ▼272.2×	·	·	·	·	·	·	·	·	·	·	·
R79	MPS / Vulkan Scalar-DP / KSW2	·	·	·	·	1h 57m 21.3s ▼246.5× 1h 57m 7.9s ▼272.1×	·	·	·	·	·	·	·	·	·	·	·
R80	MPS / Vulkan Vulkan-DP / Block	·	·	·	·	1h 57m 33.2s ▼246.9× 1h 57m 22.2s ▼272.6×	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R2 BioseqDB	MPF / SIMD-ref Scalar-DP / Block	5.3s ▲6.9× 4.5s ▲7.8×	4.9s ▲7.8× 4s ▲9.0×	11.6s ▲7.9× 9s ▲9.9×	35.1s ▲11.2× 34s ▲11.4×	14.8s ▲2.1× 10.3s ▲2.9×	15s ▲2.3× 10.9s ▲2.9×	26.1s ▲2.7× 17.6s ▲3.8×	22.9s ▲7.9× 20.4s ▲8.4×	4m 50.9s ▼1.7× 4m 39.8s ▼1.6×	4m 54.9s ▼1.7× 4m 41.8s ▼1.7×	·	·	·	·	·	·
R3	MPF / Two-stack Scalar-DP / Block	5.3s ▲6.8× 4.5s ▲7.7×	4.9s ▲7.8× 4s ▲8.9×	10.6s ▲8.7× 7.4s ▲11.9×	34.5s ▲11.3× 33.4s ▲11.6×	15.4s ▲2.0× 11.3s ▲2.7×	11.8s ▲2.9× 8.8s ▲3.7×	23.4s ▲3.0× 16.1s ▲4.1×	·	5m 6.4s ▼1.7× 4m 54.3s ▼1.7×	4m 10.4s ▼1.5× 3m 58.2s ▼1.4×	·	·	10m 12.8s ▼5.7× 4m 6.6s ▼2.3×	8m 47.2s ▼4.9× 3m 42.1s ▼2.1×	·	·
R4	MPF / Scalar Scalar-DP / Block	5.4s ▲6.7× 4.6s ▲7.6×	4.9s ▲7.7× 4s ▲8.9×	11.6s ▲8.0× 8.6s ▲10.3×	37.7s ▲10.4× 36.6s ▲10.6×	19.1s ▲1.7× 14.6s ▲2.1×	16.9s ▲2.0× 13.7s ▲2.3×	28s ▲2.5× 20.6s ▲3.2×	49.5s ▲3.7× 45.9s ▲3.7×	5m 20.4s ▼1.8× 5m 8s ▼1.8×	5m 8.8s ▼1.8× 4m 57s ▼1.8×	·	·	11m 9.4s ▼6.3× 5m 3.9s ▼2.9×	9m 52.5s ▼5.5× 5m 21.2s ▼3.1×	·	·
R5	MPF / SIMD-ref RMQ / Block	5.4s ▲6.7× 4.5s ▲7.8×	5s ▲7.7× 4.1s ▲8.8×	1m 11.9s ▲22% 1m 8.9s ▲22%	36s ▲10.9× 34.6s ▲11.2×	26.6s ▲16% 21.2s ▲30%	22.9s ▲1.5× 17.8s ▲1.8×	27.7s ▲2.5× 17.3s ▲3.8×	50.2s ▲3.6× 47.7s ▲3.6×	5m 0.9s ▼1.7× 4m 49.9s ▼1.7×	5m 32s ▼1.9× 5m 18.9s ▼1.9×	·	·	·	·	·	·
R6	MPF / Two-stack RMQ / Block	5.3s ▲6.8× 4.4s ▲7.9×	5s ▲7.7× 4.1s ▲8.9×	1m 21s ▲12% 1m 17.7s ▲12%	35s ▲11.2× 33.7s ▲11.5×	21.2s ▲1.5× 16.9s ▲1.8×	21.7s ▲1.6× 18.6s ▲1.7×	30.6s ▲2.3× 23s ▲2.9×	·	4m 31.5s ▼1.5× 4m 22.4s ▼1.5×	4m 45s ▼1.7× 4m 33.2s ▼1.6×	·	·	10m 15.4s ▼5.7× 4m 7.2s ▼2.3×	8m 43.3s ▼4.9× 3m 39s ▼2.1×	·	·
R7	MPF / Scalar RMQ / Block	5.5s ▲6.6× 4.6s ▲7.6×	5s ▲7.6× 4.1s ▲8.8×	22.8s ▲4.0× 19.7s ▲4.5×	35.2s ▲11.1× 34s ▲11.4×	35s ▼11% 30.5s ▶ 1%	35.6s ▶ 4% 32.4s ▶ 1%	34s ▲2.0× 26.4s ▲2.5×	56.3s ▲3.2× 53s ▲3.2×	5m 26.3s ▼1.9× 5m 17.1s ▼1.8×	5m 17.7s ▼1.9× 5m 5.9s ▼1.8×	·	·	11m 28.1s ▼6.4× 5m 21.3s ▼3.0×	10m 39.7s ▼6.0× 5m 34.9s ▼3.2×	·	·
R8	MPF / Two-stack SIMD-DP / Block	5.4s ▲6.7× 4.5s ▲7.8×	5s ▲7.6× 4s ▲8.9×	43.8s ▲2.1× 40.6s ▲2.2×	35.7s ▲11.0× 34.6s ▲11.2×	17.7s ▲1.8× 13.5s ▲2.2×	22s ▲1.6× 18.6s ▲1.7×	23.1s ▲3.0× 15.6s ▲4.2×	36.6s ▲5.0× 33.2s ▲5.2×	4m 54.2s ▼1.7× 4m 41.8s ▼1.6×	4m 16.1s ▼1.5× 4m 3.7s ▼1.4×	·	·	10m 20.1s ▼5.8× 4m 12.8s ▼2.4×	8m 41.2s ▼4.9× 3m 38.4s ▼2.1×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R9 BioseqDB	MPF / Scalar SIMD-DP / Block	5.5s ▲6.6× 4.6s ▲7.6×	5.1s ▲7.5× 4.1s ▲8.8×	32.2s ▲2.9× 29.2s ▲3.0×	35.7s ▲11.0× 34.6s ▲11.2×	23.2s ▲27% 18.8s ▲1.6×	32.3s ▲5% 28.5s ▲11%	1m 28.5s ▼27% 1m 20.8s ▼22%	57.3s ▲3.2× 53.9s ▲3.2×	5m 24.9s ▼1.9× 5m 16s ▼1.8×	5m 22s ▼1.9× 5m 9.2s ▼1.8×	·	·	11m 28.2s ▼6.4× 5m 22.9s ▼3.1×	9m 48.5s ▼5.5× 5m 17.7s ▼3.0×	·	·
R10	MPF / Bounded Scalar-DP / Block	5.3s ▲6.8× 4.6s ▲7.7×	5.5s ▲6.9× 4.6s ▲7.8×	10.8s ▲8.5× 7.8s ▲11.4×	34.1s ▲11.5× 33s ▲11.7×	18.7s ▲1.7× 14.3s ▲2.1×	18.3s ▲1.9× 14.8s ▲2.2×	23.6s ▲3.0× 16s ▲4.1×	54s ▲3.3× 50.6s ▲3.4×	3m 40.6s ▼26% 3m 31.8s ▼22%	4m 25.9s ▼1.6× 4m 13.9s ▼1.5×	·	·	10m 4.6s ▼5.6× 3m 56.7s ▼2.2×	14m 49s ▼8.3× 8m 18.7s ▼4.7×	17m 59.1s ▼4.9× 9m 9.4s ▼2.5×	·
R11	MPF / Bounded RMQ / Block	5.4s ▲6.8× 4.6s ▲7.6×	9.9s ▲3.9× 9.1s ▲4.0×	1m 5.5s ▲29% 1m 2.5s ▲29%	37.3s ▲10.5× 36.1s ▲10.7×	25.7s ▲19% 21.3s ▲30%	25.7s ▲25% 22.1s ▲1.5×	23.6s ▲2.9× 15.9s ▲4.2×	53s ▲3.4× 49.5s ▲3.5×	4m 2.1s ▼1.4× 3m 53.3s ▼1.3×	3m 42.6s ▼1.3× 3m 31.1s ▼25%	·	·	10m 3.7s ▼5.6× 3m 53.5s ▼2.2×	14m 40.9s ▼8.2× 8m 18.9s ▼4.7×	·	·
R12	MPF / SIMD-ref SIMD-DP / Block	5.4s ▲6.7× 4.5s ▲7.8×	17.6s ▲2.2× 16.5s ▲2.2×	8.7s ▲10.7× 6.5s ▲13.5×	35.9s ▲10.9× 34.7s ▲11.1×	29.2s ▲8% 22.8s ▲25%	33s ▶3% 27s ▲16%	54.5s ▲22% 43.2s ▲1.5×	37.7s ▲4.8× 35.3s ▲4.9×	4m 54.9s ▼1.7× 4m 43.5s ▼1.6×	4m 42.7s ▼1.7× 4m 29.7s ▼1.6×	·	·	·	·	·	·
R13	MPF / SIMD-ref Scalar-DP / KSW2	13.8s ▲2.6× 11.3s ▲3.1×	6.3s ▲6.0× 3.9s ▲9.3×	17.9s ▲5.2× 10.1s ▲8.8×	39s ▲10.0× 35.1s ▲11.0×	14.8s ▲2.1× 10.2s ▲3.0×	16s ▲2.1× 11.5s ▲2.8×	26.6s ▲2.6× 17.2s ▲3.8×	23.6s ▲7.7× 20.4s ▲8.4×	4m 52.7s ▼1.7× 4m 39.7s ▼1.6×	5m 1.6s ▼1.8× 4m 46.5s ▼1.7×	·	·	·	·	·	·
R14	MPF / Two-stack RMQ / KSW2	6.9s ▲5.2× 4.4s ▲8.0×	6.4s ▲5.9× 3.9s ▲9.3×	32.1s ▲2.9× 23.8s ▲3.7×	35.5s ▲11.0× 31.4s ▲12.3×	17.2s ▲1.8× 13s ▲2.3×	17.3s ▲2.0× 13.9s ▲2.3×	24.4s ▲2.8× 16.4s ▲4.0×	·	4m 33.3s ▼1.6× 4m 19.4s ▼1.5×	4m 13.5s ▼1.5× 4m 0s ▼1.4×	·	·	10m 11.1s ▼5.7× 4m 1.3s ▼2.3×	8m 47.2s ▼4.9× 3m 42.1s ▼2.1×	·	·
R15	MPF / SIMD-ref RMQ / KSW2	6.9s ▲5.3× 4.4s ▲8.0×	6.5s ▲5.9× 3.9s ▲9.3×	19.3s ▲4.8× 11.4s ▲7.8×	39.9s ▲9.8× 35.7s ▲10.8×	15.9s ▲2.0× 10.4s ▲2.9×	17.9s ▲1.9× 12.3s ▲2.6×	29.4s ▲2.4× 17.7s ▲3.7×	23.4s ▲7.7× 20s ▲8.6×	4m 58.1s ▼1.7× 4m 45.3s ▼1.6×	5m 2.1s ▼1.8× 4m 47.2s ▼1.7×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.21: (B): AWS: 4 Gbp, k=15, w=10

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R16 BioseqDB	MPF / Scalar SIMD-DP / KSW2	8.2s ▲4.4× 5.6s ▲6.2×	6.5s ▲5.9× 3.9s ▲9.1×	2m 15.5s ▼1.5× 2m 7.7s ▼1.4×	37.8s ▲10.3× 33.9s ▲11.4×	42.2s ▼1.3× 37.6s ▼24%	26.9s ▲21% 23.3s ▲27%	1m 59.3s ▼1.7× 1m 51.3s ▼1.7×	55s ▲3.3× 50.9s ▲3.4×	5m 27.2s ▼1.9× 5m 16.6s ▼1.8×	5m 50s ▼2.1× 5m 36.1s ▼2.0×	·	·	11m 14.3s ▼6.3× 5m 8.1s ▼2.9×	9m 48.4s ▼5.5× 5m 16.4s ▼3.0×	·	·
R17	MPF / SIMD-ref SIMD-DP / KSW2	6.9s ▲5.2× 4.4s ▲8.0×	6.5s ▲5.9× 3.9s ▲9.2×	53.6s ▲1.7× 45.3s ▲2.0×	39.8s ▲9.8× 35.9s ▲10.8×	23.3s ▲26% 17.1s ▲1.8×	38.6s ▼13% 32.2s ▶ 1%	46.7s ▲1.5× 34.1s ▲1.9×	51.3s ▲3.5× 47.9s ▲3.6×	4m 57.4s ▼1.7× 4m 44.6s ▼1.6×	5m 23.4s ▼1.9× 5m 7.5s ▼1.8×	·	·	·	·	·	·
R18	MPF / Scalar RMQ / KSW2	7s ▲5.1× 4.5s ▲7.7×	6.5s ▲5.9× 3.9s ▲9.1×	17.7s ▲5.2× 9.5s ▲9.3×	36.3s ▲10.8× 32.3s ▲12.0×	26.4s ▲17% 21.9s ▲28%	21s ▲1.6× 17.6s ▲1.8×	35.4s ▲2.0× 27.1s ▲2.4×	51.9s ▲3.5× 48s ▲3.6×	5m 15.8s ▼1.8× 5m 2.5s ▼1.7×	5m 19.7s ▼1.9× 5m 5.9s ▼1.8×	·	·	11m 18.9s ▼6.3× 5m 12.1s ▼2.9×	9m 52.6s ▼5.5× 5m 20s ▼3.0×	·	·
R19	MPF / Bounded Scalar-DP / KSW2	6.9s ▲5.3× 4.4s ▲7.9×	7.6s ▲5.0× 5.2s ▲6.9×	14.7s ▲6.3× 7s ▲12.7×	39.1s ▲10.0× 35.3s ▲11.0×	22.4s ▲29% 18.1s ▲1.7×	14.1s ▲2.4× 10s ▲3.2×	24.1s ▲2.9× 16s ▲4.1×	52s ▲3.5× 48.1s ▲3.6×	3m 56.3s ▼1.3× 3m 44.9s ▼29%	4m 6.8s ▼1.4× 3m 52.3s ▼1.4×	·	·	10m 13.7s ▼5.7× 4m 6.2s ▼2.3×	14m 38.1s ▼8.2× 8m 47s ▼5.0×	·	·
R20	MPF / Two-stack SIMD-DP / KSW2	6.9s ▲5.3× 4.4s ▲8.0×	7.3s ▲5.2× 4.8s ▲7.5×	12.5s ▲7.4× 6.8s ▲13.1×	41.9s ▲9.4× 37.8s ▲10.2×	28.6s ▲9% 24.4s ▲19%	24s ▲30% 20.6s ▲1.6×	57.1s ▲18% 49.1s ▲26%	·	4m 37.5s ▼1.6× 4m 25.9s ▼1.5×	4m 48.3s ▼1.7× 4m 34.5s ▼1.6×	·	·	10m 6.4s ▼5.7× 3m 55.9s ▼2.2×	8m 51.6s ▼5.0× 3m 47s ▼2.2×	·	·
R21	MPF / Bounded RMQ / KSW2	6.9s ▲5.2× 4.5s ▲7.9×	9.6s ▲3.9× 7.2s ▲5.0×	14.6s ▲6.3× 6.7s ▲13.1×	39.9s ▲9.8× 35.8s ▲10.8×	30.9s ▶ 2% 26.5s ▲12%	37.1s ▼9% 33.3s ▶ 4%	24.1s ▲2.9× 15.8s ▲4.2×	58.6s ▲3.1× 54.5s ▲3.1×	3m 41.3s ▼26% 3m 30.7s ▼21%	3m 59.5s ▼1.4× 3m 45.5s ▼1.3×	·	·	10m 2.7s ▼5.6× 3m 55.6s ▼2.2×	15m 4.3s ▼8.4× 8m 30.7s ▼4.9×	·	·
R22	MPF / Bounded SIMD-DP / KSW2	14.7s ▲2.5× 12.1s ▲2.9×	8.1s ▲4.7× 5.5s ▲6.5×	24.9s ▲3.7× 16.9s ▲5.2×	34.6s ▲11.3× 30.4s ▲12.7×	30s ▲5% 25.5s ▲16%	38.5s ▼13% 34.3s ▼7%	51.7s ▲26% 43.6s ▲1.5×	55.6s ▲3.3× 51.7s ▲3.3×	3m 57.7s ▼1.4× 3m 46.9s ▼1.3×	4m 0.1s ▼1.4× 3m 46s ▼1.3×	·	·	10m 21.9s ▼5.8× 4m 12s ▼2.4×	11m 12.6s ▼6.3× 5m 0.9s ▼2.9×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R23 BioseqDB	MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×	9.4s ▲9.8× 7s ▲12.7×	35s ▲11.2× 33.8s ▲11.4×	23.1s ▲27% 18.6s ▲1.6×	25.2s ▲26% 21.2s ▲1.5×	54.4s ▲22% 46.7s ▲29%	49.8s ▲3.6× 46.4s ▲3.7×	3m 44.5s ▼28% 3m 35.9s ▼24%	3m 57.4s ▼1.4× 3m 45.7s ▼1.3×	6m 50.6s ▼22% 6m 30.7s ▼18%	·	9m 45.5s ▼5.5× 3m 41.5s ▼2.1×	8m 25.6s ▼4.7× 3m 56.7s ▼2.3×	15m 24.9s ▼4.2× 6m 0.2s ▼1.7×	·
R24	MPF / Scalar Scalar-DP / KSW2	15s ▲2.4× 12.5s ▲2.8×	10.6s ▲3.6× 8.2s ▲4.4×	15.3s ▲6.1× 7.3s ▲12.1×	36.2s ▲10.8× 32.2s ▲12.0×	20s ▲1.6× 15.6s ▲1.9×	12.4s ▲2.8× 9.1s ▲3.5×	25.3s ▲2.7× 17.1s ▲3.9×	57.6s ▲3.1× 53.6s ▲3.2×	5m 25.6s ▼1.9× 5m 12.1s ▼1.8×	5m 5.9s ▼1.8× 4m 52.6s ▼1.7×	·	·	11m 14.1s ▼6.3× 5m 5.9s ▼2.9×	9m 54.4s ▼5.5× 5m 20.4s ▼3.0×	·	·
R25	MPS / Bounded Scalar-DP / Block	10.9s ▲3.3× 10.3s ▲3.4×	11.1s ▲3.4× 10.3s ▲3.5×	21.5s ▲4.3× 18.9s ▲4.7×	·	8m 24.2s ▼16.0× 59.7s ▼2.0×	6m 45.3s ▼11.9× 1m 3.2s ▼2.0×	·	·	4m 17.5s ▼1.5× 4m 8.2s ▼1.4×	4m 7.1s ▼1.4× 3m 55.3s ▼1.4×	·	·	10m 38s ▼6.0× 4m 13.2s ▼2.4×	9m 41.8s ▼5.4× 4m 18.5s ▼2.5×	·	·
R26	MPS / SIMD-ref Scalar-DP / Block	11.9s ▲3.0× 10.9s ▲3.2×	10.9s ▲3.5× 10s ▲3.6×	·	·	6m 13.5s ▼11.8× 39.8s ▼1.3×	5m 21s ▼9.4× 35.2s ▼10%	·	·	·	·	·	·	·	·	·	·
R27	MPS / SIMD-ref RMQ / Block	12s ▲3.0× 11s ▲3.2×	11s ▲3.5× 10s ▲3.6×	·	·	6m 15.3s ▼11.9× 41.2s ▼1.4×	5m 22.7s ▼9.5× 37s ▼15%	·	·	·	·	·	·	·	·	·	·
R28	MPS / Bounded RMQ / Block	11s ▲3.3× 10.3s ▲3.4×	11.5s ▲3.3× 10.7s ▲3.4×	21.8s ▲4.2× 18.8s ▲4.7×	·	8m 23.3s ▼15.9× 59.1s ▼1.9×	7m 3.6s ▼12.4× 1m 6.2s ▼2.1×	·	·	4m 10.2s ▼1.4× 4m 1s ▼1.4×	4m 11.6s ▼1.5× 3m 59.9s ▼1.4×	6m 51.1s ▼22% 6m 31.1s ▼18%	·	10m 27.7s ▼5.9× 4m 6s ▼2.3×	9m 42.6s ▼5.4× 4m 14.7s ▼2.4×	·	·
R29	MPS / Bounded SIMD-DP / Block	11.1s ▲3.3× 10.5s ▲3.3×	11.4s ▲3.4× 10.5s ▲3.4×	30.1s ▲3.1× 27.5s ▲3.2×	·	8m 12s ▼15.6× 1m 0.9s ▼2.0×	6m 45.6s ▼11.9× 1m 1.4s ▼1.9×	·	·	4m 19.3s ▼1.5× 4m 10.3s ▼1.4×	4m 29.2s ▼1.6× 4m 17.1s ▼1.5×	·	·	10m 25.7s ▼5.8× 4m 1.1s ▼2.3×	9m 26.4s ▼5.3× 4m 6.2s ▼2.3×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.21: (B): AWS: 4 Gbp, k=15, w=10

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R30 BioseqDB	MPS / Two-stack SIMD-DP / Block	11.5s ▲3.1× 10.9s ▲3.2×	11.2s ▲3.4× 10.4s ▲3.5×	·	·	8m 2s ▼15.3× 48.3s ▼1.6×	7m 1.6s ▼12.4× 48s ▼1.5×	·	·	4m 39.5s ▼1.6× 4m 30.1s ▼1.6×	4m 24.7s ▼1.6× 4m 13s ▼1.5×	·	·	·	·	·	·
R31	MPS / Scalar SIMD-DP / Block	11.3s ▲3.2× 10.7s ▲3.3×	11.4s ▲3.3× 10.7s ▲3.4×	·	·	8m 11.6s ▼15.6× 1m 1.8s ▼2.0×	6m 34.1s ▼11.6× 58.8s ▼1.8×	·	·	5m 48.1s ▼2.0× 5m 36.4s ▼1.9×	5m 51.7s ▼2.1× 5m 39.9s ▼2.0×	·	·	11m 45.1s ▼6.6× 5m 20.1s ▼3.0×	10m 10.3s ▼5.7× 5m 30.2s ▼3.1×	·	·
R32	MPS / Two-stack RMQ / Block	11.4s ▲3.2× 10.9s ▲3.2×	11.8s ▲3.2× 10.9s ▲3.3×	·	·	8m 8.4s ▼15.5× 45.9s ▼1.5×	7m 22.7s ▼13.0× 47.8s ▼1.5×	·	·	4m 44.1s ▼1.6× 4m 35.1s ▼1.6×	4m 42.8s ▼1.7× 4m 30.9s ▼1.6×	·	·	·	·	·	·
R33	MPS / Scalar RMQ / Block	11.5s ▲3.2× 10.9s ▲3.2×	11.9s ▲3.2× 11s ▲3.3×	·	·	7m 41.6s ▼14.6× 59.8s ▼2.0×	6m 41.2s ▼11.8× 59.4s ▼1.9×	·	·	5m 40.1s ▼1.9× 5m 31s ▼1.9×	6m 11.3s ▼2.2× 5m 59.5s ▼2.1×	·	·	11m 59.3s ▼6.7× 5m 36.2s ▼3.2×	·	·	·
R34	MPS / Two-stack Scalar-DP / Block	12s ▲3.0× 11s ▲3.2×	11.6s ▲3.3× 10.6s ▲3.4×	·	·	8m 0.3s ▼15.2× 45.7s ▼1.5×	6m 34.2s ▼11.6× 45.6s ▼1.4×	·	·	·	·	·	·	·	·	·	·
R35	MPS / Scalar Scalar-DP / Block	15.6s ▲2.3× 14.7s ▲2.4×	12.1s ▲3.1× 11s ▲3.3×	·	·	7m 42.1s ▼14.6× 58.9s ▼1.9×	6m 19.9s ▼11.1× 54.2s ▼1.7×	·	·	5m 51.9s ▼2.0× 5m 39.9s ▼2.0×	·	·	·	11m 44.1s ▼6.6× 5m 20.6s ▼3.0×	10m 13.1s ▼5.7× 5m 32.9s ▼3.2×	·	·
R36	MPS / Bounded SIMD-DP / KSW2	12.3s ▲3.0× 9.7s ▲3.6×	12.1s ▲3.1× 9.1s ▲4.0×	26.1s ▲3.5× 16.6s ▲5.3×	·	8m 21.9s ▼15.9× 57.7s ▼1.9×	7m 3s ▼12.4× 1m 0.6s ▼1.9×	·	·	4m 19.7s ▼1.5× 4m 8.9s ▼1.4×	3m 58.7s ▼1.4× 3m 45.4s ▼1.3×	10m 17.4s ▼1.8× 9m 56s ▼1.8×	·	10m 43.5s ▼6.0× 4m 18.3s ▼2.4×	9m 31.6s ▼5.3× 4m 7.6s ▼2.4×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R37 BioseqDB	MPS / Bounded Scalar-DP / KSW2	12.3s ▲2.9× 9.7s ▲3.6×	13.5s ▲2.8× 9.6s ▲3.7×	26.1s ▲3.5× 16.5s ▲5.4×	·	8m 23.3s ▼15.9× 58.5s ▼1.9×	6m 59.2s ▼12.3× 1m 4.7s ▼2.0×	·	·	4m 19.1s ▼1.5× 4m 8.7s ▼1.4×	4m 29.2s ▼1.6× 4m 15s ▼1.5×	·	·	10m 32.3s ▼5.9× 4m 7.7s ▼2.3×	9m 53.4s ▼5.5× 4m 23.7s ▼2.5×	·	·
R38	MPS / SIMD-ref SIMD-DP / KSW2	14.6s ▲2.5× 9.8s ▲3.6×	12.6s ▲3.0× 8.7s ▲4.1×	·	·	6m 16.2s ▼11.9× 40.5s ▼1.3×	5m 23.1s ▼9.5× 36.2s ▼13%	·	·	·	·	·	·	·	·	·	·
R39	MPS / Bounded RMQ / KSW2	12.6s ▲2.9× 9.7s ▲3.6×	13.7s ▲2.8× 9.5s ▲3.8×	26.4s ▲3.5× 16.6s ▲5.3×	·	8m 35.7s ▼16.3× 58.8s ▼1.9×	6m 49.4s ▼12.0× 1m 4.5s ▼2.0×	·	·	4m 20.8s ▼1.5× 4m 9.9s ▼1.4×	3m 58.2s ▼1.4× 3m 44.9s ▼1.3×	6m 49.2s ▼22% 6m 25.6s ▼16%	·	10m 29.6s ▼5.9× 4m 5.4s ▼2.3×	9m 40.1s ▼5.4× 4m 14.5s ▼2.4×	·	·
R40	MPS / SIMD-ref Scalar-DP / KSW2	14.5s ▲2.5× 10s ▲3.5×	12.7s ▲3.0× 8.7s ▲4.1×	·	·	6m 13.5s ▼11.8× 39.6s ▼1.3×	5m 21.7s ▼9.4× 35.3s ▼10%	·	·	·	·	·	·	·	·	·	·
R41	MPS / Scalar SIMD-DP / KSW2	12.7s ▲2.9× 9.9s ▲3.6×	13.2s ▲2.9× 9.7s ▲3.7×	·	·	8m 31.6s ▼16.2× 1m 3.7s ▼2.1×	6m 40.8s ▼11.8× 56.5s ▼1.8×	·	·	5m 39.7s ▼1.9× 5m 28.2s ▼1.9×	5m 42.5s ▼2.0× 5m 29s ▼2.0×	·	·	11m 45s ▼6.6× 5m 19.1s ▼3.0×	10m 13.5s ▼5.7× 5m 31.5s ▼3.2×	·	·
R42	MPS / SIMD-ref RMQ / KSW2	14.5s ▲2.5× 10s ▲3.5×	12.8s ▲3.0× 8.7s ▲4.1×	·	·	6m 21.1s ▼12.1× 41.4s ▼1.4×	5m 23.8s ▼9.5× 37s ▼16%	·	·	·	·	·	·	·	·	·	·
R43	MPS / Two-stack SIMD-DP / KSW2	13.1s ▲2.8× 10.5s ▲3.3×	13.4s ▲2.8× 9.7s ▲3.7×	·	·	8m 20.3s ▼15.8× 45.6s ▼1.5×	6m 45s ▼11.9× 45.4s ▼1.4×	·	·	4m 54.4s ▼1.7× 4m 44.1s ▼1.6×	4m 39.9s ▼1.6× 4m 26.4s ▼1.6×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.21: (B): AWS: 4 Gbp, k=15, w=10

Engine	Execution	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼6.9× 35s ▼8.0×	38.1s ▼7.8× 36s ▼9.3×	1m 32.4s ▼10.7× 1m 28.5s ▼13.5×	6m 31.5s ▼11.5× 6m 26.3s ▼12.7×	31.6s ▼2.1× 30.3s ▼3.0×	34.1s ▼2.9× 32s ▼3.7×	1m 9.5s ▼3.0× 1m 6.1s ▼4.3×	3m 1s ▼7.9× 2m 51.5s ▼8.6×	2m 55.5s ▲20% 2m 54s ▲17%	2m 50.4s ▲23% 2m 48.2s ▲20%	5m 35.7s ▲18% 5m 31.9s ▲14%	44m 20.5s n/a 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.6× 1m 45.1s ▲2.1×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a 29m 38.5s n/a
R44 BioseqDB	MPS / Two-stack Scalar-DP / KSW2	14.6s ▲2.5× 9.9s ▲3.5×	13.2s ▲2.9× 9.2s ▲3.9×	·	·	8m 22s ▼15.9× 46.5s ▼1.5×	6m 54.6s ▼12.2× 45.8s ▼1.4×	·	·	·	·	·	·	10m 24.5s ▼5.8× 4m 4.1s ▼2.3×	8m 30.6s ▼4.8× 3m 55.3s ▼2.2×	·	·
R45	MPS / Scalar RMQ / KSW2	13.3s ▲2.7× 10.8s ▲3.3×	18.6s ▲2.0× 13.7s ▲2.6×	·	·	7m 41.4s ▼14.6× 59.5s ▼2.0×	6m 30.2s ▼11.5× 54.7s ▼1.7×	·	·	5m 34.3s ▼1.9× 5m 20.2s ▼1.8×	·	·	·	12m 14s ▼6.9× 5m 48.2s ▼3.3×	·	·	·
R46	MPS / Two-stack RMQ / KSW2	13.3s ▲2.7× 10.8s ▲3.2×	17.9s ▲2.1× 13.5s ▲2.7×	·	·	8m 11.3s ▼15.6× 46s ▼1.5×	6m 46s ▼11.9× 45.8s ▼1.4×	·	·	·	·	·	·	·	·	·	·
R47	MPF / Two-stack Scalar-DP / KSW2	20.1s ▲1.8× 17.6s ▲2.0×	16.8s ▲2.3× 14.4s ▲2.5×	17.5s ▲5.3× 9.2s ▲9.6×	35.7s ▲11.0× 31.7s ▲12.2×	15.2s ▲2.1× 11s ▲2.7×	14.2s ▲2.4× 10.7s ▲3.0×	23.3s ▲3.0× 15.5s ▲4.3×	·	4m 31.8s ▼1.5× 4m 18.1s ▼1.5×	4m 18s ▼1.5× 4m 3.8s ▼1.4×	·	·	10m 1.7s ▼5.6× 3m 53.1s ▼2.2×	8m 16.7s ▼4.6× 3m 46.2s ▼2.2×	·	·
R48	MPS / Scalar Scalar-DP / KSW2	14.5s ▲2.5× 10.1s ▲3.5×	14.3s ▲2.7× 9.5s ▲3.8×	·	·	7m 42s ▼14.6× 58.8s ▼1.9×	6m 31.7s ▼11.5× 54.5s ▼1.7×	·	·	5m 38.2s ▼1.9× 5m 24.8s ▼1.9×	·	·	·	12m 12s ▼6.8× 5m 47s ▼3.3×	·	·	·
R49	MPS / SIMD-ref SIMD-DP / Block	15.4s ▲2.3× 14.5s ▲2.4×	15.1s ▲2.5× 14.2s ▲2.5×	·	·	6m 17.1s ▼11.9× 42.5s ▼1.4×	5m 21.5s ▼9.4× 37.9s ▼18%	·	·	·	·	8m 12.5s ▼1.5× 7m 55.2s ▼1.4×	·	·	·	·	·
R50 BioseqDB	Vulkan (all modes)	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.22: (C): AWS: 4 Gbp, k=20, w=15

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	9.2s ▲18.3× 8.5s ▲19.0×	11.4s ▲15.0× 10.7s ▲15.3×	6.8s ▲43.2× 5.1s ▲54.6×	39.1s ▲38.5× 38s ▲39.5×	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×	18.6s ▲3.7× 14.3s ▲4.6×	57.2s ▲3.3× 53.9s ▲3.4×	2m 38.5s ▲24% 2m 29.8s ▲26%	2m 49s ▲28% 2m 37s ▲30%	5m 3.3s ▲1.4× 4m 42.7s ▲1.5×	·	8m 52.3s ▼7.6× 2m 45.5s ▼2.4×	7m 22.2s ▼6.4× 2m 48.2s ▼2.5×	14m 43.3s ▼6.2× 5m 9.5s ▼2.2×	·
R3	MPF / Bounded Scalar-DP / Block	10.7s ▲15.7× 10.1s ▲16.1×	11.5s ▲14.8× 10.8s ▲15.2×	7.6s ▲38.5× 5.4s ▲51.4×	36.6s ▲41.1× 35.5s ▲42.2×	17s ▲2.0× 12.8s ▲2.6×	17.3s ▲2.0× 14.1s ▲2.3×	23s ▲3.0× 15.8s ▲4.1×	51.2s ▲3.7× 47.8s ▲3.8×	2m 50s ▲18% 2m 41s ▲21%	2m 48.3s ▲28% 2m 35.9s ▲1.4×	4m 47.5s ▲1.5× 4m 26.9s ▲1.6×	·	9m 21.4s ▼8.0× 3m 11.2s ▼2.8×	7m 45.7s ▼6.8× 2m 38.2s ▼2.4×	·	·
R4	MPF / Bounded RMQ / Block	8.3s ▲20.4× 7.6s ▲21.3×	8.6s ▲19.8× 7.8s ▲20.8×	35s ▲8.4× 32.9s ▲8.5×	38.3s ▲39.3× 37.2s ▲40.3×	35.9s ▼6% 31.8s ▶ 3%	33.2s ▶ 4% 30.1s ▲8%	46.2s ▲1.5× 39.4s ▲1.7×	49.2s ▲3.9× 45.8s ▲4.0×	2m 53.8s ▲17% 2m 43.7s ▲19%	3m 4.5s ▲21% 2m 52.1s ▲23%	·	·	9m 6.6s ▼7.8× 2m 56.5s ▼2.6×	7m 59.6s ▼7.0× 2m 51.2s ▼2.6×	·	·
R5	MPF / Bounded SIMD-DP / KSW2	9.6s ▲17.6× 7.1s ▲22.6×	9.6s ▲17.7× 7.3s ▲22.2×	38.3s ▲7.7× 30.9s ▲9.0×	40.5s ▲37.2× 36.4s ▲41.2×	34.6s ▶ 2% 30.4s ▲8%	21.3s ▲1.6× 17.7s ▲1.8×	37s ▲1.8× 29.3s ▲2.2×	55.9s ▲3.4× 52s ▲3.5×	2m 51.2s ▲18% 2m 39.2s ▲21%	3m 0s ▲23% 2m 45.6s ▲26%	·	·	9m 4.2s ▼7.7× 2m 55.2s ▼2.5×	7m 44.4s ▼6.8× 2m 37.2s ▼2.3×	·	·
R6	MPF / Bounded RMQ / KSW2	10.6s ▲15.9× 8.3s ▲19.5×	14.6s ▲11.7× 12.5s ▲13.0×	12.5s ▲23.5× 5.1s ▲54.4×	36s ▲41.8× 32.2s ▲46.6×	26.8s ▲21% 22.7s ▲1.5×	34.6s ▶ 1% 31.2s ▶ 4%	26s ▲2.6× 18.5s ▲3.5×	53.3s ▲3.6× 49.4s ▲3.7×	2m 51.8s ▲18% 2m 40.6s ▲21%	2m 50.2s ▲27% 2m 36.4s ▲1.4×	·	·	9m 12.4s ▼7.9× 2m 59.4s ▼2.6×	7m 51.1s ▼6.9× 2m 41.8s ▼2.4×	·	·
R7	MPS / Bounded SIMD-DP / Block	11.7s ▲14.3× 11s ▲14.7×	11.9s ▲14.3× 11.2s ▲14.6×	·	·	8m 5.8s ▼14.3× 38.1s ▼16%	5m 56.7s ▼10.4× 35.4s ▼8%	·	·	2m 43.8s ▲21% 2m 34.7s ▲24%	2m 58.9s ▲23% 2m 46.3s ▲26%	·	·	9m 21.2s ▼8.0× 3m 2.3s ▼2.6×	8m 1.2s ▼7.0× 2m 44.6s ▼2.5×	·	·
R8	MPS / Bounded RMQ / Block	11.7s ▲14.3× 11s ▲14.6×	11.8s ▲14.5× 11.1s ▲14.7×	·	·	7m 41.1s ▼13.6× 35.7s ▼8%	5m 57.8s ▼10.4× 34.4s ▼5%	·	·	2m 43.8s ▲21% 2m 34.7s ▲24%	2m 52.1s ▲26% 2m 39.6s ▲29%	·	·	9m 17.3s ▼7.9× 2m 59.8s ▼2.6×	·	14m 16s ▼6.0× 5m 26.4s ▼2.4×	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R9 BioseqDB	MPS / Bounded Scalar-DP / Block	11.8s ▲14.2× 11.1s ▲14.6×	12.7s ▲13.4× 11.9s ▲13.7×	·	·	7m 27.3s ▼13.2× 36.3s ▼10%	5m 57.5s ▼10.4× 34.3s ▶ 5%	·	·	2m 52.6s ▲17% 2m 42.8s ▲20%	2m 58.9s ▲23% 2m 46.9s ▲25%	·	·	9m 15.5s ▼7.9× 3m 0.5s ▼2.6×	·	·	·
R10	MPF / Bounded Scalar-DP / KSW2	14.5s ▲11.6× 12.2s ▲13.3×	14.2s ▲12.0× 11.9s ▲13.7×	12.7s ▲23.2× 5.2s ▲53.2×	36.8s ▲40.9× 32.8s ▲45.7×	14.8s ▲2.3× 10.7s ▲3.1×	17.7s ▲1.9× 14.3s ▲2.3×	27s ▲2.5× 19.4s ▲3.4×	48.6s ▲3.9× 44.8s ▲4.1×	2m 40.6s ▲23% 2m 29.9s ▲26%	3m 2.9s ▲22% 2m 48.3s ▲25%	·	·	9m 11s ▼7.8× 2m 56.6s ▼2.6×	7m 47.4s ▼6.8× 2m 37.9s ▼2.4×	·	·
R11	MPS / Bounded RMQ / KSW2	13.5s ▲12.4× 11s ▲14.7×	12.8s ▲13.3× 10.6s ▲15.4×	·	·	7m 20.4s ▼13.0× 35.7s ▼8%	6m 8.8s ▼10.7× 34.6s ▼6%	·	·	2m 56.8s ▲15% 2m 44.8s ▲19%	3m 11.4s ▲18% 2m 56.8s ▲21%	·	·	9m 19.1s ▼8.0× 3m 4s ▼2.7×	·	·	·
R12	MPS / Bounded Scalar-DP / KSW2	13.4s ▲12.6× 10.9s ▲14.9×	13s ▲13.1× 10.7s ▲15.2×	·	·	7m 44.9s ▼13.7× 35.9s ▼9%	5m 59.4s ▼10.4× 34s ▶ 4%	·	·	2m 59.1s ▲14% 2m 47.7s ▲17%	2m 50.8s ▲27% 2m 36.9s ▲30%	·	·	9m 18.6s ▼8.0× 3m 3.8s ▼2.7×	·	·	·
R13	MPS / Bounded SIMD-DP / KSW2	13.3s ▲12.7× 10.8s ▲15.0×	13.6s ▲12.5× 11.1s ▲14.7×	·	·	7m 20.3s ▼13.0× 35.3s ▼7%	5m 56.2s ▼10.3× 34s ▶ 4%	·	·	2m 59s ▲14% 2m 48.5s ▲17%	2m 52.5s ▲26% 2m 38.8s ▲29%	·	·	9m 21.1s ▼8.0× 3m 3.5s ▼2.7×	8m 2.9s ▼7.0× 2m 44s ▼2.5×	·	·
R14	MPS / Two-stack SIMD-DP / Block	51.6s ▲3.3× 9.8s ▲16.6×	44.7s ▲3.8× 9.6s ▲17.0×	1m 16.1s ▲3.9× 15.7s ▲17.7×	·	·	·	·	·	·	·	·	·	·	·	·	·
R15	MPS / Two-stack RMQ / Block	50.7s ▲3.3× 9.4s ▲17.2×	44.9s ▲3.8× 9.9s ▲16.5×	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R16 BioseqDB	MPS / Two-stack RMQ / KSW2	53.5s ▲3.1× 9.3s ▲17.4×	47.1s ▲3.6× 9s ▲18.2×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R17	MPS / Two-stack SIMD-DP / KSW2	55.3s ▲3.0× 9.6s ▲16.8×	48.6s ▲3.5× 9.6s ▲17.0×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R18	MPS / Two-stack Scalar-DP / KSW2	57.2s ▲2.9× 9.8s ▲16.5×	49.4s ▲3.4× 9.2s ▲17.8×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R19	MPS / Scalar SIMD-DP / Block	57s ▲3.0× 14.3s ▲11.3×	50.6s ▲3.4× 15.1s ▲10.8×	·	·	7m 14.5s ▼12.8× 41.9s ▼27%	6m 19.3s ▼11.0× 41.3s ▼26%	·	·	5m 10.1s ▼1.5× 4m 27.6s ▼1.3×	5m 3.5s ▼1.3× 4m 28.9s ▼20%	·	·	·	·	·	·
R20	MPS / SIMD-ref Scalar-DP / Block	1m 1.1s ▲2.7× 14.9s ▲10.8×	50.8s ▲3.4× 14.6s ▲11.2×	·	·	7m 13.1s ▼12.8× 52.7s ▼1.6×	6m 7.7s ▼10.7× 50.4s ▼1.5×	·	·	·	·	·	·	·	·	·	·
R21	MPS / SIMD-ref RMQ / Block	1m 0.8s ▲2.8× 15s ▲10.8×	50.9s ▲3.3× 14.7s ▲11.1×	·	·	7m 13.3s ▼12.8× 53.4s ▼1.6×	6m 11.5s ▼10.8× 54s ▼1.7×	·	·	·	·	·	·	·	·	·	·
R22	MPS / Scalar RMQ / Block	57.4s ▲2.9× 14.6s ▲11.1×	51.1s ▲3.3× 15.9s ▲10.3×	·	·	7m 2.9s ▼12.5× 38.9s ▼18%	6m 25.9s ▼11.2× 44s ▼1.3×	·	·	5m 10.6s ▼1.5× 4m 28.1s ▼1.3×	5m 37s ▼1.4× 5m 0.7s ▼1.3×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R23 BioseqDB	MPS / SIMD-ref SIMD-DP / Block	1m 5.1s ▲2.6× 15s ▲10.8×	51.5s ▲3.3× 14.6s ▲11.1×	·	·	7m 16s ▼12.8× 56.4s ▼1.7×	6m 11.8s ▼10.8× 53s ▼1.6×	·	·	·	·	6m 45.8s ▲7% 5m 46.6s ▲17%	·	·	·	·	·
R24	MPS / Scalar Scalar-DP / Block	1m 0s ▲2.8× 14.7s ▲11.0×	51.7s ▲3.3× 14.7s ▲11.1×	·	·	7m 3s ▼12.5× 38.5s ▼17%	6m 18s ▼11.0× 41.1s ▼26%	·	·	5m 22.6s ▼1.5× 4m 36.5s ▼1.4×	·	·	·	·	·	·	·
R25	MPS / Two-stack Scalar-DP / Block	56.5s ▲3.0× 9.7s ▲16.7×	51.7s ▲3.3× 9.9s ▲16.5×	·	·	·	·	·	·	·	·	·	·	·	·	·	·
R26	MPS / SIMD-ref SIMD-DP / KSW2	1m 2.9s ▲2.7× 14.9s ▲10.8×	52s ▲3.3× 14.4s ▲11.3×	·	·	7m 15.8s ▼12.8× 53.3s ▼1.6×	6m 10.8s ▼10.8× 51.3s ▼1.6×	·	·	·	·	·	·	·	·	·	·
R27	MPS / Scalar SIMD-DP / KSW2	58.6s ▲2.9× 14.4s ▲11.2×	52.3s ▲3.3× 15.4s ▲10.6×	·	·	7m 14.6s ▼12.8× 39s ▼19%	6m 25.1s ▼11.2× 39s ▼19%	·	·	5m 10.5s ▼1.5× 4m 25.5s ▼1.3×	4m 45.4s ▼22% 4m 9.3s ▼12%	·	·	·	·	·	·
R28	MPS / Scalar Scalar-DP / KSW2	1m 1.8s ▲2.7× 14.7s ▲11.0×	52.7s ▲3.2× 14.6s ▲11.2×	·	·	7m 1.1s ▼12.4× 38.9s ▼18%	6m 4.9s ▼10.6× 39.9s ▼22%	·	·	5m 9.5s ▼1.5× 4m 23s ▼30%	·	·	·	·	·	·	·
R29	MPS / SIMD-ref RMQ / KSW2	1m 2.5s ▲2.7× 15s ▲10.8×	53.6s ▲3.2× 15.3s ▲10.7×	·	·	7m 16s ▼12.8× 53.4s ▼1.6×	6m 11.5s ▼10.8× 52.1s ▼1.6×	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.22: (C): AWS: 4 Gbp, k=20, w=15

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R30 BioseqDB	MPS / SIMD-ref Scalar-DP / KSW2	1m 2.8s ▲2.7× 15s ▲10.8×	53.9s ▲3.2× 15.2s ▲10.7×	·	·	7m 17.8s ▼12.9× 53.3s ▼1.6×	6m 9.3s ▼10.7× 50.2s ▼1.5×	·	·	·	·	·	·	·	·	·	·
R31	MPS / Scalar RMQ / KSW2	59.3s ▲2.8× 14.7s ▲11.0×	56.8s ▲3.0× 14.7s ▲11.1×	·	·	7m 5s ▼12.5× 39.2s ▼19%	6m 26.9s ▼11.2× 41.2s ▼26%	·	·	5m 14.2s ▼1.5× 4m 26.6s ▼1.3×	·	·	·	·	·	·	·
R32	MPF / Two-stack RMQ / KSW2	9m 9.8s ▼3.3× 9m 7.7s ▼3.4×	15m 43.9s ▼5.5× 15m 40.9s ▼5.8×	33m 26.8s ▼6.8× 33m 20.7s ▼7.2×	·	·	·	·	·	3m 22.2s ▶ 3% 2m 35.6s ▲23%	3m 3.8s ▲21% 2m 25s ▲1.5×	·	·	·	·	·	·
R33	MPF / Two-stack RMQ / Block	9m 11s ▼3.3× 9m 10.5s ▼3.4×	13m 20.9s ▼4.7× 13m 20.1s ▼4.9×	33m 23s ▼6.8× 33m 21.5s ▼7.2×	·	·	·	·	·	3m 21.7s ▶ 3% 2m 38.8s ▲22%	3m 0.8s ▲22% 2m 25s ▲1.5×	·	·	·	·	·	·
R34	MPF / Two-stack SIMD-DP / Block	9m 11.2s ▼3.3× 9m 10.7s ▼3.4×	15m 34.3s ▼5.5× 15m 33.4s ▼5.7×	33m 48.6s ▼6.9× 33m 47.1s ▼7.3×	·	·	·	·	·	3m 19s ▶ 4% 2m 36.1s ▲23%	3m 6.1s ▲20% 2m 30.7s ▲1.5×	·	·	·	·	·	·
R35	MPF / Two-stack SIMD-DP / KSW2	9m 11.4s ▼3.3× 9m 9.5s ▼3.4×	13m 22s ▼4.7× 13m 19.3s ▼4.9×	33m 35.9s ▼6.9× 33m 29.9s ▼7.2×	·	·	·	·	·	3m 21.5s ▶ 3% 2m 35s ▲24%	3m 4.9s ▲21% 2m 25.8s ▲1.5×	·	·	·	·	·	·
R36	MPF / Two-stack Scalar-DP / Block	9m 15.8s ▼3.3× 9m 15s ▼3.4×	15m 26.5s ▼5.4× 15m 25.5s ▼5.7×	33m 22.2s ▼6.8× 33m 20.6s ▼7.2×	·	·	·	·	·	3m 18.4s ▶ 5% 2m 36s ▲23%	3m 4.1s ▲21% 2m 28.5s ▲1.5×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R37 BioseqDB	MPF / Two-stack Scalar-DP / KSW2	9m 16.3s ▼3.3× 9m 13.5s ▼3.4×	16m 8.5s ▼5.7× 16m 5.1s ▼5.9×	33m 26.6s ▼6.8× 33m 20.5s ▼7.2×	·	·	·	·	·	3m 21.4s ▶ 3% 2m 35s ▲24%	3m 4.9s ▲21% 2m 25.7s ▲1.5×	·	·	·	·	·	·
R38	MPF / Scalar Scalar-DP / KSW2	11m 11.3s ▼4.0× 11m 7.6s ▼4.1×	24m 35.8s ▼8.7× 24m 32.7s ▼9.0×	49m 58.6s ▼10.2× 49m 52.4s ▼10.7×	3h 11m 57.8s ▼7.7× 3h 11m 53.5s ▼7.7×	16m 14.9s ▼28.7× 16m 11.2s ▼29.5×	38m 2.4s ▼66.3× 37m 59.5s ▼69.7×	1h 11m 44.1s ▼63.1× 1h 11m 37.9s ▼66.0×	2h 23m 50.8s ▼45.5× 2h 23m 46.4s ▼47.0×	5m 8.2s ▼1.5× 4m 20.4s ▼28%	4m 17.9s ▼11% 3m 42.6s ▶ 0%	·	·	12m 31.3s ▼10.7× 4m 43.5s ▼4.1×	11m 42.9s ▼10.2× 5m 21.5s ▼4.8×	·	·
R39	MPF / Scalar Scalar-DP / Block	11m 13.8s ▼4.0× 11m 13s ▼4.2×	26m 53.1s ▼9.5× 26m 52.3s ▼9.9×	50m 56.1s ▼10.4× 50m 54.6s ▼10.9×	3h 12m 16.6s ▼7.7× 3h 12m 14.4s ▼7.7×	14m 50.9s ▼26.2× 14m 47.2s ▼26.9×	29m 53.2s ▼52.1× 29m 50.6s ▼54.7×	1h 12m 1.2s ▼63.4× 1h 11m 55.6s ▼66.3×	2h 25m 29.9s ▼46.0× 2h 25m 25.6s ▼47.5×	5m 21.8s ▼1.5× 4m 34.8s ▼1.4×	4m 17.1s ▼10% 3m 43.8s ▶ 0%	·	·	12m 28.5s ▼10.7× 4m 39.8s ▼4.1×	11m 56.7s ▼10.4× 5m 31.3s ▼5.0×	·	·
R40	MPF / Scalar SIMD-DP / KSW2	11m 14.3s ▼4.0× 11m 12.2s ▼4.2×	24m 37.5s ▼8.7× 24m 35.6s ▼9.1×	44m 15.3s ▼9.0× 44m 9.6s ▼9.5×	1h 34m 25.1s ▼3.8× 1h 34m 20.6s ▼3.8×	14m 52.8s ▼26.3× 14m 49.1s ▼27.0×	32m 52.9s ▼57.3× 32m 50s ▼60.2×	1h 8m 24.8s ▼60.2× 1h 8m 18.7s ▼62.9×	1h 26m 14.3s ▼27.3× 1h 26m 9.8s ▼28.1×	4m 53s ▼1.4× 4m 8.2s ▼22%	5m 9.3s ▼1.3× 4m 31.7s ▼22%	·	·	12m 34.5s ▼10.7× 4m 44.3s ▼4.1×	11m 58.3s ▼10.5× 5m 31.6s ▼5.0×	·	·
R41	MPF / Scalar RMQ / KSW2	11m 14.6s ▼4.0× 11m 12.5s ▼4.2×	25m 12.4s ▼8.9× 25m 10.5s ▼9.3×	53m 45.8s ▼11.0× 53m 39.5s ▼11.5×	3h 11m 29.7s ▼7.6× 3h 11m 25.2s ▼7.7×	15m 59.1s ▼28.3× 15m 55.5s ▼29.0×	29m 55.2s ▼52.2× 29m 52.6s ▼54.8×	1h 11m 28.9s ▼62.9× 1h 11m 22.8s ▼65.8×	1h 56m 30.8s ▼36.8× 1h 56m 26.1s ▼38.0×	5m 17.4s ▼1.5× 4m 30.6s ▼1.3×	4m 16.1s ▼10% 3m 41.1s ▶ 1%	·	·	12m 25.5s ▼10.6× 4m 33.9s ▼4.0×	12m 4.7s ▼10.6× 5m 35.2s ▼5.0×	·	·
R42	MPF / SIMD-ref RMQ / Block	11m 19.5s ▼4.0× 11m 18.8s ▼4.2×	23m 45.6s ▼8.4× 23m 44.9s ▼8.7×	45m 51.5s ▼9.3× 45m 50.2s ▼9.9×	2h 47m 50.9s ▼6.7× 2h 47m 48.7s ▼6.7×	12m 10.8s ▼21.5× 12m 7.1s ▼22.1×	31m 7.5s ▼54.3× 31m 4.5s ▼57.0×	47m 1.1s ▼41.4× 46m 55.3s ▼43.2×	·	4m 39.8s ▼1.3× 3m 55.2s ▼16%	4m 28.8s ▼15% 3m 52s ▶ 4%	·	·	10m 49.3s ▼9.2× 4m 37.1s ▼4.0×	10m 38.4s ▼9.3× 5m 1.8s ▼4.5×	·	·
R43	MPF / SIMD-ref RMQ / KSW2	11m 21.2s ▼4.1× 11m 17.6s ▼4.2×	23m 50.3s ▼8.4× 23m 47.4s ▼8.8×	47m 6.3s ▼9.6× 47m 0.3s ▼10.1×	3h 11m 12.9s ▼7.6× 3h 11m 8.5s ▼7.6×	12m 7.2s ▼21.4× 12m 3.6s ▼22.0×	25m 37s ▼44.7× 25m 34.2s ▼46.9×	47m 19.6s ▼41.6× 47m 13.3s ▼43.5×	·	4m 43.9s ▼1.4× 3m 57.8s ▼17%	4m 55.1s ▼27% 4m 16.8s ▼15%	·	·	10m 52.3s ▼9.3× 4m 28.1s ▼3.9×	10m 29.8s ▼9.2× 4m 56.7s ▼4.4×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.22: (C): AWS: 4 Gbp, k=20, w=15

Engine	Execution	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼20.4× 2m 41.7s ▼22.6×	2m 50.3s ▼19.8× 2m 43s ▼22.2×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼41.8× 25m 0.7s ▼46.6×	33.9s ▼2.3× 32.9s ▼3.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.9× 3m 3.7s ▼4.1×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.5×	7m 14.6s ▼1.5× 6m 57.2s ▼1.6×	51m 22s n/a 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.3×	2m 21.5s ▲6.0× 2m 18.4s ▲2.2×	15m 56.8s n/a 15m 51.9s n/a
R44 BioseqDB	MPF / SIMD-ref SIMD-DP / KSW2	11m 21.3s ▼4.1× 11m 17.7s ▼4.2×	24m 2.7s ▼8.5× 23m 59.7s ▼8.8×	46m 8s ▼9.4× 46m 1.9s ▼9.9×	1h 35m 48.4s ▼3.8× 1h 35m 44.2s ▼3.8×	12m 1.2s ▼21.2× 11m 57.5s ▼21.8×	35m 19s ▼61.6× 35m 15.4s ▼64.7×	46m 59.7s ▼41.3× 46m 53.4s ▼43.2×	·	4m 42.6s ▼1.4× 3m 56.3s ▼17%	4m 36.4s ▼19% 3m 58.1s ▼7%	·	·	11m 3.4s ▼9.4× 4m 43.3s ▼4.1×	10m 45.6s ▼9.4× 5m 9.2s ▼4.6×	·	·
R45	MPF / SIMD-ref Scalar-DP / Block	11m 22.1s ▼4.1× 11m 21.3s ▼4.2×	24m 22.6s ▼8.6× 24m 21.8s ▼9.0×	46m 17.2s ▼9.4× 46m 15.7s ▼9.9×	2h 59m 10.5s ▼7.1× 2h 59m 8.2s ▼7.2×	12m 25.5s ▼22.0× 12m 21.9s ▼22.5×	25m 32.6s ▼44.5× 25m 30s ▼46.8×	47m 21.3s ▼41.7× 47m 15.6s ▼43.5×	·	4m 45.1s ▼1.4× 4m 0.5s ▼19%	4m 33.4s ▼17% 3m 57.3s ▼6%	·	·	11m 4.3s ▼9.5× 4m 42.4s ▼4.1×	10m 27.9s ▼9.2× 4m 57.1s ▼4.4×	·	·
R46	MPF / Scalar RMQ / Block	11m 22.5s ▼4.1× 11m 22s ▼4.2×	24m 52.9s ▼8.8× 24m 52.4s ▼9.2×	46m 1.7s ▼9.4× 46m 0.4s ▼9.9×	2h 57m 27.5s ▼7.1× 2h 57m 25.3s ▼7.1×	14m 45.6s ▼26.1× 14m 42s ▼26.8×	31m 31s ▼54.9× 31m 28.4s ▼57.7×	1h 7m 40.8s ▼59.5× 1h 7m 35.3s ▼62.3×	1h 26m 25.6s ▼27.3× 1h 26m 20.7s ▼28.2×	5m 16.6s ▼1.5× 4m 33.9s ▼1.4×	4m 53.5s ▼26% 4m 17.9s ▼15%	·	·	12m 47.3s ▼10.9× 4m 55.6s ▼4.3×	12m 11.3s ▼10.7× 5m 42.7s ▼5.1×	·	·
R47	MPF / SIMD-ref Scalar-DP / KSW2	11m 28.4s ▼4.1× 11m 24.7s ▼4.2×	24m 37.6s ▼8.7× 24m 34.9s ▼9.0×	46m 9.8s ▼9.4× 46m 3.7s ▼9.9×	·	12m 9.4s ▼21.5× 12m 5.8s ▼22.0×	25m 52.5s ▼45.1× 25m 49.6s ▼47.4×	47m 13.9s ▼41.6× 47m 7.6s ▼43.4×	·	4m 48.9s ▼1.4× 4m 2.5s ▼20%	4m 34s ▼17% 3m 55.9s ▼6%	·	·	10m 57s ▼9.4× 4m 39.3s ▼4.0×	10m 35.2s ▼9.3× 5m 2s ▼4.5×	·	·
R48	MPF / Scalar SIMD-DP / Block	11m 30.6s ▼4.1× 11m 29.9s ▼4.3×	25m 2.6s ▼8.8× 25m 1.8s ▼9.2×	45m 36s ▼9.3× 45m 34.7s ▼9.8×	1h 36m 16.5s ▼3.8× 1h 36m 14.3s ▼3.8×	15m 49.6s ▼28.0× 15m 45.8s ▼28.7×	32m 50.4s ▼57.2× 32m 47.7s ▼60.2×	1h 6m 46.8s ▼58.8× 1h 6m 41.3s ▼61.4×	1h 30m 32.9s ▼28.6× 1h 30m 28.4s ▼29.6×	4m 51s ▼1.4× 4m 8.2s ▼22%	4m 40s ▼20% 4m 5.7s ▼10%	·	·	12m 25.7s ▼10.6× 4m 36.4s ▼4.0×	11m 44.4s ▼10.3× 5m 24.7s ▼4.9×	·	·
R49	MPF / SIMD-ref SIMD-DP / Block	12m 1.2s ▼4.3× 12m 0.4s ▼4.5×	24m 25.1s ▼8.6× 24m 24.2s ▼9.0×	46m 40.1s ▼9.5× 46m 38.8s ▼10.0×	1h 35m 35.8s ▼3.8× 1h 35m 33.7s ▼3.8×	12m 28.9s ▼22.1× 12m 25.2s ▼22.6×	34m 56.4s ▼60.9× 34m 53.2s ▼64.0×	49m 6.1s ▼43.2× 49m 0.3s ▼45.2×	·	4m 45.5s ▼1.4× 4m 1.2s ▼19%	4m 32.2s ▼17% 3m 55.6s ▼5%	·	·	10m 51.7s ▼9.3× 4m 28.8s ▼3.9×	10m 57.3s ▼9.6× 5m 24.8s ▼4.9×	·	·
R50 BioseqDB	Vulkan (all modes)	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.23: (D): AWS: 2 Gbp, k=15, w=10

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R2 BioseqDB	MPF / Bounded Scalar-DP / Block	3.6s ▲3.1× 2.8s ▲3.5×	3.4s ▲3.1× 2.6s ▲3.3×	6.4s ▲3.5× 4.4s ▲4.2×	3.2s ▲22.5× 2.4s ▲27.7×	10.2s ▶ 3% 6s ▲1.5×	8.6s ▲10% 5.5s ▲28%	14.6s ▲11% 8.4s ▲1.5×	12.4s ▲2.3× 9.2s ▲2.9×	1m 56.2s ▼2.2× 1m 47.8s ▼2.1×	2m 7.4s ▼2.3× 1m 56.8s ▼2.1×	·	2m 1.9s ▲3.0× 1m 56.1s ▲3.1×	6m 7.7s ▼7.7× 2m 6.1s ▼2.7×	6m 48.2s ▼9.2× 3m 14.7s ▼4.6×	·	11m 10.4s ▼4.0× 2m 8s ▲21%
R3	MPF / SIMD-ref Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.5×	3.4s ▲3.2× 2.6s ▲3.3×	6.5s ▲3.5× 4.4s ▲4.2×	3.3s ▲22.1× 2.5s ▲26.2×	12.7s ▼29% 7.4s ▲15%	13s ▼1.4× 7.4s ▶ 3%	24s ▼1.5× 13.6s ▼5%	16.5s ▲1.7× 13.6s ▲1.9×	2m 51.6s ▼3.2× 2m 26.6s ▼2.8×	2m 23.6s ▼2.5× 2m 2.9s ▼2.3×	·	·	·	·	·	·
R4	MPF / Scalar Scalar-DP / Block	3.8s ▲3.0× 3.1s ▲3.3×	3.5s ▲3.1× 2.7s ▲3.2×	6.5s ▲3.4× 4.8s ▲3.9×	3.4s ▲21.8× 2.5s ▲26.2×	10.1s ▶ 2% 5.9s ▲1.5×	8.3s ▲13% 5.2s ▲1.5×	15.9s ▶ 4% 9.6s ▲26%	14.5s ▲1.9× 11.3s ▲2.3×	2m 51.4s ▼3.2× 2m 29.1s ▼2.9×	2m 59.3s ▼3.2× 2m 39.2s ▼2.9×	·	·	7m 9.6s ▼8.9× 2m 48s ▼3.6×	6m 28.6s ▼8.7× 2m 48.4s ▼4.0×	·	·
R5	MPF / Two-stack Scalar-DP / Block	3.6s ▲3.1× 2.9s ▲3.4×	3.4s ▲3.2× 2.6s ▲3.3×	6.3s ▲3.6× 4.6s ▲4.1×	7.9s ▲9.2× 7.1s ▲9.3×	9.7s ▶ 3% 5.5s ▲1.6×	7.6s ▲20% 4.7s ▲1.6×	15.8s ▶ 4% 8.8s ▲1.5×	12.4s ▲2.3× 8.9s ▲3.0×	2m 41.5s ▼3.0× 2m 17s ▼2.6×	2m 37.7s ▼2.8× 2m 16.7s ▼2.5×	·	·	6m 32.3s ▼8.2× 2m 11.7s ▼2.8×	5m 45.5s ▼7.8× 2m 3.6s ▼2.9×	·	·
R6	MPF / Two-stack SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.6s ▲3.3×	6.7s ▲3.4× 4.7s ▲4.0×	3.4s ▲21.5× 2.5s ▲25.8×	11.8s ▼19% 7.6s ▲14%	8.6s ▲10% 5s ▲1.5×	16s ▶ 3% 9.1s ▲30%	13.5s ▲2.1× 11s ▲2.4×	2m 38.5s ▼3.0× 2m 16s ▼2.6×	2m 34.1s ▼2.7× 2m 13.1s ▼2.4×	·	3m 52.6s ▲1.6× 3m 35.8s ▲1.6×	6m 24.3s ▼8.0× 2m 4.9s ▼2.7×	5m 40.8s ▼7.7× 1m 59.2s ▼2.8×	·	13m 14.7s ▼4.8× 3m 54.6s ▼1.4×
R7	MPF / SIMD-ref SIMD-DP / Block	3.7s ▲3.0× 2.9s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.9s ▲3.3× 4.5s ▲4.2×	3.4s ▲21.4× 2.5s ▲26.3×	13.2s ▼1.3× 7.7s ▲13%	13.9s ▼1.5× 8s ▶ 5%	28.7s ▼1.7× 14.5s ▼13%	26.2s ▲7% 24.4s ▲7%	2m 47.2s ▼3.2× 2m 22.2s ▼2.7×	2m 32.6s ▼2.7× 2m 12.9s ▼2.4×	·	3m 6.8s ▲1.9× 2m 49.2s ▲2.1×	·	·	·	·
R8	MPF / Bounded RMQ / Block	3.7s ▲3.0× 2.9s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.5s ▲3.5× 4.5s ▲4.2×	3.4s ▲21.2× 2.5s ▲26.1×	18.7s ▼1.9× 14.5s ▼1.6×	20s ▼2.1× 16.6s ▼2.2×	21.3s ▼29% 14.6s ▼13%	12.4s ▲2.3× 9s ▲2.9×	1m 55.5s ▼2.2× 1m 47.1s ▼2.1×	2m 21s ▼2.5× 2m 10.3s ▼2.4×	·	2m 4s ▲2.9× 1m 58.1s ▲3.0×	6m 8.6s ▼7.7× 2m 7s ▼2.7×	5m 25.3s ▼7.3× 1m 58.4s ▼2.8×	·	11m 12.6s ▼4.0× 2m 10.3s ▲20%

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R9 BioseqDB	MPF / SIMD-ref RMQ / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.4× 4.6s ▲4.1×	3.5s ▲21.0× 2.6s ▲25.5×	13.6s ▼1.4× 7.8s ▲12%	13.2s ▼1.4× 8.1s ▼6%	25.7s ▼1.6× 14s ▼9%	18.3s ▲1.5× 15.5s ▲1.7×	2m 42.5s ▼3.1× 2m 17.6s ▼2.7×	2m 24.5s ▼2.6× 2m 4.8s ▼2.3×	·	·	·	·	·	·
R10	MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×	8.8s ▲8.3× 7.9s ▲8.3×	8.2s ▲17% 5.8s ▲1.5×	6.6s ▲1.4× 4.6s ▲1.6×	13.7s ▲17% 8.8s ▲1.5×	20.2s ▲28% 16.8s ▲1.6×	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×	3m 42.9s ▼1.9× 3m 25s ▼1.8×	2m 5.6s ▲2.9× 1m 59.7s ▲3.0×	5m 52.1s ▼7.3× 1m 53.5s ▼2.4×	4m 49.2s ▼6.5× 1m 56.8s ▼2.7×	9m 39.5s ▼6.4× 3m 18.1s ▼2.3×	11m 24.8s ▼4.1× 2m 20.4s ▲14%
R11	MPF / Scalar RMQ / Block	4s ▲2.8× 3.1s ▲3.2×	3.7s ▲2.9× 2.8s ▲3.1×	7.1s ▲3.2× 4.8s ▲3.9×	3.5s ▲20.6× 2.6s ▲25.3×	10s ▶ 1% 5.9s ▲1.5×	8.7s ▲8% 5.7s ▲25%	9.5s ▲1.7× 6.5s ▲2.0×	23s ▲18% 19.8s ▲25%	2m 51.5s ▼3.2× 2m 29.1s ▼2.9×	3m 4.8s ▼3.3× 2m 44.5s ▼3.0×	·	·	7m 6.3s ▼8.9× 2m 44.6s ▼3.5×	6m 52.5s ▼9.3× 3m 7.3s ▼4.4×	·	·
R12	MPF / Two-stack RMQ / Block	3.8s ▲2.9× 3s ▲3.3×	3.6s ▲3.0× 2.7s ▲3.2×	6.8s ▲3.3× 4.7s ▲4.0×	3.6s ▲20.1× 2.7s ▲24.2×	9.6s ▶ 3% 5.5s ▲1.6×	8.2s ▲14% 5s ▲1.5×	12.8s ▲23% 8.6s ▲1.5×	14.3s ▲2.0× 11.1s ▲2.4×	2m 38.4s ▼3.0× 2m 16.3s ▼2.6×	2m 38s ▼2.8× 2m 17.8s ▼2.5×	·	·	6m 27.6s ▼8.1× 2m 7.9s ▼2.7×	6m 2.1s ▼8.1× 2m 19.9s ▼3.3×	·	·
R13	MPF / Scalar SIMD-DP / Block	3.9s ▲2.9× 3.1s ▲3.3×	4.5s ▲2.4× 3.6s ▲2.4×	6.6s ▲3.4× 4.7s ▲4.0×	9.1s ▲8.1× 8.1s ▲8.1×	10.4s ▼5% 6s ▲1.5×	11.1s ▼17% 7.8s ▶ 2%	16.6s ▶ 1% 10.1s ▲22%	14.3s ▲2.0× 12.1s ▲2.2×	3m 8.5s ▼3.6× 2m 46.3s ▼3.2×	3m 1.2s ▼3.2× 2m 41.3s ▼3.0×	·	3m 2.9s ▲2.0× 2m 46s ▲2.1×	7m 10.9s ▼9.0× 2m 49.7s ▼3.6×	6m 59s ▼9.4× 3m 11.3s ▼4.5×	·	12m 21.6s ▼4.5× 3m 1s ▼11%
R14	MPF / Bounded SIMD-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.9s ▲2.3× 4.3s ▲4.4×	7.4s ▲9.9× 4.4s ▲15.1×	10.2s ▶ 3% 5.9s ▲1.5×	12.5s ▼1.3× 8.7s ▼14%	14.1s ▲14% 8.5s ▲1.5×	21.5s ▲24% 17.6s ▲1.5×	2m 11.6s ▼2.5× 2m 1.1s ▼2.3×	2m 7.5s ▼2.3× 1m 54.8s ▼2.1×	·	2m 20.7s ▲2.6× 2m 10.4s ▲2.7×	6m 9.1s ▼7.7× 2m 5.6s ▼2.7×	5m 32.3s ▼7.5× 1m 56.1s ▼2.7×	·	11m 23.6s ▼4.1× 2m 22.7s ▲12%
R15	MPF / SIMD-ref Scalar-DP / KSW2	5.1s ▲2.2× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.4×	9.8s ▲2.3× 4.3s ▲4.3×	7.2s ▲10.1× 4.4s ▲15.1×	12.8s ▼29% 7.3s ▲17%	13s ▼1.4× 7.3s ▶ 4%	24.5s ▼1.5× 13.4s ▶ 4%	16.2s ▲1.7× 12.9s ▲2.0×	2m 52.8s ▼3.3× 2m 25.9s ▼2.8×	2m 31.6s ▼2.7× 2m 9.7s ▼2.4×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R16 BioseqDB	MPF / Bounded Scalar-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.6×	4.7s ▲2.3× 2.5s ▲3.5×	9.7s ▲2.3× 4.3s ▲4.4×	7.2s ▲10.1× 4.3s ▲15.4×	9.9s ▶ 0% 5.8s ▲1.5×	8.5s ▲11% 5.1s ▲1.5×	15.2s ▲8% 8.5s ▲1.5×	13s ▲2.2× 9.4s ▲2.8×	2m 8.2s ▼2.4× 1m 58.3s ▼2.3×	2m 3.7s ▼2.2× 1m 51.2s ▼2.0×	·	2m 25.1s ▲2.5× 2m 14.9s ▲2.6×	6m 7.9s ▼7.7× 2m 3.4s ▼2.6×	5m 36.8s ▼7.6× 2m 6.4s ▼3.0×	9m 39.3s ▼6.4× 3m 13.2s ▼2.2×	11m 21s ▼4.1× 2m 19.3s ▲14%
R17	MPF / Two-stack Scalar-DP / KSW2	5.2s ▲2.1× 2.8s ▲3.5×	4.7s ▲2.3× 2.5s ▲3.4×	9.5s ▲2.4× 4.5s ▲4.2×	7.2s ▲10.1× 4.4s ▲14.9×	9.7s ▶ 2% 5.6s ▲1.6×	7.8s ▲18% 4.7s ▲1.6×	16.5s ▶ 0% 8.9s ▲1.5×	12.8s ▲2.2× 9s ▲2.9×	2m 46.6s ▼3.1× 2m 20.5s ▼2.7×	2m 46.1s ▼2.9× 2m 22.8s ▼2.6×	·	·	6m 29s ▼8.1× 2m 7.9s ▼2.7×	5m 48.7s ▼7.8× 2m 5.1s ▼2.9×	·	·
R18	MPF / Bounded RMQ / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.8s ▲2.2× 2.5s ▲3.5×	9.8s ▲2.3× 4.2s ▲4.4×	7.7s ▲9.5× 4.6s ▲14.2×	12s ▼21% 7.7s ▲13%	11.5s ▼21% 7.6s ▶ 0%	17.2s ▶ 4% 10s ▲22%	13.2s ▲2.1× 9.4s ▲2.8×	2m 17.7s ▼2.6× 2m 7s ▼2.4×	1m 59.4s ▼2.1× 1m 47.5s ▼2.0×	·	2m 24s ▲2.5× 2m 13.9s ▲2.7×	6m 7.2s ▼7.6× 2m 3.6s ▼2.6×	5m 26.8s ▼7.3× 1m 56.2s ▼2.7×	·	·
R19	MPF / Scalar Scalar-DP / KSW2	5.2s ▲2.1× 2.9s ▲3.5×	4.8s ▲2.2× 2.6s ▲3.3×	9.5s ▲2.4× 4.5s ▲4.2×	8.6s ▲8.5× 5.7s ▲11.6×	10.1s ▶ 2% 5.9s ▲1.5×	8.8s ▲8% 5.3s ▲1.4×	15.9s ▶ 4% 9.2s ▲29%	13.1s ▲2.1× 9.4s ▲2.8×	2m 52.3s ▼3.3× 2m 28.6s ▼2.9×	3m 4.9s ▼3.3× 2m 43.4s ▼3.0×	·	·	7m 3.4s ▼8.8× 2m 40.4s ▼3.4×	6m 32.5s ▼8.8× 2m 49.9s ▼4.0×	·	·
R20	MPF / SIMD-ref SIMD-DP / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.8s ▲2.2× 2.5s ▲3.4×	9.7s ▲2.3× 4.3s ▲4.4×	9s ▲8.1× 6.2s ▲10.6×	16.7s ▼1.7× 11.2s ▼27%	16.4s ▼1.7× 9.7s ▼27%	30.5s ▼1.8× 14.9s ▼15%	17.3s ▲1.6× 13.9s ▲1.9×	2m 51.2s ▼3.2× 2m 24.3s ▼2.8×	2m 22.1s ▼2.5× 2m 1.6s ▼2.2×	3m 23.3s ▼1.8× 2m 50.3s ▼1.5×	·	·	·	·	·
R21	MPF / Two-stack SIMD-DP / KSW2	5.3s ▲2.1× 2.9s ▲3.5×	4.8s ▲2.2× 2.5s ▲3.4×	9.8s ▲2.3× 4.5s ▲4.2×	7.1s ▲10.3× 4.2s ▲15.6×	9.7s ▶ 2% 5.5s ▲1.6×	9.6s ▶ 1% 5.2s ▲1.5×	16.4s ▶ 0% 8.8s ▲1.5×	14.9s ▲1.9× 11s ▲2.4×	2m 27.4s ▼2.8× 2m 3.1s ▼2.4×	2m 26.8s ▼2.6× 2m 5.2s ▼2.3×	·	·	6m 28.7s ▼8.1× 2m 7.4s ▼2.7×	5m 46.6s ▼7.8× 2m 1.5s ▼2.9×	·	·
R22	MPF / SIMD-ref RMQ / KSW2	5.3s ▲2.1× 2.8s ▲3.6×	4.9s ▲2.2× 2.5s ▲3.4×	9.9s ▲2.3× 4.3s ▲4.4×	5.5s ▲13.3× 2.4s ▲26.8×	13.6s ▼1.4× 7.5s ▲15%	13.8s ▼1.5× 7.7s ▶ 0%	26.7s ▼1.6× 13.8s ▼7%	31.9s ▼13% 28.3s ▼8%	2m 45.3s ▼3.1× 2m 18.5s ▼2.7×	2m 24.6s ▼2.6× 2m 3.8s ▼2.3×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R23 BioseqDB	MPF / Scalar SIMD-DP / KSW2	5.4s ▲2.1× 2.9s ▲3.5×	4.9s ▲2.2× 2.6s ▲3.3×	10.3s ▲2.2× 4.6s ▲4.1×	7.8s ▲9.3× 5s ▲13.2×	10.2s ▶ 3% 5.9s ▲1.5×	8.9s ▲6% 5.3s ▲1.5×	17.8s ▼8% 10.8s ▲16%	15.6s ▲1.8× 12.5s ▲2.1×	3m 4.9s ▼3.5× 2m 40.5s ▼3.1×	3m 7.6s ▼3.3× 2m 46s ▼3.0×	·	·	7m 6.6s ▼8.9× 2m 43.2s ▼3.5×	6m 54s ▼9.3× 3m 6.6s ▼4.4×	·	·
R24	MPF / Two-stack RMQ / KSW2	5.3s ▲2.1× 2.9s ▲3.5×	5s ▲2.1× 2.5s ▲3.4×	10.1s ▲2.2× 4.5s ▲4.2×	5.6s ▲12.9× 2.6s ▲25.5×	9.6s ▶ 4% 5.5s ▲1.6×	8.9s ▲6% 5.1s ▲1.5×	16.4s ▶ 1% 8.9s ▲1.5×	12.7s ▲2.2× 8.8s ▲3.0×	2m 37s ▼3.0× 2m 12.9s ▼2.6×	2m 34.6s ▼2.7× 2m 12.9s ▼2.4×	·	·	6m 27.3s ▼8.1× 2m 5.7s ▼2.7×	5m 42s ▼7.7× 1m 58.6s ▼2.8×	·	·
R25	MPF / Scalar RMQ / KSW2	5.5s ▲2.0× 2.9s ▲3.4×	5.1s ▲2.1× 2.7s ▲3.2×	10.3s ▲2.2× 4.6s ▲4.1×	5.6s ▲13.1× 2.5s ▲26.3×	10s ▶ 1% 5.9s ▲1.5×	8.7s ▲8% 5.3s ▲1.4×	16s ▶ 3% 9.3s ▲28%	21.1s ▲25% 17.5s ▲1.5×	3m 1.5s ▼3.4× 2m 37.7s ▼3.0×	2m 54.6s ▼3.1× 2m 33.3s ▼2.8×	·	·	7m 3s ▼8.8× 2m 38.7s ▼3.4×	6m 59.2s ▼9.4× 3m 13.2s ▼4.5×	·	·
R26	MPS / Bounded Scalar-DP / Block	6.4s ▲1.7× 5.7s ▲1.8×	6.6s ▲1.6× 5.7s ▲1.5×	12.5s ▲1.8× 10.1s ▲1.9×	5.9s ▲12.3× 4.9s ▲13.4×	4m 54s ▼29.7× 32s ▼3.6×	4m 0.6s ▼25.4× 28.5s ▼3.7×	7m 33.3s ▼27.5× 53.3s ▼4.1×	·	2m 24.9s ▼2.7× 2m 2.4s ▼2.4×	2m 39.7s ▼2.8× 2m 18.8s ▼2.5×	·	2m 43s ▲2.2× 2m 26.5s ▲2.4×	6m 32.4s ▼8.2× 2m 10.2s ▼2.8×	5m 33.1s ▼7.5× 1m 56s ▼2.7×	·	11m 41.8s ▼4.2× 2m 22.1s ▲13%
R27	MPS / Bounded SIMD-DP / Block	6.7s ▲1.7× 6s ▲1.7×	6.9s ▲1.5× 6s ▲1.4×	12.5s ▲1.8× 10.4s ▲1.8×	6s ▲12.1× 5s ▲13.0×	4m 54.5s ▼29.7× 33.3s ▼3.8×	4m 26.3s ▼28.1× 33.8s ▼4.4×	8m 17s ▼30.1× 54.4s ▼4.2×	·	2m 20.8s ▼2.7× 1m 57.4s ▼2.3×	2m 25.9s ▼2.6× 2m 5.5s ▼2.3×	·	2m 42.2s ▲2.2× 2m 23.6s ▲2.5×	6m 30.1s ▼8.1× 2m 18.5s ▼2.9×	5m 34.2s ▼7.5× 1m 56.9s ▼2.7×	·	11m 44s ▼4.2× 2m 21.6s ▲13%
R28	MPS / Bounded RMQ / Block	6.5s ▲1.7× 5.7s ▲1.8×	6.5s ▲1.6× 5.8s ▲1.5×	12.7s ▲1.8× 10.2s ▲1.8×	6.1s ▲12.0× 5.1s ▲12.8×	4m 55.9s ▼29.9× 33.3s ▼3.8×	4m 1.2s ▼25.4× 29.1s ▼3.8×	7m 32.9s ▼27.5× 53.3s ▼4.1×	·	2m 31.3s ▼2.9× 2m 9.3s ▼2.5×	2m 16.4s ▼2.4× 1m 56.6s ▼2.1×	·	2m 44.7s ▲2.2× 2m 26.9s ▲2.4×	6m 30.9s ▼8.1× 2m 9.4s ▼2.8×	5m 39.3s ▼7.6× 2m 0.7s ▼2.8×	·	11m 44.3s ▼4.2× 2m 23.5s ▲12%
R29	MPS / SIMD-ref SIMD-DP / Block	7.2s ▲1.5× 6.1s ▲1.6×	6.7s ▲1.6× 5.7s ▲1.5×	·	6.4s ▲11.4× 5.5s ▲12.0×	4m 55.8s ▼29.9× 34.7s ▼3.9×	4m 10.3s ▼26.4× 31.6s ▼4.1×	·	·	·	·	4m 15.9s ▼2.2× 3m 41.9s ▼2.0×	3m 19.6s ▲1.8× 3m 1.1s ▲2.0×	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.23: (D): AWS: 2 Gbp, k=15, w=10

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R30 BioseqDB	MPS / Two-stack SIMD-DP / Block	6.8s ▲1.6× 6.1s ▲1.7×	6.8s ▲1.6× 6s ▲1.4×	12.6s ▲1.8× 10.6s ▲1.8×	6.5s ▲11.2× 5.7s ▲11.6×	4m 47.8s ▼29.1× 26.2s ▼3.0×	3m 58.5s ▼25.1× 24.4s ▼3.2×	·	·	2m 46.7s ▼3.1× 2m 24.6s ▼2.8×	2m 47.4s ▼3.0× 2m 27.3s ▼2.7×	·	3m 50.1s ▲1.6× 3m 32.7s ▲1.7×	·	·	·	·
R31	MPS / Scalar SIMD-DP / Block	6.8s ▲1.6× 6.2s ▲1.6×	7.1s ▲1.5× 6.2s ▲28%	·	6.6s ▲11.1× 5.6s ▲11.7×	4m 53.5s ▼29.6× 32.6s ▼3.7×	4m 10s ▼26.4× 31.4s ▼4.1×	·	·	3m 9.8s ▼3.6× 2m 47.5s ▼3.2×	3m 6.4s ▼3.3× 2m 46.4s ▼3.0×	·	3m 9.9s ▲1.9× 2m 52.7s ▲2.1×	7m 6.1s ▼8.9× 2m 43.9s ▼3.5×	·	·	12m 35.2s ▼4.5× 3m 14.5s ▼20%
R32	MPS / SIMD-ref Scalar-DP / Block	7.2s ▲1.6× 6.1s ▲1.6×	6.8s ▲1.6× 5.8s ▲1.5×	·	·	4m 54.2s ▼29.7× 32.4s ▼3.7×	4m 9.1s ▼26.3× 31.3s ▼4.1×	·	·	·	·	·	·	·	·	·	·
R33	MPS / SIMD-ref RMQ / Block	7.1s ▲1.6× 6s ▲1.7×	6.8s ▲1.6× 5.8s ▲1.5×	·	·	4m 56.7s ▼29.9× 35.7s ▼4.1×	4m 11.8s ▼26.6× 33.7s ▼4.4×	·	·	·	·	3m 24.8s ▼1.8× 2m 59.9s ▼1.6×	·	·	·	·	·
R34	MPS / Two-stack RMQ / Block	6.9s ▲1.6× 6.2s ▲1.6×	7.5s ▲29% 6.6s ▲23%	·	·	4m 45.1s ▼28.8× 25.1s ▼2.8×	3m 56.8s ▼25.0× 23.9s ▼3.1×	·	·	2m 47.3s ▼3.2× 2m 24.6s ▼2.8×	2m 30.3s ▼2.7× 2m 10.8s ▼2.4×	4m 45s ▼2.5× 4m 11.3s ▼2.2×	·	·	·	·	·
R35	MPS / Two-stack Scalar-DP / Block	6.9s ▲1.6× 6.2s ▲1.6×	7.2s ▲1.5× 6.3s ▲26%	·	·	4m 44.1s ▼28.7× 24.5s ▼2.8×	3m 57.3s ▼25.0× 23.7s ▼3.1×	·	·	·	·	·	·	·	·	·	·
R36	MPS / Scalar RMQ / Block	6.9s ▲1.6× 6.2s ▲1.6×	7s ▲1.5× 6.2s ▲28%	·	·	4m 52.3s ▼29.5× 31.8s ▼3.6×	4m 9.2s ▼26.3× 30.9s ▼4.0×	·	·	3m 15.8s ▼3.7× 2m 53.2s ▼3.3×	3m 13.2s ▼3.4× 2m 53s ▼3.2×	·	·	7m 7.4s ▼8.9× 2m 45.6s ▼3.5×	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R37 BioseqDB	MPS / Scalar Scalar-DP / Block	7.5s ▲1.5× 6.3s ▲1.6×	7.2s ▲1.5× 6.4s ▲26%	·	·	4m 52s ▼29.5× 31.3s ▼3.6×	4m 9.3s ▼26.3× 30.8s ▼4.0×	·	·	3m 13.5s ▼3.6× 2m 47.5s ▼3.2×	3m 16.5s ▼3.5× 2m 55.3s ▼3.2×	·	·	7m 3.9s ▼8.8× 2m 41.8s ▼3.4×	·	·	·
R38	MPS / Bounded Scalar-DP / KSW2	7.9s ▲29% 4.9s ▲2.0×	8.6s ▲20% 5s ▲1.7×	17.2s ▲24% 8.9s ▲2.1×	8s ▲9.2× 4.6s ▲14.2×	4m 56.3s ▼29.9× 32.9s ▼3.7×	4m 2.1s ▼25.5× 28.6s ▼3.7×	8m 11.2s ▼29.8× 53.8s ▼4.2×	·	2m 37s ▼3.0× 2m 12s ▼2.5×	2m 30.6s ▼2.7× 2m 7.1s ▼2.3×	·	2m 29s ▲2.4× 2m 7.2s ▲2.8×	6m 26.9s ▼8.1× 2m 3.3s ▼2.6×	5m 37.5s ▼7.6× 1m 58.7s ▼2.8×	·	12m 0.3s ▼4.3× 2m 37s ▶ 4%
R39	MPS / Bounded RMQ / KSW2	7.9s ▲29% 4.9s ▲2.0×	8.5s ▲20% 4.9s ▲1.7×	17.4s ▲23% 8.8s ▲2.1×	8.1s ▲9.1× 4.6s ▲14.2×	4m 56.6s ▼29.9× 32.7s ▼3.7×	4m 2.8s ▼25.6× 28.9s ▼3.8×	7m 36.6s ▼27.7× 53.2s ▼4.1×	·	2m 21.9s ▼2.7× 1m 57.6s ▼2.3×	2m 27.9s ▼2.6× 2m 5.4s ▼2.3×	·	2m 29.1s ▲2.4× 2m 8.2s ▲2.8×	6m 41.2s ▼8.3× 2m 18.2s ▼2.9×	5m 35.7s ▼7.5× 1m 59.6s ▼2.8×	·	·
R40	MPS / Two-stack RMQ / KSW2	8.3s ▲26% 5.3s ▲1.9×	8.1s ▲23% 5.1s ▲1.7×	·	·	4m 47.6s ▼29.0× 25.3s ▼2.9×	3m 58.5s ▼25.2× 23.4s ▼3.1×	·	·	2m 51.1s ▼3.2× 2m 26.7s ▼2.8×	2m 59.5s ▼3.2× 2m 36.8s ▼2.9×	·	·	·	·	·	·
R41	MPS / Bounded SIMD-DP / KSW2	8.2s ▲26% 5.4s ▲1.8×	8.4s ▲21% 4.9s ▲1.8×	17.8s ▲21% 8.9s ▲2.1×	9.6s ▲7.6× 6s ▲10.9×	4m 56.1s ▼29.9× 32.4s ▼3.7×	4m 12.1s ▼26.6× 30.4s ▼4.0×	8m 4.9s ▼29.4× 52.6s ▼4.1×	·	2m 20.7s ▼2.7× 1m 56.8s ▼2.3×	2m 19.7s ▼2.5× 1m 57.9s ▼2.2×	·	2m 42.4s ▲2.2× 2m 21.1s ▲2.5×	6m 37.2s ▼8.3× 2m 13.5s ▼2.8×	5m 36.2s ▼7.6× 1m 59.7s ▼2.8×	·	11m 54s ▼4.3× 2m 27.8s ▲9%
R42	MPS / Scalar RMQ / KSW2	8.2s ▲26% 5.4s ▲1.9×	8.7s ▲18% 5.4s ▲1.6×	·	·	4m 53.7s ▼29.7× 31.6s ▼3.6×	4m 11.2s ▼26.5× 31.2s ▼4.1×	·	·	3m 14.2s ▼3.7× 2m 49.7s ▼3.3×	3m 39.1s ▼3.9× 3m 16.1s ▼3.6×	·	·	7m 9s ▼8.9× 2m 43.5s ▼3.5×	·	·	·
R43	MPS / Two-stack SIMD-DP / KSW2	8.3s ▲26% 5.4s ▲1.8×	8.4s ▲21% 5.2s ▲1.7×	16.2s ▲28% 9.2s ▲2.0×	·	4m 46.4s ▼28.9× 24.9s ▼2.8×	3m 58.8s ▼25.2× 23.6s ▼3.1×	·	·	2m 40.2s ▼3.0× 2m 15.4s ▼2.6×	2m 42.5s ▼2.9× 2m 20.9s ▼2.6×	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.1× 10s ▼3.6×	10.7s ▼3.2× 8.6s ▼3.5×	22.5s ▼3.6× 18.8s ▼4.4×	1m 13.1s ▼22.5× 1m 5.7s ▼27.7×	9.9s ▼21% 8.8s ▼1.6×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼1.7× 12.9s ▼2.0×	28.2s ▼2.3× 26.2s ▼3.0×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.8× 1m 51.8s ▲1.5×	6m 1.3s ▼3.0× 5m 55.2s ▼3.1×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.2×	2m 46.4s ▲4.0× 2m 42.7s ▼27%
R44 BioseqDB	MPS / Scalar SIMD-DP / KSW2	8.3s ▲26% 5.5s ▲1.8×	8.3s ▲22% 5.2s ▲1.6×	·	·	4m 52.9s ▼29.6× 31.1s ▼3.5×	4m 10.6s ▼26.4× 30.7s ▼4.0×	·	·	3m 1.5s ▼3.4× 2m 37.6s ▼3.0×	3m 21.1s ▼3.6× 2m 58.7s ▼3.3×	·	·	7m 6.4s ▼8.9× 2m 43.1s ▼3.5×	·	·	·
R45	MPS / Scalar Scalar-DP / KSW2	8.3s ▲26% 5.4s ▲1.8×	8.7s ▲18% 5.3s ▲1.6×	·	·	4m 54.2s ▼29.7× 31.5s ▼3.6×	4m 11s ▼26.5× 31s ▼4.1×	·	·	3m 21.3s ▼3.8× 2m 53.7s ▼3.3×	·	·	·	7m 19.5s ▼9.1× 2m 56s ▼3.7×	·	·	·
R46	MPS / Two-stack Scalar-DP / KSW2	8.4s ▲25% 5.4s ▲1.8×	9.4s ▲12% 5.5s ▲1.6×	·	·	4m 46.5s ▼28.9× 24.8s ▼2.8×	3m 58s ▼25.1× 23s ▼3.0×	·	·	·	·	·	·	·	·	·	·
R47	MPS / SIMD-ref SIMD-DP / KSW2	9.8s ▲13% 5.3s ▲1.9×	8.6s ▲20% 5s ▲1.7×	15.7s ▲1.4× 9.3s ▲2.0×	·	4m 55.9s ▼29.9× 33.3s ▼3.8×	4m 11.3s ▼26.5× 31.3s ▼4.1×	·	·	·	·	·	·	·	·	·	·
R48	MPS / SIMD-ref RMQ / KSW2	9.8s ▲12% 5.2s ▲1.9×	8.6s ▲19% 4.9s ▲1.7×	·	·	4m 58.8s ▼30.2× 36.1s ▼4.1×	4m 12.9s ▼26.7× 33.1s ▼4.3×	·	·	·	·	·	·	·	·	·	·
R49	MPS / SIMD-ref Scalar-DP / KSW2	9.8s ▲12% 5.2s ▲1.9×	8.7s ▲19% 5s ▲1.7×	16.4s ▲27% 8.9s ▲2.1×	·	4m 55s ▼29.8× 32s ▼3.6×	4m 9.4s ▼26.3× 29.7s ▼3.9×	·	·	·	·	·	·	·	·	·	·
R50 BioseqDB	Vulkan (all modes)	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]	—[g]

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.24: (E): AWS: 2 Gbp, k=20, w=15

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.6s ▲6.9× 1.9s ▲8.6×	5.4s ▲7.9× 3.4s ▲11.7×	4.5s ▲86.7× 3.6s ▲97.9×	7.5s ▲19% 5.2s ▲1.6×	5.8s ▲1.5× 4s ▲1.7×	12.6s ▲21% 7.9s ▲1.6×	12.9s ▲1.4× 10.6s ▲1.6×	1m 24.8s ▼2.0× 1m 16.9s ▼1.9×	1m 25.4s ▼2.0× 1m 15.7s ▼1.8×	2m 44s ▼1.9× 2m 26.2s ▼1.7×	1m 42s ▲2.9× 1m 35.7s ▲3.0×	5m 1.5s ▼8.6× 1m 24.2s ▼2.5×	4m 0.6s ▼6.7× 1m 24.3s ▼2.5×	8m 31.7s ▼6.9× 2m 34.1s ▼2.2×	10m 27.6s ▼10.4× 1m 51.4s ▼1.9×
R3	MPF / Bounded RMQ / Block	2.9s ▲6.7× 2.2s ▲8.3×	2.7s ▲6.7× 2s ▲8.3×	5.2s ▲8.2× 3.3s ▲12.1×	4.7s ▲82.6× 3.9s ▲92.2×	11.1s ▼19% 7.1s ▲16%	8.8s ▶ 1% 5.7s ▲19%	13.5s ▲15% 7.5s ▲1.7×	11.6s ▲1.6× 9.4s ▲1.8×	1m 34s ▼2.2× 1m 24.9s ▼2.1×	1m 33s ▼2.2× 1m 22.3s ▼2.0×	·	1m 44.1s ▲2.8× 1m 38.1s ▲2.9×	5m 11.1s ▼8.9× 1m 29.9s ▼2.6×	4m 41.1s ▼7.8× 1m 27.4s ▼2.6×	·	10m 23.5s ▼10.4× 1m 47.3s ▼1.8×
R4	MPF / Bounded Scalar-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.8s ▲6.5× 2s ▲8.2×	5.3s ▲8.0× 3.3s ▲11.8×	4.2s ▲92.8× 3.3s ▲106.7×	9.5s ▶ 2% 5.3s ▲1.6×	8.1s ▲7% 4.7s ▲1.5×	13.7s ▲14% 7.5s ▲1.7×	12.8s ▲1.4× 10.6s ▲1.6×	1m 32.9s ▼2.2× 1m 23.4s ▼2.0×	1m 31.2s ▼2.1× 1m 21s ▼2.0×	·	1m 43.1s ▲2.8× 1m 36.4s ▲3.0×	5m 11.8s ▼8.9× 1m 30.8s ▼2.7×	4m 49.9s ▼8.1× 1m 33.6s ▼2.7×	·	10m 26.2s ▼10.4× 1m 48.4s ▼1.9×
R5	MPF / Bounded SIMD-DP / KSW2	4.6s ▲4.3× 2.2s ▲8.3×	4.1s ▲4.5× 1.9s ▲8.5×	9.9s ▲4.3× 3.3s ▲12.1×	4.7s ▲83.2× 1.9s ▲184.9×	11.5s ▼23% 7.4s ▲12%	10.8s ▼23% 7.1s ▶ 1%	15.4s ▶ 2% 8.6s ▲1.5×	12.5s ▲1.5× 9.7s ▲1.8×	1m 34.1s ▼2.2× 1m 24s ▼2.0×	1m 32.2s ▼2.2× 1m 20s ▼1.9×	·	1m 46.2s ▲2.7× 1m 35.2s ▲3.0×	5m 10.2s ▼8.9× 1m 27.3s ▼2.6×	4m 45.4s ▼7.9× 1m 30.5s ▼2.6×	·	10m 25.5s ▼10.4× 1m 44.7s ▼1.8×
R6	MPF / Bounded Scalar-DP / KSW2	4.6s ▲4.3× 2.2s ▲8.3×	4.1s ▲4.4× 1.9s ▲8.6×	9.7s ▲4.4× 3.1s ▲12.6×	5.8s ▲67.2× 3s ▲119.5×	9.5s ▶ 2% 5.3s ▲1.6×	8.3s ▶ 5% 4.6s ▲1.5×	14.2s ▲10% 7.5s ▲1.7×	13.3s ▲28% 10.5s ▲1.6×	1m 27.2s ▼2.1× 1m 17s ▼1.9×	1m 39.6s ▼2.3× 1m 26.7s ▼2.1×	·	1m 46.4s ▲2.7× 1m 36.3s ▲3.0×	5m 10.4s ▼8.9× 1m 27.2s ▼2.6×	4m 48.1s ▼8.0× 1m 29.7s ▼2.6×	·	10m 29.2s ▼10.5× 1m 47.6s ▼1.8×
R7	MPF / Bounded RMQ / KSW2	4.5s ▲4.3× 2.2s ▲8.4×	4.2s ▲4.3× 2s ▲8.4×	9.7s ▲4.4× 3.2s ▲12.4×	6.2s ▲63.0× 3.4s ▲104.6×	9.4s ▶ 1% 5.3s ▲1.6×	8.1s ▲8% 4.6s ▲1.5×	15.6s ▶ 1% 9.1s ▲28%	12.6s ▲1.5× 9.8s ▲1.7×	1m 32.9s ▼2.2× 1m 23.1s ▼2.0×	1m 41.2s ▼2.4× 1m 28.4s ▼2.2×	·	1m 46.6s ▲2.7× 1m 35.5s ▲3.0×	5m 9.8s ▼8.9× 1m 27.1s ▼2.6×	4m 52.9s ▼8.2× 1m 32.9s ▼2.7×	·	10m 33.7s ▼10.5× 1m 55.3s ▼2.0×
R8	MPS / Bounded Scalar-DP / Block	6.6s ▲3.0× 5.9s ▲3.2×	6.6s ▲2.7× 5.9s ▲2.8×	12.8s ▲3.3× 10.5s ▲3.8×	6.3s ▲61.6× 5.4s ▲65.7×	4m 37.1s ▼29.7× 20s ▼2.4×	3m 55.5s ▼27.0× 19.7s ▼2.8×	·	·	1m 42.1s ▼2.4× 1m 20s ▼1.9×	1m 37.4s ▼2.3× 1m 18s ▼1.9×	·	1m 56.9s ▲2.5× 1m 39.6s ▲2.9×	5m 28.5s ▼9.4× 1m 29.3s ▼2.6×	4m 45.1s ▼7.9× 1m 22.8s ▼2.4×	·	10m 46.5s ▼10.8× 1m 52.9s ▼1.9×

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R9 BioseqDB	MPS / Bounded RMQ / Block	6.6s ▲3.0× 5.9s ▲3.2×	6.5s ▲2.8× 5.8s ▲2.8×	12.6s ▲3.4× 10.3s ▲3.8×	6.4s ▲61.0× 5.5s ▲65.2×	4m 37.8s ▼29.8× 20.1s ▼2.4×	3m 55.4s ▼26.9× 19.8s ▼2.8×	·	·	1m 43.2s ▼2.4× 1m 21s ▼2.0×	1m 41.8s ▼2.4× 1m 21.9s ▼2.0×	2m 55s ▼2.0× 2m 21s ▼1.7×	1m 57.1s ▲2.5× 1m 39.5s ▲2.9×	5m 28.1s ▼9.4× 1m 28.8s ▼2.6×	4m 44.5s ▼7.9× 1m 22.4s ▼2.4×	·	10m 46.5s ▼10.8× 1m 51.9s ▼1.9×
R10	MPS / Bounded SIMD-DP / Block	6.5s ▲3.0× 5.8s ▲3.2×	6.5s ▲2.8× 5.8s ▲2.8×	16.8s ▲2.5× 14.2s ▲2.8×	6.4s ▲60.5× 5.5s ▲64.7×	4m 39s ▼29.9× 20.9s ▼2.5×	3m 55.7s ▼27.0× 20.1s ▼2.9×	7m 43.9s ▼29.3× 35.6s ▼2.8×	·	1m 54.2s ▼2.7× 1m 29.4s ▼2.2×	1m 40.7s ▼2.4× 1m 20.8s ▼2.0×	·	1m 51.5s ▲2.6× 1m 35.2s ▲3.0×	5m 38.6s ▼9.7× 1m 39.5s ▼2.9×	4m 46.5s ▼8.0× 1m 23.5s ▼2.4×	8m 50.5s ▼7.2× 2m 35s ▼2.2×	10m 48.8s ▼10.8× 1m 54.4s ▼2.0×
R11	MPS / Bounded RMQ / KSW2	8.2s ▲2.4× 5.7s ▲3.3×	8s ▲2.3× 5.6s ▲2.9×	17.4s ▲2.4× 10.1s ▲3.9×	8.3s ▲46.6× 5.3s ▲67.2×	4m 39.5s ▼30.0× 20.2s ▼2.4×	3m 57.2s ▼27.2× 19.8s ▼2.8×	·	·	1m 48.4s ▼2.6× 1m 24.7s ▼2.0×	1m 39.1s ▼2.3× 1m 18s ▼1.9×	·	2m 0.6s ▲2.4× 1m 39.5s ▲2.9×	5m 38.2s ▼9.7× 1m 37.1s ▼2.9×	4m 45.8s ▼8.0× 1m 22.4s ▼2.4×	·	10m 55.2s ▼10.9× 1m 53.8s ▼1.9×
R12	MPS / Bounded SIMD-DP / KSW2	8.4s ▲2.3× 5.7s ▲3.3×	8s ▲2.3× 5.6s ▲2.9×	17.5s ▲2.4× 10.1s ▲3.9×	8.2s ▲47.3× 5.3s ▲67.7×	4m 39.1s ▼29.9× 20.1s ▼2.4×	3m 56.9s ▼27.1× 19.4s ▼2.8×	7m 26s ▼28.2× 34.2s ▼2.7×	·	1m 43.1s ▼2.4× 1m 19.2s ▼1.9×	1m 39.2s ▼2.3× 1m 17.9s ▼1.9×	·	1m 59.9s ▲2.4× 1m 38.2s ▲2.9×	5m 30.8s ▼9.5× 1m 29.4s ▼2.6×	4m 45.4s ▼7.9× 1m 22.1s ▼2.4×	·	10m 48.3s ▼10.8× 1m 48.9s ▼1.9×
R13	MPS / Bounded Scalar-DP / KSW2	8.2s ▲2.4× 5.7s ▲3.3×	8.1s ▲2.3× 5.6s ▲2.9×	17.7s ▲2.4× 10.1s ▲3.9×	8.4s ▲46.5× 5.3s ▲66.8×	4m 39.7s ▼30.0× 20.4s ▼2.4×	3m 56.6s ▼27.1× 19.5s ▼2.8×	·	·	1m 50.8s ▼2.6× 1m 27.1s ▼2.1×	1m 43.6s ▼2.4× 1m 21.8s ▼2.0×	·	2m 0.5s ▲2.4× 1m 38.7s ▲2.9×	5m 29.2s ▼9.4× 1m 28s ▼2.6×	·	·	10m 47.5s ▼10.8× 1m 51.8s ▼1.9×
R14	MPS / Two-stack SIMD-DP / Block	39.5s ▼2.0× 6.8s ▲2.7×	35.7s ▼2.0× 7.2s ▲2.3×	·	33.9s ▲11.5× 6.9s ▲51.8×	4m 25.9s ▼28.5× 13.3s ▼1.6×	3m 42s ▼25.4× 12s ▼1.7×	·	·	·	·	·	·	·	·	·	·
R15	MPS / SIMD-ref SIMD-DP / Block	46s ▼2.3× 8.9s ▲2.1×	40.5s ▼2.2× 8.8s ▲1.9×	·	34.8s ▲11.2× 8.2s ▲43.6×	4m 48.1s ▼30.9× 30.2s ▼3.6×	4m 3.1s ▼27.8× 28.3s ▼4.0×	·	·	·	·	·	2m 54.8s ▲1.7× 2m 26.4s ▲2.0×	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R16 BioseqDB	MPS / Two-stack RMQ / Block	39.9s ▼2.0× 6.8s ▲2.7×	35.5s ▼1.9× 7s ▲2.4×	·	·	4m 24.9s ▼28.4× 12.5s ▼1.5×	3m 41.4s ▼25.3× 11.5s ▼1.6×	·	·	·	·	·	·	·	·	·	·
R17	MPS / Two-stack Scalar-DP / Block	39.8s ▼2.0× 6.8s ▲2.7×	35.8s ▼2.0× 6.5s ▲2.5×	·	·	4m 25.7s ▼28.5× 12.7s ▼1.5×	3m 41.4s ▼25.3× 11.5s ▼1.6×	·	·	·	·	·	·	·	·	·	·
R18	MPS / Scalar SIMD-DP / Block	43.8s ▼2.2× 8.6s ▲2.2×	38s ▼2.1× 8.7s ▲1.9×	·	36.6s ▲10.6× 8.2s ▲43.4×	4m 40.3s ▼30.1× 23.1s ▼2.8×	4m 0.1s ▼27.5× 22.3s ▼3.2×	·	·	2m 52.9s ▼4.1× 2m 16.2s ▼3.3×	2m 34.3s ▼3.6× 2m 5.6s ▼3.1×	·	2m 43.8s ▲1.8× 2m 16.9s ▲2.1×	6m 48.5s ▼11.7× 2m 32s ▼4.5×	·	·	·
R19	MPS / Scalar RMQ / Block	43.6s ▼2.2× 8.8s ▲2.1×	37.4s ▼2.1× 8.8s ▲1.9×	·	·	4m 39.7s ▼30.0× 22s ▼2.6×	3m 57.3s ▼27.2× 21.1s ▼3.0×	·	·	2m 53.3s ▼4.1× 2m 17.7s ▼3.3×	2m 48s ▼3.9× 2m 18.9s ▼3.4×	3m 24s ▼2.3× 2m 47.8s ▼2.0×	·	6m 48.4s ▼11.7× 2m 33s ▼4.5×	·	·	·
R20	MPS / Scalar Scalar-DP / Block	43.3s ▼2.2× 8.7s ▲2.2×	37.5s ▼2.1× 8.9s ▲1.9×	·	·	4m 39.8s ▼30.0× 22s ▼2.6×	3m 57.2s ▼27.1× 20.8s ▼3.0×	·	·	2m 57.9s ▼4.2× 2m 19.7s ▼3.4×	·	·	·	·	·	·	·
R21	MPS / Scalar SIMD-DP / KSW2	45.1s ▼2.3× 8.4s ▲2.2×	37.9s ▼2.1× 8.2s ▲2.0×	1m 6.6s ▼1.6× 14.5s ▲2.7×	40.5s ▲9.6× 8.3s ▲43.2×	4m 40.4s ▼30.1× 21.8s ▼2.6×	3m 58.8s ▼27.3× 20.9s ▼3.0×	·	·	2m 54.8s ▼4.1× 2m 18.1s ▼3.3×	2m 39.7s ▼3.7× 2m 8.9s ▼3.1×	4m 27.6s ▼3.1× 3m 34.6s ▼2.6×	·	6m 52.2s ▼11.8× 2m 34.3s ▼4.5×	·	·	·
R22	MPS / SIMD-ref RMQ / Block	46.5s ▼2.4× 9.2s ▲2.0×	38.3s ▼2.1× 8.8s ▲1.9×	·	·	4m 47s ▼30.8× 28.9s ▼3.4×	4m 2.7s ▼27.8× 28.4s ▼4.0×	·	·	·	·	3m 51.4s ▼2.7× 3m 1.9s ▼2.2×	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R23 BioseqDB	MPS / SIMD-ref Scalar-DP / Block	46.3s ▼2.4× 9.2s ▲2.0×	38.4s ▼2.1× 8.8s ▲1.9×	·	·	4m 46.5s ▼30.7× 28.7s ▼3.4×	4m 0.9s ▼27.6× 26.6s ▼3.8×	·	·	·	·	·	·	·	·	·	·
R24	MPS / Scalar Scalar-DP / KSW2	45.2s ▼2.3× 8.7s ▲2.1×	39.2s ▼2.2× 8.8s ▲1.9×	1m 6.9s ▼1.6× 14.8s ▲2.7×	·	4m 40.9s ▼30.1× 21.8s ▼2.6×	3m 58.9s ▼27.3× 20.7s ▼2.9×	·	·	2m 57.6s ▼4.2× 2m 18.2s ▼3.3×	·	·	·	·	·	·	·
R25	MPS / Two-stack Scalar-DP / KSW2	43.6s ▼2.2× 6.8s ▲2.7×	39.5s ▼2.2× 7.2s ▲2.3×	·	·	4m 31.6s ▼29.1× 12.4s ▼1.5×	3m 44.6s ▼25.7× 11.3s ▼1.6×	·	·	·	·	·	·	·	·	·	·
R26	MPS / Two-stack SIMD-DP / KSW2	43.7s ▼2.2× 6.8s ▲2.7×	39.6s ▼2.2× 7.2s ▲2.3×	·	·	4m 30s ▼29.0× 12.4s ▼1.5×	3m 45.3s ▼25.8× 11.5s ▼1.6×	·	·	·	·	·	·	·	·	·	·
R27	MPS / Two-stack RMQ / KSW2	43.4s ▼2.2× 6.8s ▲2.7×	39.6s ▼2.2× 6.7s ▲2.4×	·	·	4m 29.8s ▼28.9× 12.6s ▼1.5×	3m 45.3s ▼25.8× 11.5s ▼1.6×	·	·	·	·	·	·	·	·	·	·
R28	MPS / Scalar RMQ / KSW2	45.4s ▼2.3× 8.8s ▲2.1×	39.7s ▼2.2× 9s ▲1.8×	·	·	4m 41.5s ▼30.2× 22.4s ▼2.7×	4m 0.1s ▼27.5× 21.1s ▼3.0×	·	·	2m 52.9s ▼4.1× 2m 15.4s ▼3.3×	2m 47.4s ▼3.9× 2m 15.8s ▼3.3×	·	·	·	·	·	·
R29	MPS / SIMD-ref Scalar-DP / KSW2	47.8s ▼2.4× 9.1s ▲2.0×	39.9s ▼2.2× 8.7s ▲1.9×	·	·	4m 48.7s ▼31.0× 28.7s ▼3.4×	4m 3.3s ▼27.8× 27s ▼3.8×	·	·	·	·	·	·	·	·	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.24: (E): AWS: 2 Gbp, k=20, w=15

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R30 BioseqDB	MPS / SIMD-ref RMQ / KSW2	48.4s ▼2.5× 9.1s ▲2.0×	40.4s ▼2.2× 8.8s ▲1.9×	·	·	4m 48.1s ▼30.9× 28.9s ▼3.4×	4m 3.5s ▼27.9× 27.1s ▼3.9×	·	·	·	·	·	·	·	·	·	·
R31	MPS / SIMD-ref SIMD-DP / KSW2	47.5s ▼2.4× 8.9s ▲2.1×	40.7s ▼2.2× 8.9s ▲1.9×	1m 6.7s ▼1.6× 14.4s ▲2.7×	·	4m 47.7s ▼30.9× 28.5s ▼3.4×	4m 3.9s ▼27.9× 27.7s ▼3.9×	·	·	·	·	·	·	·	·	·	·
R32	MPF / Two-stack RMQ / Block	4m 37.6s ▼14.2× 4m 37.1s ▼14.9×	7m 30s ▼24.7× 7m 29.2s ▼27.2×	17m 24.5s ▼24.6× 17m 22.9s ▼26.5×	·	3m 23s ▼21.8× 3m 21.6s ▼24.0×	5m 46.9s ▼39.7× 5m 45.4s ▼49.2×	11m 57.4s ▼45.4× 11m 54.8s ▼56.1×	·	1m 54.6s ▼2.7× 1m 19s ▼1.9×	1m 45.2s ▼2.5× 1m 14.4s ▼1.8×	·	·	5m 31.5s ▼9.5× 1m 18.5s ▼2.3×	4m 59.4s ▼8.3× 1m 13.4s ▼2.1×	·	·
R33	MPF / Two-stack RMQ / KSW2	4m 34.2s ▼14.0× 4m 32.2s ▼14.6×	6m 52.3s ▼22.7× 6m 49.6s ▼24.8×	17m 29.3s ▼24.7× 17m 23.1s ▼26.5×	·	3m 23.9s ▼21.9× 3m 21.3s ▼24.0×	5m 43s ▼39.3× 5m 39.6s ▼48.3×	12m 2.7s ▼45.7× 11m 55.6s ▼56.2×	·	1m 59.6s ▼2.8× 1m 19.6s ▼1.9×	1m 48.5s ▼2.5× 1m 14.5s ▼1.8×	·	·	5m 35.5s ▼9.6× 1m 17.7s ▼2.3×	5m 2s ▼8.4× 1m 12.5s ▼2.1×	·	·
R34	MPF / Two-stack SIMD-DP / Block	4m 36.1s ▼14.1× 4m 35.5s ▼14.8×	7m 3.9s ▼23.3× 7m 3s ▼25.6×	17m 28.7s ▼24.7× 17m 27.2s ▼26.6×	5m 27s ▲16% 5m 26s ▲9%	3m 24.2s ▼21.9× 3m 22.7s ▼24.2×	5m 44.1s ▼39.4× 5m 42.7s ▼48.8×	11m 58.9s ▼45.5× 11m 56.2s ▼56.2×	4m 0.4s ▼13.0× 3m 59s ▼14.0×	1m 57.3s ▼2.8× 1m 21.6s ▼2.0×	1m 45.1s ▼2.5× 1m 15.1s ▼1.8×	·	·	5m 30.7s ▼9.5× 1m 17.9s ▼2.3×	4m 57.2s ▼8.3× 1m 13.2s ▼2.1×	·	·
R35	MPF / Two-stack SIMD-DP / KSW2	4m 37.8s ▼14.2× 4m 35.9s ▼14.8×	7m 37.6s ▼25.1× 7m 34.4s ▼27.5×	17m 30.1s ▼24.7× 17m 23.9s ▼26.5×	·	3m 24.7s ▼22.0× 3m 22.1s ▼24.1×	5m 44.4s ▼39.4× 5m 40.9s ▼48.5×	12m 4s ▼45.8× 11m 56.8s ▼56.3×	·	1m 58.9s ▼2.8× 1m 18.9s ▼1.9×	1m 47.5s ▼2.5× 1m 13.8s ▼1.8×	·	·	5m 34.8s ▼9.6× 1m 17.6s ▼2.3×	5m 3.8s ▼8.5× 1m 12.8s ▼2.1×	·	·
R36	MPF / Two-stack Scalar-DP / KSW2	4m 38.1s ▼14.2× 4m 36.2s ▼14.8×	7m 34.7s ▼25.0× 7m 31.6s ▼27.3×	17m 28.2s ▼24.7× 17m 22.1s ▼26.5×	·	3m 24.7s ▼22.0× 3m 22.1s ▼24.1×	5m 43.5s ▼39.3× 5m 40.1s ▼48.4×	12m 2.2s ▼45.7× 11m 55s ▼56.1×	·	1m 59.1s ▼2.8× 1m 19.3s ▼1.9×	1m 47.1s ▼2.5× 1m 13.3s ▼1.8×	·	·	5m 34.9s ▼9.6× 1m 17.2s ▼2.3×	4m 58.5s ▼8.3× 1m 10.5s ▼2.1×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —[g] = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R37 BioseqDB	MPF / Two-stack Scalar-DP / Block	4m 36s ▼14.1× 4m 35.5s ▼14.8×	6m 53.8s ▼22.7× 6m 52.9s ▼25.0×	17m 25.1s ▼24.6× 17m 23.5s ▼26.5×	·	3m 24.8s ▼22.0× 3m 23.4s ▼24.3×	5m 38s ▼38.7× 5m 36.6s ▼47.9×	11m 57.4s ▼45.4× 11m 54.9s ▼56.1×	·	1m 55.2s ▼2.7× 1m 18.8s ▼1.9×	1m 43.5s ▼2.4× 1m 13.7s ▼1.8×	·	·	5m 30.7s ▼9.5× 1m 17.1s ▼2.3×	4m 55s ▼8.2× 1m 11.7s ▼2.1×	·	·
R38	MPF / Scalar Scalar-DP / Block	5m 39.5s ▼17.3× 5m 39s ▼18.2×	12m 20.4s ▼40.7× 12m 19.8s ▼44.8×	22m 35.5s ▼31.9× 22m 34.5s ▼34.4×	12m 31.2s ▼1.9× 12m 30.6s ▼2.1×	7m 58.4s ▼51.3× 7m 54.7s ▼56.6×	17m 10s ▼117.9× 17m 7.3s ▼146.2×	29m 8.1s ▼110.6× 29m 2.5s ▼136.8×	15m 18.5s ▼49.7× 15m 15.3s ▼53.8×	2m 53.1s ▼4.1× 2m 14.9s ▼3.3×	2m 43.4s ▼3.8× 2m 13.1s ▼3.2×	·	·	6m 44s ▼11.6× 2m 28s ▼4.4×	6m 4.7s ▼10.2× 2m 28s ▼4.3×	·	·
R39	MPF / Scalar Scalar-DP / KSW2	5m 39.7s ▼17.3× 5m 37.2s ▼18.1×	12m 8.9s ▼40.0× 12m 7.2s ▼44.0×	22m 21.6s ▼31.6× 22m 15.8s ▼33.9×	10m 39.3s ▼1.6× 10m 36.8s ▼1.8×	7m 46.7s ▼50.1× 7m 43s ▼55.2×	16m 59.7s ▼116.7× 16m 56.8s ▼144.7×	28m 56.8s ▼109.8× 28m 50.8s ▼135.9×	15m 4.5s ▼48.9× 15m 0.9s ▼52.9×	2m 57.1s ▼4.2× 2m 16.9s ▼3.3×	3m 14.4s ▼4.5× 2m 40.8s ▼3.9×	·	·	6m 47s ▼11.6× 2m 28.7s ▼4.4×	6m 3s ▼10.1× 2m 24.7s ▼4.2×	·	·
R40	MPF / Scalar SIMD-DP / Block	5m 39.7s ▼17.3× 5m 39.2s ▼18.2×	12m 5.2s ▼39.8× 12m 4.6s ▼43.9×	22m 32.6s ▼31.8× 22m 31.6s ▼34.3×	12m 48.6s ▼2.0× 12m 47.9s ▼2.2×	7m 52s ▼50.6× 7m 48.3s ▼55.9×	16m 56.1s ▼116.3× 16m 53.4s ▼144.2×	29m 42.8s ▼112.8× 29m 37.4s ▼139.5×	15m 28s ▼50.2× 15m 24.8s ▼54.3×	2m 45.2s ▼3.9× 2m 8.9s ▼3.1×	2m 44.3s ▼3.8× 2m 14.4s ▼3.3×	·	2m 41s ▲1.8× 2m 13.3s ▲2.2×	6m 49.3s ▼11.7× 2m 33s ▼4.5×	6m 44.7s ▼11.3× 2m 51.6s ▼5.0×	·	11m 45.1s ▼11.7× 2m 31.9s ▼2.6×
R41	MPF / Scalar RMQ / Block	5m 39.9s ▼17.3× 5m 39.5s ▼18.2×	12m 19.6s ▼40.6× 12m 19.1s ▼44.8×	22m 41.7s ▼32.0× 22m 40.7s ▼34.6×	12m 57s ▼2.0× 12m 56.4s ▼2.2×	7m 52.2s ▼50.7× 7m 48.5s ▼55.9×	17m 9.4s ▼117.8× 17m 6.8s ▼146.1×	21m 1.2s ▼79.8× 20m 58.9s ▼98.8×	15m 4.2s ▼48.9× 15m 1.3s ▼52.9×	2m 49.9s ▼4.0× 2m 13.4s ▼3.2×	2m 29.9s ▼3.5× 2m 1.1s ▼2.9×	·	·	6m 44.4s ▼11.6× 2m 28.2s ▼4.4×	6m 31.8s ▼10.9× 2m 38.1s ▼4.6×	·	·
R42	MPF / SIMD-ref Scalar-DP / KSW2	5m 40.6s ▼17.4× 5m 36.9s ▼18.1×	12m 16.8s ▼40.5× 12m 13.7s ▼44.4×	22m 52.9s ▼32.3× 22m 46.8s ▼34.7×	12m 8.2s ▼1.9× 12m 4.8s ▼2.0×	6m 5.2s ▼39.2× 6m 1.5s ▼43.1×	13m 19.6s ▼91.5× 13m 16.8s ▼113.4×	24m 19.3s ▼92.3× 24m 13.1s ▼114.1×	11m 40.3s ▼37.9× 11m 36.8s ▼40.9×	2m 42s ▼3.8× 2m 2.1s ▼2.9×	2m 30.1s ▼3.5× 1m 57.8s ▼2.9×	·	·	6m 17.7s ▼10.8× 2m 16.5s ▼4.0×	6m 21.8s ▼10.6× 2m 44.5s ▼4.8×	·	·
R43	MPF / Scalar SIMD-DP / KSW2	5m 42.8s ▼17.5× 5m 40.6s ▼18.3×	12m 19.8s ▼40.6× 12m 18s ▼44.7×	22m 43.4s ▼32.1× 22m 39.5s ▼34.5×	11m 32.1s ▼1.8× 11m 29.4s ▼1.9×	8m 18.5s ▼53.5× 8m 14.8s ▼59.0×	16m 50.3s ▼115.6× 16m 47.4s ▼143.4×	29m 22.4s ▼111.5× 29m 16.4s ▼137.9×	15m 17.5s ▼49.6× 15m 13.9s ▼53.7×	2m 52.8s ▼4.1× 2m 15.7s ▼3.3×	2m 32.2s ▼3.6× 2m 0.8s ▼2.9×	·	·	6m 48.6s ▼11.7× 2m 28.4s ▼4.4×	6m 49.3s ▼11.4× 2m 53.7s ▼5.1×	·	·

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

Table 10.24: (E): AWS: 2 Gbp, k=20, w=15

Engine	Execution	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.7× 18.6s ▼8.4×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼8.2× 39.4s ▼12.6×	6m 29.2s ▼92.8× 5m 56.9s ▼184.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.7×	18.5s ▼1.6× 17s ▼1.8×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9× 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.3×	35.9s ▲6.7× 34.2s ▲2.1×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4× 58.6s ▲1.8×
R44 BioseqDB	MPF / SIMD-ref RMQ / Block	5m 44s ▼17.6× 5m 43.3s ▼18.4×	12m 19.6s ▼40.6× 12m 18.9s ▼44.7×	23m 12.3s ▼32.7× 23m 11s ▼35.3×	11m 15.2s ▼1.7× 11m 14.4s ▼1.9×	6m 4.6s ▼39.1× 6m 1s ▼43.1×	13m 18.8s ▼91.4× 13m 16.2s ▼113.3×	23m 19.7s ▼88.5× 23m 16.1s ▼109.6×	12m 38s ▼41.0× 12m 35s ▼44.4×	2m 40.3s ▼3.8× 2m 2.4s ▼3.0×	2m 36.1s ▼3.6× 2m 4.7s ▼3.0×	·	·	6m 15.1s ▼10.7× 2m 16.3s ▼4.0×	6m 12.9s ▼10.4× 2m 40.4s ▼4.7×	·	·
R45	MPF / Scalar RMQ / KSW2	5m 46.6s ▼17.7× 5m 44.5s ▼18.5×	12m 26.1s ▼41.0× 12m 24.3s ▼45.1×	22m 22.5s ▼31.6× 22m 16.8s ▼33.9×	11m 45.8s ▼1.8× 11m 43.3s ▼2.0×	7m 49.2s ▼50.3× 7m 45.6s ▼55.5×	17m 0.3s ▼116.8× 16m 57.4s ▼144.8×	29m 26.3s ▼111.7× 29m 20.3s ▼138.2×	15m 2.9s ▼48.8× 14m 59.6s ▼52.8×	2m 53.6s ▼4.1× 2m 15.9s ▼3.3×	2m 44s ▼3.8× 2m 12.1s ▼3.2×	·	·	6m 46.2s ▼11.6× 2m 28.4s ▼4.4×	6m 33.5s ▼11.0× 2m 49.4s ▼5.0×	·	·
R46	MPF / SIMD-ref SIMD-DP / KSW2	5m 50.5s ▼17.9× 5m 46.9s ▼18.6×	12m 30.9s ▼41.3× 12m 27.6s ▼45.3×	22m 56s ▼32.4× 22m 50s ▼34.8×	11m 32.8s ▼1.8× 11m 29.4s ▼1.9×	6m 4.7s ▼39.1× 6m 1.1s ▼43.1×	13m 22.2s ▼91.8× 13m 19.2s ▼113.7×	25m 24.5s ▼96.4× 25m 18.3s ▼119.2×	12m 52.2s ▼41.8× 12m 48.8s ▼45.2×	2m 40.4s ▼3.8× 2m 0.9s ▼2.9×	2m 41.5s ▼3.8× 2m 8.1s ▼3.1×	·	·	6m 16.9s ▼10.8× 2m 16s ▼4.0×	6m 11s ▼10.3× 2m 39.5s ▼4.7×	·	·
R47	MPF / SIMD-ref RMQ / KSW2	5m 50.6s ▼17.9× 5m 47s ▼18.6×	12m 12.8s ▼40.3× 12m 9.8s ▼44.2×	23m 7.9s ▼32.6× 23m 1.9s ▼35.1×	10m 52.1s ▼1.7× 10m 49.7s ▼1.8×	6m 6.7s ▼39.3× 6m 3s ▼43.3×	13m 19.1s ▼91.5× 13m 16.2s ▼113.3×	24m 33.5s ▼93.2× 24m 27.3s ▼115.2×	12m 1.2s ▼39.0× 11m 57.8s ▼42.2×	2m 41.8s ▼3.8× 2m 2.1s ▼2.9×	2m 37.2s ▼3.7× 2m 3.8s ▼3.0×	·	·	6m 18.7s ▼10.8× 2m 18s ▼4.1×	6m 21.7s ▼10.6× 2m 44.6s ▼4.8×	·	·
R48	MPF / SIMD-ref SIMD-DP / Block	5m 50.7s ▼17.9× 5m 49.9s ▼18.8×	12m 26.8s ▼41.0× 12m 25.9s ▼45.2×	22m 58.7s ▼32.4× 22m 57.3s ▼35.0×	13m 2.2s ▼2.0× 13m 1.5s ▼2.2×	6m 8.9s ▼39.6× 6m 5.3s ▼43.6×	13m 31.2s ▼92.8× 13m 28.4s ▼115.0×	24m 58.1s ▼94.7× 24m 52.4s ▼117.1×	13m 3.8s ▼42.4× 13m 0.9s ▼45.9×	2m 38.4s ▼3.7× 2m 0.1s ▼2.9×	2m 35.4s ▼3.6× 2m 4.3s ▼3.0×	·	2m 49.5s ▲1.7× 2m 21.9s ▲2.0×	6m 15.5s ▼10.7× 2m 16.4s ▼4.0×	5m 52.6s ▼9.8× 2m 26.6s ▼4.3×	·	12m 1.9s ▼12.0× 3m 7.8s ▼3.2×
R49	MPF / SIMD-ref Scalar-DP / Block	5m 51.9s ▼18.0× 5m 51.1s ▼18.9×	12m 10.6s ▼40.1× 12m 9.8s ▼44.2×	23m 42.4s ▼33.5× 23m 41s ▼36.1×	11m 56.4s ▼1.8× 11m 55.6s ▼2.0×	6m 3.8s ▼39.0× 6m 0.1s ▼43.0×	13m 15.2s ▼91.0× 13m 12.6s ▼112.8×	24m 21.9s ▼92.5× 24m 16.2s ▼114.3×	11m 53.1s ▼38.6× 11m 50s ▼41.7×	2m 40s ▼3.8× 2m 1.7s ▼2.9×	2m 29.9s ▼3.5× 1m 58.9s ▼2.9×	·	·	6m 18.4s ▼10.8× 2m 19.3s ▼4.1×	6m 19.9s ▼10.6× 2m 45.3s ▼4.8×	·	·
R50 BioseqDB	Vulkan (all modes)	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g	—g

Each cell: A, comparison, B, comparison. · = no measurement. —g = GPU unavailable. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
MPF/MPS = minimizers per fragment/sequence. SIMD-ref = reference minimizer. Bounded = bounded ntHash.

10.1.4. Comparison with other aligners

The following adjustments were needed to run the compared aligners:

- Minimap2. A single FASTA record must fit within its signed 32-bit coordinate limit. We therefore split the 4 Gbp reference into two records of roughly 2 Gbp each. Returned positions are converted back to coordinates in the original reference. The 2 Gbp reference remains one record.

- Winnowmap. We re-enabled saving its native index and changed loading to rebuild the weighting Bloom filter from the saved list of frequent k-mers produced by Meryl. We also initialized a repeat-length variable that otherwise produced undefined MAPQ values. The 4 Gbp references used index batches of 1,999 million bases. Where memory was still insufficient, we split the reference into 250 Mb records, overlapping by the longest query plus 64 bases, and reduced index batches to 499 million bases. Results from the index parts are merged, which can affect MAPQ.

- blastn. We disabled low-complexity masking and required at least 80% query coverage to remove short chance matches. Scoring uses match reward 2, mismatch penalty 4 and gap open/extension penalties 4/2, matching the first affine-gap branch of minimap2. BLAST has no second branch and scores N as -3 rather than -1. The default E-value threshold of 10 is retained.

- LexicMap. Its 28-bit coordinates require records of at most 268,435,455 bases. We split the reference accordingly, with overlap equal to the longest query, and restore original coordinates in the results. On machines with less than 10 GiB RAM, the 4 Gbp reference was indexed one record per batch. We kept its native 20,000 LexicHash masks and alignment scoring. Query coverage is at least 80%. Its reference encoding replaces N with A, while query bases are unchanged.

Aligner	Configuration	Index 4 Gbp k=15, w=10								No index 4 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	36.2s ▼3.6× 35s ▼3.9×	38.1s ▼2.9× 36s ▼3.0×	1m 32.4s ▼9.8× 1m 28.5s ▼12.7×	6m 31.5s ▼11.2×[g] 6m 26.3s ▼11.4×	31.6s ▼1.4× 30.3s ▼1.6×	34.1s ▼1.4× 32s ▼1.5×	1m 9.5s ▼28% 1m 6.1s ▼1.4×	3m 1s ▼3.6×[g] 2m 51.5s ▼3.7×	2m 55.5s ▲22% 2m 54s ▲19%	2m 50.4s ▲28% 2m 48.2s ▲25%	5m 35.7s ▲18% 5m 31.9s ▲15%	44m 20.5s n/a[g] 44m 15.6s n/a	1m 47.1s ▲5.5× 1m 45.8s ▲2.1×	1m 47.2s ▲4.7× 1m 45.1s ▲2.3×	3m 40.3s ▲4.2× 3m 36.6s ▲1.7×	29m 43.4s n/a[g] 29m 38.5s n/a
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	9.9s ▲3.6× 9s ▲3.9×	13s ▲2.9× 11.9s ▲3.0×	9.4s ▲9.8× 7s ▲12.7×	35s ▲11.2× 33.8s ▲11.4×	23.1s ▲27% 18.6s ▲1.6×	25.2s ▲26% 21.2s ▲1.5×	54.4s ▲22% 46.7s ▲29%	49.8s ▲3.6× 46.4s ▲3.7×	3m 44.5s ▼28% 3m 35.9s ▼24%	3m 57.4s ▼1.4× 3m 45.7s ▼1.3×	6m 50.6s ▼22% 6m 30.7s ▼18%	·	9m 45.5s ▼5.5× 3m 41.5s ▼2.1×	8m 25.6s ▼4.7× 3m 56.7s ▼2.3×	15m 24.9s ▼4.2× 6m 0.2s ▼1.7×	·
R3 blastn	Native*	11m 14.7s ▼18.6×[c] 11m 9.5s ▼19.1×	16m 40.6s ▼26.3×[c] 16m 29.1s ▼27.5×	22m 6.7s ▼14.4×[c] 21m 51.5s ▼14.8×	12m 25.4s ▼1.9×[c] 12m 19.3s ▼1.9×	11m 40.1s ▼22.2×[a,c] 9m 50.3s ▼19.5×	16m 52.8s ▼29.7×[a,c] 13m 48.2s ▼25.9×	23m 30.3s ▼20.3×[a,c] 18m 49.3s ▼17.1×	12m 50.3s ▼4.3×[a,c] 10m 46.6s ▼3.8×	11m 28s ▼3.9×[c] 11m 22.8s ▼3.9×	16m 55s ▼6.0×[c] 16m 43.4s ▼6.0×	22m 36.2s ▼4.0×[c] 22m 20.8s ▼4.0×	13m 1.6s ▲3.4×[c] 12m 55.8s ▲3.4×	12m 18.6s ▼6.9×[a,c] 10m 29.2s ▼5.9×	17m 25.7s ▼9.8×[a,c] 14m 22.8s ▼8.2×	24m 29.4s ▼6.7×[a,c] 19m 49.6s ▼5.5×	13m 43.9s ▲2.2×[a,c] 11m 39.5s ▲2.5×
R4 winnowmap	Native*	52.9s ▼1.5×[c,e] 50.6s ▼1.4×	59.4s ▼1.6×[c,e] 55.2s ▼1.5×	1m 49.1s ▼18%[c,e] 1m 41.3s ▼14%	1m 16.1s ▲5.1×[c,d] 1m 8.2s ▲5.7×	31.6s ▶ 0%[c,e] 29.4s ▶ 3%	44.2s ▼30%[c,e] 40.3s ▼26%	59s ▲15%[c,e] 51.8s ▲22%	1m 8s ▲2.7×[c,d] 1m 0.3s ▲2.8×	8m 6s ▼2.8×[c,e] 8m 3.7s ▼2.8×	7m 59.6s ▼2.8×[c,e] 7m 55.3s ▼2.8×	15m 50.3s ▼2.8×[c,e] 15m 42.3s ▼2.8×	7m 57s ▲5.6×[c,d] 7m 48.9s ▲5.7×	6m 51.2s ▼3.8×[c,e] 6m 49.1s ▼3.9×	6m 41.1s ▼3.7×[c,e] 6m 37.2s ▼3.8×	12m 42.6s ▼3.5×[c,e] 12m 35.1s ▼3.5×	7m 10.6s ▲4.1×[c,d] 7m 3.1s ▲4.2×
R5 lexicmap	Native*	4m 7.1s ▼6.8×[c] 4m 7s ▼7.0×	6m 29.9s ▼10.2×[c] 6m 29.7s ▼10.8×	9m 24.6s ▼6.1×[c] 9m 24.4s ▼6.4×	7m 16.7s ▼12%[c] 7m 16.4s ▼13%	4m 7.8s ▼7.9×[b,c] 4m 7.5s ▼8.2×	6m 29.2s ▼11.4×[b,c] 6m 28.8s ▼12.1×	11m 26.4s ▼9.9×[b,c] 11m 25.5s ▼10.4×	7m 15.7s ▼2.4×[b,c] 7m 13s ▼2.5×	7m 34.2s ▼2.6×[c] 7m 34.1s ▼2.6×	11m 38.9s ▼4.1×[c] 11m 38.7s ▼4.2×	17m 11.5s ▼3.1×[c] 17m 11.2s ▼3.1×	15m 28s ▲2.9×[c] 15m 27.2s ▲2.9×	7m 48.7s ▼4.4×[b,c] 7m 48.4s ▼4.4×	11m 34.4s ▼6.5×[b,c] 11m 34s ▼6.6×	17m 36.1s ▼4.8×[b,c] 17m 35.3s ▼4.9×	16m 6s ▲1.8×[b,c] 16m 3.7s ▲1.8×

Each cell: A, comparison, B, comparison. · = no measurement. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
Comparisons use minimap2. The minimap2 row compares with BioseqDB.
*A includes startup and output. B removes different work for each tool.
[a]BLAST scores N as -3, compared with -1 in minimap2.
[b]LexicMap stores reference N as A.
[c]The 4 Gbp reference is split into two input records.
[d]Winnowmap uses overlapping 250 Mb records and 499M index batches. Merging results can affect MAPQ.
[e]Winnowmap merges results from 1999M index batches.
[g]Paging pressure was observed on C7G.

Aligner	Configuration	Index 4 Gbp k=20, w=15								No index 4 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	2m 48.1s ▼18.3× 2m 41.7s ▼19.0×	2m 50.3s ▼15.0× 2m 43s ▼15.3×	4m 54.3s ▼43.2× 4m 39s ▼54.6×	25m 5.5s ▼38.5×[g] 25m 0.7s ▼39.5×	33.9s ▼1.9× 32.9s ▼2.1×	34.4s ▼2.6× 32.7s ▼2.9×	1m 8.2s ▼3.7× 1m 5.1s ▼4.6×	3m 9.8s ▼3.3×[g] 3m 3.7s ▼3.4×	3m 28.3s ▼1.3× 3m 22.8s ▼1.4×	3m 53.2s ▼1.4× 3m 43.5s ▼1.4×	7m 14.6s ▼1.4× 6m 57.2s ▼1.5×	51m 22s n/a[g] 51m 17.2s n/a	1m 10.3s ▲7.6× 1m 9s ▲2.4×	1m 8.6s ▲6.4× 1m 6.9s ▲2.5×	2m 21.5s ▲6.2× 2m 18.4s ▲2.2×	15m 56.8s n/a[g] 15m 51.9s n/a
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	9.2s ▲18.3× 8.5s ▲19.0×	11.4s ▲15.0× 10.7s ▲15.3×	6.8s ▲43.2× 5.1s ▲54.6×	39.1s ▲38.5× 38s ▲39.5×	18.2s ▲1.9× 16s ▲2.1×	13.2s ▲2.6× 11.4s ▲2.9×	18.6s ▲3.7× 14.3s ▲4.6×	57.2s ▲3.3× 53.9s ▲3.4×	2m 38.5s ▲24% 2m 29.8s ▲26%	2m 49s ▲28% 2m 37s ▲30%	5m 3.3s ▲1.4× 4m 42.7s ▲1.5×	·	8m 52.3s ▼7.6× 2m 45.5s ▼2.4×	7m 22.2s ▼6.4× 2m 48.2s ▼2.5×	14m 43.3s ▼6.2× 5m 9.5s ▼2.2×	·
R3 blastn	Native*	4m 35.4s ▼1.6×[c] 4m 30.3s ▼1.7×	6m 21.2s ▼2.2×[c] 6m 10.8s ▼2.3×	8m 55.3s ▼1.8×[c] 8m 41.3s ▼1.9×	5m 9.4s ▲4.9×[c] 5m 4.2s ▲4.9×	5m 39.4s ▼10.0×[a,c] 4m 13.6s ▼7.7×	7m 32.4s ▼13.1×[a,c] 5m 27.1s ▼10.0×	11m 29.3s ▼10.1×[a,c] 8m 4s ▼7.4×	6m 10.2s ▼2.0×[a,c] 4m 39s ▼1.5×	4m 58.3s ▼1.4×[c] 4m 53.3s ▼1.4×	6m 40.8s ▼1.7×[c] 6m 30.3s ▼1.7×	9m 34s ▼1.3×[c] 9m 19.8s ▼1.3×	5m 40.5s ▲9.1×[c] 5m 35.1s ▲9.2×	6m 15.6s ▼5.3×[a,c] 4m 49.2s ▼4.2×	8m 11.1s ▼7.2×[a,c] 6m 2.7s ▼5.4×	12m 29.6s ▼5.3×[a,c] 9m 3.4s ▼3.9×	7m 4.7s ▲2.3×[a,c] 5m 34.7s ▲2.8×
R4 winnowmap	Native*	55.6s ▲3.0×[c,e] 53.8s ▲3.0×	59.6s ▲2.9×[c,e] 56.2s ▲2.9×	1m 43s ▲2.9×[c,e] 1m 36.3s ▲2.9×	1m 0.7s ▲24.8×[c,d] 54.1s ▲27.7×	24.6s ▲28%[c,e] 22.8s ▲1.4×	23.7s ▲1.4×[c,e] 20.4s ▲1.6×	36.1s ▲1.9×[c,e] 29.6s ▲2.2×	41.4s ▲4.6×[c,d] 34.7s ▲5.3×	10m 41.8s ▼3.1×[c,e] 10m 40s ▼3.2×	11m 28.6s ▼3.0×[c,e] 11m 25.2s ▼3.1×	33m 6.8s ▼4.6×[c,e] 33m 0.1s ▼4.7×	12m 28.4s ▲4.1×[c,d] 12m 21.9s ▲4.1×	8m 28.7s ▼7.2×[c,e] 8m 26.9s ▼7.3×	8m 59.5s ▼7.9×[c,e] 8m 56.1s ▼8.0×	24m 58.6s ▼10.6×[c,e] 24m 52s ▼10.8×	10m 19s ▲1.5×[c,d] 10m 12.1s ▲1.6×
R5 lexicmap	Native*	56.9s ▲3.0×[c] 56.7s ▲2.9×	1m 21.3s ▲2.1×[c] 1m 21.1s ▲2.0×	2m 1.2s ▲2.4×[c] 2m 0.9s ▲2.3×	1m 16.5s ▲19.7×[c] 1m 16.1s ▲19.7×	56.4s ▼1.7×[b,c] 55.8s ▼1.7×	1m 20.1s ▼2.3×[b,c] 1m 19.6s ▼2.4×	2m 9.7s ▼1.9×[b,c] 2m 8.5s ▼2.0×	1m 14.6s ▲2.5×[b,c] 1m 12.8s ▲2.5×	4m 52.5s ▼1.4×[c] 4m 52.3s ▼1.4×	6m 45.3s ▼1.7×[c] 6m 45.1s ▼1.8×	10m 57.1s ▼1.5×[c] 10m 56.8s ▼1.6×	9m 46.1s ▲5.3×[c] 9m 45.6s ▲5.3×	4m 52.2s ▼4.2×[b,c] 4m 51.8s ▼4.2×	6m 46.8s ▼5.9×[b,c] 6m 46.3s ▼6.1×	10m 14.1s ▼4.3×[b,c] 10m 13.1s ▼4.4×	9m 49.2s ▲1.6×[b,c] 9m 47.7s ▲1.6×

Each cell: A, comparison, B, comparison. · = no measurement. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
Comparisons use minimap2. The minimap2 row compares with BioseqDB.
*A includes startup and output. B removes different work for each tool.
[a]BLAST scores N as -3, compared with -1 in minimap2.
[b]LexicMap stores reference N as A.
[c]The 4 Gbp reference is split into two input records.
[d]Winnowmap uses overlapping 250 Mb records and 499M index batches. Merging results can affect MAPQ.
[e]Winnowmap merges results from 1999M index batches.
[g]Paging pressure was observed on C7G.

Aligner	Configuration	Index 2 Gbp k=15, w=10								No index 2 Gbp k=15, w=10							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	11.2s ▼3.0× 10s ▼3.4×	10.7s ▼3.0× 8.6s ▼3.2×	22.5s ▼3.3× 18.8s ▼4.2×	1m 13.1s ▼8.3×[g] 1m 5.7s ▼8.3×	9.9s ▼21% 8.8s ▼1.5×	9.5s ▼1.4× 7.6s ▼1.6×	16.5s ▼20% 12.9s ▼1.5×	28.2s ▼1.4×[g] 26.2s ▼1.6×	53s ▲2.2× 51.9s ▲2.1×	56.6s ▲2.1× 54.6s ▲1.9×	1m 55.5s ▲1.9× 1m 51.8s ▲1.8×	6m 1.3s ▼2.9×[g] 5m 55.2s ▼3.0×	48.1s ▲7.3× 47s ▲2.4×	44.5s ▲6.5× 42.6s ▲2.7×	1m 30.2s ▲6.4× 1m 26.7s ▲2.3×	2m 46.4s ▲4.1×[g] 2m 42.7s ▼16%
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	3.7s ▲3.0× 3s ▲3.4×	3.5s ▲3.0× 2.7s ▲3.2×	6.7s ▲3.3× 4.5s ▲4.2×	8.8s ▲8.3×[g] 7.9s ▲8.3×	8.2s ▲17% 5.8s ▲1.5×	6.6s ▲1.4× 4.6s ▲1.6×	13.7s ▲17% 8.8s ▲1.5×	20.2s ▲28% 16.8s ▲1.6×	1m 56.4s ▼2.2× 1m 48.1s ▼2.1×	1m 56.2s ▼2.1× 1m 46.2s ▼1.9×	3m 42.9s ▼1.9× 3m 25s ▼1.8×	2m 5.6s ▲2.9×[g] 1m 59.7s ▲3.0×	5m 52.1s ▼7.3× 1m 53.5s ▼2.4×	4m 49.2s ▼6.5× 1m 56.8s ▼2.7×	9m 39.5s ▼6.4× 3m 18.1s ▼2.3×	11m 24.8s ▼4.1× 2m 20.4s ▲14%
R3 blastn	Native*	5m 36.2s ▼30.1×[f] 5m 30.7s ▼33.0×	8m 14.3s ▼46.4×[f] 8m 2.4s ▼56.0×	10m 51.3s ▼28.9× 10m 35.3s ▼33.8×	5m 54.4s ▼4.9×[f] 5m 48.6s ▼5.3×	6m 37.9s ▼40.2×[a] 4m 57.9s ▼33.9×	9m 32.3s ▼60.3×[a] 6m 18s ▼49.6×	13m 27.4s ▼49.0×[a] 8m 52.1s ▼41.2×	6m 53.6s ▼14.7×[a] 4m 51.8s ▼11.1×	5m 39.9s ▼6.4× 5m 34.5s ▼6.4×	8m 22.6s ▼8.9× 8m 10.6s ▼9.0×	11m 6.9s ▼5.8× 10m 50.9s ▼5.8×	6m 6.4s ▶ 1% 6m 0.6s ▶ 2%	6m 51.8s ▼8.6×[a] 5m 12.2s ▼6.6×	9m 48.4s ▼13.2×[a] 6m 35.3s ▼9.3×	13m 55.3s ▼9.3×[a] 9m 23.3s ▼6.5×	7m 14.5s ▼2.6×[a] 5m 12.1s ▼1.9×
R4 winnowmap	Native*	12.9s ▼15% 11.1s ▼11%	13.5s ▼27% 10.3s ▼20%	23.2s ▶ 3% 17.1s ▲9%	44.3s ▲1.7×[d] 37.8s ▲1.7×	12.3s ▼24% 10.6s ▼21%	13s ▼1.4× 10s ▼1.3×	23.8s ▼1.4× 17.6s ▼1.4×	37.3s ▼1.3×[d] 31.4s ▼20%	3m 33.2s ▼4.0× 3m 31.5s ▼4.1×	3m 22.6s ▼3.6× 3m 19.3s ▼3.7×	6m 21.3s ▼3.3× 6m 15s ▼3.4×	3m 56.6s ▲1.5×[d] 3m 50.3s ▲1.5×	3m 7.4s ▼3.9× 3m 5.8s ▼4.0×	3m 2.2s ▼4.1× 2m 59.3s ▼4.2×	6m 35.9s ▼4.4× 6m 29.8s ▼4.5×	3m 33.9s ▼29%[d] 3m 28.1s ▼28%
R5 lexicmap	Native*	2m 14.1s ▼12.0× 2m 14s ▼13.4×	3m 26.7s ▼19.4× 3m 26.6s ▼24.0×	4m 40.1s ▼12.4× 4m 39.9s ▼14.9×	3m 24.3s ▼2.8× 3m 24.2s ▼3.1×	2m 12.9s ▼13.4×[b] 2m 12.5s ▼15.1×	3m 24.1s ▼21.5×[b] 3m 23.6s ▼26.7×	4m 56.4s ▼18.0×[b] 4m 55.6s ▼22.9×	3m 22.8s ▼7.2×[b] 3m 22.5s ▼7.7×	3m 18.9s ▼3.8× 3m 18.7s ▼3.8×	5m 10.5s ▼5.5× 5m 10.3s ▼5.7×	8m 19.4s ▼4.3× 8m 19.2s ▼4.5×	5m 43.1s ▲5% 5m 43s ▶ 3%	3m 16.5s ▼4.1×[b] 3m 16.1s ▼4.2×	5m 5s ▼6.9×[b] 5m 4.5s ▼7.2×	7m 40.8s ▼5.1×[b] 7m 39.9s ▼5.3×	5m 52.3s ▼2.1×[b] 5m 51.9s ▼2.2×

Each cell: A, comparison, B, comparison. · = no measurement. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
Comparisons use minimap2. The minimap2 row compares with BioseqDB.
*A includes startup and output. B removes different work for each tool.
[a]BLAST scores N as -3, compared with -1 in minimap2.
[b]LexicMap stores reference N as A.
[d]Winnowmap uses overlapping 250 Mb records and 499M index batches. Merging results can affect MAPQ.
[f]Swap settings were not recorded.
[g]Paging pressure was observed on C7G.

Aligner	Configuration	Index 2 Gbp k=20, w=15								No index 2 Gbp k=20, w=15							
		DNA4				DNA5				DNA4				DNA5			
		M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G	M7A	M7I	M4	C7G
		C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
R1 minimap2	Baseline	19.6s ▼6.6× 18.6s ▼8.1×	18.2s ▼6.9× 16.5s ▼8.6×	42.5s ▼7.9× 39.4s ▼11.7×	6m 29.2s ▼86.7× 5m 56.9s ▼97.9×	9.3s ▼24% 8.4s ▼1.6×	8.7s ▼1.5× 7s ▼1.7×	15.8s ▼26% 12.7s ▼1.6×	18.5s ▼1.4×[g] 17s ▼1.6×	42.4s ▲2.0× 41.4s ▲1.9×	42.9s ▲2.0× 41.1s ▲1.8×	1m 26.9s ▲1.9× 1m 23.7s ▲1.7×	4m 51.6s ▼2.9×[g] 4m 46.6s ▼3.0×	35s ▲8.6× 34s ▲2.5×	35.9s ▲6.7× 34.2s ▲2.5×	1m 14.1s ▲6.9× 1m 11.1s ▲2.2×	1m 0.1s ▲10.4×[g] 58.6s ▲1.9×
R2 BioseqDB	MPF / Bounded SIMD-DP / Block	3s ▲6.6× 2.3s ▲8.1×	2.6s ▲6.9× 1.9s ▲8.6×	5.4s ▲7.9× 3.4s ▲11.7×	4.5s ▲86.7×[g] 3.6s ▲97.9×	7.5s ▲19% 5.2s ▲1.6×	5.8s ▲1.5× 4s ▲1.7×	12.6s ▲21% 7.9s ▲1.6×	12.9s ▲1.4× 10.6s ▲1.6×	1m 24.8s ▼2.0× 1m 16.9s ▼1.9×	1m 25.4s ▼2.0× 1m 15.7s ▼1.8×	2m 44s ▼1.9× 2m 26.2s ▼1.7×	1m 42s ▲2.9× 1m 35.7s ▲3.0×	5m 1.5s ▼8.6× 1m 24.2s ▼2.5×	4m 0.6s ▼6.7× 1m 24.3s ▼2.5×	8m 31.7s ▼6.9× 2m 34.1s ▼2.2×	10m 27.6s ▼10.4× 1m 51.4s ▼1.9×
R3 blastn	Native[*]	2m 19.5s ▼7.1× 2m 14.5s ▼7.2×	3m 12.3s ▼10.6× 3m 1.8s ▼11.0×	4m 30.1s ▼6.4× 4m 16s ▼6.5×	2m 27.9s ▲2.6× 2m 22.7s ▲2.5×	3m 27.6s ▼22.3×[a] 2m 22.7s ▼17.0×	4m 37.9s ▼31.8×[a] 2m 47s ▼23.8×	7m 14.2s ▼27.5×[a] 4m 26.5s ▼20.9×	3m 30.4s ▼11.4×[a] 2m 14.4s ▼7.9×	2m 28.5s ▼3.5× 2m 23.4s ▼3.5×	3m 20.9s ▼4.7× 3m 10.4s ▼4.6×	4m 46.7s ▼3.3× 4m 32.6s ▼3.3×	2m 38.7s ▲1.8× 2m 33.5s ▲1.9×	3m 43.6s ▼6.4×[a] 2m 37.7s ▼4.6×	4m 51.4s ▼8.1×[a] 2m 59.6s ▼5.3×	7m 39.4s ▼6.2×[a] 4m 49.6s ▼4.1×	3m 50.8s ▼3.8×[a] 2m 36.2s ▼2.7×
R4 winnowmap	Native[*]	28.4s ▼1.4× 26.9s ▼1.4×	27.6s ▼1.5× 24.8s ▼1.5×	1m 2.8s ▼1.5× 55s ▼1.4×	8m 35.8s ▼1.3× 8m 4.4s ▼1.4×	10.8s ▼16% 9.4s ▼12%	10.9s ▼24% 8s ▼14%	20.1s ▼27% 14.5s ▼14%	21s ▼14% 17.1s ▶ 0%	4m 16.8s ▼6.1× 4m 15.2s ▼6.2×	5m 0.6s ▼7.0× 4m 57.9s ▼7.2×	10m 23.6s ▼7.2× 10m 18s ▼7.4×	16m 47.8s ▼3.5× 16m 11s ▼3.4×	3m 21.4s ▼5.8× 3m 19.9s ▼5.9×	4m 0.5s ▼6.7× 3m 57.6s ▼7.0×	8m 56.2s ▼7.2× 8m 50.6s ▼7.5×	4m 43.2s ▼4.7× 4m 39.3s ▼4.8×
R5 lexicmap	Native[*]	38.8s ▼2.0× 38.6s ▼2.1×	54.3s ▼3.0× 54s ▼3.3×	1m 29.8s ▼2.1× 1m 29.4s ▼2.3×	49.9s ▲7.8× 49.7s ▲7.2×	37.8s ▼4.1×[b] 37.3s ▼4.4×	53.5s ▼6.1×[b] 52.8s ▼7.5×	1m 18.6s ▼5.0×[b] 1m 17.5s ▼6.1×	49.7s ▼2.7×[b] 49.3s ▼2.9×	1m 41.8s ▼2.4× 1m 41.7s ▼2.5×	2m 35.6s ▼3.6× 2m 35.4s ▼3.8×	4m 6.7s ▼2.8× 4m 6.3s ▼2.9×	3m 25.5s ▲30% 3m 25.3s ▲28%	1m 43.4s ▼3.0×[b] 1m 42.6s ▼3.0×	2m 37.7s ▼4.4×[b] 2m 37.1s ▼4.6×	3m 45.3s ▼3.0×[b] 3m 43.9s ▼3.2×	3m 16.3s ▼3.3×[b] 3m 15.8s ▼3.3×

Each cell: A, comparison, B, comparison. · = no measurement. n/a = no comparison.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
Light gray = minimap2. Pale yellow = preferred MPF / Bounded / SIMD-DP / Block configuration.
Comparisons use minimap2. The minimap2 row compares with BioseqDB.
[*]A includes startup and output. B removes different work for each tool.
[a]BLAST scores N as -3, compared with -1 in minimap2.
[b]LexicMap stores reference N as A.
[g]Paging pressure was observed on C7G.

10.1.5. Vulkan on a Vast.ai GPU instance

Table 10.29: Vast.ai machine specs

Property	Value
CPU	AMD EPYC 7282 16-Core Processor
CPU quota	7.68 cores
Memory limit	120.8 GiB
GPU	NVIDIA GeForce RTX 5090
GPU memory	32,607 MiB
OS	Ubuntu 24.04
NVIDIA driver	595.84
Vulkan API	1.4.329

Engine	Execution	Index			
		4 Gbp		2 Gbp	
		k=15 w=10 DNA4	k=20 w=15 DNA4	k=15 w=10 DNA4	k=20 w=15 DNA4
		C1	C2	C3	C4
R1 minimap2	Baseline	19.5s ▲21.1× 18s ▲19.3×	42.5s ▲7.6× 41.5s ▲6.3×	14.1s ▲7.7× 13s ▲4.5×	1m 34.6s ▶ 4% 1m 33.6s ▼2.1×
R2 BioseqDB	MPS / Vulkan Vulkan-DP / KSW2	×[2]	×[2]	1m 54.9s ▼8.2× 1m 2.3s ▼4.8×	1m 38.5s ▶ 4% 44.1s ▲2.1×
R3 BioseqDB	MPS / Vulkan Vulkan-DP / Block	×[2]	×[1]	1m 48.7s ▼7.7× 58.9s ▼4.5×	1m 38.5s ▶ 4% 47s ▲2.0×
R4 BioseqDB	MPF / Vulkan Vulkan-DP / KSW2	6m 50.8s ▼21.1× 5m 48.5s ▼19.3×	5m 24.2s ▼7.6× 4m 22.1s ▼6.3×	3m 38.5s ▼15.5× 2m 49.8s ▼13.0×	2m 58.3s ▼1.9× 2m 9.4s ▼1.4×
R5 BioseqDB	MPF / Vulkan Vulkan-DP / Block	7m 7.5s ▼21.9× 6m 8.1s ▼20.4×	5m 48s ▼8.2× 4m 48s ▼6.9×	4m 37s ▼19.7× 3m 47s ▼17.4×	3m 8.4s ▼2.0× 2m 21.2s ▼1.5×

Each cell: A, comparison, B, comparison. Light gray = minimap2.
Blue = one lowest A and one lowest B measurement for each column for each configuration.
MPF/MPS = minimizers per fragment/sequence. Vulkan rows compare with minimap2 on the same machine.
· = no result. n/a = no comparison. ◯ = setup rejected. × = execution failed.
[1]The GPU index exceeds the available memory budget.
[2]The MPS control buffer exceeds the Vulkan buffer limit.

Chapter 11

Conclusions

BioSeqDB demonstrates that sequence search can be represented as a PostgreSQL execution operation rather than as a scalar call to an external mapper. The system stores packed biological sequences, constructs persistent minimizer indexes, and runs a minimap2-derived pipeline for seeding, chaining, filtering, and final extension. Planner support and custom scans preserve the batch structure required by SIMD and GPU implementations while presenting typed SQL results that can participate in filters, projections, and joins.

The first research question concerned a shared representation for the algorithmic stages. Alphabet-generic compact ranges cover resident values, while lazy sequence views represent fragmented, sliced, and composed sources. Algorithms read the required region without first converting the complete value to a byte array. The second question concerned heterogeneous execution. Minimizer extraction and chaining provide regular work for the implemented SIMD and GPU routes. Dispatch follows explicit algorithm settings, input compatibility, index form, and resource availability. Final alignment uses banded or unbanded KSW2, adaptive Block Aligner, and GPU routes that may return specialized scripts before device dynamic programming. Optional prefilters may reject unsuitable candidates but do not replace alignment.

The third question concerned database execution. A custom scan owns the search expression, batches input rows, and substitutes match fields into planned expressions. This design also permits multiple alignments for one input row. The fourth question concerned the surrounding data system. Physical sequence storage, persistent indexes, bounded ingestion, external-provider relation definitions, cache coordination, and source-driven SQL generation give the kernels explicit storage and lifecycle contracts.

The remaining limits are operational and algorithmic. Some remote retrieval paths still require record-sized or archive-sized memory. Provider quotas are not coordinated across all PostgreSQL backends. Hardware-specific routes have batch-size thresholds and require broader evaluation on additional devices. GPU chaining may examine a bounded predecessor set, and seed and alignment backends retain route-specific contracts. The present mapper is centered on linear references rather than pangenome graphs. A relational database can nevertheless plan and execute batched sequence analysis while retaining typed data, bounded resources, persistent derived state, and explicit algorithm selection.

Authorship, development and AI assistance

I developed BioSeqDB over approximately three years with most of that time was dedicated to learning how alignment software works in general. Early work included Python aligner prototypes written approximately two years ago. I designed the system and implemented the BioSeqDB-specific algorithms, representations and optimizations presented as my contributions in the preceding chapters. Moving from those prototypes to a PostgreSQL extension also required substantial integration, testing and performance work.

The entire project was authored only by me:

- Created ports of existing algorithms based on their high-level design into portable GPU kernels and SIMD kernels including my own custom optimisations (all the algorithms are custom implementations authored by me)
- Designed and implemented C++ interfaces, Postgres interfaces
- Ported Citus/Hydra columnar storage to new PostgreSQL versions and adapted it to my custom needs

With contributions from AI tools in the only non-core areas that helped me focus on the important problems:

- Development tools including source generator that helped me to generate SQL boilerplate
- Benchmarking utilities to measure performance
- Automatic refactor of my files that included only: moving functions to different files, renaming functions, variables and files and making sure that code follows clang tidy formatter

References

Abadi, Daniel, Samuel Madden, and Miguel Ferreira. "Integrating compression and execution in column-oriented database systems." In: Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data. SIGMOD '06. Association for Computing Machinery, 2006, pp. 671–682. doi: 10.1145/1142473.1142548. url: https://doi.org/10.1145/1142473.1142548.

cp-algorithms contributors. Range Minimum Query. url: https://cp-algorithms.com/sequences/rmq.html (visited on 09/14/2026).

— Segment Tree. url: https://cp-algorithms.com/data_structures/segment_tree.html (visited on 09/14/2026).

Alser, Mohammed et al. "SneakySnake: a fast and accurate universal genome pre-alignment filter for CPUs, GPUs and FPGAs." In: Bioinformatics 36.22-23 (2020), pp. 5282–5290. doi: 10.1093/bioinformatics/btaa1015. arXiv: 1910.09020 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/btaa1015.

Amazon Web Services. Amazon EBS-optimized instance types. url: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-optimized.html (visited on 09/25/2026).

Chernishev, George A. et al. "A Comprehensive Study of Late Materialization Strategies for a Disk-Based Column-Store." In: Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data. Vol. 3130. CEUR Workshop Proceedings. 2022, pp. 21–30. url: https://ceur-ws.org/Vol-3130/paper3.pdf.

Citus Data contributors. Citus Columnar storage implementation. Version 14.0.0. AGPL-3.0. Feb. 17, 2026. url: https://github.com/citusdata/citus (visited on 07/28/2026).

ClickHouse contributors. ClickBench: analytical database benchmark. Historical PostgreSQL, Citus, and Hydra submissions. Sept. 20, 2023. url: https://github.com/ClickHouse/ClickBench/tree/2e8146c397f81e3e98937205d09e331313b991e3 (visited on 09/09/2026).

Conway, Neil. Inside the PostgreSQL Query Optimizer. Slide deck. Undated online source. Fujitsu Australia Software Technology. url: https://www.neilconway.org/talks/optimizer/optimizer.pdf (visited on 09/17/2026).

cppreference.com contributors. C++ named requirements: StandardLayoutType. Undated online source. url: https://en.cppreference.com/cpp/named_req/StandardLayoutType (visited on 09/15/2026).

Crick, Francis. "Central Dogma of Molecular Biology." In: Nature 227.5258 (Aug. 8, 1970), pp. 561–563. doi: 10.1038/227561a0. url: https://doi.org/10.1038/227561a0.

Cubukcu, Umur et al. “Citus: Distributed PostgreSQL for Data-Intensive Applications.” In: Proceedings of the 2021 International Conference on Management of Data. SIGMOD ’21. Association for Computing Machinery, 2021, pp. 2490–2502. doi: 10.1145/3448016.3457551. url: https://doi.org/10.1145/3448016.3457551.

Davis, Jeff. Citus 10 brings columnar compression to Postgres. Mar. 6, 2021. url: https://www.citusdata.com/blog/2021/03/06/citus-10-columnar-compression-for-postgres/ (visited on 09/08/2026).

Feintzeig, Jake. SIMD + Parallelism: Ray Tracer Optimization Part 3. Undated online source. url: https://www.jakef.science/posts/simd-parallelism/ (visited on 08/19/2026).

Gotoh, Osamu. “An improved algorithm for matching biological sequences.” In: Journal of Molecular Biology 162.3 (1982), pp. 705–708. doi: 10.1016/0022-2836(82)90398-9. url: https://doi.org/10.1016/0022-2836(82)90398-9.

Groot Koerkamp, Ragnar and Igor Martayan. packed-seq: Bitpacked sequence trait and implementation. Version 5.0.0. MIT license. Rust crate developed for SimdMinimizers (SEA 2025). url: https://github.com/rust-seq/packed-seq (visited on 09/15/2026).

— packed_seq crate documentation. Version 5.0.0. July 7, 2026. url: https://docs.rs/packed-seq/latest/packed_seq/ (visited on 09/15/2026).

— “SimdMinimizers: Computing Random Minimizers, fast.” In: 23rd International Symposium on Experimental Algorithms. Vol. 338. Leibniz International Proceedings in Informatics. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 20:1–20:19. doi: 10.4230/LIPIcs.SEA.2025.20. url: https://doi.org/10.4230/LIPIcs.SEA.2025.20.

Han, Yeejoo et al. “G^{3}SA: A GPU-Accelerated Gold Standard Genomics Library for End-to-End Sequence Alignment.” In: Proceedings of the 39th ACM International Conference on Supercomputing. ICS ’25. Salt Lake City, Utah, USA: Association for Computing Machinery, 2025, pp. 173–188. doi: 10.1145/3721145.3729516. url: https://doi.org/10.1145/3721145.3729516.

Hydra contributors. Hydra Columnar: PostgreSQL columnar storage extension. Source snapshot d1f7657ee822. Feb. 10, 2025. url: https://github.com/hydradatabase/columnar (visited on 09/09/2026).

Institute, National Human Genome Research. The Cost of Sequencing a Human Genome. Version 17. Undated online source. url: https://www.genome.gov/about-genomics/fact-sheets/Sequencing-Human-Genome-cost (visited on 07/28/2026).

Juliusz K Mateusz C, Szymon D and Damian W. “Master thesis: An integrated and practical database solution for storage and efficient processing of large genomic datasets.” In: MIM UW 1.1 (2022). url: https://mimuw.edu.pl/en/.

Kallenborn, Felix et al. “GPU-accelerated homology search with MMseqs2.” In: Nature Methods 22.10 (Oct. 2025), pp. 2024–2027. issn: 1548-7105. doi: 10.1038/s41592-025-02819-8. url: https://doi.org/10.1038/s41592-025-02819-8.

— “GPU-accelerated homology search with MMseqs2.” In: Nature Methods 22.10 (Sept. 18, 2025), pp. 2024–2027. doi: 10.1038/s41592-025-02819-8. url: https://doi.org/10.1038/s41592-025-02819-8.

Knuth, Donald E. The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd ed. Reading, Massachusetts: Addison-Wesley, 1998. isbn: 0-201-89685-0. url: https://cs.stanford.edu/~knuth/taocp.html.

Kreis, Lara et al. GPU-RMQ: Accelerating Range Minimum Queries on Modern GPUs. Preprint. Apr. 2, 2026. doi: 10.48550/arXiv.2604.01811. arXiv: 2604.01811.

Kryukov, Kirill et al. “Nucleotide Archival Format (NAF) enables efficient lossless reference-free compression of DNA sequences.” In: Bioinformatics 35.19 (Oct. 2019),

pp. 3826–3828. issn: 1367-4803. doi: 10.1093/bioinformatics/btz144. eprint: https://academic.oup.com/bioinformatics/article-pdf/35/19/3826/48977341/bioinformatics_35_19_3826.pdf. url: https://doi.org/10.1093/bioinformatics/btz144.

Lane, Tom. Re: what exactly is a PlaceHolderVar? Reply to Robert Haas. Message-ID: 23076.1277179362@sss.pgh.pa.us. PostgreSQL Hackers mailing list. June 22, 2010. url: https://www.postgresql.org/message-id/23076.1277179362@sss.pgh.pa.us (visited on 09/17/2026).

Lanius, Christian. parse_mmi.py: Python parser and lookup functions for minimap2 indexes. Sept. 27, 2021. url: https://gist.github.com/christian-lanius/8b8d6a38e35b93783ff7d6236211ff5e (visited on 09/08/2026).

Li, Heng. minimap2. Version 2.30. url: https://github.com/lh3/minimap2/tree/79c9cc186b95f50bd899f69b48eba995ced810c6 (visited on 07/28/2026).

— Minimap2 manual. url: https://lh3.github.io/minimap2/minimap2.html (visited on 09/08/2026).

— minimap2: mapping-quality implementation in *hit.c*. Function `mm_set_mapq2`; commit 3c28777e7e2dcc90f825de1b9f17a89cca7d4452. url: https://github.com/lh3/minimap2/blob/3c28777e7e2dcc90f825de1b9f17a89cca7d4452/hit.c#L432-L485 (visited on 09/27/2026).

— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (2018), pp. 3094–3100. doi: 10.1093/bioinformatics/bty191. arXiv: 1708.01492 [q-bio.GN]. url: https://doi.org/10.1093/bioinformatics/bty191.

— "Minimap2: pairwise alignment for nucleotide sequences." In: Bioinformatics 34.18 (Sept. 2018), pp. 3094–3100. issn: 1367-4803. doi: 10.1093/bioinformatics/bty191. eprint: https://academic.oup.com/bioinformatics/article-pdf/34/18/3094/48919122/bioinformatics_34_18_3094.pdf. url: https://doi.org/10.1093/bioinformatics/bty191.

Li, Heng, Xiaowen Feng, and Chong Chu. "The design and construction of reference pangenome graphs with minigraph." In: Genome Biology 21 (Oct. 2020), p. 265. doi: 10.1186/s13059-020-02168-z.

Liu, Daniel. Block Aligner: official project README. url: https://github.com/Daniel-Liu-c0deb0t/block-aligner#readme (visited on 09/27/2026).

Liu, Daniel and Martin Steinegger. "Block Aligner: an adaptive SIMD-accelerated aligner for sequences and position-specific scoring matrices." In: Bioinformatics 39.8, btad487 (2023). doi: 10.1093/bioinformatics/btad487. url: https://doi.org/10.1093/bioinformatics/btad487.

minimap2 contributors. How to interpret the mmi index file? Issue 820. Discussion updated in 2025. Sept. 26, 2021. url: https://github.com/lh3/minimap2/issues/820 (visited on 09/08/2026).

Mirdita, M. et al. "Fast and sensitive taxonomic assignment to metagenomic contigs." In: Bioinformatics 37.18 (2021), pp. 3029–3031. doi: 10.1093/bioinformatics/btab184. url: https://doi.org/10.1093/bioinformatics/btab184.

Mirdita, Milot, Martin Steinegger, and Johannes Söding. "MMseqs2 desktop and local web server app for fast, interactive sequence searches." In: Bioinformatics 35.16 (2019), pp. 2856–2858. doi: 10.1093/bioinformatics/bty1057. url: https://doi.org/10.1093/bioinformatics/bty1057.

Needleman, Saul B. and Christian D. Wunsch. “A general method applicable to the search for similarities in the amino acid sequence of two proteins.” In: Journal of Molecular Biology 48.3 (1970), pp. 443–453. doi: 10.1016/0022-2836(70)90057-4. url: https://doi.org/10.1016/0022-2836(70)90057-4.

Neon. The pg_search extension. Undated online source. Documents ParadeDB pg_search on Neon. url: https://neon.com/docs/extensions/pg_search (visited on 09/17/2026).

Pigsty. pg_search: full text search for PostgreSQL using BM25. Version 0.25.2. Undated online source. Last modified 2026-08-21. Aug. 21, 2026. url: https://pigsty.io/ext/e/pg_search/ (visited on 09/17/2026).

PostgreSQL Global Development Group. PostgreSQL 12 Release Notes. Version 12. url: https://www.postgresql.org/docs/12/release-12.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Appendix K. PostgreSQL Limits. Version 17. url: https://www.postgresql.org/docs/17/limits.html (visited on 09/21/2026).

— PostgreSQL 17 Documentation: Architectural Fundamentals. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/tutorial-arch.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Background Worker Processes. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/bgworker.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Behavior in Threaded Programs. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/libpq-threading.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Database Page Layout. Version 17. url: https://www.postgresql.org/docs/17/storage-page-layout.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: Foreign Data. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/ddl-foreign-data.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Function Optimization Information. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/xfunc-optimization.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Generic WAL Records. Version 17. url: https://www.postgresql.org/docs/17/generic-wal.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: How Parallel Query Works. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/how-parallel-query-works.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Introduction to Concurrency Control. Version 17. url: https://www.postgresql.org/docs/17/mvcc-intro.html (visited on 09/08/2026).

— PostgreSQL 17 Documentation: pg_settings. Version 17. url: https://www.postgresql.org/docs/17/view-pg-settings.html (visited on 09/09/2026).

— PostgreSQL 17 Documentation: *postgres_fdw*. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/postgres-fdw.html (visited on 09/16/2026).

— PostgreSQL 17 Documentation: Set Returning Functions. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/functions-srf.html (visited on 07/28/2026).

— PostgreSQL 17 Documentation: Table Access Method Interface Definition. Version 17. url: https://www.postgresql.org/docs/17/tableam.html (visited on 09/08/2026).
— PostgreSQL 17 Documentation: TOAST. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/storage-toast.html (visited on 07/28/2026).
— PostgreSQL 17 Documentation: Writing a Custom Scan Provider. Version 17. Undated online source. url: https://www.postgresql.org/docs/17/custom-scan.html (visited on 07/28/2026).
— PostgreSQL 17 source: tuple table slots. Version 17. Header `src/include/executor/tuptable.h` on tag `REL_17_10`. url: https://github.com/postgres/postgres/blob/REL_17_10/src/include/executor/tuptable.h (visited on 09/18/2026).
— PostgreSQL 17: Configuration Inspection Functions. url: https://github.com/postgres/postgres/blob/REL_17_STABLE/src/backend/utils/misc/guc_funcs.c (visited on 09/09/2026).
— PostgreSQL 9.1 Release Notes. Version 9.1. url: https://www.postgresql.org/docs/9.1/release-9-1.html (visited on 09/16/2026).
— PostgreSQL 9.3 Release Notes. Version 9.3. url: https://www.postgresql.org/docs/9.3/release-9-3.html (visited on 09/16/2026).
— PostgreSQL 9.5 Release Notes. Version 9.5. url: https://www.postgresql.org/docs/9.5/release-9-5.html (visited on 09/16/2026).
— PostgreSQL Documentation: Planner/Optimizer. Undated online source. Section on generating possible plans, including the Genetic Query Optimizer. url: https://www.postgresql.org/docs/current/planner-optimizer.html#PLANNER-OPTIMIZER-GENERATING-POSSIBLE-PLANS (visited on 09/17/2026).

Python Software Foundation. Python 3 Documentation: Common Sequence Operations. Version 3.14.7. Slice clamping rules are given in note 4 of the operation table. Sept. 15, 2026. url: https://docs.python.org/3/builtins/stdtypes.html#common-sequence-operations (visited on 09/15/2026).

Reinert, Knut et al. "The SeqAn C++ template library for efficient sequence analysis: A resource for programmers." In: Journal of Biotechnology 261 (Nov. 10, 2017), pp. 157–168. doi: 10.1016/j.jbiotec.2017.07.017. url: https://www.sciencedirect.com/science/article/pii/S0168165617315420.

Roberts, Michael et al. "Reducing storage requirements for biological sequence comparison." In: Bioinformatics 20.18 (2004), pp. 3363–3369. doi: 10.1093/bioinformatics/bth408. url: https://doi.org/10.1093/bioinformatics/bth408.

Sadasivan H Maric M, Dawson E et all. mm2-gpu: Accelerating Minimap2 for Accurate Long Read Alignment on GPUs. Undated online source. doi: https://doi.org/10.26502/jbb.2642-91280067. url: https://github.com/Minimap2onGPU/mm2-gb (visited on 08/19/2026).

SAFARI Research Group. SneakySnake: official project README. url: https://github.com/CMU-SAFARI/SneakySnake#readme (visited on 09/27/2026).

SAM/BAM Format Specification Working Group. Sequence Alignment/Map Format Specification. Version 1.6. Undated online source. url: https://samtools.github.io/hts-specs/SAMv1.pdf (visited on 07/28/2026).

Saurabh Kalikar Chirag Jain, Md Vasimuddin et all. mm2-fast: Accelerating minimap2 for long-read sequencing applications on modern CPUs. Undated online source. doi: https://doi.org/10.1038/s43588-022-00201-8. url: https://github.com/bwa-mem2/mm2-fast (visited on 08/19/2026).

SeqAn Project. Alphabets in SeqAn. Version 3.4.0. url: https://docs.seqan.de/seqan3/3.4.0/tutorial_alphabets.html (visited on 09/09/2026).
— SeqAn Manual. url: https://seqan.readthedocs.io/en/main/ (visited on 09/09/2026).
— SeqAn Tutorial: Sequences In-Depth. Version 1.4.2. String specializations, including packed and external strings. url: https://seqan.readthedocs.io/en/seqan-v1.4.2/Tutorial/SequencesInDepth.html (visited on 09/15/2026).
Shen, Wei. LexicMap FAQs. Undated online source. url: https://bioinf.shenwei.me/LexicMap/faqs/ (visited on 08/19/2026).
Shen, Wei, Botond Sipos, and Liuyang Zhao. SeqKit usage documentation: the subseq subcommand. Undated online source. Documentation for the tool described in SeqKit2, iMeta 3(3):e191, 2024. url: https://bioinf.shenwei.me/seqkit/usage/#subseq (visited on 09/15/2026).
Shur, Arseny. On Minimizers of Minimum Density. June 5, 2025. doi: 10.48550/arXiv.2506.05277. arXiv: 2506.05277 [cs.DS]. url: https://arxiv.org/abs/2506.05277 (visited on 08/31/2026).
Smith, Temple F. and Michael S. Waterman. "Identification of common molecular subsequences." In: Journal of Molecular Biology 147.1 (1981), pp. 195–197. doi: 10.1016/0022-2836(81)90087-5. url: https://doi.org/10.1016/0022-2836(81)90087-5.
Steinegger, Martin and Johannes Söding. "Clustering huge protein sequence sets in linear time." In: Nature Communications 9, 2542 (June 29, 2018). doi: 10.1038/s41467-018-04964-5. url: https://doi.org/10.1038/s41467-018-04964-5.
— "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets." In: Nature Biotechnology 35.11 (Oct. 16, 2017), pp. 1026–1028. doi: 10.1038/nbt.3988. url: https://doi.org/10.1038/nbt.3988.
— "MMseqs2: sensitive protein sequence searching for the analysis of massive data sets." Version 5. In: bioRxiv, 079681 (June 7, 2017). Preprint. doi: 10.1101/079681. url: https://www.biorxiv.org/content/early/2017/06/07/079681.
Stopa, Natasha. MMseqs2 Github issue 323. Undated online source. url: https://github.com/lh3/minimap2/issues/323 (visited on 08/19/2026).
Suzuki, Hajime and Masahiro Kasahara. "Introducing difference recurrence relations for faster semi-global alignment of long sequences." In: BMC Bioinformatics 19, 45 (2018). doi: 10.1186/s12859-018-2014-8. url: https://doi.org/10.1186/s12859-018-2014-8.
Sweden, NBIS - National Bioinformatics Infrastructure. The Cost of Sequencing a Human Genome. Version 1. Undated online source. url: https://github.com/NBISweden/GAAS/blob/master/annotation/knowledge/cigar.md (visited on 07/28/2026).
Uhl, Tim Niklas, Matthias Schimek, and Daniel Brommer. Concepts in Practice: C++ MPI Bindings for the HPC Ecosystem. From a Standardizable Core to a Composable Interface. 2026. arXiv: 2606.09102 [cs.DC]. url: https://arxiv.org/abs/2606.09102.
Youngblut, Nick. MMseqs2 Github issue 338. Undated online source. url: https://github.com/soedinglab/MMseqs2/issues/338 (visited on 08/19/2026).
Zhan, Xiaowei. MMseqs2 Github issue 517. Undated online source. url: https://github.com/soedinglab/MMseqs2/issues/517 (visited on 08/19/2026).

Appendix 1

A runnable SQL walkthrough

This appendix shows SQL showing fucntionalities of BioseqDB (I executed it on built Macos package binary with the default settings 2).

Alpha-version limitations

Please note that the current software is an alpha version and still has numerous issues. The memory checks can be too restrictive. Currently when there's less than 2GiB available BioseqDB reports the following error: `transient query execution stats disagree with its generated backend`, which does not explain the actual issue. Because memory cannot be allocated, the query reports that index cannot be built, so in stats the index access strategy sets the flag as unavilable. I check the "transient query execution stats" to confirm that what user chose to do is actually done (detects GPU fallbacks, scalar fallbacks and other hidden behaviour). In the future this error needs to detect that the problem was memory allocation, so UX is much better. Additionally please `bioseq.index.insert_maintenance.enabled` set to off as currently BioseqDB defaults to grouped-MPF which does not support automatic index update on insert. You can disable grouped-MPF to allow automatic index update on insert.

Currently grouped-MPF which was measured in benchmarks for case of query/target table join doesn't work for single table queries correctly and causes minimizers to be always recomputed. To prevent this you need to change minimizer algorithm (the example below shows incremental automatic updates to index on inserts as well):

Listing 1.1: How to avoid index-read bug (always recompute minimizers)

```
-- Disabled grouped-MPF and change minimizer algorithm to something else not
    bounded SIMD
SET bioseq.aligner.grouped_soa_generation.enabled = off;
SET bioseq.aligner.minimizer.algorithm = 'two_stack_nthash';
-- Autmatic index update on each insert:
SET bioseq.index.insert_maintenance.enabled = on;

-- Your INSERT statements go here.
-- No update_index() call is needed for these new rows.
```

This code snippet avoid recomputing minimizer each time. Note that joins without these defaults (grouped-MPF for bounded SIMD) still require `aligner_options(index_read_requirement=>2)` to work correctly.

Please note that in this example I use sequences longer than >30 bases. Due to how filtering works 3.13?? (also in minimap2) extremally short sequences might not produce any hits.

Listing 1.2: A continuous SQL walkthrough of BioseqDB.

SQL or execution plan	Explanation

`\set ON_ERROR_STOP on`
`\pset pager off`
`CREATE EXTENSION bioseq;`
 	
`CREATE TABLE samples (`
`  id integer PRIMARY KEY,`
`  upload_date date NOT NULL,`
`  patient_name text NOT NULL,`
`  age integer NOT NULL,`
`  seq bioseq.seq('dna4', 64) NOT NULL`
`) USING bioseq;`
	1. Create a sequence table. The four ordinary columns hold some patient metadata. The sequences themselves are DNA4 so "N" is not allowed. The base unit width is 64 bits (the value is not maximum sequence size like in case of VARCHAR)
	
	`USING bioseq` makes sure that table is stored in columnar format using BioseqDB. Plese check Figure 8.2 to see how storage layout works and section 6.2 for explanation of bit-packing of sequences.

`WITH dna AS (`
`  SELECT string_agg(translate(md5(i::text),`
`  '0123456789abcdef',`
`  'ACGTACGTACGTACGT'), '' ORDER BY i) AS s`
`  FROM generate_series(1, 64) AS g(i)`
`)`
`INSERT INTO samples`
`SELECT 1, DATE '2026-09-01', 'Anna', 34,`
`  (s || s)::bioseq.seq('dna4', 64) FROM dna`
`UNION ALL`
`SELECT 2, DATE '2026-09-02', 'Ben', 52,`
`  substring(s FROM 257 FOR 512)`
`  ::bioseq.seq('dna4', 64) FROM dna`
`UNION ALL`
`SELECT 3, DATE '2026-09-03', 'Cara', 29,`
`  s::bioseq.seq('dna4', 64) FROM dna;`
	2. Insert patient data. This is the most subjectively ugly way to generate random sequences. I use md5 to generate 32-length hex string that then is traslated to valid DNA4 sequence and concatenated 64 times (giving us 2048 bases). Then I insert data of 3 patients with sequence sizes (4096 - two copies, 512 - substring, 2048 - full sequence)

`SELECT id, upload_date, patient_name, age` `FROM samples;`	3. Read metadata (non-seqence) columns. We can query the table as normally done in Postgres, but the underlying data is currently stored in columnar format

`SELECT id, upper(patient_name) AS patient,`
`  age, bioseq.len(seq) AS bases,`
`  seq[:16]::text AS prefix`
`FROM samples`
`WHERE age BETWEEN 30 AND 60`
`  AND upload_date >= DATE '2026-09-01'`
`  AND (patient_name LIKE 'A%'`
`  OR patient_name LIKE 'B%')`
`ORDER BY age DESC, id;`
	4. Reading sequences. We combine the filter on metadata with extraction of sequence length and one slicing. This returns Ben and Anna, in that order.
	
	`len` reports sequence size, while the slice `[:16]` returns the first sixteen symbols (note that text cast is optional and the coordinates for sequences always start at 0, which is different to Postgres array indexing 6.2)

SQL or execution plan	Explanation

`EXPLAIN (ANALYZE, COSTS OFF)`
`SELECT id, upload_date, patient_name, age`
`FROM samples;`
	5. Read plan for metadata query. You can call `ANALYZE` to see how the simplest BioseqDB scan works under the hood. `COSTS OFF` is to hide the cost estimates of the query as they aren't relevant here.

Custom Scan (**BioseqScan**) on samples (actual time =0.010..0.015 rows=3 loops=1) Bioseq Projected Columns: id, upload_date, patient_name, age –> Cache Cache Hits: 0 Cache Misses: 4 Cache Evictions: 0 Cache Writes: 4 Cache Maximum Size: 60 Cache Ending Size: 60 Total Cache Entries: 4 Planning Time: 0.028 ms Execution Time: 0.031 ms	`BioseqScan` is the scan node thatreads the table. `Projected Columns` lists which columns are required for this query and what the BioseqDB needs to return. The cache stores decompressed column chunks, as described in Table 8.1. Hits and misses count buffer lookups in the cache. Writes count buffers added or replaced, evictions count removed entries from the chunk cache, and entries count current buffers stored in the cache. Maximum and ending sizes are peak and current payload bytes. There is no counter here used for chunk-skipping (min/max pruning), because naturally there is no filter in the query, so the scan cannot skip chunks.

`SELECT seq::text AS probe`
`FROM samples WHERE id = 2 \gset`
 	
`SELECT id, patient_name,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30`
`  AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64));`
	6. Alignmment query `\gset` saves Ben's DNA in the `psql` variable `probe`.
	`mmsearch` here will try to align each patient sequence against Ben's sequence. This returns two matches for Anna and one for Ben. Because they are exact occurences in this very simplistic case, each CIGAR string is `512M`.
	Requesting `cigar` enables additional pipeline stage for base alignment, because the pipeline needs to know exactly which bases were subsituted/inserted or deleted. However, `score` alone does not enable it and can return zero (less exact score is used based only on chaining). (check base alignment Section 5.1 for more details). If you encounter `"transient query execution stats disagree with its generated backend"` error here or in the next step, please check the warning 1 - you should have at least 2GiB of free memory available

SQL or execution plan	Explanation
`SELECT bioseq.update_index(` `  'samples', 'minimizers_per_fragment');`	7. Trigger index rebuild This processes three sequence values and returns 3 as the number of sequences in the index. It extracts minimizers using the current algorithm and k, w, groups their positions, and saves (publishes) the data. Table 9.2 summarizes when indexes are updated. The default uses grouped-MPF format 9.2.

`SELECT id, bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30`
`  AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64));`
	8. Repeated search. The same three matches remain, but now the query uses the index we built

`SELECT id,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30 AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64),`
`  aligner_options(`
`  kmer_size => 15, window_size => 10,`
`  minimizer_algorithm => 4,`
`  chaining_algorithm => 1,`
`  base_alignment_algorithm => 1));`
	Change algorithms for one call I wanted to have an example with explicit algorithm settings to show how we can change either the algorithm for specific stage like chaining or the k/w parameters. You need to pass `aligner_options` to the `mmsearch` function. The codes choose bounded ntHash SIMD minimizers (4), SIMD minimap2 chaining (1), and BlockAligner (1). Each minimizer window contains ten consecutive 15-mers. Each of the parameters correspond to the BioseqDB current defaults so the pipeline works exactly the same as before.
	Unfortunately the global GUC settings use strings for enums, while the `aligner_options` type uses numeric code s (due to my lazyness). Available codes are listed in Appendix 2

`EXPLAIN (ANALYZE, COSTS OFF)`
`SELECT id,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30 AND mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64),`
`  aligner_options(`
`  kmer_size => 15, window_size => 10,`
`  minimizer_algorithm => 4,`
`  chaining_algorithm => 1,`
`  base_alignment_algorithm => 1));`
	9. Explaining the search (lots of fields!) I wanted to present entire plan of the query. It contains a lot of statistics and settings to allow me to more easily troubleshoot BioseqDB behaviour

SQL or execution plan	Explanation
Result (actual time=1.281..1.295 rows=3 loops=1) −> Custom Scan (BioseqScan) on samples (actual time =1.281..1.294 rows=3 loops=1) Bioseq Projected Columns: id, age, seq Bioseq Chunk Group Filters: (age >= 30) Bioseq Chunk Groups Removed by Filter: 0 Bioseq Vectorized Filter: (age >= 30) Bioseq Index Read Requirement: allow Bioseq Recomputed: minimizers 100%	`Result` is just a projection of the `BioseqScan` The scan reads the identifier, age and sequence. A chunk-group filter compares the age condition with stored minima and maxima, but couldn't skip any chunk (because we have so few rows, there's only one chunk). The vectorized filter tests individual ages in the remaining batches 2. `allow` means that we can either read the index or recompute minimizers from scratch 2.14. The planned share is 100% recomputed - this is the estimate of how many minimziers in index do not have a corresponding minimizers stored. In this case BioseqDB have buggy const-query implementation which causes index to always be rebuilt which significantly degrades performance. For better performance you should use join between query and target tables, so both of query/target arguments are red from tables.
Bioseq Scheduler: supervisor=local mode= task_parallel workers=6/6 batches=8/8 awaiting=0 queries=8/0 qps=0.000 async_persist_waiting=0 index_memory_reserved=0 One−to−many alignment: Task−parallel [mode=task_parallel, workers=6, index_memory_reserved=0 bytes]: Planned Architecture: NEON_BF16 CPU−Supported Targets: NEON_BF16, NEON, NEON_WITHOUT_AES, EMU128, SCALAR Highway Compiled Targets: NEON_BF16, NEON, NEON_WITHOUT_AES, EMU128 Runnable Targets: NEON_BF16, NEON, NEON_WITHOUT_AES, EMU128 ∗ Executed Architecture: NEON_BF16 | QuerySketchStage: Bounded u32 ntHash SIMD Minimizers Highway Function: minimizer::bounded_nthash [planned architecture: NEON_BF16]	This part is created by the Postgres process, which executes this query. It reports workers in thread pool, processed/scheduled batches, waiting work and processed/waiting queries. The local supervisor leaves the query-rate field at zero. You can find more about work distribution in Section 7.3.3. Below we have list of architectures for which the binary was compiled. Runnable targets are the intersection of compiled and CPU-supported targets (can be used only if built for this architecture and the current hardware supports it). `QuerySketchStage` lists used minimizer algorithm, and `Highway Function` lists the C++ function name. See Section 3.13.

SQL or execution plan	Explanation
| index_stage: pipeline configuration [cep.window_len: 0] [cep.kmer_len: 0] [cep.sequence_len_log2: 63] [cep.specialised: fallback] | seed_hit_stage: Seed hits + canonical CPU ordering max_occ: 500 query_len: 0 [reserved: 64]	Internally BioseqDB is implemented as type-driven heavily-templated code, so CEP means "constant expression parameter". The zero `cep.window_len`, `cep.kmer_len` and `query_len` values represent the fact that generic implementaion is used (internally I have parameter that can allow instantiation of template specifically for k/w parameters or for specific sequence length, for specific minimizer lengths. Specialized false with all zeros means that these parameters are runtime, not compile-time optimised constexpr parameters - this is default as enabling instantiation produces very large binaries the linker msut work with). `max_occ` limits max counts of minimizers per hash, while `reserved` is a planned anchor-buffer capacity in elements (how many anchors I predict to have - in reality this allocation by default is const, but can grow)
| chain_stage: Minimap2 SIMD min_chain_score: 40 min_seed_count: 3 max_chain_skip: 25 max_chain_iter: 5000 bandwidth: 500 max_gap: 5000 simd_chaining_policy: bounded_batch_approximate [vectorised: true] [requires_zero_skip_penalty: false] [specialised: fallback] Highway **Function**: chaining::chain_anchors_simd [planned architecture: NEON_BF16] | region_stage: Region generation query_len: 0	Chaining joins seed anchors: `min_chain_score` and `min_seed_count`, `max_chain_skip` and `max_chain_iter` are parameters specifically discussed in chaining section ?? `vectorized` in this case means that the C++ function can accept batch instead of single element (all the properties in square brackets are compile-time properties of the code itself)
| primary_secondary_stage: Classify primary/secondary mask_level: 0.500000 mask_len: 2147483647 sub_diff: 8 | secondary_filter_stage: **Filter** secondary regions pri_ratio: 0.800000 min_diff: 30 best_n: 5	Regions are classified as primary or secondary by their overlap. See Section 5.3.

SQL or execution plan	Explanation
| align_stage: Block Aligner match_score: 2 mismatch_pen: 4 gap_open1: 4 gap_ext1: 2 block_min_size: 32 block_max_size: 256 [vectorised: block−affine] [single_affine_only: true] [specialised: fallback] Highway **Function**: alignment::block_aligner [planned architecture: NEON_BF16]	BlockAligner computes the base-level alignment and CIGAR. See Section 5.1.
| primary_secondary_stage: Classify primary/secondary mask_level: 0.500000 mask_len: 2147483647 sub_diff: 8 | secondary_filter_stage: **Filter** secondary regions pri_ratio: 0.800000 min_diff: 30 best_n: 5 ∗ Submit results [targets=2 persisted_index=0 recomputed=2]	Primary/secondary classification and filtering run again after base alignment. `Submit results` reports two processed targets. Both used recomputed minimizers and neither used a persisted index
∗ Execution statistics [requested_backend=simd_cpu actual_backend=mixed_invalid requested_index_source=persisted actual_index_source=recomputed attempted=2 fully_attested=0 unattested=2 environment_mixed=false persisted_identity_mixed=false identity_conflict=false saturated=false] ∗ First unattested [target_row=1 query_ordinal=0] ∗ **Index** outcomes [targets=2 persisted=0 missing_persisted=2 recomputed=2]	These diagnostics are useful to see what is happening inside the BioseqDB and how it chooses available algorithms. `attempted` counts how many target/query jobs were attempted. `fully_attested` and `unattested` and `mixed_invalid` are internally used special flag with counters that aren't useful at the moment. (their purpose was to - after connecting additional validation code - check if the result is valid and if it came from the backend that test code expected) `missing_persisted` is important here as it shows that BioseqDB for queries without join between query and target tables will always use recomputed minimizers for grouped MPF index. This is currently a major bug, that needs to be addressed.
∗ GPU dispatch [seed=0 chain=0 chain_backtrack=0 coordinate_vote=0 extend=0 backend_declines=0 backend_fallbacks=0] GPU Chain Dispatches: 0 GPU Chain Backtrack Dispatches: 0 GPU Coordinate Vote Dispatches: 0 ∗ Host stats [query_sketch_ns=0 target_index_build_ns=0 query_hash_readbacks=0 query_symbol_packing=0 target_symbol_packing=0 alignment_classification=0 cigar_construction=0]	I won't go into details on the GPU counters as the most important is the job dispatch counter. All other were internally used and some of them are just zeroed out currently.

SQL or execution plan	Explanation
−> Cache Cache Hits: 0 Cache Misses: 5 Cache Evictions: 0 Cache Writes: 5 Cache Maximum Size: 16608 Cache Ending Size: 16608 Total Cache Entries: 5 Planning Time: 0.241 ms Execution Time: 1.432 ms	The cache fields have the same meanings as in the first plan. This search needs sequence data as well as metadata, so the buffers and their sizes differ. The final two lines give statement planning and execution time in milliseconds. They include work outside the alignment stages.

`SELECT id,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE mmsearch(seq, seq)`
`ORDER BY id;`
	10. Self-alignment query Each of the three rows produces a hit as natuarally expected sequence should align to itself

`SELECT t.id AS target_id, q.id AS query_id,`
`  bioseq.target_start(t.seq) AS target_start,`
`  bioseq.target_end(t.seq) AS target_end,`
`  bioseq.query_start(t.seq) AS query_start,`
`  bioseq.query_end(t.seq) AS query_end,`
`  bioseq.score(t.seq) AS score,`
`  bioseq.cigar(t.seq) AS cigar`
`FROM samples AS t JOIN samples AS q`
`  ON mmsearch(t.seq, q.seq, aligner_options(`
`  index_read_requirement => 2))`
`ORDER BY target_id, query_id, target_start,`
`  query_start;`
	Compare rows using a self-join. As self-alignment is not realistically useful, we can compare various sequences from our dataset using self-join 7.4
	For target 1 and query 2, the same score occurs at both $[256, 768)$ and $[2304, 2816)$. These are different occurrences (hits) 7.1, even though both have the same cigar `512M`
	`index_read_requirement => 2` is used because otherwise bug in BioseqDB prevents index to be used, which slows down the query.

`COPY (`
`  SELECT line FROM samples`
`  CROSS JOIN LATERAL (VALUES`
`  (0, '>sample_' || id || ' synthetic DNA'),`
`  (1, seq::text)`
`  ) AS fasta(part, line)`
`  ORDER BY id, part`
`) TO '/tmp/bioseq-walkthrough.fasta';`
 	
`CREATE TABLE imported (`
`  header text,`
`  seq bioseq.seq('dna4', 64) NOT NULL`
`) USING bioseq;`
	11. Load a FASTA file into another table. We can load FASTA file with 3 records into the tables. `load` will return the number of records in FASTA file, in this case 3. With one text column, one sequence column and no explicit parameters, the default `auto` mode keeps each full header without the leading `>` in the text column.

SQL or execution plan	Explanation
`SELECT bioseq.load('imported',` `  '/tmp/bioseq-walkthrough.fasta');`	Header parsing options. The header parser for FASTA can accept header format name that supports various databases: `header_format` parmeter accepts `auto`, `plain`, `uni_prot`, `uni_ref`, `uni_parc`, `uni_mes`, `spd`, `ncbi`, `ncbi_gi`, `ncbi_pdb`, `refseq`, `genbank`, `gisaid` and `imgt`. Auto can detect which format the header uses automatically. The last argument of load functionmaps parsed fields to table columns using `bioseq.column_mapping`, with destination first and source second, so if you want to map certain fields of the header to the table column, you can do that!

`SELECT header, bioseq.len(seq) AS bases,`
`  seq[:16]::text AS prefix`
`FROM imported ORDER BY header;`
 	
`SELECT seq[128:384]::text AS short_probe`
`FROM imported`
`WHERE header LIKE 'sample_2 %' \gset`
 	
`SELECT header,`
`  bioseq.target_start(seq) AS target_start,`
`  bioseq.target_end(seq) AS target_end,`
`  bioseq.cigar(seq) AS cigar`
`FROM imported`
`WHERE mmsearch(seq,`
`  :'short_probe'::bioseq.seq('dna4', 64))`
`ORDER BY header, target_start;`
	12. Slices The first query returns three headers, lengths and 16bp slices and the next saves a 256-base slice of the loaded Ben record as `short_probe`
	
	Then we do each in that table.

`SELECT t.id AS target_id, q.header,`
`  bioseq.score(t.seq) AS score,`
`  bioseq.dp_score(t.seq) AS dp_score,`
`  bioseq.dp_max_score(t.seq) AS dp_max_score,`
`  bioseq.chain_score(t.seq) AS chain_score,`
`  bioseq.secondary_chain_score(t.seq)`
`  AS secondary_chain_score,`
`  bioseq.query_len(t.seq) AS query_len,`
`  bioseq.target_len(t.seq) AS target_len,`
`  bioseq.query_start(t.seq) AS query_start,`
`  bioseq.query_end(t.seq) AS query_end,`
`  bioseq.target_start(t.seq) AS target_start,`
`  bioseq.target_end(t.seq) AS target_end,`
`  bioseq.num_matches(t.seq) AS num_matches,`
`  bioseq.block_length(t.seq) AS block_length,`
	13. Every possible alignment field This example shows each possible value we can get from the aligner pipeline: `score` and `dp_score` are aliases for the same value which is computed minimap2 score. `chain_score` describes the seed chain score.
	
	`block_length` is similar to PAF in minimap2 - total alignment block length i.e length of the expanded CIGAR string.
	
	`num_matches` - number of residue matches in minimap2 so practically number of M in CIGAR string

SQL or execution plan	Explanation

`  bioseq.edit(t.seq) AS edits,`
`  bioseq.mapq(t.seq) AS mapq,`
`  bioseq.seed_count(t.seq) AS seed_count,`
`  bioseq.is_reverse(t.seq) AS is_reverse,`
`  bioseq.is_primary(t.seq) AS is_primary,`
`  bioseq.has_cigar(t.seq) AS has_cigar,`
`  bioseq.cigar(t.seq) AS cigar,`
`  bioseq.divergence(t.seq) AS divergence`
	`edit` counts alignment edits. `mapq` estimates mapping quality (similar to mapq minimap2), while `divergence` is the score estimated from seeds. `seed_count` counts how many anchors were there before chaining started.
	The Boolean fields report strand orientation, chain classification and CIGAR.

`FROM samples AS t JOIN imported AS q`
`  ON mmsearch(t.seq, q.seq,`
`    aligner_options(index_read_requirement => 2))`
`ORDER BY score DESC, target_id, q.header,`
`  target_start, query_start;`
	This join requires the compatible persisted target index built in step 7 9.7.1 and as mentioned before currently due to bugs in implementation of grouped-MPF index this JOIN correctly reads index, while default read policy ignores it. Moreover as mentioned alignment without join will also ignore index.

`SELECT count(*) AS patients,`
`  min(age) AS youngest, max(age) AS oldest,`
`  sum(age) AS total_age, avg(age) AS mean_age,`
`  min(upload_date) AS first_upload,`
`  max(upload_date) AS last_upload`
`FROM samples`
`WHERE age >= 30`
`  AND upload_date >= DATE '2026-09-01';`
	14. Aggregate metadata This returns two patients, minimum age 34, maximum age 52, sum 86 and average 43. The dates span 1–2 September. By default aggregates will filter-out chunks due to min/max pruning.

`EXPLAIN (ANALYZE, COSTS OFF)`
`SELECT count(*) AS patients,`
`  min(age) AS youngest, max(age) AS oldest,`
`  sum(age) AS total_age, avg(age) AS mean_age,`
`  min(upload_date) AS first_upload,`
`  max(upload_date) AS last_upload`
`FROM samples`
`WHERE age >= 30`
`  AND upload_date >= DATE '2026-09-01';`
	Explain now the query.

`Custom Scan (VectorAggNode) (actual time=0.067..0.067`
`  rows=1 loops=1)`
`  -> Custom Scan (BioseqScan) on samples (actual time`
`  =0.058..0.063 rows=1 loops=1)`
`  Bioseq Projected Columns: upload_date, age`
`  Bioseq Chunk Group Filters: ((age >= 30) AND (`
`  upload_date >= '2026-09-01'::date))`
`  Bioseq Chunk Groups Removed by Filter: 0`
`  Bioseq Vectorized Filter: ((age >= 30) AND (`
`  upload_date >= '2026-09-01'::date))`
`...`
	`VectorAggNode` is the BioseqDB vector aggregate. Row number doesn't reflect number of rows in the result.
	Because number of rows in small the chunk filter didn't filter any chunks. The vectorized filter allowed to evaluate filter on each row in the batch. 7.3.6

SQL or execution plan	Explanation

`UPDATE samples SET age = age + 1`
`WHERE id = 1`
`RETURNING id, patient_name, age;`
 	
`INSERT INTO samples`
`VALUES (4, DATE '2026-09-04', 'Daria', 41,`
`  :'probe'::bioseq.seq('dna4', 64))`
`RETURNING id, patient_name, bioseq.len(seq);`
 	
`SELECT bioseq.update_index(`
`  'samples', 'minimizers_per_fragment');`
`SELECT bioseq.update_index(`
`  'samples', 'symbol_counts');`
	15. Update/insert and index rebuild Anna becomes 35 and Daria receives Ben's sequence. There are now four stored sequences.
	
	With insert maintenance off, these writes do not build new minimizer indices, so unfortunately you need to manually rebuild indices. The two explicit calls prepare MPF and symbol-count indexes. See Figure 8.9 and Table 9.2.

`SELECT index_name, update_policy,`
`  storage_size, time_since_update,`
`  bioseq.index_table_row_count(`
`    'samples', index_name) AS physical_rows`
`FROM bioseq.list_metadata()`
`WHERE table_name = 'samples'::regclass`
`ORDER BY index_name;`
	MPF and symbol counts have stored data, while we don't touch MPS in this example.
	
	`storage_size` measures the index relations, while physical rows count internal index records (not patients!) The update age is the time since the recorded build. The update policy here will return default 9.3 `UPDATE_ON_INSERT`

`SELECT id, bioseq.score(seq) AS score,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE mmsearch(seq,`
`  :'probe'::bioseq.seq('dna4', 64),`
`  aligner_options(`
`    minimizer_algorithm => 0,`
`    chaining_algorithm => 0,`
`    base_alignment_algorithm => 0,`
`    index_read_requirement => 1))`
`ORDER BY id;`
	Try scalar minimizers, scalar chaining and KSW2. The three algorithm codes are all 0. The query returns five matches.
	
	Adding read requirement 1 forbids persisted target-index reads for this call. That matters because the stored index was built with a different minimizer algorithm and otherwise BioseqDB can produce an error

`SELECT id,`
`  bioseq.symbol_count(seq, 'A') AS adenines,`
`  (seq[:8] OPERATOR(bioseq.||) seq[-8:])`
`    ::text AS joined_ends,`
`  seq[0]::text AS first_letter,`
`  bioseq.reverse(seq[:8])::text AS reversed,`
`  bioseq.reverse_complement(seq[:8])::text`
`    AS reverse_complement,`
`  bioseq.reverse(bioseq.reverse_complement(`
`    seq[:8]))::text AS complemented,`
`  bioseq.edit(seq) AS edits,`
`  bioseq.cigar(seq) AS cigar`
`FROM samples`
`WHERE age >= 30`
`  AND mmsearch(seq,`
`    :'probe'::bioseq.seq('dna4', 64))`
`  AND bioseq.edit(seq) < 5`
`ORDER BY id;`
	Combine sequence operations and an edit filter. This returns four alignment rows for Anna, Ben and Daria. It counts A bases, joins two 8bp slices, reads first letter of sequence, and reverses or reverse-complements a prefix.
	
	The predicate `bioseq.edit(seq) < 5` enables experimental separate stage in aligner pipeline to estimate edit distance before alignement and filter out candidates as exampled in Section 5.4

Appendix 2

Configuration reference

This appendix lists all runtime settings registered by BioseqDB (GUCs) - their types, defaults, possible values and descriptions. Each setting can be viewed and set with the `SHOW` and `SET` commands.[1] Some settings are hidden so they're not visible with `SHOW ALL`.

Listing 2.1: Inspecting and settingBioseqDB config values

```
SHOW bioseq.aligner.minimizer.algorithm;

SELECT name, setting, boot_val, reset_val,
       vartype, context, min_val, max_val, enumvals
FROM pg_settings
WHERE name = 'bioseq.aligner.minimizer.algorithm';

SET bioseq.aligner.minimizer.algorithm = 'scalar';
```

2.1. Physical storage

Table 2.1: Physical storage settings.

GUC name	Type	GUC values	Description
`bioseq.storage.compression.algorithm`	enum	`zstd` (default)	Choose the compression algorithm used for storing data columns Use Zstandard when compiled in. This is the preferred build-time default. Changing the default does not rewrite any existing indices!
		`none`	Store column chunks without compression.
		`pglz`	Use Postgres built-in coded. This becomes the default when neither of previous two is available.
		`lz4`	Use LZ4 when compiled in. This becomes the default when Zstandard is unavailable.

Continued on the next page

[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation: pg_settings; PostgreSQL Global Development Group, PostgreSQL 17: Configuration Inspection Functions.

GUC name	Type	GUC values	Description
`bioseq.storage.compression.level`	int	`3` (default) 1…19	Set the zstd compression level if zstd is used.
`bioseq.storage.stripe.row_limit`	int	`150000` (default) 1000…100000000 rows	Sets the maximum rows in a new table's column stripe, when writing rows. Existing table options and stored stripes are unchanged.
`bioseq.storage.chunk_group.row_limit`	int	`10000` (default) 1000…100000000 rows	Sets the maximum rows in a new table's chunk group, when writing rows. Chunk groups divide a stripe into smaller column-reading units.
`bioseq.storage.dml.enabled` hidden	bool	`on` (default)	Special setting that controls whether reads apply deleted-row masks. Default, shouldn't be changed. Normally deleted rows that are marked as such, should not be returned in queries.
		`off`	Bypass any deletion filtering. Useful for debugging, the queries will return all rows including deleted ones. Note that if the table is rewritten using VACCUM FULL, then deleted rows will physically dissapear!
`bioseq.column_cache.enabled`	bool	`on` (default)	Controls the backend's decompressed column-chunk cache. After decompression save the buffer, so it's avaialable for reuse Decompression cache only applies to one query. After completion of the query, the cache is cleared.
		`off`	Read and decompress but do not save buffers in cache.
`bioseq.column_cache.capacity_mb`	int	`200` (default) 20…20000 MiB	Limits the decompressed column-chunk cache BioseqDB will still allocate buffers for decompression, so this setting does not control the total memory used during decompression
`bioseq.load.write_overlap.enabled`	bool	`off` (default)	Experimental setting to use parallelized writing and decoding of data chunks through queue when loading FASTA/FASTQ files. Default reader perform decoding and writing (placing sequences in buffer) sequentially.
		`on`	Enable parallelized writing and decoding of data chunks through queue.
`bioseq.storage.sequence_shard.max_size_gib` superuser	int	`1024` (default) 0…2147483647 GiB	Sets the sequence-shard size limit. If the shard table exceeds that size, then new table will be created. Setting it to 0 means that each sequence is stored in a separate table (useful for debugging, but extremally ineffective in practice)

2.2. Planning and execution

Table 2.2: Planning and execution settings.

GUC name	Type	GUC values	Description
`bioseq.execution.parallel.enabled` hidden	bool	`on` (default)	Enable Postgres parallel queries Allow possible parallel scan paths (actual worker availability still depends on PostgreSQL planner, so this setting does not force the planner to use parallel scans). Additionally, any query using mmsearch won't be parallelized, due to current code architecture. The parallel scans however are possible for metadata queries. Alignment won't use parallel scans even with this setting enabled.
		`off`	Disable all parallel scans on BioseqDB tables.
`bioseq.execution.parallel.minimum_processes`	int	`8` (default) 1…32	Requests the PostgreSQL worker count for possbible Bioseq parallel scan paths. This is soft minimum and the actual worker count is determined by the PostgreSQL planner.
`bioseq.vectorization.enabled` hidden	bool	`on` (default)	Can disable vectorized SQL operations for aggregates like SUM(), COUNT(), etc. Use supported vectorized SQL operations when the plan allows them. Vectorized operations only apply to non-sequence aggregates, not alignment pipeline.
		`off`	Use normal SQL processing for those operations.
`bioseq.planner.index_scan.enabled` hidden	bool	`on` (default)	Can allow usage of non-BioseqDB indices with BioseqDB tables. Allow using non-BioseqDB indices during queries, for example it becomes possible to use B-tree index on metadata column. This settings does not control any BioseqDB indices like MPF/MPS etc.
		`off`	Do not allow using any non-BioseqDB indices on BioseqDB tables. This can heavily degrade performance as Postgres will select plan that ignores existing B-trees, hash indexes etc.
`bioseq.planner.custom_scan.enabled` hidden	bool	`on` (default)	Entirely disable all BioseqDB custom scans. Allow the planner to install Bioseq custom scans. Useful to force Postgres to select non-BioseqDB plans. Do not use this unless you know what you're doing.
		`off`	Disable custom scans. This means that it won't be possible to use 'mmsearch' function anymore as it requires custom scan path. All other metadata queries still will be possible.

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.planner.custom_scan.max_paths hidden	int	64 (default) 1…1024	Limit paths explored by GQO for BioseqDB custom scans. Increasing the number may yield better plans, but increase planning time.
bioseq.planner.qual_pushdown.enabled hidden	bool	on (default)	Controls behaviour of predicate pushdown. Normally filters can be pushed down into the scan, which improved performance. The predicates that Postgres reports as highly-correlated will be used to possibly reduce number of loaded chunks (min/max prunning mechanism) Pushdown uses correlation statistics for column reported by Postgres to decide if this predicate can be useful to avoid loading chunks
		off	Leave predicates outside the scan, which may severely degrade performance.
bioseq.planner.qual_pushdown.correlation_threshold hidden	real	0.4 (default) 0…1	Sets the minimum absolute column correlation value (reported by Postgres table statistics) used by chunk prunning logic described for bioseq.planner.qual_pushdown.enabled. Values closer to one, mean stronger correlation.
bioseq.planner.sequence_sort_scan.enabled hidden	bool	off (default)	Enable or disable custom sort for BioseqDB sequences. Sort works only if there's single sorting key that is sequence column itself. Use default Postgres sort.
		on	Consider the custom batch-sort where possible.

2.3. Task scheduling

Table 2.3: Task scheduling settings.

GUC name	Type	GUC values	Description
bioseq.scheduler.backend.max_concurrency reload	int	0 (default) 0…1024	Limits maximum number of workers in TBB worker pool (this is separate to Postgres max workers setting)
bioseq.scheduler.pipeline.mode	enum	task_parallel (default)	Changes how the pipeline execution is performed Each query dispatches separate worker (exxtension of parallel_dispatch) and additionally each stage of pipeline can submit independent tasks to TBB worker pool In the benchmark I use task_parallel, because it enables the highest degree of parallelism
		single_thread	Each query dispatches separate worker, but one pipeline itself isn't parallelized.
		parallel_dispatch	

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.scheduler.postgres_parallel.enabled`	bool	`off` (default)	Reduce tasks slots to 1 Disable any parallelism completely (note that if bioseq.scheduler.pipeline.mode is set to task_parallel, then pipeline will be executed via task queues, but only one worker will be active at the time. This setting is the highest level limiter on parallelism of workers) This setting effectively forces bioseq.scheduler.backend.max_concurrency to be 1
		`on`	Allow >1 parallel tasks limit

2.4. Index construction and reading

Table 2.4: Index construction and reading settings.

GUC name	Type	GUC values	Description
`bioseq.index.insert_maintenance.enabled`	bool	`off` (default)	Enables automatic index incremental update when row is inserted into the table. Save new source rows without automatic index builds. You can always still manually update index via update_index function.
		`on`	Currently grouped MPF index (the default one) does not support this function and reports an error.
`bioseq.index.sequence_read.enabled`	bool	`on` (default)	Controls whether the pipeline executor can read indexes. If the option is disabled, then pipeline will always recompute all seeds from scratch. Normally allow reading BioseqDB indices. There's also per-search option passed via aligner_options: index_read_requirement codes 1 and 2 override this setting.
		`off`	Force pipeline to always recompute all seeds from scratch.

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.index.async_persistence.enabled`	bool	`off` (default)	Very experimental setting that allows main process to asynchronously update missing indices for sequences: If query does not have seeds in index for certain sequence and needs to compute them, after the query the index will be updated. Do not stage asynchronous persistence work. Grouped MPF index (default) does not implement this feature and using it with this option enabled will produce an error. Additionally you need to enable bioseq.index.recompute_async_persistence.enabled as well.
		`on`	Allow the limited ordinary-MPS submission path when its other requirements are met.
`bioseq.index.recompute_async_persistence.enabled`	bool	`off` (default)	Separate setting, but effectively to enable async persistence you need to enable bioseq.index.async_persistence.enabled together with this one. Same as bioseq.index.async_persistence.enabled Set bioseq.index.async_persistence.enabled to the same value as this one.
		`on`	Same as bioseq.index.async_persistence.enabled
`bioseq.index.memory_cap_mb`	int	`0` (default) `0`…`2147483647` MiB	Specialized limit on the memory allocated by index builders/readers in MB. Using 0 means that they share the same memory budget as other query buffers.
`bioseq.index.sequence_minimizer_sort.algorithm` hidden	enum	`std_sort` (default)	Change sorting algorithm used for non-grouped MPF index builds Use the standard-library sort SIMD versions weren't tested by me in terms of correctness and they were slower than the default
		`simd_u128_vqsort`	Use Highway VQSort on packed 128-bit keys (implementation from C++ Highway repo).
		`simd_u128_vec_stream_1`	Use the custom SIMD sorter with one vector stream.
		`simd_u128_vec_stream_2`	Use the custom SIMD sorter with two vector streams.
		`simd_u128_vec_stream_4`	Use the custom SIMD sorter with four vector streams.
`bioseq.index_build.run_buffer_bytes` hidden, superuser	int	`0` (default) `0`…`2147483647` bytes	For MPS overrides the external-sort run buffer size. Zero computes a size from the common BioseqDB memory budget.

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.aligner.fragment_minimizer.lookup_route	enum	direct_compact_hash_lookup (default)	Changes how grouped MPF works when reading is performed By default read hash directly from the hash bucket. The directory row should be faster in theory, but it's disabled, because I'm not sure if it's correct
		control_block_scan	Search the directory row for the hash bucket first
bioseq.aligner.grouped_soa_generation.enabled	bool	on (default)	Selects if we want to use grouped MPF version of index or non-grouped one (for writing only) Prefer grouped version of MPF index over non-grouped (for writing only) Changing this does not rewrite existing indexes, only applies to new ones and does not affect reading!
		off	Use only non-grouped MPF when possible (for writing only)

2.5. Minimizers and seed filtering

Table 2.5: Minimizers and seed filtering settings.

GUC name	Type	GUC values	Description
bioseq.aligner.minimizer.kmer_size	int	15 (default) 1…64 symbols	Sets the number of sequence symbols in each k-mer Per search: kmer_size.
bioseq.aligner.minimizer.window_size	int	10 (default) 1…1024 k-mers	Sets the number of consecutive k-mers in one minimizer window Per search: window_size.
bioseq.aligner.minimizer.algorithm	enum	bounded_nthash_simd code 4 (default)	Chooses how minimizers are calculated Use the bounded SIMD BioseqDB algorithm Changing the algorithm can invalidate compatibility with saved indexes! Per search: minimizer_algorithm.
		scalar code 0	Minimap2 scalar algorithm
		simd_minimap2_reference code 1	Use the naive version of SIMD implementation of minimap2's
		two_stack_nthash code 2	Use rolling ntHash and two-stack window-minimum inspired by simd-minimizers paper.
		vulkan_minimap_integer code 3	Use BioseqDB GPU algorithm

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.aligner.seeding. maximum_occurrence	int	500 (default) 1…16777216 positions	Excludes minimizer keys with more target occurrences than this limit. Per search: max_occ.
bioseq.aligner.seeding. intermediate_occurrence_ threshold	int	500 (default) 1…16777216 positions	Currently unused Per search: mid_occ.

2.6. Chaining

Table 2.6: Chaining settings.

GUC name	Type	GUC values	Description
bioseq.aligner.chaining. maximum_gap	int	5000 (default) 1…1073741824 positions	Sets the requested maximum anchor gap used by chaining. Scalar (minimap2)chaining raises this limit to at least the bandwidth value. Per search: max_gap.
bioseq.aligner.chaining. maximum_reference_gap	int	-1 (default) −1…1073741824 positions	Currently unused Per search: max_gap_ref.
bioseq.aligner.chaining. bandwidth	int	500 (default) 1…1048576 positions	Despite the name it's also used by aligner stage, limits maximum distane to diagonal. Per search: bandwidth.
bioseq.aligner.chaining. long_join_bandwidth	int	0 (default) 0…1048576 positions	Currently unused. Per search: bandwidth_long.
bioseq.aligner.chaining. maximum_skipped_anchors	int	25 (default) 0…1048576	Limit of consecutive updates for anchors that do not improve score (only for supported algorithms like scalar) Per search: max_chain_skip.
bioseq.aligner.chaining. maximum_predecessors	int	5000 (default) 1…4194304	Limits how many predecessors chainer can consider, but scalar chaining (depending on the actual algorithm) can examine one additional candidate. RMQ does not use this field. Per search: max_chain_iter.
bioseq.aligner.chaining. minimum_seed_count	int	3 (default) 0…1048576	Sets the minimum seed count for an accepted chain. Per search: min_seed_count.
bioseq.aligner.chaining. minimum_score	int	40 (default) 0…1048576	Sets the minimum accepted chain score (the score is computed by the selected chaining algorithm) Per search: min_chain_score.

Continued on the next page

GUC name	Type	GUC values	Description
bioseq.aligner.chaining.gap_penalty_scale	real	0.8 (default) 0…100	Scales the gap penalty used by chaining (larger values penalize that contribution more strongly) Per search: chain_gap_scale.
bioseq.aligner.chaining.skip_penalty_scale	real	0.0 (default) 0…100	Scales the chaining skip-distance penalty. Per search: chain_skip_scale.
bioseq.aligner.chaining.algorithm	enum	simd_minimap2 code 1 (default)	Selects algorithm used for chaining BioseqDB implementation of accelerated minimap2-inspired chaining Selected algorithm might still be rejected if the hardware does not support it Per search: chaining_algorithm.
		scalar_minimap2 code 0	Minimap2 scalar chaining
		range_minimum_query code 2	BioseqDB RMQ on AVL trees similar to what minimap2 uses
		vulkan_tiled_dynamic_programming code 3	Use tiled GPU chaining

2.7. Primary and secondary alignments

Table 2.7: Primary and secondary alignments settings.

GUC name	Type	GUC values	Description
bioseq.aligner.secondary.overlap_threshold	real	0.5 (default) 0…1	Sets the primary/secondary classification thresholds Per search: mask_level.
bioseq.aligner.secondary.uncovered_length_limit	int	2147483647 (default) 0…2147483647 symbols	Limits the uncovered query length used when classifying an alignment as secondary to an existing primary. Per search: mask_len.
bioseq.aligner.secondary.score_ratio_threshold	real	0.8 (default) 0…1	Sets the secondary-to-primary score-ratio threshold. Zero bypasses this filtering step, including its secondary-count limit. Per search: pri_ratio.
bioseq.aligner.secondary.maximum_count	int	5 (default) 0…1024	Limits retained secondary alignments across the complete list passed to the filtering stage Per search: best_n.

2.8. Base alignment

Table 2.8: Base alignment settings.

GUC name	Type	GUC values	Description
`bioseq.aligner.scoring.match_score`	int	`2` (default) 1…127	Sets the score added for an aligned matching symbol. Per search: `match_score`.
`bioseq.aligner.scoring.mismatch_penalty`	int	`4` (default) 1…127	Sets the score penalty for an aligned mismatch. Per search: `mismatch_pen`.
`bioseq.aligner.scoring.short_gap_open_penalty`	int	`4` (default) 0…127	Sets the first gap-opening penalty. KSW2 and BlockAligner use this pair. BlockAligner requires positive opening and extension penalties whose sum is at most 128. Per search: `gap_open1`.
`bioseq.aligner.scoring.short_gap_extension_penalty`	int	`2` (default) 0…127	Sets the first gap-extension penalty. BlockAligner requires it to be positive and its sum with the first opening penalty to be at most 128. Per search: `gap_ext1`.
`bioseq.aligner.scoring.long_gap_open_penalty`	int	`24` (default) 0…127	Sets KSW2's second gap-opening penalty. Native BlockAligner uses only the first gap-penalty pair. Per search: `gap_open2`.
`bioseq.aligner.scoring.long_gap_extension_penalty`	int	`1` (default) 0…127	Sets KSW2's second gap-extension penalty. Native BlockAligner does not use this second pair. Per search: `gap_ext2`.
`bioseq.aligner.scoring.ambiguous_base_penalty`	int	`1` (default) 0…127	Sets the alignment penalty for ambiguous symbols on paths that support them. Per search: `sc_ambi`.
`bioseq.aligner.scoring.z_drop_threshold`	int	`400` (default) −1…1048576	Sets KSW2's Z-drop threshold or BlockAligner's X-drop threshold for end extensions. BlockAligner extensions require a nonnegative value. Per search: `zdrop`.
`bioseq.aligner.scoring.end_bonus`	int	`-1` (default) −1…1048576	Sets KSW2's sequence-end bonus. Negative one disables the bonus. Native BlockAligner does not use it. Per search: `end_bonus`.
`bioseq.aligner.block_alignment.minimum_block_size`	int	`32` (default) 8…4096	Sets BlockAligner's minimum adaptive block size. (block sizes must be powers of two and the minimum cannot exceed the maximum) Per search: `block_min_size`.
`bioseq.aligner.block_alignment.maximum_block_size`	int	`256` (default) 8…16384	Sets BlockAligner's maximum adaptive block size. (block sizes must be powers of two and the minimum cannot exceed the maximum) Per search: `block_max_size`.

`bioseq.aligner.base_alignment.algorithm`	enum	`block_aligner`
		code 1
		(default)
		`ksw2`
		code 0
			Selects the base alignment algorithm Adaptive block aligner Per search: `base_alignment_algorithm`.
			Use KSW2 aligner

2.9. Vulkan execution

Table 2.9: Vulkan execution settings.

GUC name	Type	GUC values	Description
`bioseq.aligner.vulkan.enabled`	bool	`on` (default)	Enables GPU execution Allow supported GPU work GPU algorithms currently are not optimised and severly increase the query time. Enabling execution doesn't mean the GPU will be used (for example software-emulated GPUs are disabled by default). The correct GPU algorithm also need to be selected by setting respective GUCs for aligner/chaining. Per search: `vulkan_enabled`.
		`off`	Setting aligner/chaining algorithms (or other) to Vulkan will immediately fail.
`bioseq.aligner.vulkan.seed_lookup.enabled`	bool	`off` (default)	Enabled GPU-acclelerated minimizers (reading) Disable GPU for reading minimizer indexes Per search: `vulkan_seed_hits_enabled`.
		`on`	Enable GPU minimizer lookup (but other GPU settings apply)
`bioseq.aligner.vulkan.index_cache.enabled`	bool	`on` (default)	Cache minimizer index data to be reused across GPU calls (applies only to single pipeline execution and transfers between GPU/CPU memory) Save buffer with minimizers Per search: `vulkan_index_cache_enabled`.
		`off`	GPU doesn't reuse minimizers during computation
`bioseq.aligner.vulkan.fused_frontend.enabled`	bool	`off` (default)	Enable experimental GPU fused kernel (this doesn't work correctly) Leave this frontend unselected.
		`on`	Enable special optimisation for GPU kernels
`bioseq.aligner.vulkan.index_build.enabled`	bool	`off` (default)	Enabled GPU-acclelerated minimizers (writing) Disable GPU for wrikting minimizer indexes
		`on`	Enable GPU minimizer index building (but other GPU settings apply)
`bioseq.aligner.vulkan.alignment_extension.enabled`	bool	`off` (default)	Enabled CIGAR computation on GPU Disbable CIGAR computation on GPU
		`on`	Allow computing CIGAR on GPU (but other GPU settings apply)
`bioseq.aligner.vulkan.ordered_sketch_tile_bp`	int	`2048` (default) 1…16777216 symbols	Currently unused

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.aligner.vulkan.radix_sort_digit_bits`	int	`8` (default) 1…16 bits	Currently unused

2.10. Diagnostics

Table 2.10: Diagnostics settings.

GUC name	Type	GUC values	Description
`bioseq.diagnostics.logging.enabled` superuser	bool	`off` (default)	Enable or dissable internal logging. BioseqDB internally supports logging to file, but current builds are all configured to use only postgres logging mechanisms. Disable internal logger.
		`on`	Enable internal logger.
`bioseq.planner.diagnostics.log_level`	enum	`debug3` (default)	Chooses the PostgreSQL severity for planner diagnostic messages (logging levels are Postgres specific). Emit them at DEBUG3 severity. PostgreSQL's message thresholds determine where the messages appear.
		`debug5`	Emit them at DEBUG5 severity.
		`debug4`	Emit them at DEBUG4 severity.
		`debug2`	Emit them at DEBUG2 severity.
		`debug1`	Emit them at DEBUG1 severity.
		`log`	Emit them at LOG severity.
		`info`	Emit them at INFO severity.
		`notice`	Emit them at NOTICE severity.
		`warning`	Emit them at WARNING severity.
`bioseq.aligner.profiling.enabled`	bool	`off` (default)	Enables internal profiling of the pipeline execution profiling measures statistics like number of minimizers/chains/timings per pipeline stage etc.
		`on`	Profiling can increase execution time

2.11. Optional query-result cache

Table 2.11: Optional query-result cache settings.

GUC name	Type	GUC values	Description
`bioseq_cache.query_cache.mode`	enum	`disabled` (default)	Enables experimental query result cache (provided by BioseqDB bioseq_cache subextension) Do not use the query result cache (disabled due to my low confidence if it works correctly) Any mode other than 'disabled' required Postgres shared libraries reload.
		`exact_query`	Reuse completed identical requests and coordinate concurrent identical computations.
		`reusable_fragments`	Permit completed-fragment reuse for providers that support it. The Bioseq alignment adapter currently supports exact reuse only (this mode is not implemented)
`bioseq_cache.query_cache.completed_result_ttl_ms`	int	`3600000` (default) 0…2147483647 ms	Maximum age of the records in the bioseq_cache tables that can be reused

2.12. Controls present only in test builds

Table 2.12: Settings available only in test builds.

GUC name	Type	GUC values	Description
`bioseq.testing.chunked_scanner.enabled` hidden, superuser	bool	`off` (default)	Forces BioseqDB to load entire sequence into memory and create 'fake' lazy-loading interface that accesses it. Used by me to test C++ interface for lazy-loading. Use the normal scanner.
		`on`	Force load entire sequence into memory and create lazy-loader interface for that RAM sequence.
`bioseq.testing.in_memory_scanner.enabled` hidden, superuser	bool	`off` (default)	Forces BioseqDB to load entire sequence into memory and passes it directly to the pipeline. (Note that this is the same for user as bioseq.testing.chunked_scanner.enabled with the only difference being internally used C++ API). Used by me to test C++ interface for lazy-loading. Use the normal scanner.
		`on`	Force load entire sequence into memory and pass it directly to the pipeline.

Continued on the next page

GUC name	Type	GUC values	Description
`bioseq.testing.persisted_index.failure_mode` hidden, superuser	enum	`none` (default) `unsupported_vector_shape`	Cause 'fake' index reader failure to be reported during queries. Do nothing. Make the index reader report an unsupported vector shape (due to wrong w/k etc. parameters used during index build).

2.13. Per-search options

Additionally instead of operating on global settings, user can also provide single object as third parameter to the `mmsearch` function call.

Parameter encoding

Currently the enum options are encoded by integers - they correspond to the global GUCs, but you can use only numbers. This is just unimplemented feature - the types of the options should be changed to strings and encoded to internal enum int values. It was requiring less effort to expose internal representation, so it was left as is, see listing below 2.2.

Encoding of parameters as integer values:

Listing 2.2: Passing options by 3rd argument of mmsearch

```
SELECT (public.aligner_options(
  minimizer_algorithm => 0,
  index_read_requirement => 1)).*;
```

Every options is available below, for meaning and usage check the GUC tables:

Table 2.13: All nullable alignment options (for `aligner_options` call) in the order as specified in the SQL API

Options field	SQL type	Inherited GUC or independent policy
`kmer_size`	int	`bioseq.aligner.minimizer.kmer_size`
`window_size`	int	`bioseq.aligner.minimizer.window_size`
`max_gap`	int	`bioseq.aligner.chaining.maximum_gap`
`max_gap_ref`	int	`bioseq.aligner.chaining.maximum_reference_gap`
`bandwidth`	int	`bioseq.aligner.chaining.bandwidth`
`bandwidth_long`	int	`bioseq.aligner.chaining.long_join_bandwidth`
`max_chain_skip`	int	`bioseq.aligner.chaining.maximum_skipped_anchors`
`max_chain_iter`	int	`bioseq.aligner.chaining.maximum_predecessors`
`min_seed_count`	int	`bioseq.aligner.chaining.minimum_seed_count`
`min_chain_score`	int	`bioseq.aligner.chaining.minimum_score`
`chain_gap_scale`	double	`bioseq.aligner.chaining.gap_penalty_scale`
`chain_skip_scale`	double	`bioseq.aligner.chaining.skip_penalty_scale`

Continued on the next page

Options field	SQL type	Inherited GUC or independent policy
mask_level	double	bioseq.aligner.secondary.overlap_threshold
mask_len	int	bioseq.aligner.secondary.uncovered_length_limit
pri_ratio	double	bioseq.aligner.secondary.score_ratio_threshold
best_n	int	bioseq.aligner.secondary.maximum_count
match_score	int	bioseq.aligner.scoring.match_score
mismatch_pen	int	bioseq.aligner.scoring.mismatch_penalty
gap_open1	int	bioseq.aligner.scoring.short_gap_open_penalty
gap_ext1	int	bioseq.aligner.scoring.short_gap_extension_penalty
gap_open2	int	bioseq.aligner.scoring.long_gap_open_penalty
gap_ext2	int	bioseq.aligner.scoring.long_gap_extension_penalty
sc_ambi	int	bioseq.aligner.scoring.ambiguous_base_penalty
zdrop	int	bioseq.aligner.scoring.z_drop_threshold
end_bonus	int	bioseq.aligner.scoring.end_bonus
max_occ	int	bioseq.aligner.seeding.maximum_occurrence
mid_occ	int	bioseq.aligner.seeding.intermediate_occurrence_threshold
block_min_size	int	bioseq.aligner.block_alignment.minimum_block_size
block_max_size	int	bioseq.aligner.block_alignment.maximum_block_size
chaining_algorithm	int	bioseq.aligner.chaining.algorithm Integer codes should be checked in the GUC table.
base_alignment_algorithm	int	bioseq.aligner.base_alignment.algorithm Integer codes should be checked in the GUC table.
minimizer_algorithm	int	bioseq.aligner.minimizer.algorithm Integer codes should be checked in the GUC table.
vulkan_enabled	bool	bioseq.aligner.vulkan.enabled
vulkan_seed_hits_enabled	bool	bioseq.aligner.vulkan.seed_lookup.enabled
vulkan_index_cache_enabled	bool	bioseq.aligner.vulkan.index_cache.enabled
index_read_requirement	int	No GUC. See Table 2.14.

2.13.1. The index-read requirement

This is special field that has no GUC equivalent, that is only available via `aligner_options`. This is special flag very useful during testing and controls how index reader behaves.

Table 2.14: Special option controling index reader

Options field	Type	Value	Description
`index_read_requirement`	int	0: **`allow`** (default)	If no compatible index is available, seeds can be recomputed from scratch. This is the most useful in practical sense as it allows the largest flexibility for user.
		1: `require_absent`	Never read index even if it exists, always recompute seeds from scratch. Additionally setting this option will produce errors if deffered index update is ON. Practically used in testing scenarios to see what happens if index does not exist.
		2: `require_compatible_persisted`	Hard requirement that index must allways exist. If the index does not exist or is incompatible (different k/w parameters etc.), the query will produce error before execution.